cell-molecular-biology-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `cell_molecular_biology_ai999x.about()` for the same answer.

Every capability cell-molecular-biology-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import cell_molecular_biology_ai999x

result = cell_molecular_biology_ai999x.solve('cmb_fick_flux', D='1e-09 m^2/s', C1='10 mol/m^3', C2='2 mol/m^3', dx='1e-05 m')
result.show()
# J = 0.0008 mol/(m^2*s), J_nmol_cm2_s = 80
```

By topic

topic	capabilities
Cell growth / cell counting	Cell concentration from a hemocytometer count, Colony-forming units per mL from a plate count, Transformation efficiency
Cell growth / cell cycle	Cell-cycle phase durations from phase fractions, Cell-cycle phase durations corrected for the exponential age distribution
Cell growth / cell size	Volume, surface area and surface-to-volume ratio of a spherical cell, Number of molecules in a cell from a concentration
Cell growth / doubling time	Doubling time and number of generations from two counts
Cell growth / exponential growth	Exponential growth of a cell population
Cell growth / growth-curve fit	Specific growth rate fitted to growth-curve data
Cell growth / logistic growth	Logistic growth of a population, Time for a logistic population to reach a size
Cell growth / Monod kinetics	Monod specific growth rate, Batch culture time under Monod kinetics
DNA and RNA / cloning	Insert mass for a ligation at an insert:vector molar ratio
DNA and RNA / genomics	Sequencing coverage and uncovered fraction (Lander-Waterman), Number of clones for a genomic library (Clarke-Carbon)
DNA and RNA / melting temperature	Primer melting temperature by the Wallace rule, Basic GC melting temperature of an oligonucleotide, Salt-adjusted melting temperature of an oligonucleotide, Nearest-neighbour melting temperature of a DNA duplex
DNA and RNA / molecular weight	Molecular weight of a DNA or RNA oligonucleotide
DNA and RNA / mutation	Mutation rate by the fluctuation-test p0 method
DNA and RNA / quantification	Extinction coefficient of a DNA oligonucleotide at 260 nm (nearest neighbour), Nucleic acid concentration from A260, A260/A280 and A260/A230 purity ratios, Protein concentration by Warburg-Christian (A280 and A260), Protein concentration from A280 and the extinction coefficient, Copy number and moles of a DNA or RNA molecule from its mass
DNA and RNA / sequence composition	GC content of a DNA or RNA sequence
DNA and RNA / sequence operations	Reverse complement of a DNA or RNA sequence, Transcription of a DNA strand into mRNA, Translation of an mRNA or DNA sequence into protein, Longest open reading frame on both strands
Enzyme kinetics / assays	Enzyme activity and specific activity from a spectrophotometric assay
Enzyme kinetics / inhibition	Competitive inhibition rate, Uncompetitive inhibition rate, Noncompetitive inhibition rate, Mixed inhibition rate, Inhibition constant from IC50 (Cheng-Prusoff)
Enzyme kinetics / Michaelis-Menten	Michaelis-Menten reaction rate, Substrate concentration for a fraction of Vmax, Turnover number kcat and catalytic efficiency kcat/Km, Km and Vmax from a Lineweaver-Burk plot, Km and Vmax by nonlinear least squares
Enzyme kinetics / temperature	Rate at another temperature from Q10
Enzyme kinetics and binding / binding	Fraction of receptor bound at a free ligand concentration, Complex concentration with ligand depletion (quadratic binding), Kd and Bmax from a Scatchard plot, Standard binding free energy from Kd, Receptor occupancy with a competing ligand
Enzyme kinetics and binding / binding kinetics	Dissociation constant from association and dissociation rates
Enzyme kinetics and binding / cooperativity	Fractional saturation by the Hill equation, Ligand concentration for a given saturation (Hill), Hill coefficient from EC10 and EC90, Fractional saturation in the MWC (concerted) model
Membranes and diffusion / diffusion coefficient	Stokes-Einstein diffusion coefficient
Membranes and diffusion / diffusion-limited reactions	Diffusion-limited association rate constant (Smoluchowski)
Membranes and diffusion / Fick's law	Fick's first law: steady diffusion flux

topic	capabilities
Membranes and diffusion / FRAP	Diffusion coefficient from a FRAP half-time (Soumpasis)
Membranes and diffusion / ion channels	Macroscopic current through ion channels
Membranes and diffusion / membrane potential	Nernst equilibrium potential of an ion across a cell membrane, Donnan equilibrium with an impermeant anion
Membranes and diffusion / osmosis	Osmotic pressure (van 't Hoff), Osmolarity of a solution and its tonicity for a cell, Cell volume after an osmotic change (Boyle-van 't Hoff)
Membranes and diffusion / permeability	Membrane permeability coefficient and flux, Equilibration of a solute across a spherical cell's membrane
Membranes and diffusion / random walk	Root-mean-square displacement of a diffusing particle, Time to diffuse a distance
Membranes and diffusion / water potential	Water potential of a plant cell or solution (solute plus pressure potential), Direction of water movement between a cell and its surroundings
Metabolism / cellular respiration	ATP yield per glucose: aerobic respiration versus fermentation
Metabolism / free energy	Free-energy change of a reaction at cellular concentrations, Free-energy change of a redox reaction from reduction potentials
Microbial control / heat and chemical killing	D-value (decimal reduction time) from survivor counts, Z-value from D-values at two temperatures, D-value at a new temperature and time to a target kill (z-value)
Microscopy / magnification and resolution	Light microscope: total magnification and resolution limit, Actual size of a specimen from a micrograph
PCR and gene expression / gene expression	Steady-state mRNA and protein numbers, MRNA and protein after switching a gene on, MRNA half-life fitted to a decay time course
PCR and gene expression / PCR	PCR amplification after n cycles, PCR cycles needed to reach a number of copies
PCR and gene expression / qPCR	QPCR relative expression by the delta-delta Ct method, QPCR relative expression with efficiency correction (Pfaffl), QPCR standard curve: slope, intercept and efficiency, Quantity from a Ct and a standard curve
PCR and gene expression / transcription	Time to transcribe a gene
PCR and gene expression / translation	Time to translate a protein
Solution biochemistry / buffers	Buffer pH by the Henderson-Hasselbalch equation, Recipe for a buffer of given pH and concentration, Buffer capacity of a weak-acid buffer, Buffer pH after adding strong acid or base
Solution biochemistry / centrifugation	Relative centrifugal force from rotor speed, Rotor speed for a required centrifugal force, Time to pellet a particle in a centrifuge (k-factor)
Solution biochemistry / dilutions	Dilution of a stock solution (C1 V1 = C2 V2), Serial dilution series
Solution biochemistry / ionic strength	Ionic strength of a solution
Solution biochemistry / molarity	Mass of solute for a solution of given molarity, Molar concentration from mass and volume, Molarity of a per cent (w/v) solution

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
cmb_fick_flux	Fick's first law: steady diffusion flux	D, C1, C2, dx	J, J_nmol_cm2_s
cmb_rms_displacement	Root-mean-square displacement of a diffusing particle	D, t, dims	rms, rms_um, msd
cmb_diffusion_time	Time to diffuse a distance	x, D, dims	t, t_min
cmb_stokes_einstein	Stokes-Einstein diffusion coefficient	T, eta, r	D, D_um2_s, friction
cmb_membrane_permeability	Membrane permeability coefficient and flux	D, K, L, C_out, C_in	P, P_cm_s, J
cmb_cell_equilibration	Equilibration of a solute across a spherical cell's membrane	r, P, C_out, C0, t	C, tau, t_half
cmb_osmotic_pressure	Osmotic pressure (van 't Hoff)	i, c, T	pi, pi_atm, pi_kPa
cmb_osmolarity_tonicity	Osmolarity of a solution and its tonicity for a cell	c, i, osm_cell	osmolarity_mOsm, difference_mOsm, tonicity
cmb_cell_volume_osmotic	Cell volume after an osmotic change (Boyle-van 't Hoff)	V0, b, osm0, osm1	V, V_fL, ratio
cmb_nernst_potential	Nernst equilibrium potential of an ion across a cell membrane	z, C_out, C_in, T	E, E_mV
cmb_donnan_equilibrium	Donnan equilibrium with an impermeant anion	A, zA, c, T	K_in, Cl_in, ratio, E, E_mV, excess_mOsm
cmb_channel_current	Macroscopic current through ion channels	N, Po, gamma, V, E	I, I_pA, G, i_single
cmb_diffusion_limited_rate	Diffusion-limited association rate constant (Smoluchowski)	DA, DB, rA, rB	k, k_per_M_s
cmb_frap_diffusion	Diffusion coefficient from a FRAP half-time (Soumpasis)	w, t_half	D, D_um2_s, tauD, gamma
cmb_exponential_growth	Exponential growth of a cell population	N0, td, t	N, generations, fold, mu, mu_per_h
cmb_doubling_time_from_counts	Doubling time and number of generations from two counts	N0, N, t	td, td_h, generations, mu, mu_per_h
cmb_growth_rate_fit	Specific growth rate fitted to growth-curve data	t, N	mu_per_h, td_h, N0_fit, r2
cmb_logistic_growth	Logistic growth of a population	N0, K, r, t	N, dNdt, dNdt_per_h, t_inflection, t_inflection_h
cmb_logistic_time_to_reach	Time for a logistic population to reach a size	N0, K, r, N	t, t_h
cmb_monod_growth_rate	Monod specific growth rate	mu_max, S, Ks	mu, mu_per_h, fraction, td_h
cmb_monod_batch_time	Batch culture time under Monod kinetics	X0, S0, Y, mu_max, Ks, X	t, t_h, S, S_g_L
cmb_cell_cycle_phase_durations	Cell-cycle phase durations from phase fractions	Tc, fractions	G1_h, S_h, G2_h, M_h
cmb_cell_cycle_age_corrected	Cell-cycle phase durations corrected for the exponential age distribution	Tc, fractions	G1_h, S_h, G2_h, M_h
cmb_hemocytometer_count	Cell concentration from a hemocytometer count	counts, dilution, V	mean_count, cells_per_mL, total_cells
cmb_plate_count	Colony-forming units per mL from a plate count	colonies, dilution, V	cfu_per_mL, log10_cfu_per_mL
cmb_transformation_efficiency	Transformation efficiency	colonies, m_DNA, V_plated, V_total	efficiency, log10_efficiency, DNA_plated_ug
cmb_cell_geometry	Volume, surface area and surface-to-volume ratio of a spherical cell	d	V, V_fL, A, A_um2, SA_V_per_um
cmb_molecules_per_cell	Number of molecules in a cell from a concentration	c, V	N, one_molecule, one_molecule_nM
cmb_gc_content	GC content of a DNA or RNA sequence	seq	gc_fraction, gc_percent, length, G, C, A, T_or_U

id	operation	inputs	outputs
cmb_tm_wallace	Primer melting temperature by the Wallace rule	seq	Tm, AT, GC
cmb_tm_basic	Basic GC melting temperature of an oligonucleotide	seq	Tm, gc_percent, length
cmb_tm_salt_adjusted	Salt-adjusted melting temperature of an oligonucleotide	seq, Na	Tm, gc_percent, salt_term
cmb_tm_nearest_neighbor	Nearest-neighbour melting temperature of a DNA duplex	seq, Na, c1, c2	Tm, dH, dS, dG37
cmb_oligo_molecular_weight	Molecular weight of a DNA or RNA oligonucleotide	seq, strands	MW, MW_g_mol, MW_hydroxyl_g_mol
cmb_oligo_extinction_coefficient	Extinction coefficient of a DNA oligonucleotide at 260 nm (nearest neighbour)	seq	eps, eps_L_mol_cm, ug_per_A260, nmol_per_A260
cmb_nucleic_acid_a260	Nucleic acid concentration from A260	A260, molecule, dilution, l, V	c, c_ug_mL, mass_ug
cmb_absorbance_ratios	A260/A280 and A260/A230 purity ratios	A260, A280, A230, A320	ratio_260_280, ratio_260_230
cmb_warburg_christian	Protein concentration by Warburg-Christian (A280 and A260)	A280, A260	protein, protein_mg_mL, ratio_280_260
cmb_protein_concentration_a280	Protein concentration from A280 and the extinction coefficient	A280, eps, MW, l	c, c_uM, c_mg_mL
cmb_dna_copy_number	Copy number and moles of a DNA or RNA molecule from its mass	m, n, molecule	MW, MW_g_mol, pmol, copies
cmb_ligation_insert_mass	Insert mass for a ligation at an insert:vector molar ratio	m_vector, L_vector, L_insert, ratio	m_insert, m_insert_ng, vector_fmol, insert_fmol
cmb_reverse_complement	Reverse complement of a DNA or RNA sequence	seq	reverse_complement, complement, reverse
cmb_transcribe	Transcription of a DNA strand into mRNA	seq, strand	mRNA, length
cmb_translate	Translation of an mRNA or DNA sequence into protein	seq	protein, full_translation, length_aa, stops
cmb_orf_finder	Longest open reading frame on both strands	seq, min_aa	strand, start, end, length_nt, length_aa, protein, n_orfs
cmb_genome_coverage	Sequencing coverage and uncovered fraction (Lander-Waterman)	L, N, G	coverage, fraction_uncovered, bases_uncovered, contigs
cmb_library_size	Number of clones for a genomic library (Clarke-Carbon)	G, l, P	clones_exact, clones
cmb_mutation_rate_p0	Mutation rate by the fluctuation-test p0 method	n0, n, Nt	p0, m, mutation_rate
cmb_pcr_amplification	PCR amplification after n cycles	N0, E, n	N, fold, log10_fold
cmb_pcr_cycles_needed	PCR cycles needed to reach a number of copies	N0, N, E	cycles_exact, cycles
cmb_qpcr_delta_delta_ct	qPCR relative expression by the delta-delta Ct method	Ct_target_treated, Ct_ref_treated, Ct_target_control, Ct_ref_control	dCt_treated, dCt_control, ddCt, fold_change, log2_fold_change
cmb_qpcr_pfaffl_ratio	qPCR relative expression with efficiency correction (Pfaffl)	E_target, E_ref, Ct_target_control, Ct_target_sample, Ct_ref_control, Ct_ref_sample	ratio, log2_ratio, dCt_target, dCt_ref
cmb_qpcr_standard_curve	qPCR standard curve: slope, intercept and efficiency	quantity, Ct	slope, intercept, r2, efficiency, amplification_factor
cmb_qpcr_quantify	Quantity from a Ct and a standard curve	slope, intercept, Ct	quantity, log10_quantity
cmb_transcription_time	Time to transcribe a gene	n, rate, k_init	t, t_min, polymerases, spacing_nt
cmb_translation_time	Time to translate a protein	n_aa, rate, k_init	t, t_min, coding_nt, ribosomes, spacing_codons
cmb_expression_steady_state	Steady-state mRNA and protein numbers	k_tx, delta_m, k_tl, delta_p	m_ss, p_ss, mRNA_half_life, protein_half_life, mRNA_half_life_min, protein_half_life_h
cmb_induction_dynamics	mRNA and protein after switching a gene on	k_tx, delta_m, k_tl, delta_p, t	m, p, fraction_of_steady_state
cmb_mrna_half_life_fit	mRNA half-life fitted to a decay time course	t, level	k_per_min, half_life_min, initial_fit, r2
cmb_michaelis_menten_rate	Michaelis-Menten reaction rate	Vmax, Km, S	v, v_uM_min, fraction
cmb_substrate_for_fraction_of_vmax	Substrate concentration for a fraction of Vmax	Km, f	S, S_uM, S_over_Km
cmb_kcat_catalytic_efficiency	Turnover number kcat and catalytic efficiency kcat/Km	Vmax, Et, Km	kcat, kcat_per_s, efficiency, efficiency_per_M_s, turnover_time

id	operation	inputs	outputs
cmb_lineweaver_burk_fit	Km and Vmax from a Lineweaver-Burk plot	S, v	Km, Vmax, slope, intercept, r2
cmb_michaelis_menten_nonlinear_fit	Km and Vmax by nonlinear least squares	S, v	Km, Vmax, rss, r2
cmb_competitive_inhibition	Competitive inhibition rate	Vmax, Km, S, I, Ki	v, Km_app, Vmax_app, percent_inhibition
cmb_uncompetitive_inhibition	Uncompetitive inhibition rate	Vmax, Km, S, I, Ki	v, Km_app, Vmax_app, percent_inhibition
cmb_noncompetitive_inhibition	Noncompetitive inhibition rate	Vmax, Km, S, I, Ki	v, Km_app, Vmax_app, percent_inhibition
cmb_mixed_inhibition	Mixed inhibition rate	Vmax, Km, S, I, Ki, Kii	v, Km_app, Vmax_app, alpha
cmb_ic50_to_ki	Inhibition constant from IC50 (Cheng-Prusoff)	mode, IC50, S, Km	Ki, Ki_uM
cmb_hill_equation	Fractional saturation by the Hill equation	L, K, n	theta, theta_percent
cmb_hill_ligand_for_fraction	Ligand concentration for a given saturation (Hill)	K, n, theta	L, L_uM
cmb_hill_coefficient_from_ec	Hill coefficient from EC10 and EC90	EC10, EC90	n, EC50, ratio
cmb_fraction_bound	Fraction of receptor bound at a free ligand concentration	L, Kd, Rt	theta, theta_percent, bound
cmb_ligand_depletion_binding	Complex concentration with ligand depletion (quadratic binding)	Rt, Lt, Kd	RL, fraction_receptor_bound, free_ligand, fraction_ligand_bound
cmb_kd_from_rates	Dissociation constant from association and dissociation rates	kon, koff	Kd, Kd_nM, Ka_per_M, half_life, residence_time
cmb_scatchard_fit	Kd and Bmax from a Scatchard plot	B, F	Kd, Bmax, slope, intercept, r2
cmb_binding_free_energy	Standard binding free energy from Kd	Kd, T	dG, dG_kJ_mol, dG_kcal_mol
cmb_competitive_occupancy	Receptor occupancy with a competing ligand	L, Kd, I, Ki	occupancy_L, occupancy_I, free_fraction, Kd_app
cmb_mwc_saturation	Fractional saturation in the MWC (concerted) model	S, KR, c, L0, n	Y, R_fraction
cmb_q10_temperature	Rate at another temperature from Q10	k1, T1, T2, Q10	k2, ratio, Ea, Ea_kJ_mol
cmb_enzyme_assay_activity	Enzyme activity and specific activity from a spectrophotometric assay	dA, dt, eps, l, V_assay, V_enzyme, protein	rate, units, units_per_mL, specific_activity
cmb_henderson_hasselbalch	Buffer pH by the Henderson-Hasselbalch equation	pKa, A, HA	pH, ratio, fraction_base
cmb_buffer_preparation	Recipe for a buffer of given pH and concentration	C, V, pH, pKa, MW_acid, MW_base	C_base, C_acid, n_base, n_acid, ratio, m_acid_g, m_base_g
cmb_buffer_capacity	Buffer capacity of a weak-acid buffer	C, pKa, pH	beta, beta_mM, beta_buffer_mM
cmb_buffer_after_addition	Buffer pH after adding strong acid or base	C, V, pH0, pKa, n_add, added	pH, delta_pH, n_base, n_acid
cmb_c1v1_dilution	Dilution of a stock solution (C1 V1 = C2 V2)	C1, C2, V2	V1, V1_uL, V_diluent, dilution_factor
cmb_serial_dilution	Serial dilution series	C0, f, n, V	C_final, total_dilution, concentrations_mM, transfer, diluent
cmb_mass_for_molarity	Mass of solute for a solution of given molarity	C, V, MW	m, m_g, m_mg, n
cmb_molarity_from_mass	Molar concentration from mass and volume	m, MW, V	C, C_mM, C_M, n
cmb_percent_wv_to_molarity	Molarity of a per cent (w/v) solution	p, MW	C, C_mM, C_M, mass_concentration, g_per_L
cmb_ionic_strength	Ionic strength of a solution	c, z	I, I_mM, I_M
cmb_centrifuge_rcf	Relative centrifugal force from rotor speed	rpm, r	RCF, rpm_value
cmb_centrifuge_rpm	Rotor speed for a required centrifugal force	RCF, r	omega, rpm_value
cmb_sedimentation_time	Time to pellet a particle in a centrifuge (k-factor)	s, rpm, r_min, r_max	t, t_h, k_factor
cmb_microscope_resolution	Light microscope: total magnification and resolution limit	M_obj, M_oc, wavelength, NA	M, d, d_nm, d_um
cmb_specimen_size	Actual size of a specimen from a micrograph	image_size, M, bar_image, bar_label	size, size_um, M_image
cmb_water_potential	Water potential of a plant cell or solution (solute plus pressure potential)	i, C, T, psi_p	psi_s, psi_s_MPa, psi, psi_MPa

id	operation	inputs	outputs
cmb_water_movement	Direction of water movement between a cell and its surroundings	i_cell, C_cell, P_cell, i_out, C_out, P_out, T	psi_cell, psi_out, dpsi, dpsi_MPa, direction
cmb_reaction_free_energy	Free-energy change of a reaction at cellular concentrations	dG0, T, products, reactants	Q, dG, dG_kJ_mol, K_eq, direction
cmb_redox_free_energy	Free-energy change of a redox reaction from reduction potentials	E_acceptor, E_donor, n	dE, dE_mV, dG, dG_kJ_mol, direction
cmb_atp_yield	ATP yield per glucose: aerobic respiration versus fermentation	P_O_NADH, P_O_FADH2, shuttle, dG_ATP, dG_glucose	ATP_aerobic, ATP_fermentation, ratio, ATP_oxidative, ATP_substrate_level, NADH_total, FADH2_total, efficiency_aerobic, efficiency_fermentation
cmb_decimal_reduction_time	D-value (decimal reduction time) from survivor counts	N0, N, t	log_reduction, D, D_min, k_per_min, survival_fraction
cmb_z_value	z-value from D-values at two temperatures	D1, T1, D2, T2	z, fold_per_10_degrees
cmb_thermal_death_time	D-value at a new temperature and time to a target kill (z-value)	D_ref, T_ref, z, T, log_reductions	D_T, D_T_min, t, t_min, lethal_rate

Fick's first law: steady diffusion flux, `cmb_fick_flux`

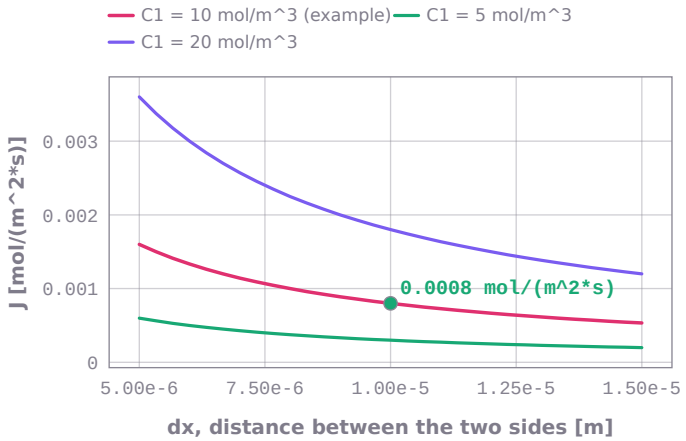
The steady one-dimensional diffusive flux between two concentrations a distance apart (positive from side 1 to side 2).

$$J = -D\frac{dC}{dx} = D\,\frac{C_1 - C_2}{\Delta x}$$

input	meaning	condition
`D`	diffusion coefficient of the solute	
`C1`	concentration on side 1	
`C2`	concentration on side 2	
`dx`	distance between the two sides	

output	meaning
`J`	molar flux from side 1 to side 2
`J_nmol_cm2_s`	flux in nmol per cm^2 per second

- **Method:** $J = D\,(C_1 - C_2)/dx$ (Fick's first law with a linear steady profile)
- **Assumes:** $D > 0$, $dx > 0$; C1, $C_2 \geq 0$; steady state, linear profile
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



J (molar flux from side 1 to side 2) against dx (distance between the two sides) from 5.0000e-06 m to 1.5000e-05 m. One curve for each value of C1 (concentration on side 1): 10 mol/m^3, 5 mol/m^3, 20 mol/m^3. Other inputs as in the worked example: D = 1e-09 m^2/s, C2 = 2.0 mol/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, dx = 1.0000e-05 m, J = 0.0008 mol/(m^2*s).

```
cell_molecular_biology_ai999x.solve('cmb_fick_flux', D=1e-09, C1=10.0, C2=2.0, dx=1e-05)
# -> J = 0.0008 mol/(m^2*s), J_nmol_cm2_s = 80   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_fick_flux', D=5e-10, C1=0.15, C2=0.0, dx=2e-06)
# -> J = 3.75e-05 mol/(m^2*s), J_nmol_cm2_s = 3.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fick diffusion flux: D = 1000 um^2/s, C1 = 10 mM, C2 = 2 mM, dx = 10 um	SOLVED	J = 0.0008 mol/(m^2*s), J_nmol_cm2_s = 80
Fick diffusion flux: D = 500 um^2/s, C1 = 0.15 mM, C2 = 0 mM, dx = 2 um	SOLVED	J = 3.75e-05 mol/(m^2*s), J_nmol_cm2_s = 3.75
Fick diffusion flux: C1 = 10 mM, C2 = 2 mM, dx = 10 um	NEEDS_INPUT	cmb_fick_flux needs D (diffusion coefficient of the solute). Example: solve('cmb_fick_flux', D='1e-09 m^2/s', C1='10 mol
Fick diffusion flux: D = 1000 um^2/s, C1 = 10 mM, C2 = 2 mM, dx = 10 um, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Diffusion flux: D = 1e-5 cm^2/s, C1 = 10 mM, C2 = 2 mM, dx = 10 um	SOLVED	J = 0.0008 mol/(m^2*s), J_nmol_cm2_s = 80

Root-mean-square displacement of a diffusing particle, `cmb_rms_displacement`

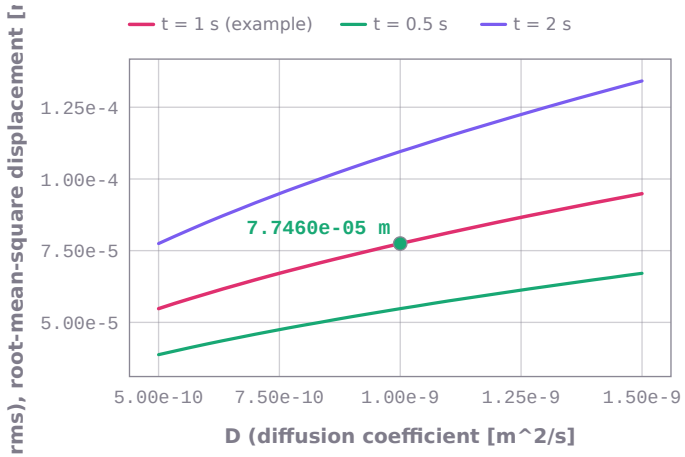
The mean-square and root-mean-square displacement of a particle diffusing freely for a time t in 1, 2 or 3 dimensions.

$$\langle r^2 \rangle = 2dDt, \quad r_{\mathrm{rms}} = \sqrt{2dDt}$$

input	meaning	condition
`D`	diffusion coefficient	
`t`	elapsed time	
`dims`	number of dimensions (1, 2 or 3)	

output	meaning
`rms`	root-mean-square displacement
`rms_um`	root-mean-square displacement in micrometres
`msd`	mean-square displacement

- **Method:** <r^2> = 2 d D t; rms = sqrt(2 d D t)
- **Assumes:** $D > 0$, $t > 0$; dims = 1, 2 or 3
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rms (root-mean-square displacement) against D (diffusion coefficient) from 5.0000e-10 m^2/s to 1.5000e-09 m^2/s. One curve for each value of t (elapsed time): 1 s, 0.5 s, 2 s. Other inputs as in the worked example: dims = 3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, D = 1.0000e-09 m^2/s, rms = 7.7460e-05 m.

```
cell_molecular_biology_ai999x.solve('cmb_rms_displacement', D=1e-09, t=1.0, dims=3)
# -> rms = 7.74597e-05 m, rms_um = 77.4597, msd = 6e-09 m^2   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_rms_displacement', D=1e-12, t=60.0, dims=2)
# -> rms = 1.54919e-05 m, rms_um = 15.4919, msd = 2.4e-10 m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Root-mean-square displacement: D = 1000 um^2/s, t = 1 s, dims = 3	SOLVED	rms = 7.74597e-05 m, rms_um = 77.4597, msd = 6e-09 m^2
Root-mean-square displacement: D = 1 um^2/s, t = 60 s, dims = 2	SOLVED	rms = 1.54919e-05 m, rms_um = 15.4919, msd = 2.4e-10 m^2
Root-mean-square displacement: t = 1 s, dims = 3	NEEDS_INPUT	cmb_rms_displacement needs D (diffusion coefficient). Example: solve('cmb_rms_displacement', D='1e-09 m^2/s', t='1 s', d
Root-mean-square displacement: D = 1000 um^2/s, t = 1 s, dims = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
RMS displacement: D = 1000 um^2/s, t = 1000 ms, dims = 3	SOLVED	rms = 7.74597e-05 m, rms_um = 77.4597, msd = 6e-09 m^2

Time to diffuse a distance, `cmb_diffusion_time`

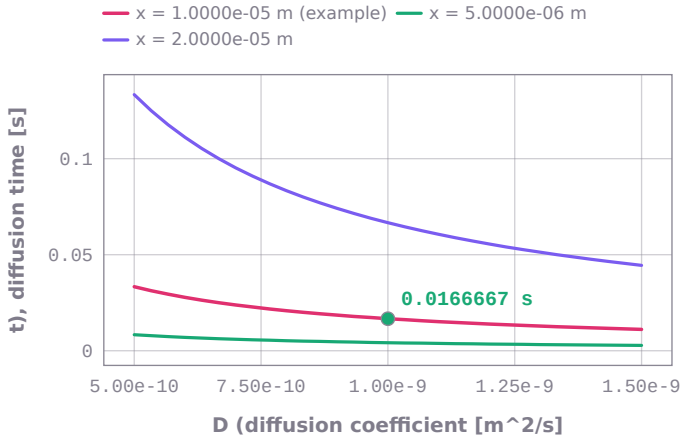
The time for the root-mean-square displacement of a diffusing molecule to reach a distance x in 1, 2 or 3 dimensions.

$$t = \frac{x^2}{2dD}$$

input	meaning	condition
`x`	distance (root-mean-square displacement to reach)	
`D`	diffusion coefficient	
`dims`	number of dimensions (1, 2 or 3)	

output	meaning
`t`	diffusion time
`t_min`	diffusion time in minutes

- **Method:** t = x^2 / (2 d D)
- **Assumes:** $x > 0$, $D > 0$; dims = 1, 2 or 3
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (diffusion time) against D (diffusion coefficient) from 5.0000e-10 m^2/s to 1.5000e-09 m^2/s. One curve for each value of x (distance (root-mean-square displacement to reach)): 1.0000e-05 m, 5.0000e-06 m, 2.0000e-05 m. Other inputs as in the worked example: dims = 3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, D = 1.0000e-09 m^2/s, t = 0.0166667 s.

```
cell_molecular_biology_ai999x.solve('cmb_diffusion_time', x=1e-05, D=1e-09, dims=3)
# -> t = 0.0166667 s, t_min = 0.000277778   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_diffusion_time', x=0.001, D=5e-10, dims=1)
# -> t = 1000 s, t_min = 16.6667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diffusion time: x = 10 um, D = 1000 um^2/s, dims = 3	SOLVED	t = 0.0166667 s, t_min = 0.000277778
Diffusion time: x = 1000 um, D = 500 um^2/s, dims = 1	SOLVED	t = 1000 s, t_min = 16.6667
Diffusion time: D = 1000 um^2/s, dims = 3	NEEDS_INPUT	cmb_diffusion_time needs x (distance (root-mean-square displacement to reach)). Example: solve('cmb_diffusion_time', x='
Diffusion time: x = 10 um, D = 1000 um^2/s, dims = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Time to diffuse: x = 0.01 mm, D = 1000 um^2/s, dims = 3	SOLVED	t = 0.0166667 s, t_min = 0.000277778

Stokes-Einstein diffusion coefficient, `cmb_stokes_einstein`

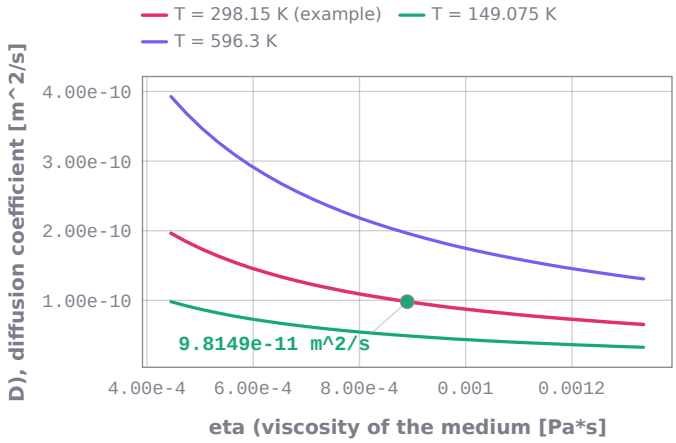
The diffusion coefficient of a sphere of hydrodynamic radius r in a fluid of viscosity eta at temperature T.

$$D = \frac{k_B T}{6\pi\eta r}$$

input	meaning	condition
`T`	absolute temperature (degC is converted)	
`eta`	viscosity of the medium	
`r`	hydrodynamic radius	

output	meaning
`D`	diffusion coefficient
`D_um2_s`	diffusion coefficient in um^2/s
`friction`	friction coefficient 6 pi eta r in kg/s

- **Method:** $D = k$ T / (6 pi eta r)
- **Assumes:** T, eta, $r > 0$; a sphere at low Reynolds number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



D (diffusion coefficient) against eta (viscosity of the medium) from 0.000445 Pa*s to 0.001335 Pa*s. One curve for each value of T (absolute temperature (degC is converted)): 298.15 K, 149.075 K, 596.3 K. Other inputs as in the worked example: r = 2.5e-09 m. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, eta = 0.00089 Pa*s, D = 9.8149e-11 m^2/s.

```
cell_molecular_biology_ai999x.solve('cmb_stokes_einstein', T=298.15, eta=0.00089, r=2.5e-09)
# -> D = 9.81492e-11 m^2/s, D_um2_s = 98.1492, friction = 4.19403e-11   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_stokes_einstein', T=310.15, eta=0.0007, r=5e-08)
# -> D = 6.49062e-12 m^2/s, D_um2_s = 6.49062, friction = 6.59734e-10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stokes-Einstein diffusion coefficient: T = 298.15 K, eta = 0.89 mPa*s, r = 2.5 nm	SOLVED	D = 9.81492e-11 m^2/s, D_um2_s = 98.1492, friction = 4.19403e-11
Stokes-Einstein diffusion coefficient: T = 310.15 K, eta = 0.7 mPa*s, r = 50 nm	SOLVED	D = 6.49062e-12 m^2/s, D_um2_s = 6.49062, friction = 6.59734e-10
Stokes-Einstein diffusion coefficient: eta = 0.89 mPa*s, r = 2.5 nm	NEEDS_INPUT	cmb_stokes_einstein needs T (absolute temperature (degC is converted)). Example: solve('cmb_stokes_einstein', T='298.15
Stokes-Einstein diffusion coefficient: T = 298.15 K, eta = 0.89 mPa*s, r = 2.5 nm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Stokes-Einstein equation: T = 25 degC, eta = 0.89 mPa*s, r = 2.5 nm	SOLVED	D = 9.81492e-11 m^2/s, D_um2_s = 98.1492, friction = 4.19403e-11

Membrane permeability coefficient and flux, `cmb_membrane_permeability`

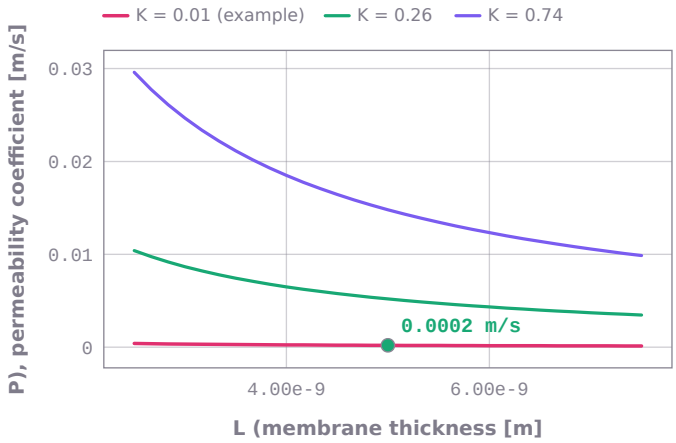
The permeability coefficient $P = K$ D / L of a lipid membrane for a solute with partition coefficient K, and the inward flux P (C_out - C_in).

$$P = \frac{KD}{L}, \quad J = P\,(C_{out} - C_{in})$$

input	meaning	condition
`D`	diffusion coefficient of the solute inside the membrane	
`K`	membrane/water partition coefficient	
`L`	membrane thickness	
`C_out`	concentration outside the cell	
`C_in`	concentration inside the cell	

output	meaning
`P`	permeability coefficient
`P_cm_s`	permeability coefficient in cm/s
`J`	inward flux (positive into the cell)

- **Method:** $P = K$ D / L; $J = P$ (C_out - C_in)
- **Assumes:** D, K, $L > 0$; concentrations >= 0; steady state, linear profile in the membrane
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (permeability coefficient) against L (membrane thickness) from 2.5000e-09 m to 7.5000e-09 m. One curve for each value of K (membrane/water partition coefficient): 0.01, 0.26, 0.74. Other inputs as in the worked example: D = 1e-10 m^2/s, C_out = 10.0 mol/m^3, C_in = 1.0 mol/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, L = 5.0000e-09 m, P = 0.0002 m/s.

```
cell_molecular_biology_ai999x.solve('cmb_membrane_permeability', D=1e-10, K=0.01, L=5e-09, C_out=10.0, C_in=1.0)
# -> P = 0.0002 m/s, P_cm_s = 0.02, J = 0.0018 mol/(m^2*s)   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_membrane_permeability', D=5e-11, K=2.0, L=4e-09, C_out=0.5, C_in=2.0)
# -> P = 0.025 m/s, P_cm_s = 2.5, J = -0.0375 mol/(m^2*s)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Membrane permeability: D = 100 um^2/s, K = 0.01, L = 5 nm, C_out = 10 mM, C_in = 1 mM	SOLVED	P = 0.0002 m/s, P_cm_s = 0.02, J = 0.0018 mol/(m^2*s)
Membrane permeability: D = 50 um^2/s, K = 2, L = 4 nm, C_out = 0.5 mM, C_in = 2 mM	SOLVED	P = 0.025 m/s, P_cm_s = 2.5, J = -0.0375 mol/(m^2*s)
Membrane permeability: K = 0.01, L = 5 nm, C_out = 10 mM, C_in = 1 mM	NEEDS_INPUT	cmb_membrane_permeability needs D (diffusion coefficient of the solute inside the membrane). Example: solve('cmb_membran
Membrane permeability: D = 100 um^2/s, K = 0.01, L = 5 nm, C_out = 10 mM, C_in = 1 mM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Permeability coefficient: D = 100 um^2/s, K = 0.01, L = 5 nm, C_out = 10 mM, C_in = 1000 uM	SOLVED	P = 0.0002 m/s, P_cm_s = 0.02, J = 0.0018 mol/(m^2*s)

Equilibration of a solute across a spherical cell's membrane, `cmb_cell_equilibration`

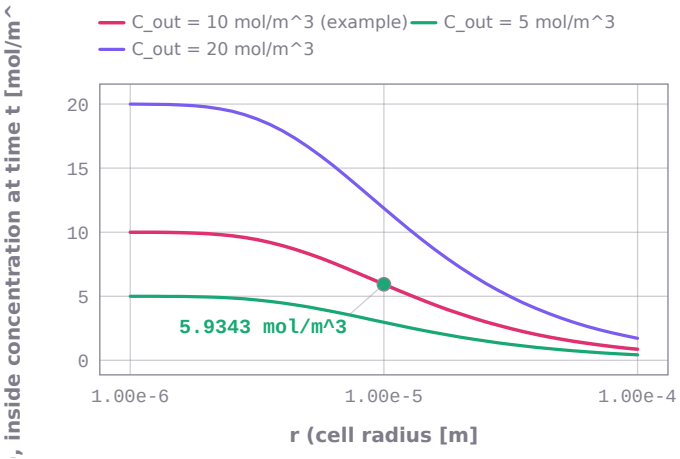
The internal concentration of a permeant solute at time t for a spherical cell $(A/V = 3/r)$ bathed at a constant outside concentration, with the time constant and half-time.

$$C(t) = C_{out} + (C_0 - C_{out})\, e^{-3Pt/r}$$

input	meaning	condition
`r`	cell radius	
`P`	membrane permeability coefficient	
`C_out`	constant outside concentration	
`C0`	initial inside concentration	
`t`	elapsed time	

output	meaning
`C`	inside concentration at time t
`tau`	time constant $r/(3P)$
`t_half`	half-time of equilibration

- **Method:** $dC/dt = P$ (A/V) (C_out - C), $A/V = 3/r$: C = C_out + (C0 - C_out) exp(-3Pt/r)
- **Assumes:** r, $P > 0$; C_out, C0, $t \geq 0$; well-mixed cell of constant volume, outside unchanged
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (inside concentration at time t) against r (cell radius) from 1.0000e-06 m to 0.0001 m on a logarithmic axis. One curve for each value of C_out (constant outside concentration): 10 mol/m^3, 5 mol/m^3, 20 mol/m^3. Other inputs as in the worked example: P = 1.0000000000000001e-07 m/s, C0 = 0.0 mol/m^3, t = 30.0 s. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, r = 1.0000e-05 m, C = 5.9343 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_cell_equilibration', r=1e-05, P=1e-07, C_out=10.0, C0=0.0, t=30.0)
# -> C = 5.9343 mol/m^3, tau = 33.3333 s, t_half = 23.1049 s   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_cell_equilibration', r=5e-06, P=2e-08, C_out=1.0, C0=5.0, t=60.0)
# -> C = 2.94701 mol/m^3, tau = 83.3333 s, t_half = 57.7623 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cell equilibration across the membrane: r = 10 um, P = 1e-05 cm/s, C_out = 10 mM, C0 = 0 mM, t = 30 s	SOLVED	C = 5.9343 mol/m^3, tau = 33.3333 s, t_half = 23.1049 s
Cell equilibration across the membrane: r = 5 um, P = 2e-06 cm/s, C_out = 1 mM, C0 = 5 mM, t = 60 s	SOLVED	C = 2.94701 mol/m^3, tau = 83.3333 s, t_half = 57.7623 s
Cell equilibration across the membrane: P = 1e-05 cm/s, C_out = 10 mM, C0 = 0 mM, t = 30 s	NEEDS_INPUT	cmb_cell_equilibration needs r (cell radius). Example: solve('cmb_cell_equilibration', r='1e-05 m', P='1e-07 m/s', C_out
Cell equilibration across the membrane: r = 10 um, P = 1e-05 cm/s, C_out = 10 mM, C0 = 0 mM, t = 30 s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Membrane equilibration: r = 10 um, P = 1e-5 cm/s, C_out = 10 mM, C0 = 0 mM, t = 0.5 min	SOLVED	C = 5.9343 mol/m^3, tau = 33.3333 s, t_half = 23.1049 s

Osmotic pressure (van 't Hoff), `cmb_osmotic_pressure`

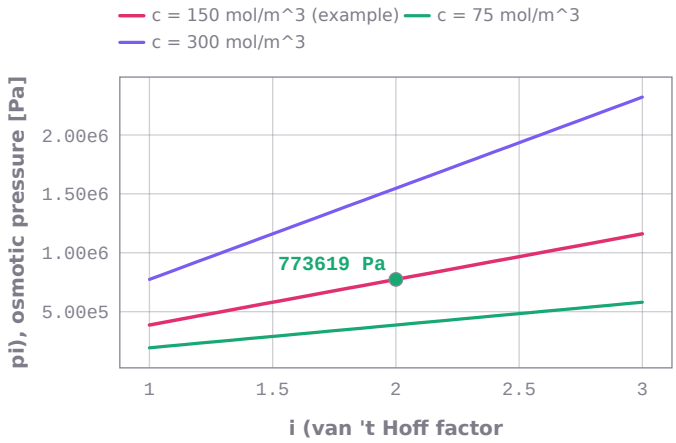
The osmotic pressure of a dilute solution from the solute concentration, the van 't Hoff factor and the temperature.

$$\Pi = i c R T$$

input	meaning	condition
`i`	van 't Hoff factor (particles per formula unit, e.g. 2 for NaCl)	
`c`	solute concentration	
`T`	absolute temperature (degC is converted)	

output	meaning
`pi`	osmotic pressure
`pi_atm`	osmotic pressure in atm
`pi_kPa`	osmotic pressure in kPa

- **Method:** $Pi = i$ c R T
- **Assumes:** i, c, $T > 0$; ideal dilute solution
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



pi (osmotic pressure) against i (van 't Hoff factor (particles per formula unit, e.g. 2 for NaCl)) from 1 to 3. One curve for each value of c (solute concentration): 150 mol/m^3, 75 mol/m^3, 300 mol/m^3. Other inputs as in the worked example: T = 310.15 K. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, i = 2, pi = 773619 Pa.

```
cell_molecular_biology_ai999x.solve('cmb_osmotic_pressure', i=2.0, c=150.0, T=310.15)
# -> pi = 773619 Pa, pi_atm = 7.63503, pi_kPa = 773.619   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_osmotic_pressure', i=1.0, c=300.0, T=298.15)
# -> pi = 743687 Pa, pi_atm = 7.33962, pi_kPa = 743.687   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Osmotic pressure: i = 2, c = 150 mM, T = 310.15 K	SOLVED	pi = 773619 Pa, pi_atm = 7.63503, pi_kPa = 773.619
Osmotic pressure: i = 1, c = 300 mM, T = 298.15 K	SOLVED	pi = 743687 Pa, pi_atm = 7.33962, pi_kPa = 743.687
Osmotic pressure: c = 150 mM, T = 310.15 K	NEEDS_INPUT	cmb_osmotic_pressure needs i (van 't Hoff factor (particles per formula unit, e.g. 2 for NaCl)). Example: solve('cmb_osm
Osmotic pressure: i = 2, c = 150 mM, T = 310.15 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Osmotic pressure: i = 2, c = 0.15 M, T = 37 degC	SOLVED	pi = 773619 Pa, pi_atm = 7.63503, pi_kPa = 773.619

Osmolarity of a solution and its tonicity for a cell, `cmb_osmolarity_tonicity`

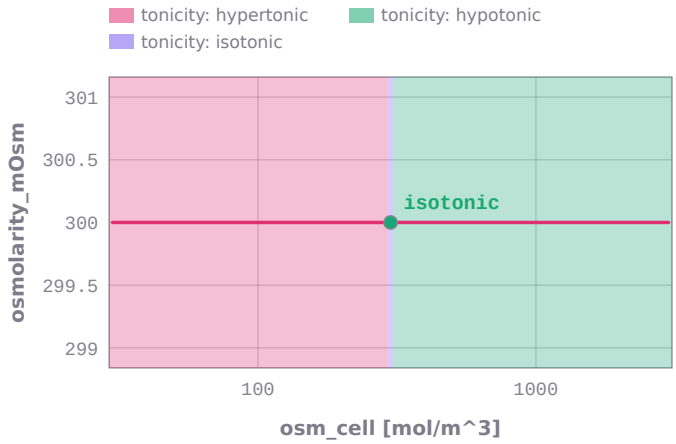
The total osmolarity of a solution of several solutes (sum of i c) and whether it is hypertonic, hypotonic or isotonic (within 1 %) relative to the cell's osmolarity, assuming impermeant solutes.

$$\mathrm{osm} = \sum_k i_k c_k$$

input	meaning	condition
`c`	solute concentrations in mM (one per solute)	
`i`	van 't Hoff factor of each solute	
`osm_cell`	osmolarity of the cell interior	

output	meaning
`osmolarity_mOsm`	osmolarity of the solution in mOsm/L
`difference_mOsm`	solution minus cell osmolarity in mOsm/L
`tonicity`	hypertonic, hypotonic or isotonic (within 1 %)

- **Method:** osmolarity = sum i_k c_k; compared with the cell's osmolarity (isotonic within 1 %)
- **Assumes:** one factor per concentration; solutes impermeant (tonicity = osmolarity)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



hypertonic, hypotonic or isotonic (within 1 %) (tonicity) along osm_cell (osmolarity of the cell interior) from 30 mol/m^3 to 3000 mol/m^3 (logarithmic axis): 81 runs of cell-molecular-biology-ai999x, each band coloured by the verdict returned (hypertonic, hypotonic, isotonic); it changes near osm_cell = 291.488 mol/m^3, 308.76 mol/m^3; the line is osmolarity_mOsm (osmolarity of the solution in mOsm/L). Other inputs as in the worked example: c = [150.0], i = [2.0]. Marked: the worked example, osm_cell = 300 mol/m^3 → isotonic.

```
cell_molecular_biology_ai999x.solve('cmb_osmolarity_tonicity', c=[150.0], i=[2.0], osm_cell=300.0)
# -> osmolarity_mOsm = 300, difference_mOsm = 0, tonicity = isotonic   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_osmolarity_tonicity', c=[100.0, 50.0], i=[2.0, 1.0], osm_cell=290.0)
# -> osmolarity_mOsm = 250, difference_mOsm = -40, tonicity = hypotonic   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Osmolarity and tonicity: c = [150], i = [2], osm_cell = 300 mOsm/L	SOLVED	osmolarity_mOsm = 300, difference_mOsm = 0, tonicity = isotonic
Osmolarity and tonicity: c = [100, 50], i = [2, 1], osm_cell = 290 mOsm/L	SOLVED	osmolarity_mOsm = 250, difference_mOsm = -40, tonicity = hypotonic
Osmolarity and tonicity: i = [2], osm_cell = 300 mOsm/L	NEEDS_INPUT	cmb_osmolarity_tonicity needs c (solute concentrations in mM (one per solute)). Example: solve('cmb_osmolarity_tonicity'
Osmolarity and tonicity: c = [150], i = [2], osm_cell = 300 mOsm/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Tonicity of a solution: c = [150], i = [2], osm_cell = 0.3 Osm/L	SOLVED	osmolarity_mOsm = 300, difference_mOsm = 0, tonicity = isotonic

Cell volume after an osmotic change (Boyle-van 't Hoff), `cmb_cell_volume_osmotic`

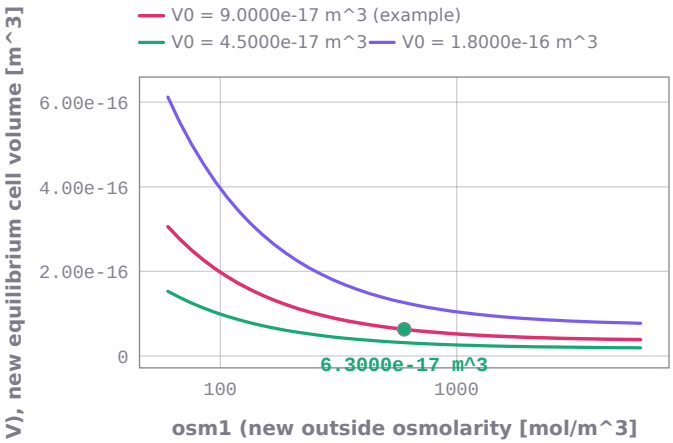
The equilibrium volume of a cell moved from osmolarity osm0 to osm1, with an osmotically inactive volume fraction b (an ideal osmometer).

$$V = V_b + (V_0 - V_b)\,\frac{\mathrm{osm}_0}{\mathrm{osm}_1}$$

input	meaning	condition
`V0`	initial cell volume	
`b`	osmotically inactive fraction of the initial volume (0 to 1)	
`osm0`	initial (isotonic) osmolarity	
`osm1`	new outside osmolarity	

output	meaning
`V`	new equilibrium cell volume
`V_fL`	new volume in femtolitres
`ratio`	new volume relative to the initial volume

- **Method:** $V = Vb + (V_0 - Vb)$ osm0 / osm1 with $Vb = b$ V0
- **Assumes:** V0, osm0, osm1 > 0; $0 \le b < 1$; impermeant solutes, water in equilibrium
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (new equilibrium cell volume) against osm1 (new outside osmolarity) from 60 mol/m^3 to 6000 mol/m^3 on a logarithmic axis. One curve for each value of V0 (initial cell volume): 9.0000e-17 m^3, 4.5000e-17 m^3, 1.8000e-16 m^3. Other inputs as in the worked example: b = 0.4, osm0 = 300.0 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, osm1 = 600 mol/m^3, V = 6.3000e-17 m^3.

```
cell_molecular_biology_ai999x.solve('cmb_cell_volume_osmotic', V0=9e-17, b=0.4, osm0=300.0, osm1=600.0)
# -> V = 6.3e-17 m^3, V_fL = 63, ratio = 0.7   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_cell_volume_osmotic', V0=2e-15, b=0.2, osm0=290.0, osm1=150.0)
# -> V = 3.49333e-15 m^3, V_fL = 3493.33, ratio = 1.74667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cell volume after an osmotic change: V0 = 90 fL, b = 40%, osm0 = 300 mOsm/L, osm1 = 600 mOsm/L	SOLVED	V = 6.3e-17 m^3, V_fL = 63, ratio = 0.7
Cell volume after an osmotic change: V0 = 2000 fL, b = 20%, osm0 = 290 mOsm/L, osm1 = 150 mOsm/L	SOLVED	V = 3.49333e-15 m^3, V_fL = 3493.33, ratio = 1.74667
Cell volume after an osmotic change: b = 40%, osm0 = 300 mOsm/L, osm1 = 600 mOsm/L	NEEDS_INPUT	cmb_cell_volume_osmotic needs V0 (initial cell volume). Example: solve('cmb_cell_volume_osmotic', V0='9e-17 m^3', b=0.4,
Cell volume after an osmotic change: V0 = 90 fL, b = 40%, osm0 = 300 mOsm/L, osm1 = 600 mOsm/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Boyle-van 't Hoff cell volume: V0 = 90 um^3, b = 40%, osm0 = 300 mOsm/L, osm1 = 0.6 Osm/L	SOLVED	V = 6.3e-17 m^3, V_fL = 63, ratio = 0.7

Nernst equilibrium potential of an ion across a cell membrane, `cmb_nernst_potential`

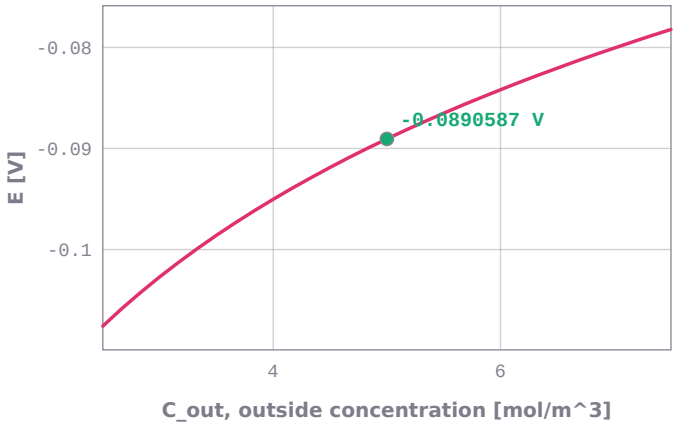
The equilibrium (reversal) potential inside relative to outside for an ion of charge z from its outside and inside concentrations.

$$E = \frac{RT}{zF} \ln \frac{C_{out}}{C_{in}}$$

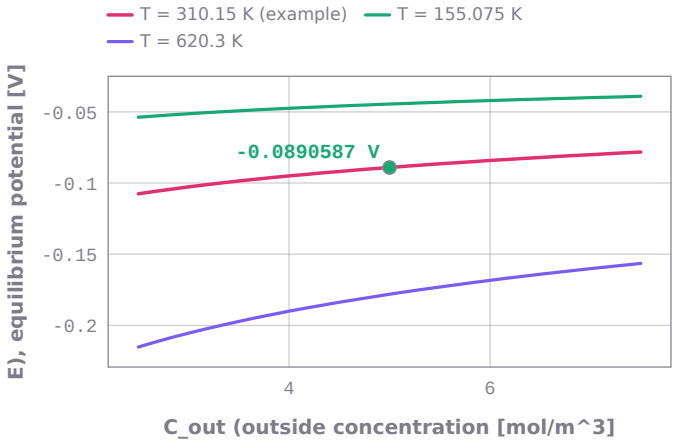
input	meaning	condition
`z`	charge number of the ion (e.g. 1 for K+, 2 for Ca2+, -1 for Cl-)	$z \neq 0$
`C_out`	outside concentration	
`C_in`	inside concentration	
`T`	absolute temperature (degC is converted)	

output	meaning
`E`	equilibrium potential (inside minus outside)
`E_mV`	equilibrium potential in mV

- **Method:** E = (R T / z F) ln(C_out / C_in)
- **Assumes:** z a non-zero whole number; concentrations > 0; $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



E (equilibrium potential (inside minus outside)) against C_out (outside concentration), with C_in = 140 mol/m^3, T = 310.15 K, z = 1. 31 points computed by cell-molecular-biology-ai999x; marked: C_out = 5, E = -0.0890587 V.



E (equilibrium potential (inside minus outside)) against C_out (outside concentration) from 2.5 mol/m^3 to 7.5 mol/m^3. One curve for each value of T (absolute temperature (degC is converted)): 310.15 K, 155.075 K, 620.3 K. Other inputs as in the worked example: z = 1, C_in = 140.0 mol/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, C_out = 5 mol/m^3, E = -0.0890587 V.

```
cell_molecular_biology_ai999x.solve('cmb_nernst_potential', z=1, C_out=5.0, C_in=140.0, T=310.15)
# -> E = -0.0890587 V, E_mV = -89.0587   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_nernst_potential', z=2, C_out=2.0, C_in=0.0001, T=310.15)
# -> E = 0.132344 V, E_mV = 132.344   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nernst potential: z = 1, C_out = 5 mM, C_in = 140 mM, T = 310.15 K	SOLVED	E = -0.0890587 V, E_mV = -89.0587
Nernst potential: z = 2, C_out = 2 mM, C_in = 0.0001 mM, T = 310.15 K	SOLVED	E = 0.132344 V, E_mV = 132.344
Nernst potential: C_out = 5 mM, C_in = 140 mM, T = 310.15 K	NEEDS_INPUT	cmb_nernst_potential needs z (charge number of the ion (e.g. 1 for K+, 2 for Ca2+, -1 for Cl-)). Example: solve('cmb_ner
Nernst potential: z = 1, C_out = 5 mM, C_in = 140 mM, T = 310.15 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Nernst potential: z = 1, C_out = 5 mM, C_in = 0.14 M, T = 37 degC	SOLVED	E = -0.0890587 V, E_mV = -89.0587
Nernst potential: z = 0, C_out = 5 mM, C_in = 140 mM, T = 310 K	INVALID_INPUT	Nernst equilibrium potential of an ion across a cell membrane: the charge number z must be a non-zero whole number.

Donnan equilibrium with an impermeant anion, `cmb_donnan_equilibrium`

The inside cation and anion concentrations, the Donnan ratio, the Donnan potential and the osmotic excess when a cell holds impermeant anions (charge -zA) bathed in a monovalent salt (KCl) of concentration c.

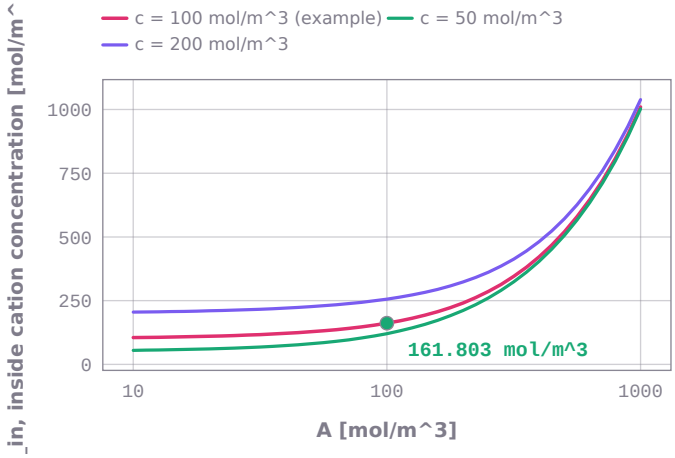
$$[\mathrm{Cl}]_i = \frac{-z_A A + \sqrt{z_A^2 A^2 + 4c^2}}{2}, \quad E = \frac{RT}{F} \ln \frac{c}{[\mathrm{K}]_i}$$

input	meaning	condition
A	concentration of impermeant anions inside	

input	meaning	condition
`zA`	magnitude of the impermeant anion's charge (1, 2 or 3)	
`c`	outside concentration of the monovalent salt	
`T`	absolute temperature (degC is converted)	

output	meaning
`K_in`	inside cation concentration
`Cl_in`	inside permeant anion concentration
`ratio`	Donnan ratio K_in / K_out = Cl_out / Cl_in
`E`	Donnan potential (inside minus outside)
`E_mV`	Donnan potential in mV
`excess_mOsm`	inside minus outside osmolarity in mOsm/L

- **Method:** K_in $Cl_in = c^2$ and $K_in = Cl_in + zA$ A: Cl_in = (-zA A + sqrt(zA^2 A^2 + 4c^2))/2; $E = (RT/F)$ ln(c/K_in)
- **Assumes:** A, c, $T > 0$; $zA = 1$, 2 or 3; ideal solutions, electroneutral compartments
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



K_in (inside cation concentration) against A (concentration of impermeant anions inside) from 10 mol/m^3 to 1000 mol/m^3 on a logarithmic axis. One curve for each value of c (outside concentration of the monovalent salt): 100 mol/m^3, 50 mol/m^3, 200 mol/m^3. Other inputs as in the worked example: zA = 1, T = 298.15 K. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, A = 100 mol/m^3, K_in = 161.803 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_donnan_equilibrium', A=100.0, zA=1, c=100.0, T=298.15)
# -> K_in = 161.803 mol/m^3, Cl_in = 61.8034 mol/m^3, ratio = 1.61803, E = -0.0123636 V, E_mV = -12.3636, excess_mOsm =
123.607   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_donnan_equilibrium', A=50.0, zA=2, c=150.0, T=310.15)
# -> K_in = 208.114 mol/m^3, Cl_in = 108.114 mol/m^3, ratio = 1.38743, E = -0.00875165 V, E_mV = -8.75165, excess_mOsm
= 66.2278   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Donnan equilibrium: A = 100 mM, zA = 1, c = 100 mM, T = 298.15 K	SOLVED	K_in = 161.803 mol/m^3, Cl_in = 61.8034 mol/m^3, ratio = 1.61803, E = -0.0123636 V, E_mV = -12.3636, excess_mOsm = 123.607
Donnan equilibrium: A = 50 mM, zA = 2, c = 150 mM, T = 310.15 K	SOLVED	K_in = 208.114 mol/m^3, Cl_in = 108.114 mol/m^3, ratio = 1.38743, E = -0.00875165 V, E_mV = -8.75165, excess_mOsm = 66.2278
Donnan equilibrium: zA = 1, c = 100 mM, T = 298.15 K	NEEDS_INPUT	cmb_donnan_equilibrium needs A (concentration of impermeant anions inside). Example: solve('cmb_donnan_equilibrium', A='
Donnan equilibrium: A = 100 mM, zA = 1, c = 100 mM, T = 298.15 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Macroscopic current through ion channels, `cmb_channel_current`

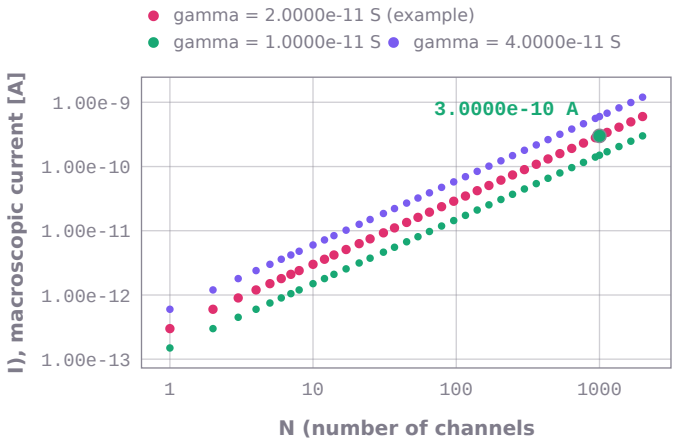
The whole-cell current $I = N$ Po gamma (V - E) through N channels of single-channel conductance gamma and open probability Po, and the single-channel current.

$$I = NP_o\gamma(V - E_{rev})$$

input	meaning	condition
`N`	number of channels	
`Po`	open probability (0 to 1)	
`gamma`	single-channel conductance	
`V`	membrane potential	
`E`	reversal potential of the channel	

output	meaning
`I`	macroscopic current (positive outward)
`I_pA`	current in pA
`G`	total open conductance N Po gamma
`i_single`	single open-channel current gamma (V - E)

- **Method:** $I = N$ Po gamma (V - E)
- **Assumes:** $N \geq 1$; $0 \leq Po \leq 1$; $\gamma > 0$; ohmic open channels
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



I (macroscopic current (positive outward)) against N (number of channels) from 1 to 2000 on a logarithmic axis, logarithmic output axis. One curve for each value of gamma (single-channel conductance): 2.0000e-11 S, 1.0000e-11 S, 4.0000e-11 S. Other inputs as in the worked example: Po = 0.3, V = -0.04 V, E = -0.09 V. 38 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, N = 1000, I = 3.0000e-10 A.

```
cell_molecular_biology_ai999x.solve('cmb_channel_current', N=1000, Po=0.3, gamma=2e-11, V=-0.04, E=-0.09)
# -> I = 3e-10 A, I_pA = 300, G = 6e-09 S, i_single = 1e-12 A   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_channel_current', N=50, Po=0.8, gamma=1e-10, V=0.0, E=0.05)
# -> I = -2e-10 A, I_pA = -200, G = 4e-09 S, i_single = -5e-12 A   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ion channel current: N = 1000, Po = 30%, gamma = 20 pS, V = -40 mV, E = -90 mV	SOLVED	I = 3e-10 A, I_pA = 300, G = 6e-09 S, i_single = 1e-12 A
Ion channel current: N = 50, Po = 80%, gamma = 100 pS, V = 0 mV, E = 50 mV	SOLVED	I = -2e-10 A, I_pA = -200, G = 4e-09 S, i_single = -5e-12 A
Ion channel current: Po = 30%, gamma = 20 pS, V = -40 mV, E = -90 mV	NEEDS_INPUT	cmb_channel_current needs N (number of channels). Example: solve('cmb_channel_current', N=1000, Po=0.3, gamma='2e-11 S',
Ion channel current: N = 1000, Po = 30%, gamma = 20 pS, V = -40 mV, E = -90 mV, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Channel current: N = 1000, Po = 30%, gamma = 0.02 nS, V = -40 mV, E = -90 mV	SOLVED	I = 3e-10 A, I_pA = 300, G = 6e-09 S, i_single = 1e-12 A

Diffusion-limited association rate constant (Smoluchowski), `cmb_diffusion_limited_rate`

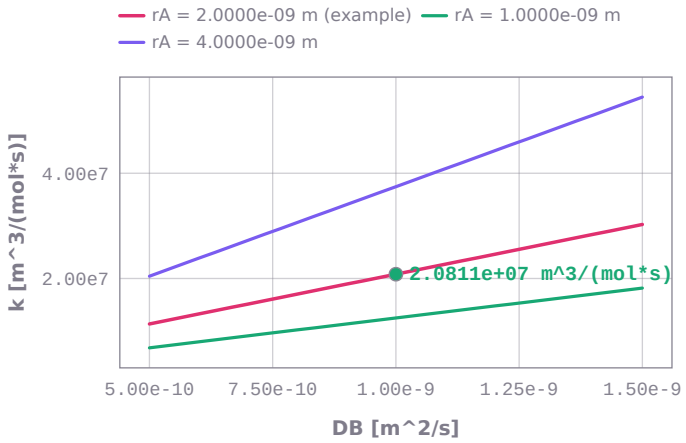
The maximum (diffusion-limited) bimolecular association rate constant of two spheres that react on contact.

$$k_D = 4\pi(D_A + D_B)(r_A + r_B)N_A$$

input	meaning	condition
`DA`	diffusion coefficient of molecule A	
`DB`	diffusion coefficient of molecule B	
`rA`	radius of molecule A	
`rB`	radius of molecule B	

output	meaning
`k`	diffusion-limited rate constant
`k_per_M_s`	rate constant in 1/(M s)

- **Method:** $k = 4$ pi (DA + DB)(rA + rB) N_A
- **Assumes:** all inputs > 0; reaction on every contact, no interaction forces
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



k (diffusion-limited rate constant) against DB (diffusion coefficient of molecule B) from 5.0000e-10 m^2/s to 1.5000e-09 m^2/s. One curve for each value of rA (radius of molecule A): 2.0000e-09 m, 1.0000e-09 m, 4.0000e-09 m. Other inputs as in the worked example: DA = 1e-10 m^2/s, rB = 5e-10 m. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, DB = 1.0000e-09 m^2/s, k = 2.0811e+07 m^3/(mol*s).

```
cell_molecular_biology_ai999x.solve('cmb_diffusion_limited_rate', DA=1e-10, DB=1e-09, rA=2e-09, rB=5e-10)
# -> k = 2.0811e+07 m^3/(mol*s), k_per_M_s = 2.0811e+10   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_diffusion_limited_rate', DA=6e-11, DB=6e-11, rA=3e-09, rB=3e-09)
# -> k = 5.4487e+06 m^3/(mol*s), k_per_M_s = 5.4487e+09   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diffusion-limited rate constant: DA = 100 um^2/s, DB = 1000 um^2/s, rA = 2 nm, rB = 0.5 nm	SOLVED	k = 2.0811e+07 m^3/(mol*s), k_per_M_s = 2.0811e+10
Diffusion-limited rate constant: DA = 60 um^2/s, DB = 60 um^2/s, rA = 3 nm, rB = 3 nm	SOLVED	k = 5.4487e+06 m^3/(mol*s), k_per_M_s = 5.4487e+09
Diffusion-limited rate constant: DB = 1000 um^2/s, rA = 2 nm, rB = 0.5 nm	NEEDS_INPUT	cmb_diffusion_limited_rate needs DA (diffusion coefficient of molecule A). Example: solve('cmb_diffusion_limited_rate',
Diffusion-limited rate constant: DA = 100 um^2/s, DB = 1000 um^2/s, rA = 2 nm, rB = 0.5 nm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Diffusion coefficient from a FRAP half-time (Soumpasis), `cmb_frap_diffusion`

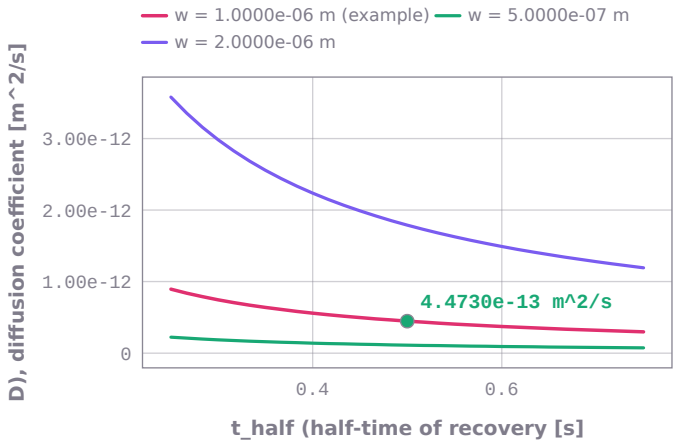
The two-dimensional diffusion coefficient from the half-time of fluorescence recovery after photobleaching a uniform circular spot of radius w (Soumpasis' exact solution, $D = 0.224$ w^2 / t_half).

$$f(t) = e^{-2\tau_D/t}\left[I_0\left(\tfrac{2\tau_D}{t}\right) + I_1\left(\tfrac{2\tau_D}{t}\right)\right], \quad D = \frac{w^2}{4\tau_D}$$

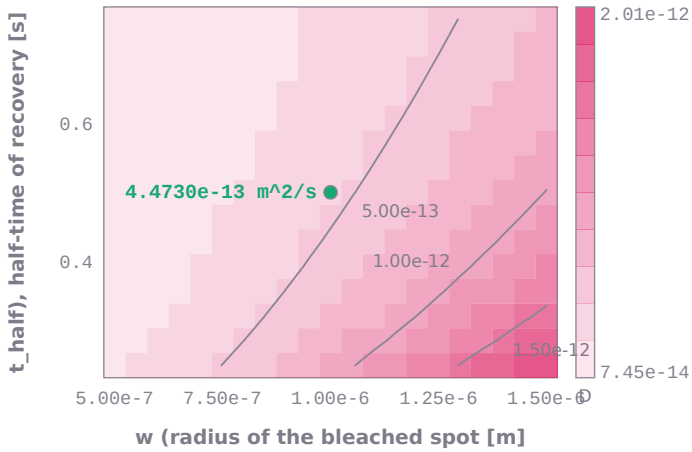
input	meaning	condition
`w`	radius of the bleached spot	
`t_half`	half-time of recovery	

output	meaning
`D`	diffusion coefficient
`D_um2_s`	D in um^2/s
`tauD`	characteristic diffusion time $w^2/(4D)$
`gamma`	tauD / t_half (about 1.12)

- **Method:** f(t) = exp(-2 tauD/t)[I0(2 tauD/t) + I1(2 tauD/t)]; solve f(t_half) = 1/2 for tauD; D = w^2/(4 tauD)
- **Assumes:** w, t_half > 0; $2 - D$ free diffusion, uniform circular bleach, full mobile fraction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



D (diffusion coefficient) against t_half (half-time of recovery) from 0.25 s to 0.75 s. One curve for each value of w (radius of the bleached spot): 1.0000e-06 m, 5.0000e-07 m, 2.0000e-06 m. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, t_half = 0.5 s, D = 4.4730e-13 m^2/s.



D (diffusion coefficient) over w (radius of the bleached spot) from 5.0000e-07 m to 1.5000e-06 m and t_half (half-time of recovery) from 0.25 s to 0.75 s: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 7.4549e-14 m^2/s to dark 2.0128e-12 m^2/s); iso-lines at D = 5.00e-13, 1.00e-12, 1.50e-12 m^2/s. Marked: the worked example, w = 1.0000e-06 m, t_half = 0.5 s, D = 4.4730e-13 m^2/s.

```
cell_molecular_biology_ai999x.solve('cmb_frap_diffusion', w=1e-06, t_half=0.5)
# -> D = 4.47296e-13 m^2/s, D_um2_s = 0.447296, tauD = 0.558915 s, gamma = 1.11783   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_frap_diffusion', w=2.5e-06, t_half=12.0)
# -> D = 1.16483e-13 m^2/s, D_um2_s = 0.116483, tauD = 13.4139 s, gamma = 1.11783   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
FRAP diffusion coefficient: w = 1 um, t_half = 0.5 s	SOLVED	D = 4.47296e-13 m^2/s, D_um2_s = 0.447296, tauD = 0.558915 s, gamma = 1.11783
FRAP diffusion coefficient: w = 2.5 um, t_half = 12 s	SOLVED	D = 1.16483e-13 m^2/s, D_um2_s = 0.116483, tauD = 13.4139 s, gamma = 1.11783
FRAP diffusion coefficient: t_half = 0.5 s	NEEDS_INPUT	cmb_frap_diffusion needs w (radius of the bleached spot). Example: solve('cmb_frap_diffusion', w='1e-06 m', t_half='0.5
FRAP diffusion coefficient: w = 1 um, t_half = 0.5 s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Exponential growth of a cell population, `cmb_exponential_growth`

The number of cells after a time t of exponential growth from N0 with doubling time td, the number of generations and the specific growth rate.

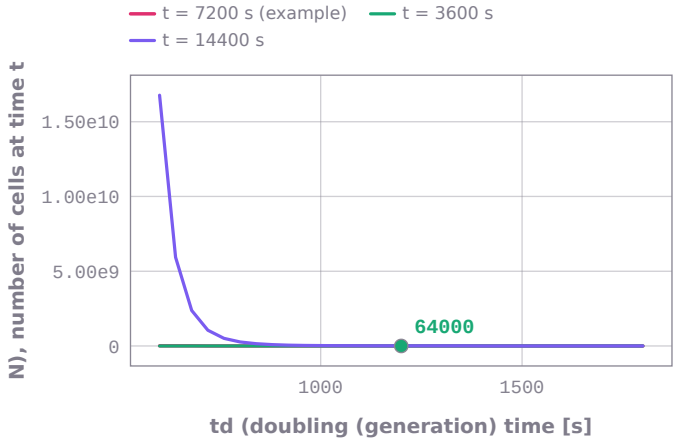
$$N = N_0\,2^{t/t_d} = N_0 e^{\mu t}, \quad \mu = \frac{\ln 2}{t_d}$$

input	meaning	condition
`N0`	initial number (or concentration) of cells	
`td`	doubling (generation) time	
`t`	elapsed time	

output	meaning
`N`	number of cells at time t

output	meaning
`generations`	number of generations t/td
`fold`	fold increase N/N_0
`mu`	specific growth rate $ln2/td$
`mu_per_h`	specific growth rate per hour

- **Method:** $N = N_0\ 2^{t/td}$; $mu = ln\ 2/td$
- **Assumes:** N0, $td > 0$; $t \geq 0$; balanced exponential growth (no lag, no limitation)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



N (number of cells at time t) against td (doubling (generation) time) from 600 s to 1800 s. One curve for each value of t (elapsed time): 7200 s, 3600 s, 14400 s. Other inputs as in the worked example: N0 = 1000.0. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, td = 1200 s, N = 64000.

```
cell_molecular_biology_ai999x.solve('cmb_exponential_growth', N0=1000.0, td=1200.0, t=7200.0)
# -> N = 64000, generations = 6, fold = 64, mu = 0.000577623 1/s, mu_per_h = 2.07944   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_exponential_growth', N0=200000.0, td=86400.0, t=259200.0)
# -> N = 1.6e+06, generations = 3, fold = 8, mu = 8.02254e-06 1/s, mu_per_h = 0.0288811   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential cell growth: N0 = 1000, td = 0.3333333333 h, t = 2 h	SOLVED	N = 64000, generations = 6, fold = 64, mu = 0.000577623 1/s, mu_per_h = 2.07944
Exponential cell growth: N0 = 200000, td = 24 h, t = 72 h	SOLVED	N = 1.6e+06, generations = 3, fold = 8, mu = 8.02254e-06 1/s, mu_per_h = 0.0288811
Exponential cell growth: td = 0.3333333333 h, t = 2 h	NEEDS_INPUT	cmb_exponential_growth needs N0 (initial number (or concentration) of cells). Example: solve('cmb_exponential_growth', N
Exponential cell growth: N0 = 1000, td = 0.3333333333 h, t = 2 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Exponential cell growth: N0 = 1000, td = 20 min, t = 2 h	SOLVED	N = 64000, generations = 6, fold = 64, mu = 0.000577623 1/s, mu_per_h = 2.07944

Doubling time and number of generations from two counts, `cmb_doubling_time_from_counts`

The doubling (generation) time, the number of generations and the specific growth rate from the cell counts at the start and end of an interval of exponential growth.

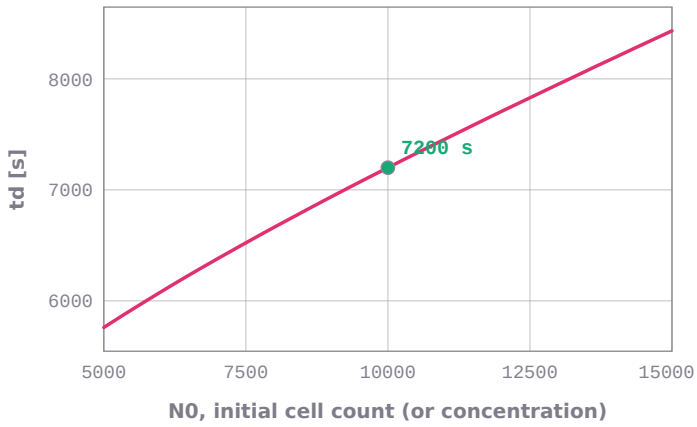
$$n = \log_2 \frac{N}{N_0}, \quad t_d = \frac{t}{n}, \quad \mu = \frac{\ln(N/N_0)}{t}$$

input	meaning	condition
`N0`	initial cell count (or concentration)	
`N`	final cell count (same unit)	
`t`	elapsed time	

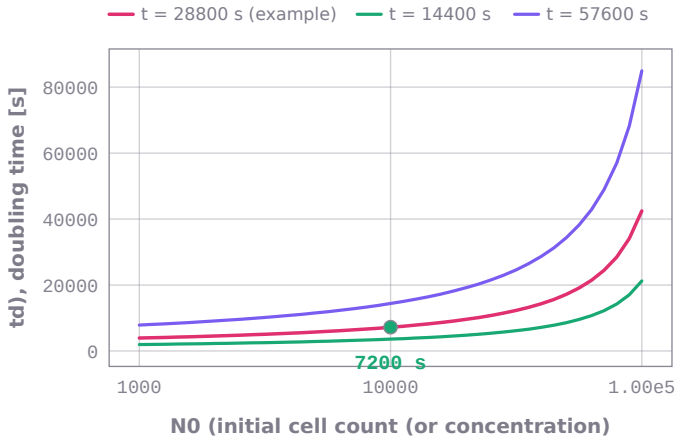
output	meaning
`td`	doubling time
`td_h`	doubling time in hours
`generations`	number of generations log2(N/N0)
`mu`	specific growth rate
`mu_per_h`	specific growth rate per hour

- **Method:** $n = log2$(N/N0) = 3.32 log10(N/N0); $td = t/n$; $mu = ln\ 2/td$

- **Assumes:** $N > N_0 > 0$; $t > 0$; exponential growth over the interval
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



td (doubling time) against N0 (initial cell count (or concentration)), with N = 1.60e5, t = 28800 s. 31 points computed by cell-molecular-biology-ai999x; marked: N0 = 10000, td = 7200 s.



td (doubling time) against N0 (initial cell count (or concentration)) from 1000 to 100000 on a logarithmic axis. One curve for each value of t (elapsed time): 28800 s, 14400 s, 57600 s. Other inputs as in the worked example: N = 160000.0. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, N0 = 10000, td = 7200 s.

```
cell_molecular_biology_ai999x.solve('cmb_doubling_time_from_counts', N0=10000.0, N=160000.0, t=28800.0)
# -> td = 7200 s, td_h = 2, generations = 4, mu = 9.62704e-05 1/s, mu_per_h = 0.346574   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_doubling_time_from_counts', N0=500000.0, N=2000000.0, t=172800.0)
# -> td = 86400 s, td_h = 24, generations = 2, mu = 8.02254e-06 1/s, mu_per_h = 0.0288811   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Doubling time from cell counts: N0 = 10000, N = 160000, t = 8 h	SOLVED	td = 7200 s, td_h = 2, generations = 4, mu = 9.62704e-05 1/s, mu_per_h = 0.346574
Doubling time from cell counts: N0 = 500000, N = 2000000, t = 48 h	SOLVED	td = 86400 s, td_h = 24, generations = 2, mu = 8.02254e-06 1/s, mu_per_h = 0.0288811
Doubling time from cell counts: N = 160000, t = 8 h	NEEDS_INPUT	cmb_doubling_time_from_counts needs N0 (initial cell count (or concentration)). Example: solve('cmb_doubling_time_from_c
Doubling time from cell counts: N0 = 10000, N = 160000, t = 8 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Doubling time: N0 = 10000, N = 160000, t = 480 min	SOLVED	td = 7200 s, td_h = 2, generations = 4, mu = 9.62704e-05 1/s, mu_per_h = 0.346574
Doubling time: N0 = 10000, N = 5000, t = 8 h	INVALID_INPUT	Doubling time and number of generations from two counts: the final count must exceed the initial count.

Specific growth rate fitted to growth-curve data, `cmb_growth_rate_fit`

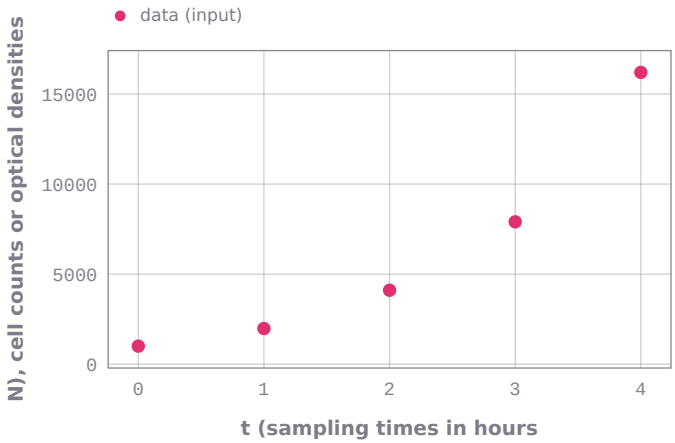
The specific growth rate, doubling time and back-extrapolated initial count from counts (or OD) measured during exponential growth, by a least-squares line through ln N against time.

$$\ln N = \ln N_0 + \mu t, \quad t_d = \frac{\ln 2}{\mu}$$

input	meaning	condition
`t`	sampling times in hours	
`N`	cell counts or optical densities (positive)	

output	meaning
`mu_per_h`	specific growth rate per hour
`td_h`	doubling time in hours
`N0_fit`	fitted count at $t = 0$
`r2`	coefficient of determination of the log-linear fit

- **Method:** least-squares line ln $N = ln\ N_0 + mu$ t; $td = ln\ 2/mu$
- **Assumes:** at least 3 points; counts > 0, increasing; exponential phase only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data N (cell counts or optical densities (positive)) against t (sampling times in hours) as given in the worked example (5 points). Also returned: mu_per_h = 0.695379, td_h = 0.996791, N0_fit = 998.435, r2 = 0.999807.

```
cell_molecular_biology_ai999x.solve('cmb_growth_rate_fit', t=[0.0, 1.0, 2.0, 3.0, 4.0], N=[1000.0, 1980.0, 4100.0,
7900.0, 16200.0])
# -> mu_per_h = 0.695379, td_h = 0.996791, N0_fit = 998.435, r2 = 0.999807   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_growth_rate_fit', t=[0.0, 2.0, 4.0, 6.0], N=[0.05, 0.11, 0.21, 0.4])
# -> mu_per_h = 0.344248, td_h = 2.01351, N0_fit = 0.0521971, r2 = 0.997406   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Growth rate fit: t = [0, 1, 2, 3, 4], N = [1000, 1980, 4100, 7900, 16200]	SOLVED	mu_per_h = 0.695379, td_h = 0.996791, N0_fit = 998.435, r2 = 0.999807
Growth rate fit: t = [0, 2, 4, 6], N = [0.05, 0.11, 0.21, 0.4]	SOLVED	mu_per_h = 0.344248, td_h = 2.01351, N0_fit = 0.0521971, r2 = 0.997406
Growth rate fit: N = [1000, 1980, 4100, 7900, 16200]	NEEDS_INPUT	cmb_growth_rate_fit needs t (sampling times in hours). Example: solve('cmb_growth_rate_fit', t=[0.0, 1.0, 2.0, 3.0, 4.0]
Growth rate fit: t = [0, 1, 2, 3, 4], N = [1000, 1980, 4100, 7900, 16200], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Logistic growth of a population, `cmb_logistic_growth`

The population at time t under logistic growth with carrying capacity K and intrinsic rate r, the growth rate dN/dt there and the time of the inflection point $(N = K/2)$.

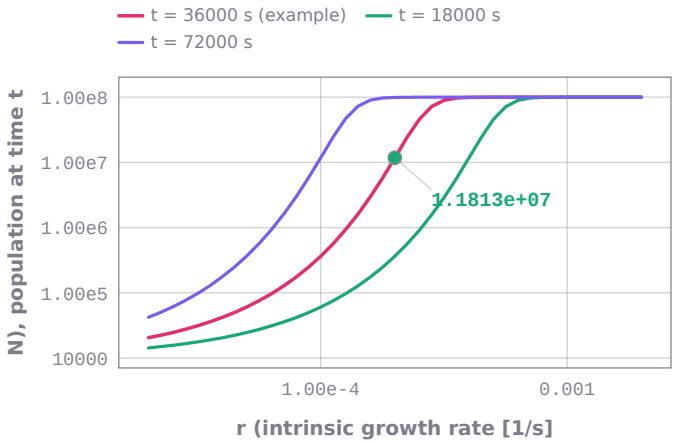
$$N(t) = \frac{K}{1 + \frac{K - N_0}{N_0} e^{-rt}}, \quad \frac{dN}{dt} = rN\left(1 - \frac{N}{K}\right)$$

input	meaning	condition
`N0`	initial population	
`K`	carrying capacity (same unit)	
`r`	intrinsic growth rate	
`t`	elapsed time	

output	meaning
`N`	population at time t
`dNdt`	growth rate at time t in cells per second
`dNdt_per_h`	growth rate at time t in cells per hour

output	meaning
`t_inflection`	time when $N = K/2$ (negative if already past)
`t_inflection_h`	time of the inflection point in hours

- **Method:** N = K / (1 + ((K - N0)/N0) e^(-r t)); $dN/dt = r$ N $(1 - N/K)$; t_infl = ln((K - N0)/N0)/r
- **Assumes:** $0 < N_0 < K$; $r > 0$; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



N (population at time t) against r (intrinsic growth rate) from 2.0000e-05 1/s to 0.002 1/s on a logarithmic axis, logarithmic output axis. One curve for each value of t (elapsed time): 36000 s, 18000 s, 72000 s. Other inputs as in the worked example: N0 = 10000.0, K = 100000000.0. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, r = 0.0002 1/s, N = 1.1813e+07.

```
cell_molecular_biology_ai999x.solve('cmb_logistic_growth', N0=10000.0, K=100000000.0, r=0.0002, t=36000.0)
# -> N = 1.18132e+07, dNdt = 2083.54, dNdt_per_h = 7.50073e+06, t_inflection = 46051.2 s, t_inflection_h = 12.792
(SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_logistic_growth', N0=50000000.0, K=1000000000.0, r=0.0001, t=18000.0)
# -> N = 2.41506e+08, dNdt = 18318.1, dNdt_per_h = 6.59452e+07, t_inflection = 29444.4 s, t_inflection_h = 8.179
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logistic growth: N0 = 10000, K = 100000000, r = 0.72 1/h, t = 10 h	SOLVED	N = 1.18132e+07, dNdt = 2083.54, dNdt_per_h = 7.50073e+06, t_inflection = 46051.2 s, t_inflection_h = 12.792
Logistic growth: N0 = 50000000, K = 1000000000, r = 0.36 1/h, t = 5 h	SOLVED	N = 2.41506e+08, dNdt = 18318.1, dNdt_per_h = 6.59452e+07, t_inflection = 29444.4 s, t_inflection_h = 8.179
Logistic growth: K = 100000000, r = 0.72 1/h, t = 10 h	NEEDS_INPUT	cmb_logistic_growth needs N0 (initial population). Example: solve('cmb_logistic_growth', N0=10000, K=1e+08, r='0.0002 1/
Logistic growth: N0 = 10000, K = 100000000, r = 0.72 1/h, t = 10 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Logistic growth: N0 = 10000, K = 100000000, r = 0.72 1/h, t = 600 min	SOLVED	N = 1.18132e+07, dNdt = 2083.54, dNdt_per_h = 7.50073e+06, t_inflection = 46051.2 s, t_inflection_h = 12.792

Time for a logistic population to reach a size, `cmb_logistic_time_to_reach`

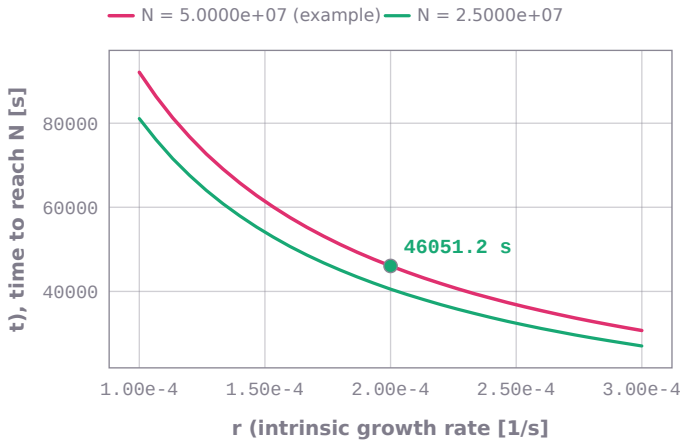
The time for a population growing logistically from N0 (carrying capacity K, rate r) to reach N.

$$t = \frac{1}{r} \ln \frac{N(K - N_0)}{N_0(K - N)}$$

input	meaning	condition
`N0`	initial population	
`K`	carrying capacity	
`r`	intrinsic growth rate	
`N`	target population	

output	meaning
`t`	time to reach N
`t_h`	time in hours

- **Method:** $t = (1/r)$ ln[N (K - N0) / (N0 (K - N))]
- **Assumes:** $0 < N_0 < N < K$; $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (time to reach N) against r (intrinsic growth rate) from 0.0001 1/s to 0.0003 1/s. One curve for each value of N (target population): 5.0000e+07, 2.5000e+07. Other inputs as in the worked example: N0 = 10000.0, K = 100000000.0. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, r = 0.0002 1/s, t = 46051.2 s.

```
cell_molecular_biology_ai999x.solve('cmb_logistic_time_to_reach', N0=10000.0, K=100000000.0, r=0.0002, N=50000000.0)
# -> t = 46051.2 s, t_h = 12.792   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_logistic_time_to_reach', N0=100.0, K=1000000.0, r=5e-05, N=900000.0)
# -> t = 228149 s, t_h = 63.3748   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logistic time to reach: N0 = 10000, K = 100000000, r = 0.72 1/h, N = 50000000	SOLVED	t = 46051.2 s, t_h = 12.792
Logistic time to reach: N0 = 100, K = 1000000, r = 0.18 1/h, N = 900000	SOLVED	t = 228149 s, t_h = 63.3748
Logistic time to reach: K = 100000000, r = 0.72 1/h, N = 50000000	NEEDS_INPUT	cmb_logistic_time_to_reach needs N0 (initial population). Example: solve('cmb_logistic_time_to_reach', N0=10000, K=1e+08
Logistic time to reach: N0 = 10000, K = 100000000, r = 0.72 1/h, N = 50000000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Logistic time to reach: N0 = 10000, K = 100000000, r = 0.72 1/h, N = 200000000	INVALID_INPUT	Time for a logistic population to reach a size: the target must lie between the initial population and the carrying capa

Monod specific growth rate, `cmb_monod_growth_rate`

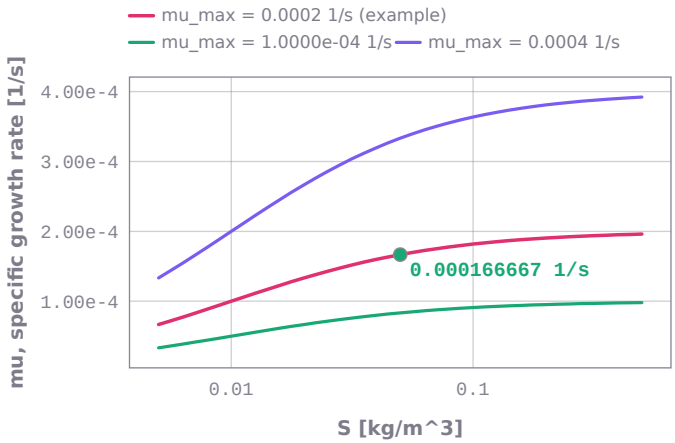
The specific growth rate of a microbial culture limited by one substrate (Monod), its fraction of the maximum and the corresponding doubling time.

$$\mu = \mu_{max}\frac{S}{K_s + S}$$

input	meaning	condition
`mu_max`	maximum specific growth rate	
`S`	limiting substrate concentration	
`Ks`	half-saturation constant	

output	meaning
`mu`	specific growth rate
`mu_per_h`	growth rate per hour
`fraction`	mu / mu_max
`td_h`	doubling time $ln2/mu$ in hours (0 when mu = 0)

- **Method:** mu = mu_max S / (Ks + S)
- **Assumes:** mu_max, $Ks > 0$; $S \geq 0$; a single limiting substrate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



mu (specific growth rate) against S (limiting substrate concentration) from 0.005 kg/m^3 to 0.5 kg/m^3 on a logarithmic axis. One curve for each value of mu_max (maximum specific growth rate): 0.0002 1/s, 1.0000e-04 1/s, 0.0004 1/s. Other inputs as in the worked example: Ks = 0.01 kg/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, S = 0.05 kg/m^3, mu = 0.000166667 1/s.

```
cell_molecular_biology_ai999x.solve('cmb_monod_growth_rate', mu_max=0.0002, S=0.05, Ks=0.01)
# -> mu = 0.000166667 1/s, mu_per_h = 0.6, fraction = 0.833333, td_h = 1.15525   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_monod_growth_rate', mu_max=0.0001, S=0.002, Ks=0.006)
# -> mu = 2.5e-05 1/s, mu_per_h = 0.09, fraction = 0.25, td_h = 7.70164   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Monod growth rate: mu_max = 0.72 1/h, S = 50 mg/L, Ks = 10 mg/L	SOLVED	mu = 0.000166667 1/s, mu_per_h = 0.6, fraction = 0.833333, td_h = 1.15525
Monod growth rate: mu_max = 0.36 1/h, S = 2 mg/L, Ks = 6 mg/L	SOLVED	mu = 2.5e-05 1/s, mu_per_h = 0.09, fraction = 0.25, td_h = 7.70164
Monod growth rate: S = 50 mg/L, Ks = 10 mg/L	NEEDS_INPUT	cmb_monod_growth_rate needs mu_max (maximum specific growth rate). Example: solve('cmb_monod_growth_rate', mu_max='0.000
Monod growth rate: mu_max = 0.72 1/h, S = 50 mg/L, Ks = 10 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Monod growth rate: mu_max = 0.72 1/h, S = 50 mg/L, Ks = 10 mg/L	SOLVED	mu = 0.000166667 1/s, mu_per_h = 0.6, fraction = 0.833333, td_h = 1.15525

Batch culture time under Monod kinetics, `cmb_monod_batch_time`

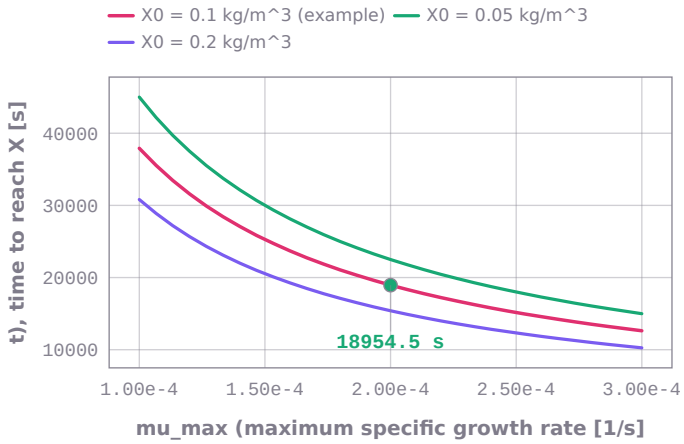
The time for a batch culture (constant yield Y, Monod growth) to grow from biomass X0 to X, and the substrate left then.

$$t = \frac{1}{\mu_{max}}\left[\left(1+\frac{K_s Y}{B}\right)\ln\frac{X}{X_0} - \frac{K_s Y}{B}\ln\frac{B-X}{B-X_0}\right]$$

input	meaning	condition
`X0`	initial biomass concentration	
`S0`	initial substrate concentration	
`Y`	biomass yield (g biomass per g substrate)	
`mu_max`	maximum specific growth rate	
`Ks`	half-saturation constant	
`X`	target biomass concentration	

output	meaning
`t`	time to reach X
`t_h`	time in hours
`S`	substrate remaining
`S_g_L`	substrate remaining in g/L

- **Method:** $S = S_0 - (X - X_0)/Y$; integrate dX/dt = mu_max S X/(Ks + S): t = [ln(X/X_0) + (Ks Y/B)(ln(X/X0) - ln($(B-X)/(B-X_0)$))]/mu_max, $B = X_0 + Y$ S0
- **Assumes:** all inputs > 0; $X_0 < X < X_0 + Y$ S0; no maintenance or death
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (time to reach X) against mu_max (maximum specific growth rate) from 0.0001 1/s to 0.0003 1/s. One curve for each value of X0 (initial biomass concentration): 0.1 kg/m^3, 0.05 kg/m^3, 0.2 kg/m^3. Other inputs as in the worked example: S0 = 10.0 kg/m^3, Y = 0.5, Ks = 0.2 kg/m^3, X = 4.0 kg/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, mu_max = 0.0002 1/s, t = 18954.5 s.

```
cell_molecular_biology_ai999x.solve('cmb_monod_batch_time', X0=0.1, S0=10.0, Y=0.5, mu_max=0.0002, Ks=0.2, X=4.0)
# -> t = 18954.5 s, t_h = 5.26514, S = 2.2 kg/m^3, S_g_L = 2.2   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_monod_batch_time', X0=0.05, S0=5.0, Y=0.4, mu_max=0.0001, Ks=1.0, X=1.0)
# -> t = 37059.9 s, t_h = 10.2944, S = 2.625 kg/m^3, S_g_L = 2.625   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Monod batch growth time: X0 = 0.1 g/L, S0 = 10 g/L, Y = 0.5, mu_max = 0.72 1/h, Ks = 0.2 g/L, X = 4 g/L	SOLVED	t = 18954.5 s, t_h = 5.26514, S = 2.2 kg/m^3, S_g_L = 2.2
Monod batch growth time: X0 = 0.05 g/L, S0 = 5 g/L, Y = 0.4, mu_max = 0.36 1/h, Ks = 1 g/L, X = 1 g/L	SOLVED	t = 37059.9 s, t_h = 10.2944, S = 2.625 kg/m^3, S_g_L = 2.625
Monod batch growth time: S0 = 10 g/L, Y = 0.5, mu_max = 0.72 1/h, Ks = 0.2 g/L, X = 4 g/L	NEEDS_INPUT	cmb_monod_batch_time needs X0 (initial biomass concentration). Example: solve('cmb_monod_batch_time', X0='0.1 kg/m^3', S
Monod batch growth time: X0 = 0.1 g/L, S0 = 10 g/L, Y = 0.5, mu_max = 0.72 1/h, Ks = 0.2 g/L, X = 4 g/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Batch culture time: X0 = 0.1 g/L, S0 = 10 g/L, Y = 0.5, mu_max = 0.72 1/h, Ks = 200 mg/L, X = 4 g/L	SOLVED	t = 18954.5 s, t_h = 5.26514, S = 2.2 kg/m^3, S_g_L = 2.2

Cell-cycle phase durations from phase fractions, `cmb_cell_cycle_phase_durations`

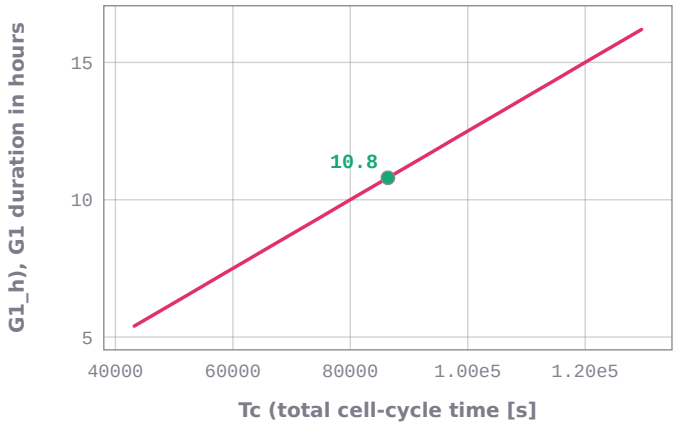
The durations of G1, S, G2 and M from the fraction of an asynchronous population in each phase (simple proportion to the cycle time).

$$t_{phase} = f_{phase}\, T_c$$

input	meaning	condition
`Tc`	total cell-cycle time	
`fractions`	fractions of cells in G1, S, G2 and M (in this order, summing to 1)	

output	meaning
`G1_h`	G1 duration in hours
`S_h`	S duration in hours
`G2_h`	G2 duration in hours
`M_h`	M duration in hours

- **Method:** duration of a phase = fraction x Tc
- **Assumes:** $Tc > 0$; four positive fractions summing to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



G1_h (G1 duration in hours) against Tc (total cell-cycle time) from 43200 s to 129600 s. Other inputs as in the worked example: fractions = [0.45, 0.35, 0.15, 0.05]. 31 points computed by cell-molecular-biology-ai999x; marked: the worked example, Tc = 86400 s, G1_h = 10.8.

```
cell_molecular_biology_ai999x.solve('cmb_cell_cycle_phase_durations', Tc=86400.0, fractions=[0.45, 0.35, 0.15, 0.05])
# -> G1_h = 10.8, S_h = 8.4, G2_h = 3.6, M_h = 1.2   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_cell_cycle_phase_durations', Tc=72000.0, fractions=[0.5, 0.3, 0.16, 0.04])
# -> G1_h = 10, S_h = 6, G2_h = 3.2, M_h = 0.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cell-cycle phase durations: Tc = 24 h, fractions = [0.45, 0.35, 0.15, 0.05]	SOLVED	G1_h = 10.8, S_h = 8.4, G2_h = 3.6, M_h = 1.2
Cell-cycle phase durations: Tc = 20 h, fractions = [0.5, 0.3, 0.16, 0.04]	SOLVED	G1_h = 10, S_h = 6, G2_h = 3.2, M_h = 0.8
Cell-cycle phase durations: fractions = [0.45, 0.35, 0.15, 0.05]	NEEDS_INPUT	cmb_cell_cycle_phase_durations needs Tc (total cell-cycle time). Example: solve('cmb_cell_cycle_phase_durations', Tc='86
Cell-cycle phase durations: Tc = 24 h, fractions = [0.45, 0.35, 0.15, 0.05], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Cell-cycle phase durations corrected for the exponential age distribution, `cmb_cell_cycle_age_corrected`

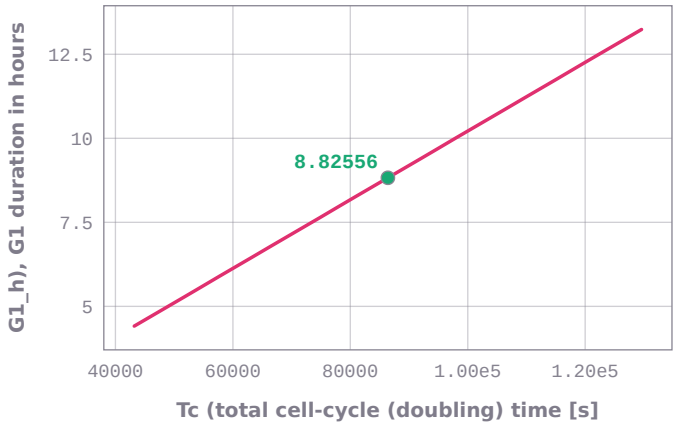
The durations of G1, S, G2 and M from phase fractions of an exponentially growing asynchronous population, using the age distribution n(a) = 2 $ln2/T\ 2^{-a/T}$ (young cells are over-represented).

$$F(a) = 2\left(1 - 2^{-a/T}\right), \quad a_k = -T\log_2\left(1 - \frac{F_k}{2}\right)$$

input	meaning	condition
`Tc`	total cell-cycle (doubling) time	
`fractions`	fractions of cells in G1, S, G2 and M (in this order, summing to 1)	

output	meaning
`G1_h`	G1 duration in hours
`S_h`	S duration in hours
`G2_h`	G2 duration in hours
`M_h`	M duration in hours

- **Method:** fraction younger than age a: F(a) = 2($\mathbf{1 - 2^{-a/T}}$); phase end ages $a_k = -T$ log2(1 - F_k/2) from cumulative fractions
- **Assumes:** $Tc > 0$; four positive fractions summing to 1; balanced exponential growth
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



G1_h (G1 duration in hours) against Tc (total cell-cycle (doubling) time) from 43200 s to 129600 s. Other inputs as in the worked example: fractions = [0.45, 0.35, 0.15, 0.05]. 31 points computed by cell-molecular-biology-ai999x; marked: the worked example, Tc = 86400 s, G1_h = 8.82556.

```
cell_molecular_biology_ai999x.solve('cmb_cell_cycle_age_corrected', Tc=86400.0, fractions=[0.45, 0.35, 0.15, 0.05])
# -> G1_h = 8.82556, S_h = 8.86161, G2_h = 4.62348, M_h = 1.68934   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_cell_cycle_age_corrected', Tc=72000.0, fractions=[0.5, 0.3, 0.16, 0.04])
# -> G1_h = 8.30075, S_h = 6.43856, G2_h = 4.12902, M_h = 1.13167   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Age-corrected cell-cycle phase durations: Tc = 24 h, fractions = [0.45, 0.35, 0.15, 0.05]	SOLVED	G1_h = 8.82556, S_h = 8.86161, G2_h = 4.62348, M_h = 1.68934
Age-corrected cell-cycle phase durations: Tc = 20 h, fractions = [0.5, 0.3, 0.16, 0.04]	SOLVED	G1_h = 8.30075, S_h = 6.43856, G2_h = 4.12902, M_h = 1.13167
Age-corrected cell-cycle phase durations: fractions = [0.45, 0.35, 0.15, 0.05]	NEEDS_INPUT	cmb_cell_cycle_age_corrected needs Tc (total cell-cycle (doubling) time). Example: solve('cmb_cell_cycle_age_corrected',
Age-corrected cell-cycle phase durations: Tc = 24 h, fractions = [0.45, 0.35, 0.15, 0.05], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Cell concentration from a hemocytometer count, `cmb_hemocytometer_count`

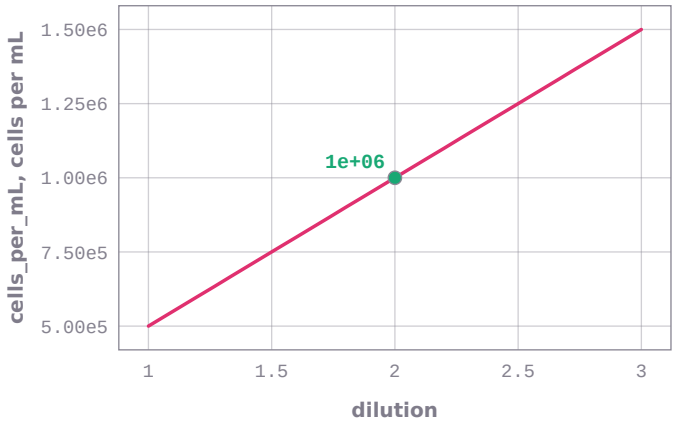
The cell concentration from counts in the large (1 mm x 1 mm x 0.1 mm) squares of a hemocytometer, the dilution factor and, optionally, the total cells in a suspension volume.

$$c = \bar{n} \times d \times 10^4 \ \mathrm{mL}^{-1}$$

input	meaning	condition
`counts`	cells counted in each large square	
`dilution`	dilution factor of the counted sample (1 for undiluted, 2 for 1:1 trypan blue)	
`V`	volume of the cell suspension (optional)	

output	meaning
`mean_count`	mean count per large square
`cells_per_mL`	cells per mL
`total_cells`	total cells in V (when V is given)

- **Method:** cells/mL = mean count x dilution x 10^4
- **Assumes:** counts >= 0; dilution >= 1; large squares of 0.1 uL
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cells_per_mL (cells per mL) against dilution (dilution factor of the counted sample (1 for undiluted, 2 for 1:1 trypan blue)) from 1 to 3. Other inputs as in the worked example: counts = [52.0, 48.0, 55.0, 45.0]. 31 points computed by cell-molecular-biology-ai999x; marked: the worked example, dilution = 2, cells_per_mL = 1e+06.

```
cell_molecular_biology_ai999x.solve('cmb_hemocytometer_count', counts=[52.0, 48.0, 55.0, 45.0], dilution=2.0)
# -> mean_count = 50, cells_per_mL = 1e+06   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_hemocytometer_count', counts=[120.0, 110.0], dilution=10.0, V=1e-05)
# -> mean_count = 115, cells_per_mL = 1.15e+07, total_cells = 1.15e+08   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hemocytometer cell count: counts = [52, 48, 55, 45], dilution = 2	SOLVED	mean_count = 50, cells_per_mL = 1e+06
Hemocytometer cell count: counts = [120, 110], dilution = 10, V = 10 mL	SOLVED	mean_count = 115, cells_per_mL = 1.15e+07, total_cells = 1.15e+08
Hemocytometer cell count: dilution = 2	NEEDS_INPUT	cmb_hemocytometer_count needs counts (cells counted in each large square). Example: solve('cmb_hemocytometer_count', cou
Hemocytometer cell count: counts = [52, 48, 55, 45], dilution = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Colony-forming units per mL from a plate count, `cmb_plate_count`

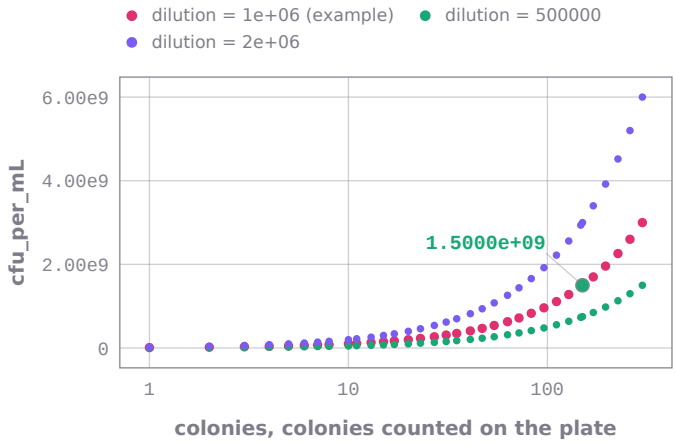
The viable count (CFU per mL) of the original culture from the colonies on a plate, the dilution factor of the plated sample and the plated volume.

$$\mathrm{CFU/mL} = \frac{n \times DF}{V_{plated}}$$

input	meaning	condition
`colonies`	colonies counted on the plate	
`dilution`	dilution factor of the plated sample (1e6 for a 10^{-6} dilution)	
`V`	volume plated	

output	meaning
`cfu_per_mL`	colony-forming units per mL of the original culture
`log10_cfu_per_mL`	log10 of CFU per mL (0 when no colonies)

- **Method:** CFU/mL = colonies x dilution factor / volume plated (mL)
- **Assumes:** colonies a whole number; dilution >= 1; $V > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cfu_per_mL (colony-forming units per mL of the original culture) against colonies (colonies counted on the plate) from 1 to 300 on a logarithmic axis. One curve for each value of dilution (dilution factor of the plated sample (1e6 for a 10^-6 dilution)): 1e+06, 500000, 2e+06. Other inputs as in the worked example: V = 1e-07 m^3. 34 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, colonies = 150, cfu_per_mL = 1.5000e+09.

```
cell_molecular_biology_ai999x.solve('cmb_plate_count', colonies=150, dilution=1000000.0, V=1e-07)
# -> cfu_per_mL = 1.5e+09, log10_cfu_per_mL = 9.17609   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_plate_count', colonies=42, dilution=10000.0, V=1e-06)
# -> cfu_per_mL = 420000, log10_cfu_per_mL = 5.62325   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plate count: colonies = 150, dilution = 1000000, V = 0.1 mL	SOLVED	cfu_per_mL = 1.5e+09, log10_cfu_per_mL = 9.17609
Plate count: colonies = 42, dilution = 10000, V = 1 mL	SOLVED	cfu_per_mL = 420000, log10_cfu_per_mL = 5.62325
Plate count: dilution = 1000000, V = 0.1 mL	NEEDS_INPUT	cmb_plate_count needs colonies (colonies counted on the plate). Example: solve('cmb_plate_count', colonies=150, dilution
Plate count: colonies = 150, dilution = 1000000, V = 0.1 mL, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
CFU per mL: colonies = 150, dilution = 1e6, V = 100 uL	SOLVED	cfu_per_mL = 1.5e+09, log10_cfu_per_mL = 9.17609

Transformation efficiency, `cmb_transformation_efficiency`

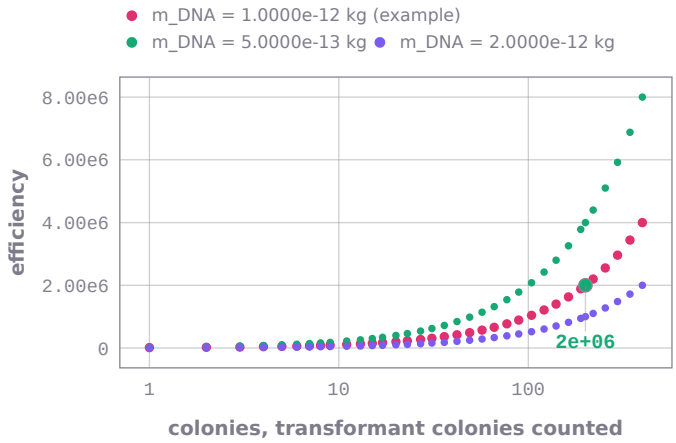
Transformants per microgram of DNA from the colonies on a plate, the DNA mass used and the fraction of the transformation plated.

$$\eta = \frac{n}{m_{DNA}\,(V_{plated}/V_{total})}$$

input	meaning	condition
`colonies`	transformant colonies counted	
`m_DNA`	DNA mass used in the transformation	
`V_plated`	volume plated	
`V_total`	total volume after recovery	

output	meaning
`efficiency`	transformants per microgram of DNA
`log10_efficiency`	log10 of the efficiency
`DNA_plated_ug`	micrograms of DNA plated

- **Method:** efficiency = colonies / (ug DNA x V_plated / V_total)
- **Assumes:** colonies a whole number; m_DNA, volumes > 0; V_plated <= V_total
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



efficiency (transformants per microgram of DNA) against colonies (transformant colonies counted) from 1 to 400 on a logarithmic axis. One curve for each value of m_DNA (DNA mass used in the transformation): 1.0000e-12 kg, 5.0000e-13 kg, 2.0000e-12 kg. Other inputs as in the worked example: V_plated = 1.0000000000000001e-07 m^3, V_total = 1.0000000000000002e-06 m^3. 35 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, colonies = 200, efficiency = 2e+06.

```
cell_molecular_biology_ai999x.solve('cmb_transformation_efficiency', colonies=200, m_DNA=1e-12, V_plated=1e-07,
V_total=1e-06)
# -> efficiency = 2e+06, log10_efficiency = 6.30103, DNA_plated_ug = 0.0001   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_transformation_efficiency', colonies=85, m_DNA=1e-11, V_plated=5e-08,
V_total=1.05e-06)
# -> efficiency = 178500, log10_efficiency = 5.25164, DNA_plated_ug = 0.00047619   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transformation efficiency: colonies = 200, m_DNA = 1 ng, V_plated = 100 uL, V_total = 1000 uL	SOLVED	efficiency = 2e+06, log10_efficiency = 6.30103, DNA_plated_ug = 0.0001
Transformation efficiency: colonies = 85, m_DNA = 10 ng, V_plated = 50 uL, V_total = 1050 uL	SOLVED	efficiency = 178500, log10_efficiency = 5.25164, DNA_plated_ug = 0.00047619
Transformation efficiency: m_DNA = 1 ng, V_plated = 100 uL, V_total = 1000 uL	NEEDS_INPUT	cmb_transformation_efficiency needs colonies (transformant colonies counted). Example: solve('cmb_transformation_efficie
Transformation efficiency: colonies = 200, m_DNA = 1 ng, V_plated = 100 uL, V_total = 1000 uL, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Volume, surface area and surface-to-volume ratio of a spherical cell, `cmb_cell_geometry`

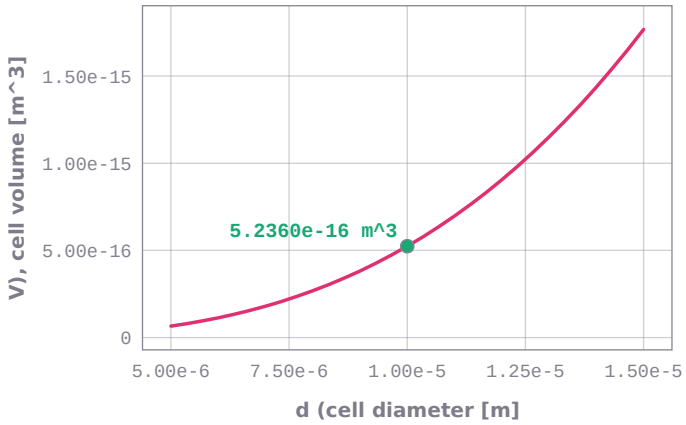
The volume (fL), surface area and surface-to-volume ratio of a spherical cell of a given diameter.

$$V = \tfrac{4}{3}\pi r^3, \quad A = 4\pi r^2, \quad \frac{A}{V} = \frac{3}{r}$$

input	meaning	condition
`d`	cell diameter	

output	meaning
`V`	cell volume
`V_fL`	volume in femtolitres (um^3)
`A`	surface area
`A_um2`	surface area in um^2
`SA_V_per_um`	surface-to-volume ratio in $1/um$

- **Method:** $V = (4/3)$ pi r^3, $A = 4$ pi r^2, $A/V = 3/r$
- **Assumes:** $d > 0$; a sphere
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (cell volume) against d (cell diameter) from 5.0000e-06 m to 1.5000e-05 m. 31 points computed by cell-molecular-biology-ai999x; marked: the worked example, d = 1.0000e-05 m, V = 5.2360e-16 m^3.

```
cell_molecular_biology_ai999x.solve('cmb_cell_geometry', d=1e-05)
# -> V = 5.23599e-16 m^3, V_fL = 523.599, A = 3.14159e-10 m^2, A_um2 = 314.159, SA_V_per_um = 0.6   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_cell_geometry', d=1e-06)
# -> V = 5.23599e-19 m^3, V_fL = 0.523599, A = 3.14159e-12 m^2, A_um2 = 3.14159, SA_V_per_um = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Spherical cell volume and surface area: d = 10 um	SOLVED	V = 5.23599e-16 m^3, V_fL = 523.599, A = 3.14159e-10 m^2, A_um2 = 314.159, SA_V_per_um = 0.6
Spherical cell volume and surface area: d = 1 um	SOLVED	V = 5.23599e-19 m^3, V_fL = 0.523599, A = 3.14159e-12 m^2, A_um2 = 3.14159, SA_V_per_um = 6
Spherical cell volume and surface area:	NEEDS_INPUT	cmb_cell_geometry needs d (cell diameter). Example: solve('cmb_cell_geometry', d='1e-05 m').
Spherical cell volume and surface area: d = 10 um, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Number of molecules in a cell from a concentration, `cmb_molecules_per_cell`

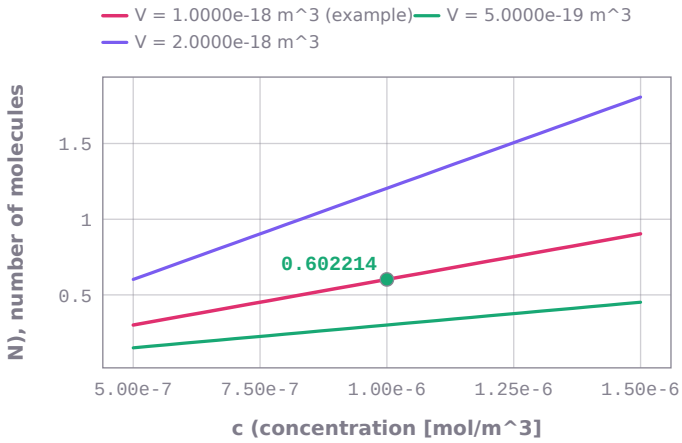
The number of molecules of a species at concentration c in a compartment of volume V, and the concentration that one molecule per compartment represents.

$$N = c V N_A$$

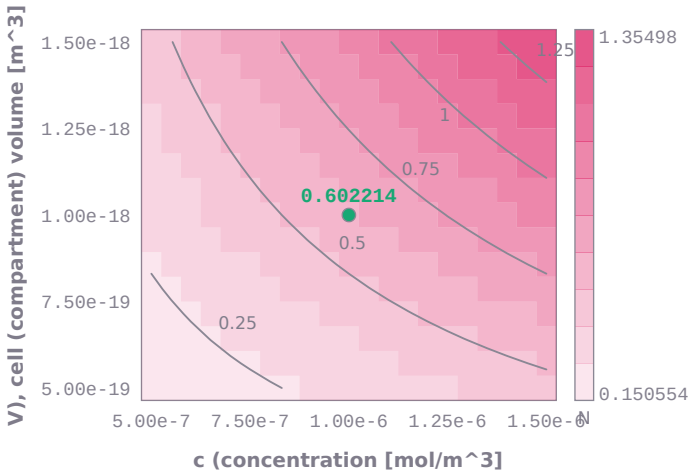
input	meaning	condition
`c`	concentration	
`V`	cell (compartment) volume	

output	meaning
`N`	number of molecules
`one_molecule`	concentration of one molecule in V
`one_molecule_nM`	one molecule per compartment in nM

- **Method:** $N = c$ V N_A
- **Assumes:** c, $V > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



N (number of molecules) against c (concentration) from 5.0000e-07 mol/m^3 to 1.5000e-06 mol/m^3. One curve for each value of V (cell (compartment) volume): 1.0000e-18 m^3, 5.0000e-19 m^3, 2.0000e-18 m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, c = 1.0000e-06 mol/m^3, N = 0.602214.



N (number of molecules) over c (concentration) from 5.0000e-07 mol/m^3 to 1.5000e-06 mol/m^3 and V (cell (compartment) volume) from 5.0000e-19 m^3 to 1.5000e-18 m^3: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 0.150554 to dark 1.35498); iso-lines at N = 0.25, 0.5, 0.75, 1, 1.25. Marked: the worked example, c = 1.0000e-06 mol/m^3, V = 1.0000e-18 m^3, N = 0.602214.

```
cell_molecular_biology_ai999x.solve('cmb_molecules_per_cell', c=1e-06, V=1e-18)
# -> N = 0.602214, one_molecule = 1.66054e-06 mol/m^3, one_molecule_nM = 1.66054   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_molecules_per_cell', c=0.1, V=2e-15)
# -> N = 1.20443e+08, one_molecule = 8.3027e-10 mol/m^3, one_molecule_nM = 0.00083027   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Molecules per cell: c = 1 nM, V = 1 fL	SOLVED	N = 0.602214, one_molecule = 1.66054e-06 mol/m^3, one_molecule_nM = 1.66054
Molecules per cell: c = 100000 nM, V = 2000 fL	SOLVED	N = 1.20443e+08, one_molecule = 8.3027e-10 mol/m^3, one_molecule_nM = 0.00083027
Molecules per cell: V = 1 fL	NEEDS_INPUT	cmb_molecules_per_cell needs c (concentration). Example: solve('cmb_molecules_per_cell', c='1e-06 mol/m^3', V='1e-18 m^3
Molecules per cell: c = 1 nM, V = 1 fL, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Molecules per cell: c = 1 nM, V = 1 fL	SOLVED	N = 0.602214, one_molecule = 1.66054e-06 mol/m^3, one_molecule_nM = 1.66054

GC content of a DNA or RNA sequence, `cmb_gc_content`

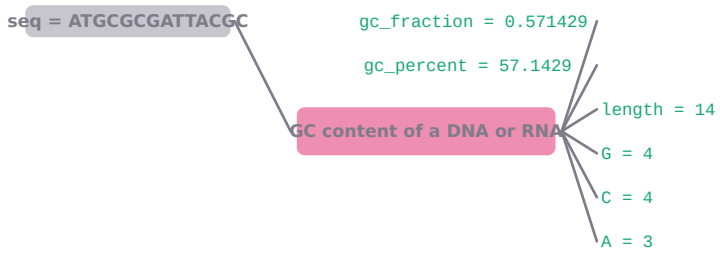
The fraction and percentage of G + C in a DNA or RNA sequence, with the base counts and the length.

$$\mathrm{GC} = \frac{n_G + n_C}{N}$$

input	meaning	condition
seq	DNA or RNA sequence (A, C, G, T or U)	

output	meaning
`gc_fraction`	fraction of G + C
`gc_percent`	GC content in per cent
`length`	length in bases
`G`	number of G
`C`	number of C
`A`	number of A
`T_or_U`	number of T (DNA) or U (RNA)

- **Method:** $GC = (n_G + n_C)/N$
- **Assumes:** only A, C, G, T or U
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of GC content of a DNA or RNA sequence as inputs and results: stated seq = ATGCGCGATTACGC; returned by cell-molecular-biology-ai999x: gc_fraction = 0.571429, gc_percent = 57.1429, length = 14, G = 4, C = 4, A = 3.

```
cell_molecular_biology_ai999x.solve('cmb_gc_content', seq='ATGCGCGATTACGC')
# -> gc_fraction = 0.571429, gc_percent = 57.1429, length = 14, G = 4, C = 4, A = 3, T_or_U = 3   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_gc_content', seq='GGAUCUUCGGAUCU')
# -> gc_fraction = 0.5, gc_percent = 50, length = 14, G = 4, C = 3, A = 2, T_or_U = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
GC content: seq = ATGCGCGATTACGC	SOLVED	gc_fraction = 0.571429, gc_percent = 57.1429, length = 14, G = 4, C = 4, A = 3, T_or_U = 3
GC content: seq = GGAUCUUCGGAUCU	SOLVED	gc_fraction = 0.5, gc_percent = 50, length = 14, G = 4, C = 3, A = 2, T_or_U = 5
GC content:	NEEDS_INPUT	cmb_gc_content needs seq (DNA or RNA sequence (A, C, G, T or U)). Example: solve('cmb_gc_content', seq='ATGCGCGATTACGC')
GC content: seq = ATGCGCGATTACGC, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Primer melting temperature by the Wallace rule, `cmb_tm_wallace`

The approximate melting temperature (degC) of a short DNA primer (about $14 - 20$ nt) by the Wallace rule, 2 degC per A/T and 4 degC per G/C.

$$T_m = 2(n_A + n_T) + 4(n_G + n_C)\ ^\circ\mathrm{C}$$

input	meaning	condition
`seq`	DNA primer sequence (A, C, G, T)	

output	meaning
`Tm`	melting temperature in degC
`AT`	number of A and T
`GC`	number of G and C

- **Method:** $Tm = 2\,(A + T) + 4$ (G + C) degC
- **Assumes:** DNA letters only; at least 2 bases (meant for $14 - 20$ nt)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Primer melting temperature by the Wallace rule as inputs and results: stated seq = ACGTTGCAATGCCGTA; returned by cell-molecular-biology-ai999x: Tm = 48, AT = 8, GC = 8.

```
cell_molecular_biology_ai999x.solve('cmb_tm_wallace', seq='ACGTTGCAATGCCGTA')
# -> Tm = 48, AT = 8, GC = 8   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_tm_wallace', seq='GCGCAATTAGCCTAGG')
# -> Tm = 50, AT = 7, GC = 9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wallace melting temperature: seq = ACGTTGCAATGCCGTA	SOLVED	Tm = 48, AT = 8, GC = 8
Wallace melting temperature: seq = GCGCAATTAGCCTAGG	SOLVED	Tm = 50, AT = 7, GC = 9
Wallace melting temperature:	NEEDS_INPUT	cmb_tm_wallace needs seq (DNA primer sequence (A, C, G, T)). Example: solve('cmb_tm_wallace', seq='ACGTTGCAATGCCGTA').
Wallace melting temperature: seq = ACGTTGCAATGCCGTA, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Basic GC melting temperature of an oligonucleotide, `cmb_tm_basic`

The basic melting temperature of a DNA oligonucleotide of 14 nt or more, $Tm = 64.9 + 41\ (G + C - 16.4)/N$ (50 mM Na+ assumed, no salt term).

$$T_m = 64.9 + 41\,\frac{n_G + n_C - 16.4}{N}$$

input	meaning	condition
`seq`	DNA sequence (A, C, G, T), at least 14 nt	

output	meaning
`Tm`	melting temperature in degC
`gc_percent`	GC content in per cent
`length`	length in nt

- **Method:** $Tm = 64.9 + 41\ (nG + nC - 16.4)/N$
- **Assumes:** DNA letters only; $N \geq 14$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Basic GC melting temperature of an oligonucleotide as inputs and results: stated seq = AGCGGATAACAATTTCACACAGGA; returned by cell-molecular-biology-ai999x: Tm = 53.9667, gc_percent = 41.6667, length = 24.

```
cell_molecular_biology_ai999x.solve('cmb_tm_basic', seq='AGCGGATAACAATTTCACACAGGA')
# -> Tm = 53.9667, gc_percent = 41.6667, length = 24   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_tm_basic', seq='GTAAAACGACGGCCAGT')
# -> Tm = 47.0529, gc_percent = 52.9412, length = 17   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Basic melting temperature: seq = AGCGGATAACAATTTCACACAGGA	SOLVED	Tm = 53.9667, gc_percent = 41.6667, length = 24
Basic melting temperature: seq = GTAAAACGACGGCCAGT	SOLVED	Tm = 47.0529, gc_percent = 52.9412, length = 17
Basic melting temperature:	NEEDS_INPUT	cmb_tm_basic needs seq (DNA sequence (A, C, G, T), at least 14 nt). Example: solve('cmb_tm_basic', seq='AGCGGATAACAATTTC
Basic melting temperature: seq = AGCGGATAACAATTTCACACAGGA, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Basic Tm: seq = ACGTACGT	INVALID_INPUT	Basic GC melting temperature of an oligonucleotide: the basic formula is for 14 nt or more (use the Wallace rule for sho

Salt-adjusted melting temperature of an oligonucleotide, `cmb_tm_salt_adjusted`

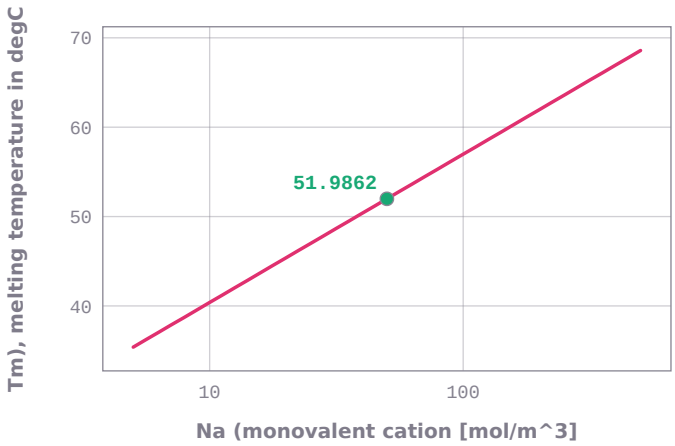
The melting temperature of a DNA oligonucleotide of 14 nt or more with a sodium correction: $Tm = 81.5 + 16.6$ log10[Na+] + 0.41 % $GC - 600/N$.

$$T_m = 81.5 + 16.6 \log_{10}[\mathrm{Na}^+] + 0.41\,\%GC - \frac{600}{N}$$

input	meaning	condition
`seq`	DNA sequence (A, C, G, T), at least 14 nt	
`Na`	monovalent cation (Na+) concentration	

output	meaning
`Tm`	melting temperature in degC
`gc_percent`	GC content in per cent
`salt_term`	salt term 16.6 log10[Na+] in degC

- **Method:** $Tm = 81.5 + 16.6$ log10([Na+]/M) + 0.41 %$GC - 600/N$
- **Assumes:** DNA letters only; $N \geq 14$; Na+ > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tm (melting temperature in degC) against Na (monovalent cation (Na+) concentration) from 5 mol/m^3 to 500 mol/m^3 on a logarithmic axis. Other inputs as in the worked example: seq = AGCGGATAACAATTTCACACAGGA. 41 points computed by cell-molecular-biology-ai999x; marked: the worked example, Na = 50 mol/m^3, Tm = 51.9862.

```
cell_molecular_biology_ai999x.solve('cmb_tm_salt_adjusted', seq='AGCGGATAACAATTTCACACAGGA', Na=50.0)
# -> Tm = 51.9862, gc_percent = 41.6667, salt_term = -21.5971   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_tm_salt_adjusted', seq='GTAAAACGACGGCCAGT', Na=100.0)
# -> Tm = 51.3118, gc_percent = 52.9412, salt_term = -16.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Salt-adjusted melting temperature: seq = AGCGGATAACAATTTCACACAGGA, Na = 50 mM	SOLVED	Tm = 51.9862, gc_percent = 41.6667, salt_term = -21.5971
Salt-adjusted melting temperature: seq = GTAAAACGACGGCCAGT, Na = 100 mM	SOLVED	Tm = 51.3118, gc_percent = 52.9412, salt_term = -16.6
Salt-adjusted melting temperature: Na = 50 mM	NEEDS_INPUT	cmb_tm_salt_adjusted needs seq (DNA sequence (A, C, G, T), at least 14 nt). Example: solve('cmb_tm_salt_adjusted', seq='
Salt-adjusted melting temperature: seq = AGCGGATAACAATTTCACACAGGA, Na = 50 mM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Salt adjusted Tm: seq = AGCGGATAACAATTTCACACAGGA, Na = 0.05 M	SOLVED	Tm = 51.9862, gc_percent = 41.6667, salt_term = -21.5971

Nearest-neighbour melting temperature of a DNA duplex, `cmb_tm_nearest_neighbor`

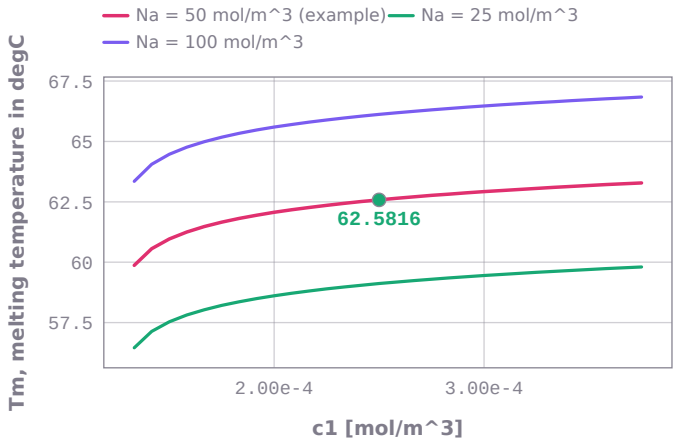
The melting temperature of a perfectly matched DNA duplex by nearest-neighbour thermodynamics (SantaLucia-Hicks 2004 unified parameters) with the SantaLucia (1998) sodium correction, and the duplex dH, dS and dG at 37 degC.

$$T_m = \frac{1000\,\Delta H^\circ}{\Delta S^\circ + 0.368(N-1)\ln[\mathrm{Na}^+] + R\ln k} - 273.15$$

input	meaning	condition
`seq`	DNA sequence 5'->3' (A, C, G, T)	
`Na`	monovalent cation (Na+) concentration	
`c1`	concentration of the more concentrated strand	
`c2`	concentration of the less concentrated strand (0 if in large excess)	

output	meaning
`Tm`	melting temperature in degC
`dH`	duplex enthalpy in kcal/mol
`dS`	salt-corrected duplex entropy in cal/(mol K)
`dG37`	free energy at 37 degC in kcal/mol

- **Method:** sum nearest-neighbour dH and dS with initiation and terminal A/T terms (+ symmetry); dS += 0.368 $(N-1)$ ln[Na+]; $Tm = 1000$ dH / (dS + R ln k) - 273.15, $k = c_1 - c_2/2$
- **Assumes:** DNA letters, length >= 2; Na+ > 0; $c_1 > c_2/2$ (self-complementary: k = c1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tm (melting temperature in degC) against c1 (concentration of the more concentrated strand) from 0.000125 mol/m^3 to 0.000375 mol/m^3. One curve for each value of Na (monovalent cation (Na+) concentration): 50 mol/m^3, 25 mol/m^3, 100 mol/m^3. Other inputs as in the worked example: seq = CGTTCCAAAGATGTGGGCATGAGCTTAC, c2 = 0.00025 mol/m^3. 30 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, c1 = 0.00025 mol/m^3, Tm = 62.5816.

```
cell_molecular_biology_ai999x.solve('cmb_tm_nearest_neighbor', seq='CGTTCCAAAGATGTGGGCATGAGCTTAC', Na=50.0, c1=0.00025,
c2=0.00025)
# -> Tm = 62.5816, dH = -221.7, dS = -628.766, dG37 = -26.6884   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_tm_nearest_neighbor', seq='GCGAATTCGC', Na=100.0, c1=0.0005, c2=0.0)
# -> Tm = 39.2579, dH = -79.4, dS = -225.326, dG37 = -9.51509   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nearest-neighbour melting temperature: seq = CGTTCCAAAGATGTGGGCATGAGCTTAC, Na = 50 mM, c1 = 250 nM, c2 = 250 nM	SOLVED	Tm = 62.5816, dH = -221.7, dS = -628.766, dG37 = -26.6884
Nearest-neighbour melting temperature: seq = GCGAATTCGC, Na = 100 mM, c1 = 500 nM, c2 = 0 nM	SOLVED	Tm = 39.2579, dH = -79.4, dS = -225.326, dG37 = -9.51509
Nearest-neighbour melting temperature: Na = 50 mM, c1 = 250 nM, c2 = 250 nM	NEEDS_INPUT	cmb_tm_nearest_neighbor needs seq (DNA sequence 5'->3' (A, C, G, T)). Example: solve('cmb_tm_nearest_neighbor', seq='CGT
Nearest-neighbour melting temperature: seq = CGTTCCAAAGATGTGGGCATGAGCTTAC, Na = 50 mM, c1 = 250 nM, c2 = 250 nM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Nearest-neighbour Tm: seq = CGTTCCAAAGATGTGGGCATGAGCTTAC, Na = 50 mM, c1 = 0.25 uM, c2 = 250 nM	SOLVED	Tm = 62.5816, dH = -221.7, dS = -628.766, dG37 = -26.6884

Molecular weight of a DNA or RNA oligonucleotide, `cmb_oligo_molecular_weight`

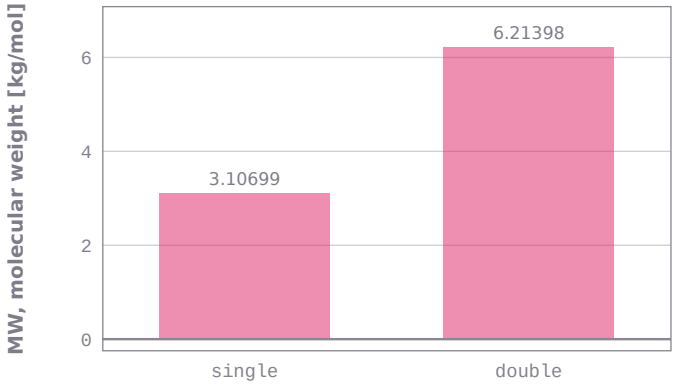
The average molecular weight of a single- or double-stranded DNA or RNA oligonucleotide, with a 5'-phosphate (sum of nucleotide monophosphates minus waters) and as the 5'-hydroxyl form of synthetic oligos.

$$M = \sum_i m_i - (n-1)\, M_{H_2O}$$

input	meaning	condition
`seq`	DNA or RNA sequence 5'->3'	
`strands`	single strand or the duplex with its complement	

output	meaning
`MW`	molecular weight (5'-phosphate)
`MW_g_mol`	molecular weight in g/mol (5'-phosphate)
`MW_hydroxyl_g_mol`	molecular weight in g/mol with 5'-hydroxyl ends

- **Method:** MW = sum of nucleotide monophosphate weights - (n - 1) x 18.0153; 5'-OH form minus HPO3 (79.98) per strand
- **Assumes:** only A, C, G, T (DNA) or A, C, G, U (RNA)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



MW (molecular weight (5'-phosphate)) for each option of strands (single strand or the duplex with its complement), one run of cell-molecular-biology-ai999x per option: single: 3.10699 kg/mol, double: 6.21398 kg/mol. The worked example uses single. Other inputs as in the worked example: seq = ATGCATGCAT.

```
cell_molecular_biology_ai999x.solve('cmb_oligo_molecular_weight', seq='ATGCATGCAT', strands='single')
# -> MW = 3.10699 kg/mol, MW_g_mol = 3106.99, MW_hydroxyl_g_mol = 3027.01   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_oligo_molecular_weight', seq='GGAUCC', strands='double')
# -> MW = 3.90832 kg/mol, MW_g_mol = 3908.32, MW_hydroxyl_g_mol = 3748.36   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Oligonucleotide molecular weight: seq = ATGCATGCAT, strands = single	SOLVED	MW = 3.10699 kg/mol, MW_g_mol = 3106.99, MW_hydroxyl_g_mol = 3027.01
Oligonucleotide molecular weight: seq = GGAUCC, strands = double	SOLVED	MW = 3.90832 kg/mol, MW_g_mol = 3908.32, MW_hydroxyl_g_mol = 3748.36
Oligonucleotide molecular weight: strands = single	NEEDS_INPUT	cmb_oligo_molecular_weight needs seq (DNA or RNA sequence 5'->3'). Example: solve('cmb_oligo_molecular_weight', seq='ATG
Oligonucleotide molecular weight: seq = ATGCATGCAT, strands = single, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Extinction coefficient of a DNA oligonucleotide at 260 nm (nearest neighbour), `cmb_oligo_extinction_coefficient`

The molar extinction coefficient at 260 nm of a single-stranded DNA oligonucleotide by the nearest-neighbour method, with micrograms and nanomoles per A260 unit (5'-hydroxyl oligo, 1 mL, 1 cm).

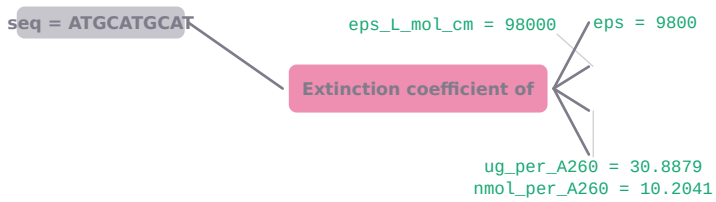
$$\varepsilon_{260} = \sum_{i=1}^{N-1} \varepsilon_{i,i+1} - \sum_{i=2}^{N-1} \varepsilon_i$$

input	meaning	condition
`seq`	DNA oligonucleotide 5'->3' (A, C, G, T)	

output	meaning
`eps`	molar extinction coefficient at 260 nm

output	meaning
`eps_L_mol_cm`	extinction coefficient in L/(mol cm)
`ug_per_A260`	micrograms per A260 unit (per mL at 1 cm)
`nmol_per_A260`	nanomoles per A260 unit (per mL at 1 cm)

- **Method:** eps = sum of nearest-neighbour pair values - sum of internal single-base values (Cantor-Warshaw-Shapiro table)
- **Assumes:** DNA letters only; single strand
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Extinction coefficient of a DNA oligonucleotide at 260 nm (nearest neighbour) as inputs and results: stated seq = ATGCATGCAT; returned by cell-molecular-biology-ai999x: eps = 9800, eps_L_mol_cm = 98000, ug_per_A260 = 30.8879, nmol_per_A260 = 10.2041.

```
cell_molecular_biology_ai999x.solve('cmb_oligo_extinction_coefficient', seq='ATGCATGCAT')
# -> eps = 9800 m^2/mol, eps_L_mol_cm = 98000, ug_per_A260 = 30.8879, nmol_per_A260 = 10.2041   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_oligo_extinction_coefficient', seq='GTAAAACGACGGCCAGT')
# -> eps = 17250 m^2/mol, eps_L_mol_cm = 172500, ug_per_A260 = 30.3097, nmol_per_A260 = 5.7971   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Oligonucleotide extinction coefficient: seq = ATGCATGCAT	SOLVED	eps = 9800 m^2/mol, eps_L_mol_cm = 98000, ug_per_A260 = 30.8879, nmol_per_A260 = 10.2041
Oligonucleotide extinction coefficient: seq = GTAAAACGACGGCCAGT	SOLVED	eps = 17250 m^2/mol, eps_L_mol_cm = 172500, ug_per_A260 = 30.3097, nmol_per_A260 = 5.7971
Oligonucleotide extinction coefficient:	NEEDS_INPUT	cmb_oligo_extinction_coefficient needs seq (DNA oligonucleotide 5'->3' (A, C, G, T)). Example: solve('cmb_oligo_extincti
Oligonucleotide extinction coefficient: seq = ATGCATGCAT, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Nucleic acid concentration from A260, `cmb_nucleic_acid_a260`

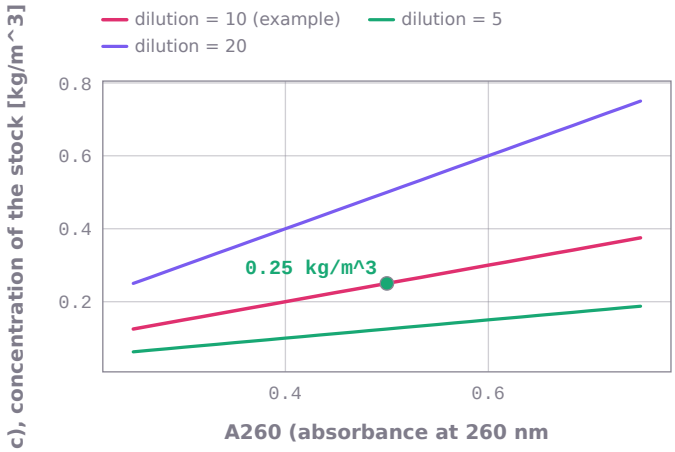
The concentration of dsDNA, ssDNA or RNA from its absorbance at 260 nm (50, 33 and 40 ug/mL per unit at 1 cm), with the dilution and path length, and the mass in a volume.

$$c = A_{260} \times F \times DF / \ell$$

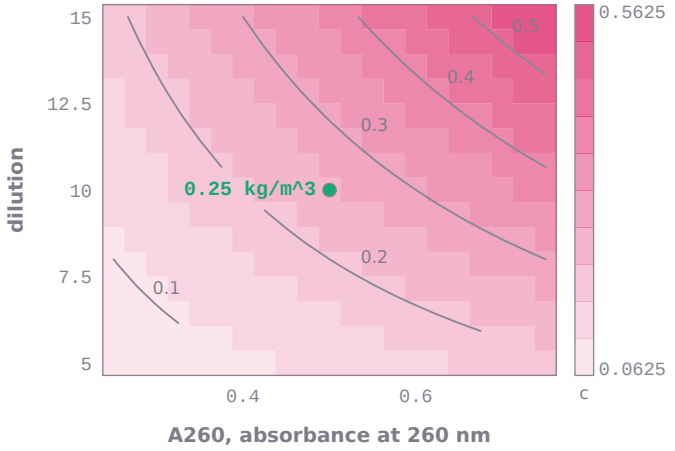
input	meaning	condition
`A260`	absorbance at 260 nm (blank-corrected)	
`molecule`	dsDNA, ssDNA or RNA	
`dilution`	dilution factor of the measured sample (default 1)	
`l`	path length (default 1 cm)	
`V`	volume of the stock (optional, for the total mass)	

output	meaning
`c`	concentration of the stock
`c_ug_mL`	concentration in ug/mL (= ng/uL)
`mass_ug`	micrograms in V (when V is given)

- **Method:** c $(ug/mL) = A260$ x factor x dilution / l(cm); factor 50 (dsDNA), 33 (ssDNA), 40 (RNA)
- **Assumes:** $A260 \geq 0$ (best within 0.1-1.0); dilution >= 1; $\ell > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



c (concentration of the stock) against A260 (absorbance at 260 nm (blank-corrected)) from 0.25 to 0.75. One curve for each value of dilution (dilution factor of the measured sample (default 1)): 10, 5, 20. Other inputs as in the worked example: molecule = dsDNA. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, A260 = 0.5, c = 0.25 kg/m^3.



c (concentration of the stock) over A260 (absorbance at 260 nm (blank-corrected)) from 0.25 to 0.75 and dilution (dilution factor of the measured sample (default 1)) from 5 to 15: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 0.0625 kg/m^3 to dark 0.5625 kg/m^3); iso-lines at c = 0.1, 0.2, 0.3, 0.4, 0.5 kg/m^3. Other inputs as in the worked example: molecule = dsDNA. Marked: the worked example, A260 = 0.5, dilution = 10, c = 0.25 kg/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_nucleic_acid_a260', A260=0.5, molecule='dsDNA', dilution=10.0)
# -> c = 0.25 kg/m^3, c_ug_mL = 250   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_nucleic_acid_a260', A260=0.25, molecule='RNA', dilution=50.0, V=5e-08)
# -> c = 0.5 kg/m^3, c_ug_mL = 500, mass_ug = 25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nucleic acid concentration from A260: A260 = 0.5, molecule = dsDNA, dilution = 10	SOLVED	c = 0.25 kg/m^3, c_ug_mL = 250
Nucleic acid concentration from A260: A260 = 0.25, molecule = RNA, dilution = 50, V = 50 uL	SOLVED	c = 0.5 kg/m^3, c_ug_mL = 500, mass_ug = 25
Nucleic acid concentration from A260: molecule = dsDNA, dilution = 10	NEEDS_INPUT	cmb_nucleic_acid_a260 needs A260 (absorbance at 260 nm (blank-corrected)). Example: solve('cmb_nucleic_acid_a260', A260=
Nucleic acid concentration from A260: A260 = 0.5, molecule = dsDNA, dilution = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
DNA concentration from A260: A260 = 0.5, molecule = ds DNA, dilution = 10	SOLVED	c = 0.25 kg/m^3, c_ug_mL = 250

A260/A280 and A260/A230 purity ratios, `cmb_absorbance_ratios`

The $A260/A280$ and $A260/A230$ ratios of a nucleic acid sample after subtracting the A320 background (about 1.8 for pure DNA, 2.0 for pure RNA; lower values suggest protein or organic contaminants).

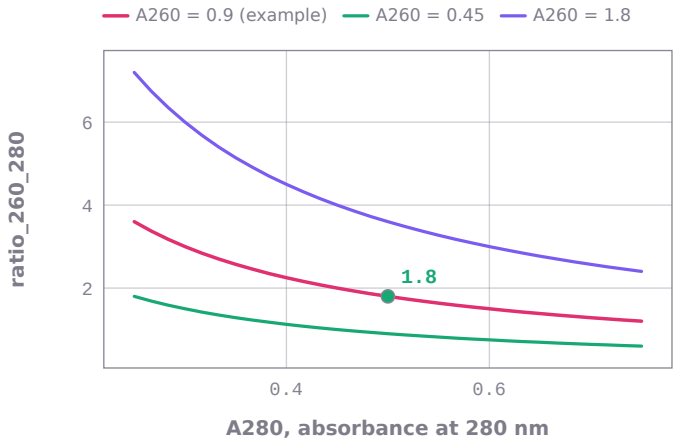
$$\frac{A_{260} - A_{320}}{A_{280} - A_{320}}$$

input	meaning	condition
`A260`	absorbance at 260 nm	
`A280`	absorbance at 280 nm	

input	meaning	condition
`A230`	absorbance at 230 nm (optional)	
`A320`	background absorbance at 320 nm (default 0)	

output	meaning
`ratio_260_280`	$A260/A280$ after background subtraction
`ratio_260_230`	$A260/A230$ after background subtraction (when A230 is given)

- **Method:** ratio = (A260 - A320)/(A280 - A320); $(A260 - A320)/(A230 - A320)$
- **Assumes:** absorbances > 0; A320 below the other readings
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ratio_260_280 (A260/A280 after background subtraction) against A280 (absorbance at 280 nm) from 0.25 to 0.75. One curve for each value of A260 (absorbance at 260 nm): 0.9, 0.45, 1.8. Other inputs as in the worked example: A230 = 0.42. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, A280 = 0.5, ratio_260_280 = 1.8.

```
cell_molecular_biology_ai999x.solve('cmb_absorbance_ratios', A260=0.9, A280=0.5, A230=0.42)
# -> ratio_260_280 = 1.8, ratio_260_230 = 2.14286   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_absorbance_ratios', A260=0.62, A280=0.33, A320=0.02)
# -> ratio_260_280 = 1.93548   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Absorbance ratios: A260 = 0.9, A280 = 0.5, A230 = 0.42	SOLVED	ratio_260_280 = 1.8, ratio_260_230 = 2.14286
Absorbance ratios: A260 = 0.62, A280 = 0.33, A320 = 0.02	SOLVED	ratio_260_280 = 1.93548
Absorbance ratios: A280 = 0.5, A230 = 0.42	NEEDS_INPUT	cmb_absorbance_ratios needs A260 (absorbance at 260 nm). Example: solve('cmb_absorbance_ratios', A260=0.9, A280=0.5, A23
Absorbance ratios: A260 = 0.9, A280 = 0.5, A230 = 0.42, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Protein concentration by Warburg-Christian (A280 and A260), `cmb_warburg_christian`

The protein concentration (mg/mL) of a sample containing nucleic acid from its absorbances at 280 and 260 nm, protein = 1.55 $A280 - 0.76$ A260 (1 cm).

$$c_{prot} = 1.55 A_{280} - 0.76 A_{260} \text{ mg/mL}$$

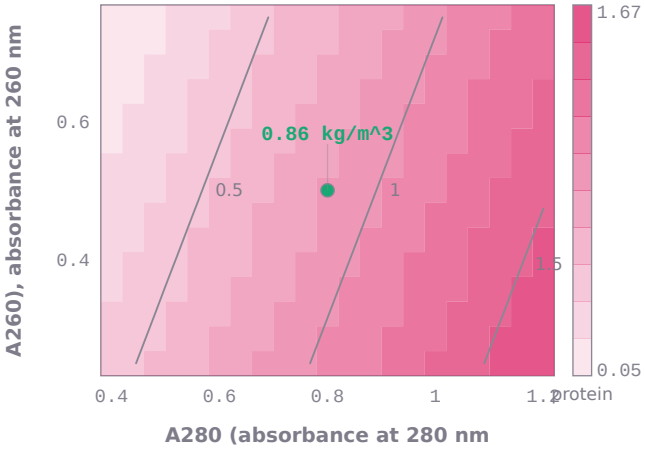
input	meaning	condition
`A280`	absorbance at 280 nm	
`A260`	absorbance at 260 nm	

output	meaning
`protein`	protein concentration
`protein_mg_mL`	protein in mg/mL
`ratio_280_260`	$A280/A260$

- **Method:** protein $(mg/mL) = 1.55\ A280 - 0.76$ A260
- **Assumes:** absorbances >= 0, 1 cm path; a positive estimate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



protein (protein concentration) against A280 (absorbance at 280 nm) from 0.4 to 1.2. One curve for each value of A260 (absorbance at 260 nm): 0.5, 0.25, 1. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, A280 = 0.8, protein = 0.86 kg/m^3.



protein (protein concentration) over A280 (absorbance at 280 nm) from 0.4 to 1.2 and A260 (absorbance at 260 nm) from 0.25 to 0.75: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 0.05 kg/m^3 to dark 1.67 kg/m^3); iso-lines at protein = 0.5, 1, 1.5 kg/m^3. Marked: the worked example, A280 = 0.8, A260 = 0.5, protein = 0.86 kg/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_warburg_christian', A280=0.8, A260=0.5)
# -> protein = 0.86 kg/m^3, protein_mg_mL = 0.86, ratio_280_260 = 1.6   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_warburg_christian', A280=1.2, A260=0.9)
# -> protein = 1.176 kg/m^3, protein_mg_mL = 1.176, ratio_280_260 = 1.33333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Warburg-Christian protein: A280 = 0.8, A260 = 0.5	SOLVED	protein = 0.86 kg/m^3, protein_mg_mL = 0.86, ratio_280_260 = 1.6
Warburg-Christian protein: A280 = 1.2, A260 = 0.9	SOLVED	protein = 1.176 kg/m^3, protein_mg_mL = 1.176, ratio_280_260 = 1.33333
Warburg-Christian protein: A260 = 0.5	NEEDS_INPUT	cmb_warburg_christian needs A280 (absorbance at 280 nm). Example: solve('cmb_warburg_christian', A280=0.8, A260=0.5).
Warburg-Christian protein: A280 = 0.8, A260 = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Protein concentration from A280 and the extinction coefficient, `cmb_protein_concentration_a280`

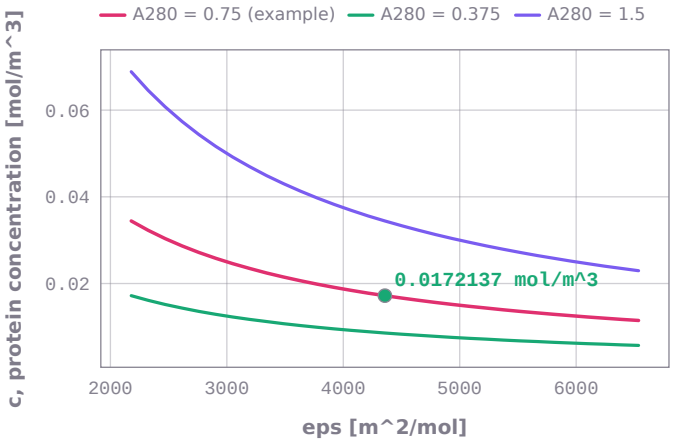
The molar (and, with the molar mass, mass) concentration of a protein from its absorbance at 280 nm and molar extinction coefficient (Beer-Lambert).

$$c = \frac{A_{280}}{\varepsilon \ell}$$

input	meaning	condition
`A280`	absorbance at 280 nm	
`eps`	molar extinction coefficient at 280 nm	
`MW`	molar mass of the protein (optional)	
`l`	path length (default 1 cm)	

output	meaning
`c`	protein concentration
`c_uM`	concentration in uM
`c_mg_mL`	concentration in mg/mL (when MW is given)

- **Method:** c = A / (eps l); mass concentration = c x MW
- **Assumes:** $A280 \geq 0$; eps, $l > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



c (protein concentration) against eps (molar extinction coefficient at 280 nm) from 2178.5 m^2/mol to 6535.5 m^2/mol. One curve for each value of A280 (absorbance at 280 nm): 0.75, 0.375, 1.5. Other inputs as in the worked example: MW = 66.43 kg/mol. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, eps = 4357 m^2/mol, c = 0.0172137 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_protein_concentration_a280', A280=0.75, eps=4357.0, MW=66.43)
# -> c = 0.0172137 mol/m^3, c_uM = 17.2137, c_mg_mL = 1.1435   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_protein_concentration_a280', A280=0.3, eps=3790.0)
# -> c = 0.00791557 mol/m^3, c_uM = 7.91557   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Protein concentration from A280: A280 = 0.75, eps = 43570 L/(mol*cm), MW = 66430 g/mol	SOLVED	c = 0.0172137 mol/m^3, c_uM = 17.2137, c_mg_mL = 1.1435
Protein concentration from A280: A280 = 0.3, eps = 37900 L/(mol*cm)	SOLVED	c = 0.00791557 mol/m^3, c_uM = 7.91557
Protein concentration from A280: eps = 43570 L/(mol*cm), MW = 66430 g/mol	NEEDS_INPUT	cmb_protein_concentration_a280 needs A280 (absorbance at 280 nm). Example: solve('cmb_protein_concentration_a280', A280=
Protein concentration from A280: A280 = 0.75, eps = 43570 L/(mol*cm), MW = 66430 g/mol, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Protein concentration from A280: A280 = 0.75, eps = 43570 L/(mol*cm), MW = 66.43 kDa	SOLVED	c = 0.0172137 mol/m^3, c_uM = 17.2137, c_mg_mL = 1.1435

Copy number and moles of a DNA or RNA molecule from its mass, `cmb_dna_copy_number`

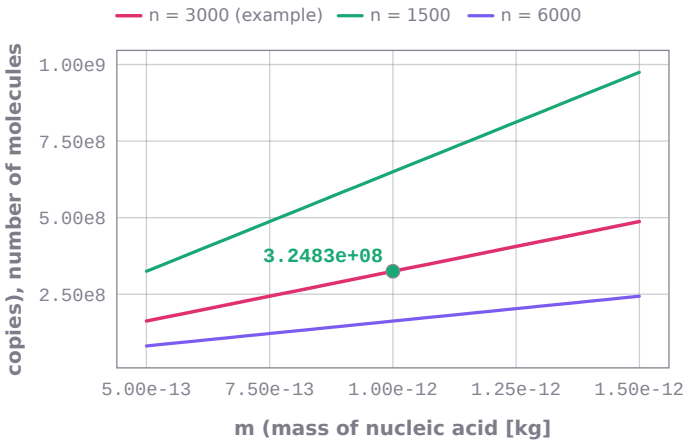
The molecular weight, picomoles and number of copies in a mass of dsDNA, ssDNA or RNA of a given length (NEB average-mass formulas).

$$N = \frac{m}{M} N_A$$

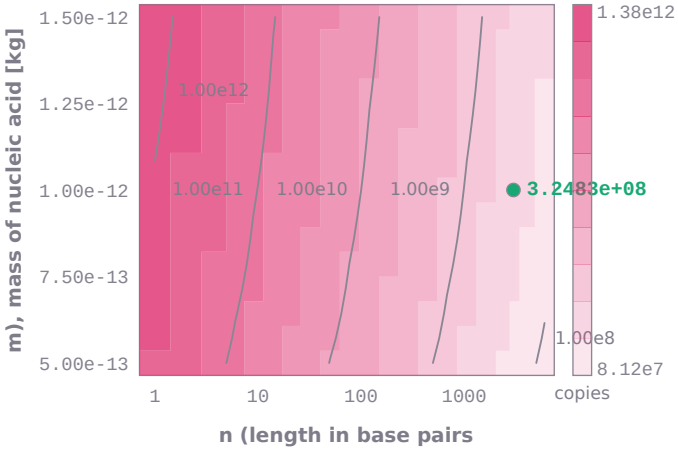
input	meaning	condition
`m`	mass of nucleic acid	
`n`	length in base pairs (dsDNA) or bases (ssDNA, RNA)	
`molecule`	dsDNA, ssDNA or RNA	

output	meaning
`MW`	molecular weight
`MW_g_mol`	molecular weight in g/mol
`pmol`	picomoles
`copies`	number of molecules

- **Method:** $MW = 617.96\,n + 36.04$ (dsDNA), 308.97 $n + 18.02$ (ssDNA), 320.5 $n + 159.0$ (RNA) g/mol; copies = m / MW x N_A
- **Assumes:** $m > 0$; n a whole number >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



copies (number of molecules) against m (mass of nucleic acid) from 5.0000e-13 kg to 1.5000e-12 kg. One curve for each value of n (length in base pairs (dsDNA) or bases (ssDNA, RNA)): 3000, 1500, 6000. Other inputs as in the worked example: molecule = dsDNA. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, m = 1.0000e-12 kg, copies = 3.2483e+08.



copies (number of molecules) over n (length in base pairs (dsDNA) or bases (ssDNA, RNA)) from 1 to 6000 and m (mass of nucleic acid) from 5.0000e-13 kg to 1.5000e-12 kg (logarithmic horizontal axis): each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 8.1209e+07 to dark 1.3812e+12, logarithmic shading); iso-lines at copies = 1.00e8, 1.00e9, 1.00e10, 1.00e11, 1.00e12. Other inputs as in the worked example: molecule = dsDNA. Marked: the worked example, n = 3000, m = 1.0000e-12 kg, copies = 3.2483e+08.

```
cell_molecular_biology_ai999x.solve('cmb_dna_copy_number', m=1e-12, n=3000, molecule='dsDNA')
# -> MW = 1853.92 kg/mol, MW_g_mol = 1.85392e+06, pmol = 0.000539399, copies = 3.24834e+08   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_dna_copy_number', m=5e-11, n=1500, molecule='RNA')
# -> MW = 480.909 kg/mol, MW_g_mol = 480909, pmol = 0.10397, copies = 6.26121e+10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
DNA copy number: m = 1 ng, n = 3000 bp, molecule = dsDNA	SOLVED	MW = 1853.92 kg/mol, MW_g_mol = 1.85392e+06, pmol = 0.000539399, copies = 3.24834e+08
DNA copy number: m = 50 ng, n = 1500 bp, molecule = RNA	SOLVED	MW = 480.909 kg/mol, MW_g_mol = 480909, pmol = 0.10397, copies = 6.26121e+10
DNA copy number: n = 3000 bp, molecule = dsDNA	NEEDS_INPUT	cmb_dna_copy_number needs m (mass of nucleic acid). Example: solve('cmb_dna_copy_number', m='1e-12 kg', n=3000, molecule
DNA copy number: m = 1 ng, n = 3000 bp, molecule = dsDNA, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
DNA copy number: m = 1 ng, n = 3000 bp, molecule = dsDNA	SOLVED	MW = 1853.92 kg/mol, MW_g_mol = 1.85392e+06, pmol = 0.000539399, copies = 3.24834e+08

Insert mass for a ligation at an insert:vector molar ratio, `cmb_ligation_insert_mass`

The mass of insert DNA needed for a ligation at a chosen insert:vector molar ratio, and the femtomoles of vector and insert (660 g/mol per bp).

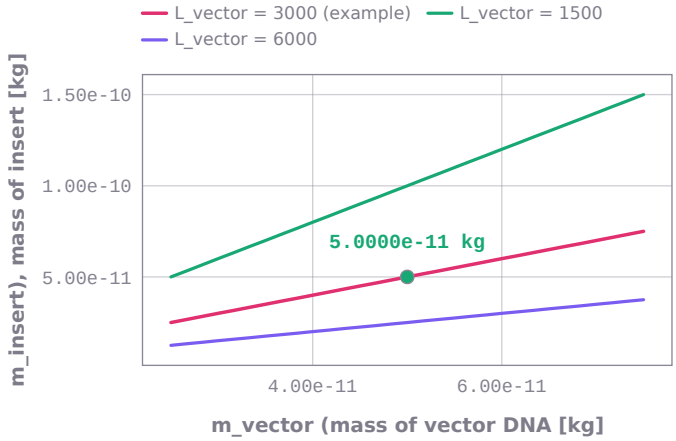
$$m_{ins} = r\, m_{vec} \frac{L_{ins}}{L_{vec}}$$

input	meaning	condition
`m_vector`	mass of vector DNA	

input	meaning	condition
`L_vector`	vector length in bp	
`L_insert`	insert length in bp	
`ratio`	insert:vector molar ratio (e.g. 3 for 3:1)	

output	meaning
`m_insert`	mass of insert
`m_insert_ng`	mass of insert in ng
`vector_fmol`	vector in fmol
`insert_fmol`	insert in fmol

- **Method:** m_insert = ratio x m_vector x L_insert / L_vector
- **Assumes:** masses, ratio > 0; lengths whole numbers >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



m_insert (mass of insert) against m_vector (mass of vector DNA) from 2.5000e-11 kg to 7.5000e-11 kg. One curve for each value of L_vector (vector length in bp): 3000, 1500, 6000. Other inputs as in the worked example: L_insert = 1000, ratio = 3.0. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, m_vector = 5.0000e-11 kg, m_insert = 5.0000e-11 kg.

```
cell_molecular_biology_ai999x.solve('cmb_ligation_insert_mass', m_vector=5e-11, L_vector=3000, L_insert=1000,
ratio=3.0)
# -> m_insert = 5e-11 kg, m_insert_ng = 50, vector_fmol = 25.2525, insert_fmol = 75.7576   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_ligation_insert_mass', m_vector=1e-10, L_vector=5400, L_insert=750, ratio=1.0)
# -> m_insert = 1.38889e-11 kg, m_insert_ng = 13.8889, vector_fmol = 28.0584, insert_fmol = 28.0584   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ligation insert mass: m_vector = 50 ng, L_vector = 3000 bp, L_insert = 1000 bp, ratio = 3	SOLVED	m_insert = 5e-11 kg, m_insert_ng = 50, vector_fmol = 25.2525, insert_fmol = 75.7576
Ligation insert mass: m_vector = 100 ng, L_vector = 5400 bp, L_insert = 750 bp, ratio = 1	SOLVED	m_insert = 1.38889e-11 kg, m_insert_ng = 13.8889, vector_fmol = 28.0584, insert_fmol = 28.0584
Ligation insert mass: L_vector = 3000 bp, L_insert = 1000 bp, ratio = 3	NEEDS_INPUT	cmb_ligation_insert_mass needs m_vector (mass of vector DNA). Example: solve('cmb_ligation_insert_mass', m_vector='5e-11
Ligation insert mass: m_vector = 50 ng, L_vector = 3000 bp, L_insert = 1000 bp, ratio = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Ligation insert mass: m_vector = 50 ng, L_vector = 3000 bp, L_insert = 1000 bp, ratio = 3	SOLVED	m_insert = 5e-11 kg, m_insert_ng = 50, vector_fmol = 25.2525, insert_fmol = 75.7576

Reverse complement of a DNA or RNA sequence, `cmb_reverse_complement`

The reverse complement (the other strand read 5'->3'), the complement and the reverse of a DNA or RNA sequence.

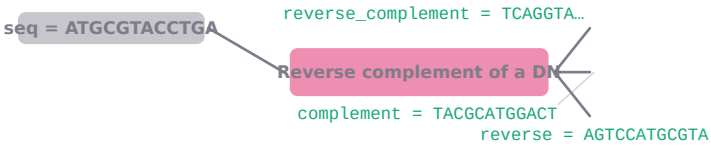
$$\mathrm{rc}(s) = \overline{s_N} \cdots \overline{s_1}$$

input	meaning	condition
`seq`	DNA or RNA sequence 5'->3'	

output	meaning
`reverse_complement`	reverse complement 5'->3'
`complement`	complement 3'->5'
`reverse`	the sequence reversed

- **Method:** complement A-T (A-U), G-C, then reverse

- **Assumes:** only A, C, G, T or U
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Reverse complement of a DNA or RNA sequence as inputs and results: stated seq = ATGCGTACCTGA; returned by cell-molecular-biology-ai999x: reverse_complement = TCAGGTACGCAT, complement = TACGCATGGACT, reverse = AGTCCATGCGTA.

```
cell_molecular_biology_ai999x.solve('cmb_reverse_complement', seq='ATGCGTACCTGA')
# -> reverse_complement = TCAGGTACGCAT, complement = TACGCATGGACT, reverse = AGTCCATGCGTA   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_reverse_complement', seq='GGAUCCUUA')
# -> reverse_complement = UAAGGAUCC, complement = CCUAGGAAU, reverse = AUUCCUAGG   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reverse complement: seq = ATGCGTACCTGA	SOLVED	reverse_complement = TCAGGTACGCAT, complement = TACGCATGGACT, reverse = AGTCCATGCGTA
Reverse complement: seq = GGAUCCUUA	SOLVED	reverse_complement = UAAGGAUCC, complement = CCUAGGAAU, reverse = AUUCCUAGG
Reverse complement:	NEEDS_INPUT	cmb_reverse_complement needs seq (DNA or RNA sequence 5'->3'). Example: solve('cmb_reverse_complement', seq='ATGCGTACCTG
Reverse complement: seq = ATGCGTACCTGA, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Transcription of a DNA strand into mRNA, `cmb_transcribe`

The mRNA (5'->3') transcribed from a DNA sequence given 5'->3' as either the coding (sense) strand or the template (antisense) strand.

$$\mathbf{mRNA} = \mathbf{coding}(T \to U) = \mathbf{rc}(\mathbf{template})(T \to U)$$

input	meaning	condition
`seq`	DNA sequence 5'->3'	
`strand`	whether the given strand is the coding or the template strand	

output	meaning
`mRNA`	mRNA sequence 5'->3'
`length`	length in nt

- **Method:** coding strand: T -> U; template strand: reverse complement, then T -> U
- **Assumes:** DNA letters only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Transcription of a DNA strand into mRNA as inputs and results: stated seq = ATGGCCTTTAAG, strand = coding; returned by cell-molecular-biology-ai999x: mRNA = AUGGCCUUUAAG, length = 12.

```
cell_molecular_biology_ai999x.solve('cmb_transcribe', seq='ATGGCCTTTAAG', strand='coding')
# -> mRNA = AUGGCCUUUAAG, length = 12   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_transcribe', seq='TACCGGAAATTC', strand='template')
# -> mRNA = GAAUUUCCGGUA, length = 12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transcribe DNA: seq = ATGGCCTTTAAG, strand = coding	SOLVED	mRNA = AUGGCCUUUAAG, length = 12
Transcribe DNA: seq = TACCGGAAATTC, strand = template	SOLVED	mRNA = GAAUUUCCGGUA, length = 12
Transcribe DNA: strand = coding	NEEDS_INPUT	cmb_transcribe needs seq (DNA sequence 5'->3'). Example: solve('cmb_transcribe', seq='ATGGCCTTTAAG', strand='coding').
Transcribe DNA: seq = ATGGCCTTTAAG, strand = coding, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Translation of an mRNA or DNA sequence into protein, `cmb_translate`

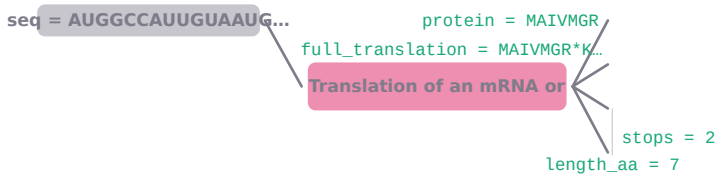
The protein encoded from the first base (frame 1) by the standard genetic code: up to the first stop codon, and the full translation with stops ***as***. A trailing incomplete codon is ignored.

$$\text{codon} \mapsto \text{amino acid (table 1)}$$

input	meaning	condition
`seq`	mRNA or coding-strand DNA 5'->3'	

output	meaning
`protein`	amino acids up to the first stop (one-letter code)
`full_translation`	full translation with * for stops
`length_aa`	length of the protein up to the first stop
`stops`	number of stop codons

- **Method:** read codons from base 1 with the standard code (NCBI table 1)
- **Assumes:** only A, C, G, T or U; at least one codon
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Translation of an mRNA or DNA sequence into protein as inputs and results: stated seq = AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG; returned by cell-molecular-biology-ai999x: protein = MAIVMGR, full_translation = MAIVMGR*KGAR*, length_aa = 7, stops = 2.

```
cell_molecular_biology_ai999x.solve('cmb_translate', seq='AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG')
# -> protein = MAIVMGR, full_translation = MAIVMGR*KGAR*, length_aa = 7, stops = 2   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_translate',
seq='ATGAAACGCATTAGCACCACCATTACCACCACCATCACCATTACCACAGGTAACGGTGCGGGCTGA')
# -> protein = MKRISTTITTTITITTGNGAG, full_translation = MKRISTTITTTITITTGNGAG*, length_aa = 21, stops = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Translate: seq = AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG	SOLVED	protein = MAIVMGR, full_translation = MAIVMGR*KGAR*, length_aa = 7, stops = 2
Translate: seq = ATGAAACGCATTAGCACCACCATTACCACCACCATCACCATTACCACAGGTAACGGTGCGGGCTGA	SOLVED	protein = MKRISTTITTTITITTGNGAG, full_translation = MKRISTTITTTITITTGNGAG*, length_aa = 21, stops = 1
Translate:	NEEDS_INPUT	cmb_translate needs seq (mRNA or coding-strand DNA 5'->3'). Example: solve('cmb_translate', seq='AUGGCCAUUGUAAUGGGCCGCU
Translate: seq = AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Longest open reading frame on both strands, `cmb_orf_finder`

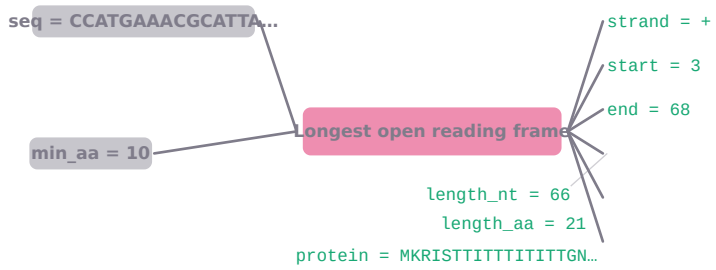
The longest open reading frame (ATG to the first in-frame stop) in all six frames of a DNA sequence with at least min_aa codons, its strand, coordinates on that strand, protein and the number of such ORFs.

$$\mathrm{ORF} = \mathrm{ATG}\,(\mathrm{NNN})^{n}\,\{\mathrm{TAA}, \mathrm{TAG}, \mathrm{TGA}\}$$

input	meaning	condition
`seq`	DNA sequence 5'->3'	
`min_aa`	minimum ORF length in codons (amino acids, stop excluded)	

output	meaning
`strand`	+ (given strand) or - (reverse complement)
`start`	1-based start of the ATG on that strand (read 5'->3')
`end`	1-based end of the stop codon on that strand
`length_nt`	ORF length in nt including the stop
`length_aa`	protein length in amino acids
`protein`	encoded protein (one-letter code)
`n_orfs`	number of ORFs of at least min_aa codons

- **Method:** scan the three frames of each strand; an ORF starts at the first ATG after the previous stop and ends at the next stop; keep the longest (ties: + strand, then the lowest start)
- **Assumes:** DNA letters only; min_aa >= 1; at least one ORF
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Longest open reading frame on both strands as inputs and results: stated seq = CCATGAAACGCATTAGCACCACCATTACCACCACCATCACCATTACCACAGGTAACGGTGCGGGCTGAGG, min_aa = 10; returned by cell-molecular-biology-ai999x: strand = +, start = 3, end = 68, length_nt = 66, length_aa = 21, protein = MKRISTTITTTTITITTGNGAG.

```
cell_molecular_biology_ai999x.solve('cmb_orf_finder',
seq='CCATGAAACGCATTAGCACCACCATTACCACCACCATCACCATTACCACAGGTAACGGTGCGGGCTGAGG', min_aa=10)
# -> strand = +, start = 3, end = 68, length_nt = 66, length_aa = 21, protein = MKRISTTITTTTITITTGNGAG, n_orfs = 1
(SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_orf_finder', seq='TTATTACTTCCAGCCAGCCATTTGAAGCTAACGG', min_aa=5)
# -> strand = -, start = 14, end = 31, length_nt = 18, length_aa = 5, protein = MAGWK, n_orfs = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the longest ORF: seq = CCATGAAACGCATTAGCACCACCATTACCACCACCATCACCATTACCACAGGTAACGGTGCGGGCTGAGG, min_aa = 10	SOLVED	strand = +, start = 3, end = 68, len = 21, protein = MKRISTTITTTTITIT
Find the longest ORF: seq = TTATTACTTCCAGCCAGCCATTTGAAGCTAACGG, min_aa = 5	SOLVED	strand = -, start = 14, end = 31, ler = 5, protein = MAGWK, n_orfs = 1
Find the longest ORF: min_aa = 10	NEEDS_INPUT	cmb_orf_finder needs seq (DNA s Example: solve('cmb_orf_finder', seq='CCATGAAACGCATTAGCAC
Find the longest ORF: seq = CCATGAAACGCATTAGCACCACCATTACCACCACCATCACCATTACCACAGGTAACGGTGCGGGCTGAGG, min_aa = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a I will not guess. Every form: cell_molecular_biology_ai999x.ten

Sequencing coverage and uncovered fraction (Lander-Waterman), `cmb_genome_coverage`

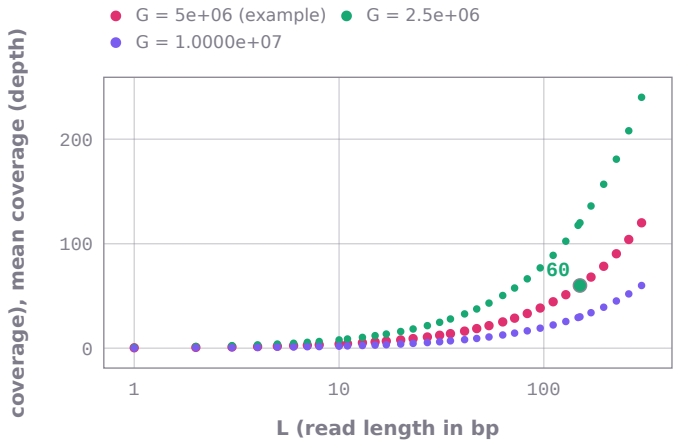
The mean sequencing depth $C = L$ N / G, the expected fraction and number of bases not covered $\left(e^{-C}\right)$ and the expected number of contigs (N e^{-C}, at least one).

$$C = \frac{LN}{G}, \quad P_0 = e^{-C}$$

input	meaning	condition
`L`	read length in bp	
`N`	number of reads	
`G`	genome size in bp	

output	meaning
`coverage`	mean coverage (depth)
`fraction_uncovered`	fraction of bases not covered e^{-C}
`bases_uncovered`	expected bases not covered
`contigs`	expected number of contigs max(1, N e^-C)

- **Method:** $C = L$ N / G; P(0 reads) = e^-C (Poisson)
- **Assumes:** L, N whole numbers >= 1; $G > 0$; reads placed uniformly at random
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



coverage (mean coverage (depth)) against L (read length in bp) from 1 to 300 on a logarithmic axis. One curve for each value of G (genome size in bp): 5e+06, 2.5e+06, 1.0000e+07. Other inputs as in the worked example: N = 2000000. 34 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, L = 150, coverage = 60.

```
cell_molecular_biology_ai999x.solve('cmb_genome_coverage', L=150, N=2000000, G=5000000.0)
# -> coverage = 60, fraction_uncovered = 8.75651e-27, bases_uncovered = 4.37826e-20, contigs = 1   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_genome_coverage', L=100, N=30000, G=1000000.0)
# -> coverage = 3, fraction_uncovered = 0.0497871, bases_uncovered = 49787.1, contigs = 1493.61   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Genome coverage: L = 150 bp, N = 2000000, G = 5000000 bp	SOLVED	coverage = 60, fraction_uncovered = 8.75651e-27, bases_uncovered = 4.37826e-20, contigs = 1
Genome coverage: L = 100 bp, N = 30000, G = 1000000 bp	SOLVED	coverage = 3, fraction_uncovered = 0.0497871, bases_uncovered = 49787.1, contigs = 1493.61
Genome coverage: N = 2000000, G = 5000000 bp	NEEDS_INPUT	cmb_genome_coverage needs L (read length in bp). Example: solve('cmb_genome_coverage', L=150, N=2e+06, G=5e+06).
Genome coverage: L = 150 bp, N = 2000000, G = 5000000 bp, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Number of clones for a genomic library (Clarke-Carbon), `cmb_library_size`

The number of random clones needed so that any given sequence is present with probability P, for a genome of size G and inserts of size I.

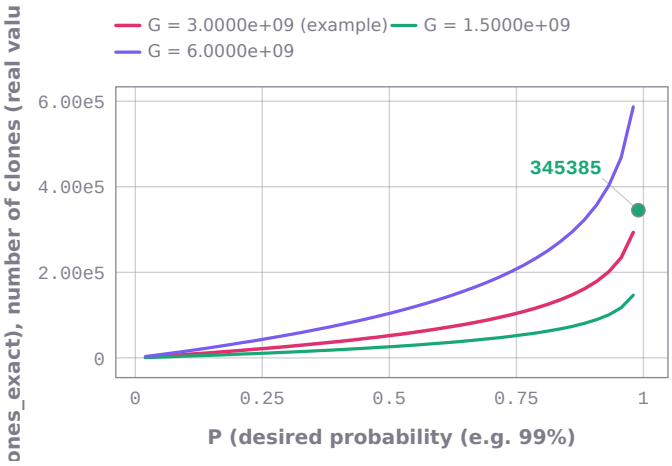
$$N = \frac{\ln(1-P)}{\ln(1-I/G)}$$

input	meaning	condition
`G`	genome size in bp	
`I`	insert size in bp	
`P`	desired probability (e.g. 99%)	

output	meaning
`clones_exact`	number of clones (real value)

output	meaning
`clones`	number of clones rounded up

- **Method:** $N = ln$(1 - P) / ln(1 - I/G)
- **Assumes:** $0 < I < G$; $0 < P < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



clones_exact (number of clones (real value)) against P (desired probability (e.g. 99%)) from 0.02 to 0.98. One curve for each value of G (genome size in bp): 3.0000e+09, 1.5000e+09, 6.0000e+09. Other inputs as in the worked example: I = 40000.0. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, P = 0.99, clones_exact = 345385.

```
cell_molecular_biology_ai999x.solve('cmb_library_size', G=3000000000.0, I=40000.0, P=0.99)
# -> clones_exact = 345385, clones = 345386   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_library_size', G=4600000.0, I=5000.0, P=0.95)
# -> clones_exact = 2754.58, clones = 2755   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Genomic library size: G = 3000000000 bp, I = 40000 bp, P = 99%	SOLVED	clones_exact = 345385, clones = 345386
Genomic library size: G = 4600000 bp, I = 5000 bp, P = 95%	SOLVED	clones_exact = 2754.58, clones = 2755
Genomic library size: I = 40000 bp, P = 99%	NEEDS_INPUT	cmb_library_size needs G (genome size in bp). Example: solve('cmb_library_size', G=3e+09, I=40000, P=0.99).
Genomic library size: G = 3000000000 bp, I = 40000 bp, P = 99%, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Genomic library size: G = 3e9 bp, I = 40000 bp, P = 99%	SOLVED	clones_exact = 345385, clones = 345386

Mutation rate by the fluctuation-test p0 method, `cmb_mutation_rate_p0`

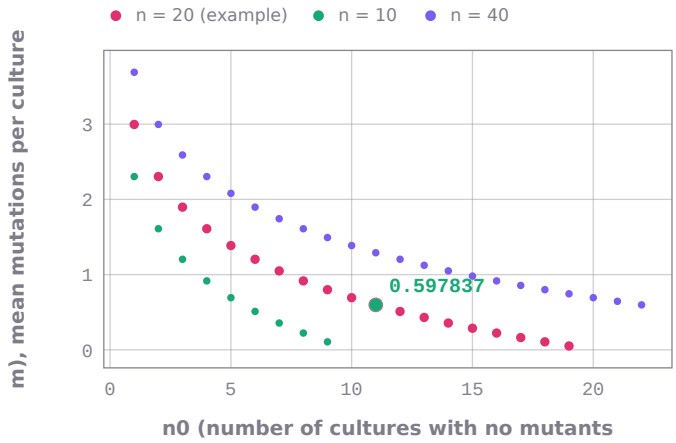
The expected number of mutations per culture $m = -ln$ p0 from the fraction of parallel cultures without mutants (Luria-Delbruck p0 method) and the mutation rate m / Nt.

$$p_0 = e^{-m}, \quad \mu = \frac{-\ln p_0}{N_t}$$

input	meaning	condition
`n0`	number of cultures with no mutants	
`n`	total number of cultures	
`Nt`	final number of cells per culture	

output	meaning
`p0`	fraction of cultures without mutants
`m`	mean mutations per culture
`mutation_rate`	mutations per cell per generation (m / Nt)

- **Method:** $p_0 = e^{-m}$, so $m = -ln$(n0/n); $mu = m/Nt$
- **Assumes:** $1 \le n_0 < n$; $Nt > 0$; Poisson number of mutational events per culture
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



m (mean mutations per culture) against n0 (number of cultures with no mutants) from 1 to 22. One curve for each value of n (total number of cultures): 20, 10, 40. Other inputs as in the worked example: Nt = 200000000.0. 19 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, n0 = 11, m = 0.597837.

```
cell_molecular_biology_ai999x.solve('cmb_mutation_rate_p0', n0=11, n=20, Nt=200000000.0)
# -> p0 = 0.55, m = 0.597837, mutation_rate = 2.98919e-09   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_mutation_rate_p0', n0=3, n=48, Nt=560000000.0)
# -> p0 = 0.0625, m = 2.77259, mutation_rate = 4.95105e-09   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mutation rate by the p0 method: n0 = 11, n = 20, Nt = 200000000	SOLVED	p0 = 0.55, m = 0.597837, mutation_rate = 2.98919e-09
Mutation rate by the p0 method: n0 = 3, n = 48, Nt = 560000000	SOLVED	p0 = 0.0625, m = 2.77259, mutation_rate = 4.95105e-09
Mutation rate by the p0 method: n = 20, Nt = 200000000	NEEDS_INPUT	cmb_mutation_rate_p0 needs n0 (number of cultures with no mutants). Example: solve('cmb_mutation_rate_p0', n0=11, n=20,
Mutation rate by the p0 method: n0 = 11, n = 20, Nt = 200000000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

PCR amplification after n cycles, `cmb_pcr_amplification`

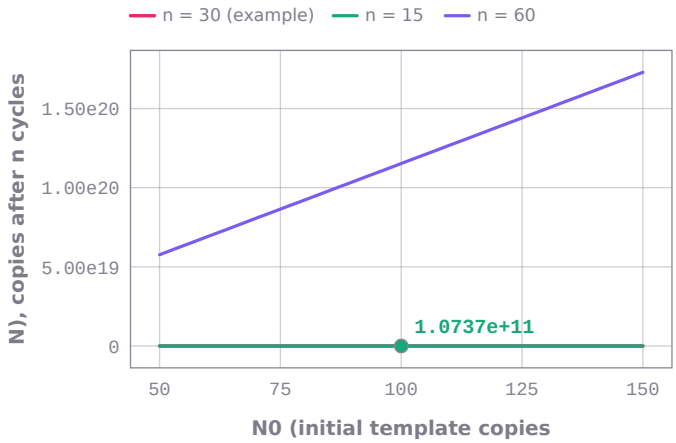
The number of copies after n PCR cycles from N0 template copies at an amplification efficiency E (1 = perfect doubling).

$$N = N_0(1+E)^n$$

input	meaning	condition
`N0`	initial template copies	
`E`	amplification efficiency per cycle (0 to 1, e.g. 95%)	
`n`	number of cycles	

output	meaning
`N`	copies after n cycles
`fold`	fold amplification $(1+E)^n$
`log10_fold`	log10 of the fold amplification

- **Method:** $N = N_0\,(1+E)^n$
- **Assumes:** $N_0 > 0$; $0 < E \le 1$; $0 \le n \le 60$; exponential phase (no plateau)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



N (copies after n cycles) against N0 (initial template copies) from 50 to 150. One curve for each value of n (number of cycles): 30, 15, 60. Other inputs as in the worked example: E = 1.0. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, N0 = 100, N = 1.0737e+11.

```
cell_molecular_biology_ai999x.solve('cmb_pcr_amplification', N0=100.0, E=1.0, n=30)
# -> N = 1.07374e+11, fold = 1.07374e+09, log10_fold = 9.0309   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_pcr_amplification', N0=1000.0, E=0.9, n=25)
# -> N = 9.30765e+09, fold = 9.30765e+06, log10_fold = 6.96884   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PCR amplification: N0 = 100, E = 100%, n = 30 cycles	SOLVED	N = 1.07374e+11, fold = 1.07374e+09, log10_fold = 9.0309
PCR amplification: N0 = 1000, E = 90%, n = 25 cycles	SOLVED	N = 9.30765e+09, fold = 9.30765e+06, log10_fold = 6.96884
PCR amplification: E = 100%, n = 30 cycles	NEEDS_INPUT	cmb_pcr_amplification needs N0 (initial template copies). Example: solve('cmb_pcr_amplification', N0=100, E=1, n=30).
PCR amplification: N0 = 100, E = 100%, n = 30 cycles, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
PCR amplification: N0 = 1000, E = 90%, n = 25 cycles	SOLVED	N = 9.30765e+09, fold = 9.30765e+06, log10_fold = 6.96884

PCR cycles needed to reach a number of copies, `cmb_pcr_cycles_needed`

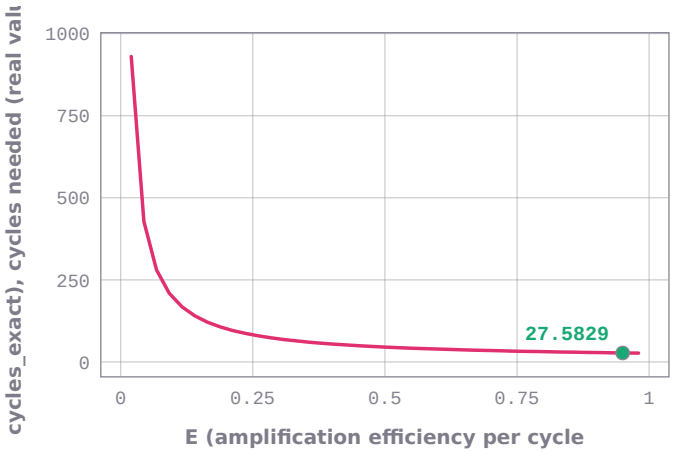
The number of PCR cycles needed to amplify N0 copies to N at efficiency E (exact value and whole cycles, rounded up).

$$n = \frac{\ln(N/N_0)}{\ln(1+E)}$$

input	meaning	condition
`N0`	initial template copies	
`N`	target number of copies	
`E`	amplification efficiency per cycle (0 to 1)	

output	meaning
`cycles_exact`	cycles needed (real value)
`cycles`	whole cycles needed

- **Method:** $n = ln$(N/N0) / ln(1 + E)
- **Assumes:** $N > N_0 > 0$; $0 < E \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cycles_exact (cycles needed (real value)) against E (amplification efficiency per cycle (0 to 1)) from 0.02 to 0.98. Other inputs as in the worked example: N0 = 100.0, N = 10000000000.0. 41 points computed by cell-molecular-biology-ai999x; marked: the worked example, E = 0.95, cycles_exact = 27.5829.

```
cell_molecular_biology_ai999x.solve('cmb_pcr_cycles_needed', N0=100.0, N=10000000000.0, E=0.95)
# -> cycles_exact = 27.5829, cycles = 28   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_pcr_cycles_needed', N0=10.0, N=1000000000.0, E=0.85)
# -> cycles_exact = 29.9433, cycles = 30   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PCR cycles needed: N0 = 100, N = 1e+10, E = 95%	SOLVED	cycles_exact = 27.5829, cycles = 28
PCR cycles needed: N0 = 10, N = 1000000000, E = 85%	SOLVED	cycles_exact = 29.9433, cycles = 30
PCR cycles needed: N = 1e+10, E = 95%	NEEDS_INPUT	cmb_pcr_cycles_needed needs N0 (initial template copies). Example: solve('cmb_pcr_cycles_needed', N0=100, N=1e+10, E=0.9
PCR cycles needed: N0 = 100, N = 1e+10, E = 95%, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

qPCR relative expression by the delta-delta Ct method, `cmb_qpcr_delta_delta_ct`

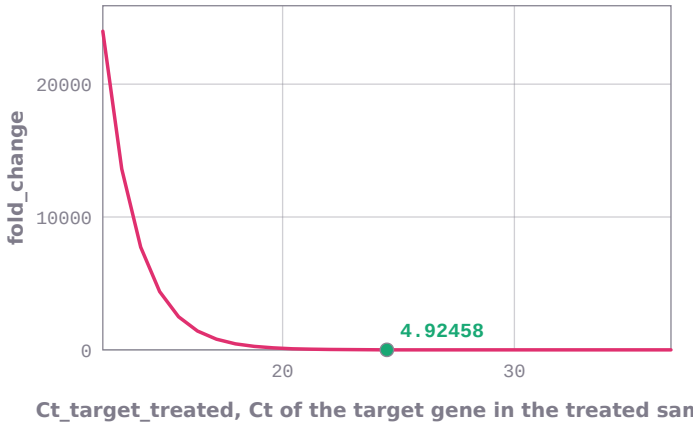
The fold change in a target gene's expression between treated and control samples, normalized to a reference gene, by 2^-ddCt (100 % efficiency assumed).

$$\Delta\Delta C_t = \Delta C_{t,treated} - \Delta C_{t,control}, \quad \text{fold} = 2^{-\Delta\Delta C_t}$$

input	meaning	condition
`Ct_target_treated`	Ct of the target gene in the treated sample	
`Ct_ref_treated`	Ct of the reference gene in the treated sample	
`Ct_target_control`	Ct of the target gene in the control sample	
`Ct_ref_control`	Ct of the reference gene in the control sample	

output	meaning
`dCt_treated`	Ct target - Ct reference, treated
`dCt_control`	Ct target - Ct reference, control
`ddCt`	dCt treated - dCt control
`fold_change`	relative expression 2^-ddCt
`log2_fold_change`	log2 fold change (-ddCt)

- **Method:** ddCt = (Ct_t - Ct_r)_treated - (Ct_t - Ct_r)_control; fold = 2^-ddCt
- **Assumes:** $0 < Ct \leq 60$; efficiencies of target and reference near 100 %
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



fold_change (relative expression 2^-ddCt) against Ct_target_treated (Ct of the target gene in the treated sample), with Ct_ref_control = 18.2, Ct_ref_treated = 18, Ct_target_control = 27. 31 points computed by cell-molecular-biology-ai999x; marked: Ct_target_treated = 24.5, fold_change = 4.92458.



fold_change (relative expression 2^-ddCt) against Ct_target_treated (Ct of the target gene in the treated sample) from 12.25 to 36.75, logarithmic output axis. One curve for each value of Ct_ref_treated (Ct of the reference gene in the treated sample): 18, 9, 36. Other inputs as in the worked example: Ct_target_control = 27.0, Ct_ref_control = 18.2. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, Ct_target_treated = 24.5, fold_change = 4.92458.

```
cell_molecular_biology_ai999x.solve('cmb_qpcr_delta_delta_ct', Ct_target_treated=24.5, Ct_ref_treated=18.0,
Ct_target_control=27.0, Ct_ref_control=18.2)
# -> dCt_treated = 6.5, dCt_control = 8.8, ddCt = -2.3, fold_change = 4.92458, log2_fold_change = 2.3   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_qpcr_delta_delta_ct', Ct_target_treated=30.1, Ct_ref_treated=20.0,
Ct_target_control=28.0, Ct_ref_control=20.3)
# -> dCt_treated = 10.1, dCt_control = 7.7, ddCt = 2.4, fold_change = 0.189465, log2_fold_change = -2.4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
qPCR delta-delta Ct: Ct_target_treated = 24.5, Ct_ref_treated = 18, Ct_target_control = 27, Ct_ref_control = 18.2	SOLVED	dCt_treated = 6.5, dCt_control = 8.8, ddCt = -2.3, fold_change = 4.92458, log2_fold_change = 2.3
qPCR delta-delta Ct: Ct_target_treated = 30.1, Ct_ref_treated = 20, Ct_target_control = 28, Ct_ref_control = 20.3	SOLVED	dCt_treated = 10.1, dCt_control = 7.7, ddCt = 2.4, fold_change = 0.189465, log2_fold_change = -2.4
qPCR delta-delta Ct: Ct_ref_treated = 18, Ct_target_control = 27, Ct_ref_control = 18.2	NEEDS_INPUT	cmb_qpcr_delta_delta_ct needs Ct_target_treated (Ct of the target gene in the treated sample). Example: solve('cmb_qpcr_
qPCR delta-delta Ct: Ct_target_treated = 24.5, Ct_ref_treated = 18, Ct_target_control = 27, Ct_ref_control = 18.2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

qPCR relative expression with efficiency correction (Pfaffl), `cmb_qpcr_pfaffl_ratio`

The efficiency-corrected expression ratio of a target gene in a sample versus a control, normalized to a reference gene (Pfaffl 2001).

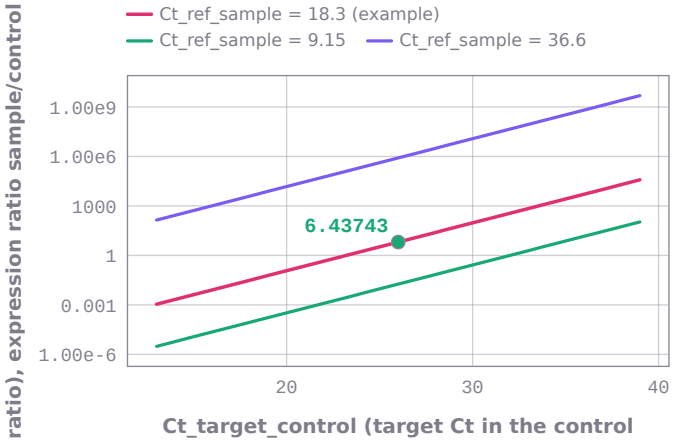
$$\text{ratio} = \frac{(1+E_t)^{\Delta C_{t,target}}}{(1+E_r)^{\Delta C_{t,ref}}}$$

input	meaning	condition
`E_target`	amplification efficiency of the target assay (0 to 1)	

input	meaning	condition
`E_ref`	amplification efficiency of the reference assay (0 to 1)	
`Ct_target_control`	target Ct in the control	
`Ct_target_sample`	target Ct in the sample	
`Ct_ref_control`	reference Ct in the control	
`Ct_ref_sample`	reference Ct in the sample	

output	meaning
`ratio`	expression ratio sample/control
`log2_ratio`	log2 of the ratio
`dCt_target`	Ct control - Ct sample for the target
`dCt_ref`	Ct control - Ct sample for the reference

- **Method:** ratio = (1 + E_t)^(Ct_t,control - Ct_t,sample) / (1 + E_r)^(Ct_r,control - Ct_r,sample)
- **Assumes:** $0 < E \leq 1; 0 < Ct \leq 60$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ratio (expression ratio sample/control) against Ct_target_control (target Ct in the control) from 13 to 39, logarithmic output axis. One curve for each value of Ct_ref_sample (reference Ct in the sample): 18.3, 9.15, 36.6. Other inputs as in the worked example: E_target = 0.9500000000000001, E_ref = 0.9, Ct_target_sample = 23.5, Ct_ref_control = 18.0. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, Ct_target_control = 26, ratio = 6.43743.

```
cell_molecular_biology_ai999x.solve('cmb_qpcr_pfaffl_ratio', E_target=0.95, E_ref=0.9, Ct_target_control=26.0,
Ct_target_sample=23.5, Ct_ref_control=18.0, Ct_ref_sample=18.3)
# -> ratio = 6.43743, log2_ratio = 2.68649, dCt_target = 2.5, dCt_ref = -0.3   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_qpcr_pfaffl_ratio', E_target=1.0, E_ref=1.0, Ct_target_control=25.0,
Ct_target_sample=27.0, Ct_ref_control=17.0, Ct_ref_sample=17.0)
# -> ratio = 0.25, log2_ratio = -2, dCt_target = -2, dCt_ref = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pfaffl expression ratio: E_target = 95%, E_ref = 90%, Ct_target_control = 26, Ct_target_sample = 23.5, Ct_ref_control = 18, Ct_ref_sample = 18.3	SOLVED	ratio = 6.43743, log2_ratio = 2.68649, dCt_target = 2.5, dCt_ref = -0.3
Pfaffl expression ratio: E_target = 100%, E_ref = 100%, Ct_target_control = 25, Ct_target_sample = 27, Ct_ref_control = 17, Ct_ref_sample = 17	SOLVED	ratio = 0.25, log2_ratio = -2, dCt_target = -2, dCt_ref = 0
Pfaffl expression ratio: E_ref = 90%, Ct_target_control = 26, Ct_target_sample = 23.5, Ct_ref_control = 18, Ct_ref_sample = 18.3	NEEDS_INPUT	cmb_qpcr_pfaffl_ratio needs E_target (amplification efficiency of the target assay (0 to 1)). Example: solve('cmb_qpcr_p
Pfaffl expression ratio: E_target = 95%, E_ref = 90%, Ct_target_control = 26, Ct_target_sample = 23.5, Ct_ref_control = 18, Ct_ref_sample = 18.3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Pfaffl expression ratio: E_target = 95%, E_ref = 90%, Ct_target_control = 26, Ct_target_sample = 23.5, Ct_ref_control = 18, Ct_ref_sample = 18.3	SOLVED	ratio = 6.43743, log2_ratio = 2.68649, dCt_target = 2.5, dCt_ref = -0.3

qPCR standard curve: slope, intercept and efficiency, `cmb_qpcr_standard_curve`

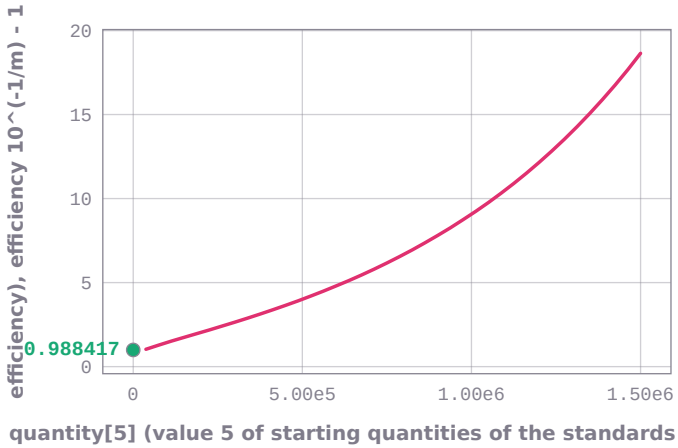
The least-squares standard curve $Ct = m$ log10(quantity) + b from a dilution series, its r^2 and the amplification efficiency $10^{-1/m} - 1$.

$$C_t = m \log_{10} q + b, \quad E = 10^{-1/m} - 1$$

input	meaning	condition
`quantity`	starting quantities of the standards (copies or ng)	
`Ct`	measured Ct of each standard	

output	meaning
`slope`	slope m (about -3.32 at 100 %)
`intercept`	intercept b (Ct of one unit)
`r2`	coefficient of determination
`efficiency`	efficiency $10^{-1/m} - 1$
`amplification_factor`	amplification per cycle $10^{-1/m}$

- **Method:** least squares $Ct = m$ log10 q + b; $E = 10^{-1/m} - 1$
- **Assumes:** at least 3 standards; quantities > 0; negative slope
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



efficiency (efficiency 10^(-1/m) - 1) as value 5 of quantity (starting quantities of the standards (copies or ng)) moves from 0 to 1.49995e+06, the other values as given ([1000000.0, 100000.0, 10000.0, 1000.0, 100.0]); 40 runs of cell-molecular-biology-ai999x. Other inputs: Ct = [15.1, 18.5, 21.8, 25.2, 28.5]. Marked: the worked example, quantity[5] = 100, efficiency = 0.988417.

```
cell_molecular_biology_ai999x.solve('cmb_qpcr_standard_curve', quantity=[1000000.0, 100000.0, 10000.0, 1000.0, 100.0],
Ct=[15.1, 18.5, 21.8, 25.2, 28.5])
# -> slope = -3.35, intercept = 35.22, r2 = 0.999973, efficiency = 0.988417, amplification_factor = 1.98842   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_qpcr_standard_curve', quantity=[100.0, 20.0, 4.0, 0.8], Ct=[20.0, 22.4, 24.7,
27.1])
# -> slope = -3.3764, intercept = 26.7628, r2 = 0.999928, efficiency = 0.977761, amplification_factor = 1.97776
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
qPCR standard curve: quantity = [1000000, 100000, 10000, 1000, 100], Ct = [15.1, 18.5, 21.8, 25.2, 28.5]	SOLVED	slope = -3.35, intercept = 35.22, r2 = 0.999973, efficiency = 0.988417, amplification_factor = 1.98842
qPCR standard curve: quantity = [100, 20, 4, 0.8], Ct = [20, 22.4, 24.7, 27.1]	SOLVED	slope = -3.3764, intercept = 26.7628, r2 = 0.999928, efficiency = 0.977761, amplification_factor = 1.97776
qPCR standard curve: Ct = [15.1, 18.5, 21.8, 25.2, 28.5]	NEEDS_INPUT	cmb_qpcr_standard_curve needs quantity (starting quantities of the standards (copies or ng)). Example: solve('cmb_qpcr_s
qPCR standard curve: quantity = [1000000, 100000, 10000, 1000, 100], Ct = [15.1, 18.5, 21.8, 25.2, 28.5], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Quantity from a Ct and a standard curve, `cmb_qpcr_quantify`

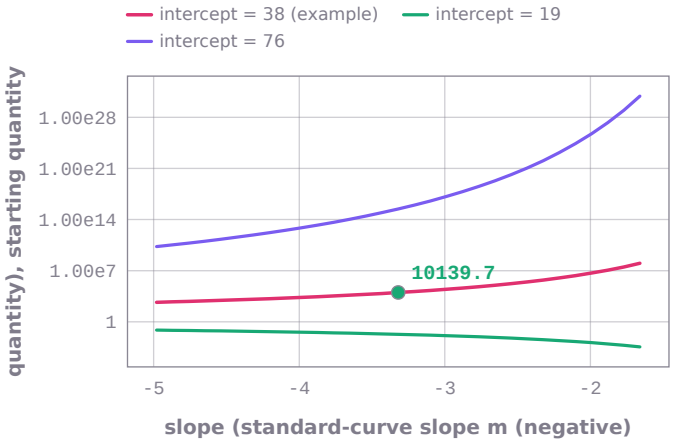
The starting quantity of an unknown from its Ct and the standard curve $Ct = m$ log10 q + b.

$$q = 10^{(C_t - b)/m}$$

input	meaning	condition
`slope`	standard-curve slope m (negative)	
`intercept`	standard-curve intercept b	
`Ct`	Ct of the unknown	

output	meaning
`quantity`	starting quantity (unit of the standards)
`log10_quantity`	log10 of the quantity

- **Method:** $q = 10$^((Ct - b)/m)
- **Assumes:** slope < 0; Ct inside the range of the standards
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



quantity (starting quantity (unit of the standards)) against slope (standard-curve slope m (negative)) from -4.98 to -1.66, logarithmic output axis. One curve for each value of intercept (standard-curve intercept b): 38, 19, 76. Other inputs as in the worked example: Ct = 24.7. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, slope = -3.32, quantity = 10139.7.

```
cell_molecular_biology_ai999x.solve('cmb_qpcr_quantify', slope=-3.32, intercept=38.0, Ct=24.7)
# -> quantity = 10139.7, log10_quantity = 4.00602   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_qpcr_quantify', slope=-3.45, intercept=36.2, Ct=30.0)
# -> quantity = 62.676, log10_quantity = 1.7971   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Quantity from a standard curve: slope = -3.32, intercept = 38, Ct = 24.7	SOLVED	quantity = 10139.7, log10_quantity = 4.00602
Quantity from a standard curve: slope = -3.45, intercept = 36.2, Ct = 30	SOLVED	quantity = 62.676, log10_quantity = 1.7971
Quantity from a standard curve: intercept = 38, Ct = 24.7	NEEDS_INPUT	cmb_qpcr_quantify needs slope (standard-curve slope m (negative)). Example: solve('cmb_qpcr_quantify', slope=-3.32, inte
Quantity from a standard curve: slope = -3.32, intercept = 38, Ct = 24.7, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Time to transcribe a gene, `cmb_transcription_time`

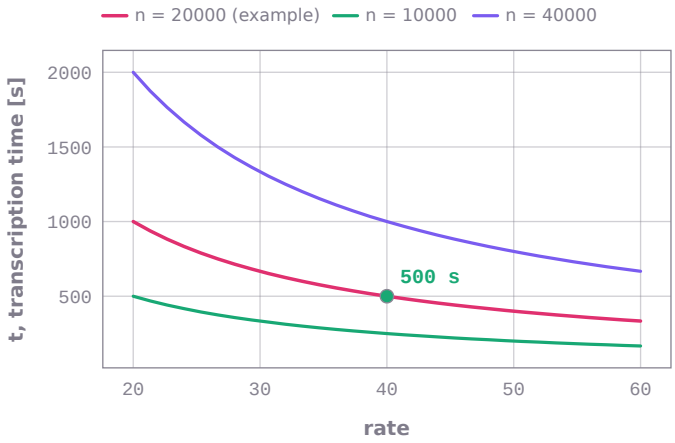
The time for RNA polymerase to transcribe a gene of n nucleotides at an elongation rate, and, with an initiation rate, the number of polymerases on the gene and their spacing.

$$t = \frac{n}{v}, \quad N_{pol} = k_{init}\, t$$

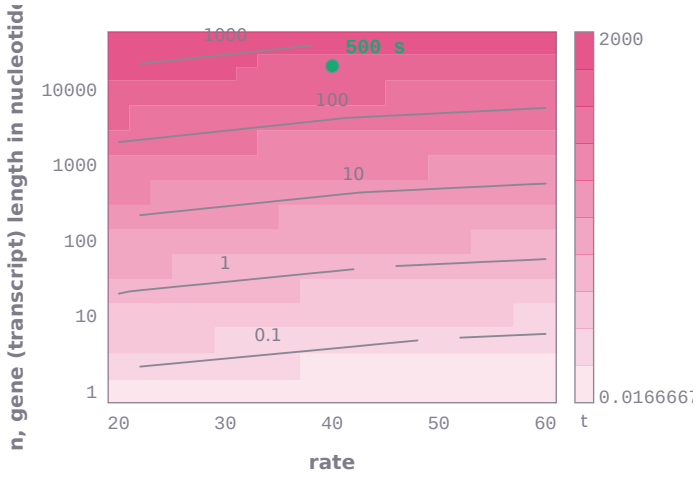
input	meaning	condition
`n`	gene (transcript) length in nucleotides	
`rate`	elongation rate in nucleotides per second	
`k_init`	initiation rate (optional)	

output	meaning
`t`	transcription time
`t_min`	time in minutes
`polymerases`	polymerases on the gene at steady state (when k_init is given)
`spacing_nt`	mean spacing between polymerases in nt (when k_init is given)

- **Method:** t = n / rate; polymerases = k_init t; spacing = rate / k_init
- **Assumes:** $n \geq 1$; rate > 0; constant elongation rate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (transcription time) against rate (elongation rate in nucleotides per second) from 20 to 60. One curve for each value of n (gene (transcript) length in nucleotides): 20000, 10000, 40000. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, rate = 40, t = 500 s.



t (transcription time) over rate (elongation rate in nucleotides per second) from 20 to 60 and n (gene (transcript) length in nucleotides) from 1 to 40000 (logarithmic vertical axis): each of the 21 × 16 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 0.0166667 s to dark 2000 s, logarithmic shading); isolines at t = 0.1, 1, 10, 100, 1000 s. Marked: the worked example, rate = 40, n = 20000, t = 500 s.

```
cell_molecular_biology_ai999x.solve('cmb_transcription_time', n=20000, rate=40.0)
# -> t = 500 s, t_min = 8.33333   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_transcription_time', n=6000, rate=50.0, k_init=0.1)
# -> t = 120 s, t_min = 2, polymerases = 12, spacing_nt = 500   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transcription time: n = 20000 nt, rate = 40 nt/s	SOLVED	t = 500 s, t_min = 8.33333
Transcription time: n = 6000 nt, rate = 50 nt/s, k_init = 6 1/min	SOLVED	t = 120 s, t_min = 2, polymerases = 12, spacing_nt = 500
Transcription time: rate = 40 nt/s	NEEDS_INPUT	cmb_transcription_time needs n (gene (transcript) length in nucleotides). Example: solve('cmb_transcription_time', n=200
Transcription time: n = 20000 nt, rate = 40 nt/s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Transcription time: n = 20000 nt, rate = 40 nt/s	SOLVED	t = 500 s, t_min = 8.33333

Time to translate a protein, `cmb_translation_time`

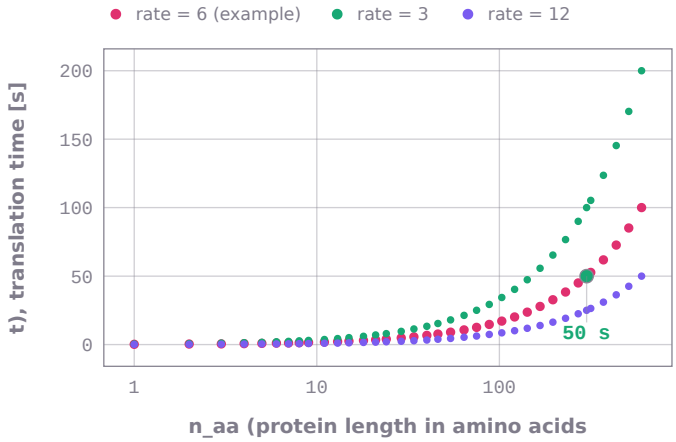
The time for a ribosome to synthesise a protein of n_aa amino acids at an elongation rate, the coding length with the stop codon, and, with an initiation rate, the ribosomes per mRNA and their spacing.

$$t = \frac{n_{aa}}{v}, \quad N_{rib} = k_{init}\, t$$

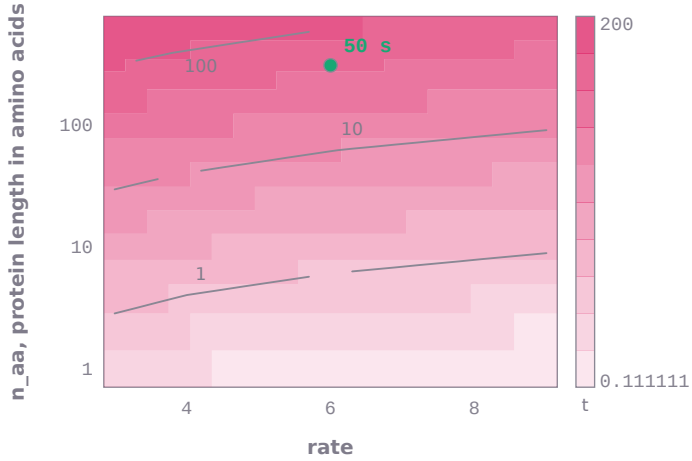
input	meaning	condition
`n_aa`	protein length in amino acids	
`rate`	elongation rate in amino acids per second	
`k_init`	initiation rate per mRNA (optional)	

output	meaning
`t`	translation time
`t_min`	time in minutes
`coding_nt`	coding sequence length in nt including the stop codon
`ribosomes`	ribosomes per mRNA at steady state (when k_init is given)
`spacing_codons`	mean spacing between ribosomes in codons (when k_init is given)

- **Method:** t = n_aa / rate; coding length = 3 $(n_a a + 1)$; ribosomes = k_init t
- **Assumes:** $n_a a \geq 1$; rate > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (translation time) against n_aa (protein length in amino acids) from 1 to 600 on a logarithmic axis. One curve for each value of rate (elongation rate in amino acids per second): 6, 3, 12. 36 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, n_aa = 300, t = 50 s.



t (translation time) over rate (elongation rate in amino acids per second) from 3 to 9 and n_aa (protein length in amino acids) from 1 to 600 (logarithmic vertical axis): each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 0.111111 s to dark 200 s, logarithmic shading); iso-lines at t = 1, 10, 100 s. Marked: the worked example, rate = 6, n_aa = 300, t = 50 s.

```
cell_molecular_biology_ai999x.solve('cmb_translation_time', n_aa=300, rate=6.0)
# -> t = 50 s, t_min = 0.833333, coding_nt = 903   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_translation_time', n_aa=450, rate=15.0, k_init=0.2)
# -> t = 30 s, t_min = 0.5, coding_nt = 1353, ribosomes = 6, spacing_codons = 75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Translation time: n_aa = 300 aa, rate = 6 aa/s	SOLVED	t = 50 s, t_min = 0.833333, coding_nt = 903
Translation time: n_aa = 450 aa, rate = 15 aa/s, k_init = 12 1/min	SOLVED	t = 30 s, t_min = 0.5, coding_nt = 1353, ribosomes = 6, spacing_codons = 75
Translation time: rate = 6 aa/s	NEEDS_INPUT	cmb_translation_time needs n_aa (protein length in amino acids). Example: solve('cmb_translation_time', n_aa=300, rate=6
Translation time: n_aa = 300 aa, rate = 6 aa/s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Translation time: n_aa = 300 aa, rate = 6 aa/s	SOLVED	t = 50 s, t_min = 0.833333, coding_nt = 903

Steady-state mRNA and protein numbers, `cmb_expression_steady_state`

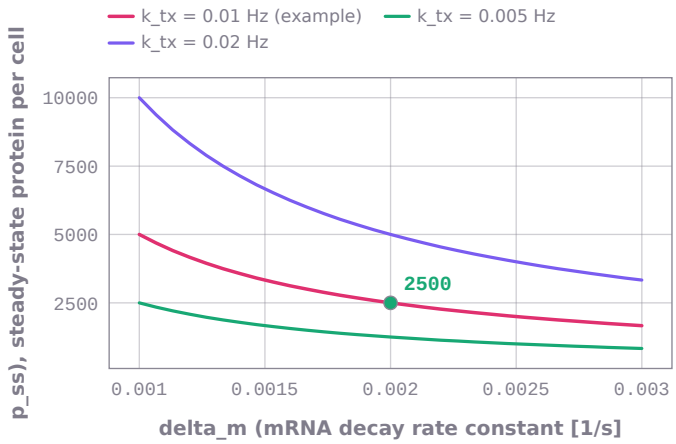
The steady-state numbers of mRNA (k_tx/delta_m) and protein (k_tl m/delta_p) per cell for constant transcription, translation and first-order decay (dilution included), with the half-lives.

$$m^* = \frac{k_{tx}}{\delta_m}, \quad p^* = \frac{k_{tl}\,k_{tx}}{\delta_m\,\delta_p}$$

input	meaning	condition
`k_tx`	transcription rate (mRNA made per cell per unit time)	
`delta_m`	mRNA decay rate constant	
`k_tl`	translation rate (proteins per mRNA per unit time)	
`delta_p`	protein decay (plus dilution) rate constant	

output	meaning
`m_ss`	steady-state mRNA per cell
`p_ss`	steady-state protein per cell
`mRNA_half_life`	mRNA half-life
`protein_half_life`	protein half-life
`mRNA_half_life_min`	mRNA half-life in minutes
`protein_half_life_h`	protein half-life in hours

- **Method:** $dm/dt = k_t x - delta_m$ m, $dp/dt = k_t l$ m - delta_p p; at steady state $m = k_t x/delta_m$, $p = k_t l$ k_tx/(delta_m delta_p)
- **Assumes:** all rates > 0; constant rates, first-order decay
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



p_ss (steady-state protein per cell) against delta_m (mRNA decay rate constant) from 0.001 1/s to 0.003 1/s. One curve for each value of k_tx (transcription rate (mRNA made per cell per unit time)): 0.01 Hz, 0.005 Hz, 0.02 Hz. Other inputs as in the worked example: k_tl = 0.05 Hz, delta_p = 9.9999999999999e-05 1/s. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, delta_m = 0.002 1/s, p_ss = 2500.

```
cell_molecular_biology_ai999x.solve('cmb_expression_steady_state', k_tx=0.01, delta_m=0.002, k_tl=0.05, delta_p=0.0001)
# -> m_ss = 5, p_ss = 2500, mRNA_half_life = 346.574 s, protein_half_life = 6931.47 s, mRNA_half_life_min = 5.77623,
protein_half_life_h = 1.92541   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_expression_steady_state', k_tx=0.016666666666666666,
delta_m=0.0033333333333333335, k_tl=0.2, delta_p=0.0002)
# -> m_ss = 5, p_ss = 5000, mRNA_half_life = 207.944 s, protein_half_life = 3465.74 s, mRNA_half_life_min = 3.46574,
protein_half_life_h = 0.962704   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gene expression steady state: k_tx = 36 1/h, delta_m = 7.2 1/h, k_tl = 180 1/h, delta_p = 0.36 1/h	SOLVED	m_ss = 5, p_ss = 2500, mRNA_half_life = 346.574 s, protein_half_life = 6931.47 s, mRNA_half_life_min = 5.77623, protein_half_life_h = 1.92541
Gene expression steady state: k_tx = 60 1/h, delta_m = 12 1/h, k_tl = 720 1/h, delta_p = 0.72 1/h	SOLVED	m_ss = 5, p_ss = 5000, mRNA_half_life = 207.944 s, protein_half_life = 3465.74 s, mRNA_half_life_min = 3.46574, protein_half_life_h = 0.962704
Gene expression steady state: delta_m = 7.2 1/h, k_tl = 180 1/h, delta_p = 0.36 1/h	NEEDS_INPUT	cmb_expression_steady_state needs k_tx (transcription rate (mRNA made per cell per unit time)). Example: solve('cmb_expr
Gene expression steady state: k_tx = 36 1/h, delta_m = 7.2 1/h, k_tl = 180 1/h, delta_p = 0.36 1/h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Gene expression steady state: k_tx = 36 1/h, delta_m = 7.2 1/h, k_tl = 180 1/h, delta_p = 0.36 1/h	SOLVED	m_ss = 5, p_ss = 2500, mRNA_half_life = 346.574 s, protein_half_life = 6931.47 s, mRNA_half_life_min = 5.77623, protein_half_life_h = 1.92541

mRNA and protein after switching a gene on, `cmb_induction_dynamics`

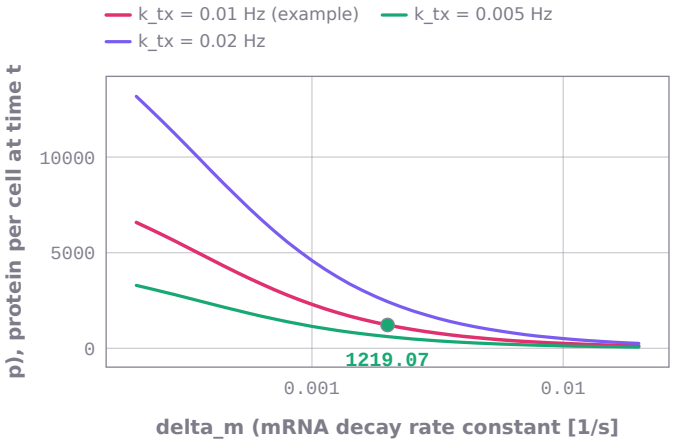
The mRNA and protein numbers a time t after a gene is switched on from zero (constant transcription and translation, first-order decay), and the protein's fraction of its steady state.

$$p(t) = \frac{k_{tl} k_{tx}}{\delta_m} \left[\frac{1 - e^{-\delta_p t}}{\delta_p} - \frac{e^{-\delta_m t} - e^{-\delta_p t}}{\delta_p - \delta_m} \right]$$

input	meaning	condition
`k_tx`	transcription rate	
`delta_m`	mRNA decay rate constant	
`k_tl`	translation rate per mRNA	
`delta_p`	protein decay (plus dilution) rate constant	
`t`	time since induction	

output	meaning
`m`	mRNA per cell at time t
`p`	protein per cell at time t
`fraction_of_steady_state`	p / p_ss

- **Method:** $m = (k_t x / delta_m)$(1 - e^(-delta_m t)); p = (k_tl k_tx/delta_m)[(1 - e^(-delta_p t))/delta_p - (e^(-delta_m $t) - e^{(- delta_p}$ t))/(delta_p - delta_m)] (t e^(-delta t) when equal)
- **Assumes:** all rates > 0; $t \geq 0$; m(0) = p(0) = 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



p (protein per cell at time t) against delta_m (mRNA decay rate constant) from 0.0002 1/s to 0.02 1/s on a logarithmic axis. One curve for each value of k_tx (transcription rate): 0.01 Hz, 0.005 Hz, 0.02 Hz. Other inputs as in the worked example: k_tl = 0.05 Hz, delta_p = 0.9999999999999999e-05 1/s, t = 7200.0 s. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, delta_m = 0.002 1/s, p = 1219.07.

```
cell_molecular_biology_ai999x.solve('cmb_induction_dynamics', k_tx=0.01, delta_m=0.002, k_tl=0.05, delta_p=0.0001,
t=7200.0)
# -> m = 5, p = 1219.07, fraction_of_steady_state = 0.487629   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_induction_dynamics', k_tx=0.02, delta_m=0.001, k_tl=0.1, delta_p=0.003,
t=1800.0)
# -> m = 16.694, p = 502.873, fraction_of_steady_state = 0.75431   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gene induction dynamics: k_tx = 36 1/h, delta_m = 7.2 1/h, k_tl = 180 1/h, delta_p = 0.36 1/h, t = 2 h	SOLVED	m = 5, p = 1219.07, fraction_of_steady_state = 0.487629
Gene induction dynamics: k_tx = 72 1/h, delta_m = 3.6 1/h, k_tl = 360 1/h, delta_p = 10.8 1/h, t = 0.5 h	SOLVED	m = 16.694, p = 502.873, fraction_of_steady_state = 0.75431
Gene induction dynamics: delta_m = 7.2 1/h, k_tl = 180 1/h, delta_p = 0.36 1/h, t = 2 h	NEEDS_INPUT	cmb_induction_dynamics needs k_tx (transcription rate). Example: solve('cmb_induction_dynamics', k_tx='0.01 Hz', delta_m
Gene induction dynamics: k_tx = 36 1/h, delta_m = 7.2 1/h, k_tl = 180 1/h, delta_p = 0.36 1/h, t = 2 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

mRNA half-life fitted to a decay time course, `cmb_mrna_half_life_fit`

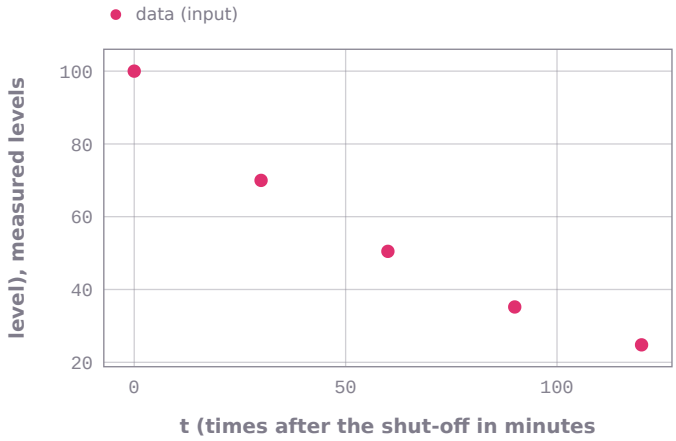
The decay rate constant and half-life of an mRNA (or protein) from levels measured after transcription is stopped, by a least-squares line through ln(level) against time.

$$\ln L = \ln L_0 - kt, \quad t_{1/2} = \frac{\ln 2}{k}$$

input	meaning	condition
`t`	times after the shut-off in minutes	
`level`	measured levels (relative units, positive)	

output	meaning
`k_per_min`	decay rate constant per minute
`half_life_min`	half-life in minutes
`initial_fit`	fitted level at $t = 0$
`r2`	coefficient of determination of the log-linear fit

- **Method:** least squares ln(level) = ln(L0) - k t; $t_1/2 = ln\,2/k$
- **Assumes:** at least 3 points; levels > 0 and decaying; first-order decay
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data level (measured levels (relative units, positive)) against t (times after the shut-off in minutes) as given in the worked example (5 points). Also returned: k_per_min = 0.011587, half_life_min = 59.8211, initial_fit = 99.9556, r2 = 0.999799.

```
cell_molecular_biology_ai999x.solve('cmb_mrna_half_life_fit', t=[0.0, 30.0, 60.0, 90.0, 120.0], level=[100.0, 70.0,
50.5, 35.2, 24.8])
# -> k_per_min = 0.011587, half_life_min = 59.8211, initial_fit = 99.9556, r2 = 0.999799   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_mrna_half_life_fit', t=[0.0, 10.0, 20.0, 40.0], level=[1.0, 0.62, 0.4, 0.15])
# -> k_per_min = 0.0473036, half_life_min = 14.6532, initial_fit = 1.00496, r2 = 0.999572   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
mRNA half-life fit: t = [0, 30, 60, 90, 120], level = [100, 70, 50.5, 35.2, 24.8]	SOLVED	k_per_min = 0.011587, half_life_min = 59.8211, initial_fit = 99.9556, r2 = 0.999799
mRNA half-life fit: t = [0, 10, 20, 40], level = [1, 0.62, 0.4, 0.15]	SOLVED	k_per_min = 0.0473036, half_life_min = 14.6532, initial_fit = 1.00496, r2 = 0.999572
mRNA half-life fit: level = [100, 70, 50.5, 35.2, 24.8]	NEEDS_INPUT	cmb_mrna_half_life_fit needs t (times after the shut-off in minutes). Example: solve('cmb_mrna_half_life_fit', t=[0.0, 3
mRNA half-life fit: t = [0, 30, 60, 90, 120], level = [100, 70, 50.5, 35.2, 24.8], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Michaelis-Menten reaction rate, `cmb_michaelis_menten_rate`

The initial rate of an enzyme-catalysed reaction at substrate concentration S from Vmax and Km, and the fraction of Vmax.

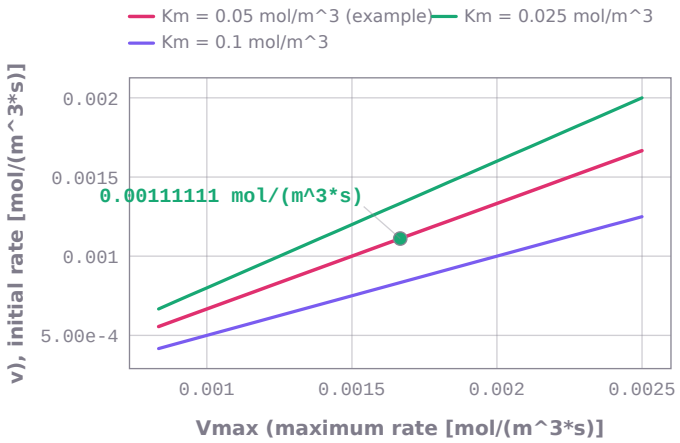
$$v = \frac{V_{max}[S]}{K_m + [S]}$$

input	meaning	condition
`Vmax`	maximum rate	
`Km`	Michaelis constant	
S	substrate concentration	

output	meaning
`v`	initial rate
`v_uM_min`	rate in uM/min

output	meaning
`fraction`	v / Vmax

- **Method:** v = Vmax S / (Km + S)
- **Assumes:** Vmax, $Km > 0$; $S \geq 0$; initial-rate conditions, S >> E
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v (initial rate) against Vmax (maximum rate) from 0.000833333 mol/(m^3*s) to 0.0025 mol/(m^3*s). One curve for each value of Km (Michaelis constant): 0.05 mol/m^3, 0.025 mol/m^3, 0.1 mol/m^3. Other inputs as in the worked example: S = 0.1 mol/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, Vmax = 0.00166667 mol/(m^3*s), v = 0.00111111 mol/(m^3*s).

```
cell_molecular_biology_ai999x.solve('cmb_michaelis_menten_rate', Vmax=0.0016666666666666668, Km=0.05, S=0.1)
# -> v = 0.00111111 mol/(m^3*s), v_uM_min = 66.6667, fraction = 0.666667   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_michaelis_menten_rate', Vmax=2e-06, Km=2.0, S=0.5)
# -> v = 4e-07 mol/(m^3*s), v_uM_min = 0.024, fraction = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Michaelis-Menten rate: Vmax = 100 uM/min, Km = 50 uM, S = 100 uM	SOLVED	v = 0.00111111 mol/(m^3*s), v_uM_min = 66.6667, fraction = 0.666667
Michaelis-Menten rate: Vmax = 0.12 uM/min, Km = 2000 uM, S = 500 uM	SOLVED	v = 4e-07 mol/(m^3*s), v_uM_min = 0.024, fraction = 0.2
Michaelis-Menten rate: Km = 50 uM, S = 100 uM	NEEDS_INPUT	cmb_michaelis_menten_rate needs Vmax (maximum rate). Example: solve('cmb_michaelis_menten_rate', Vmax='0.00166667 mol/(m
Michaelis-Menten rate: Vmax = 100 uM/min, Km = 50 uM, S = 100 uM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Michaelis-Menten rate: Vmax = 100 uM/min, Km = 50 uM, S = 0.1 mM	SOLVED	v = 0.00111111 mol/(m^3*s), v_uM_min = 66.6667, fraction = 0.666667

Substrate concentration for a fraction of Vmax, `cmb_substrate_for_fraction_of_vmax`

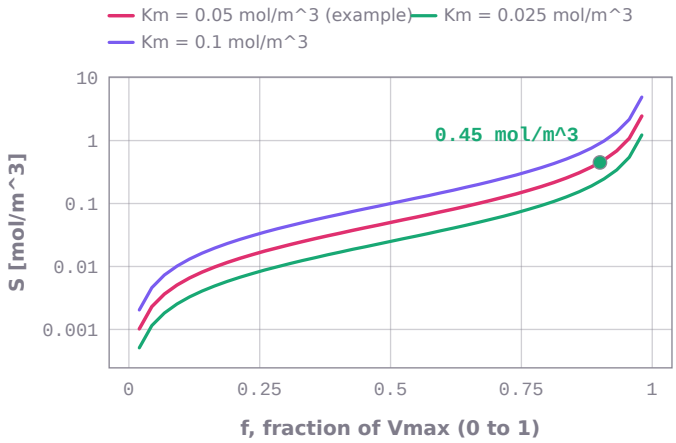
The substrate concentration at which a Michaelis-Menten enzyme runs at a fraction f of Vmax (e.g. 90 % needs 9 Km).

$$[S] = K_m \frac{f}{1 - f}$$

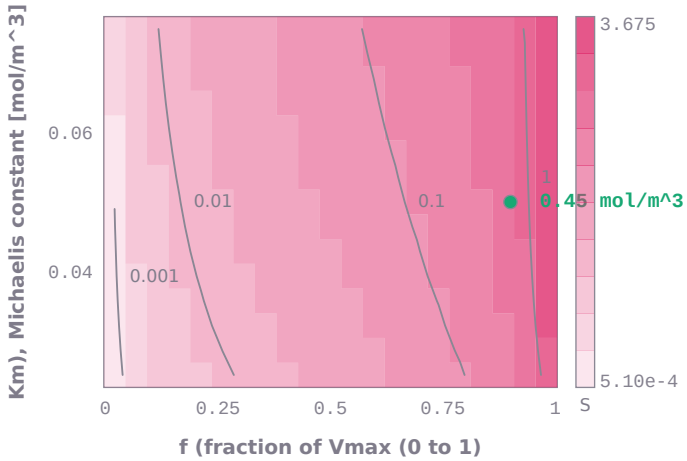
input	meaning	condition
`Km`	Michaelis constant	
`f`	fraction of Vmax (0 to 1)	

output	meaning
`S`	required substrate concentration
`S_uM`	S in uM
`S_over_Km`	S as a multiple of Km

- **Method:** $S = Km\, f/(1 - f)$
- **Assumes:** $Km > 0$; $0 < f < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



S (required substrate concentration) against f (fraction of Vmax (0 to 1)) from 0.02 to 0.98, logarithmic output axis. One curve for each value of Km (Michaelis constant): 0.05 mol/m^3, 0.025 mol/m^3, 0.1 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, f = 0.9, S = 0.45 mol/m^3.



S (required substrate concentration) over f (fraction of Vmax (0 to 1)) from 0.02 to 0.98 and Km (Michaelis constant) from 0.025 mol/m^3 to 0.075 mol/m^3: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 0.000510204 mol/m^3 to dark 3.675 mol/m^3, logarithmic shading); iso-lines at S = 0.001, 0.01, 0.1, 1 mol/m^3. Marked: the worked example, f = 0.9, Km = 0.05 mol/m^3, S = 0.45 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_substrate_for_fraction_of_vmax', Km=0.05, f=0.9)
# -> S = 0.45 mol/m^3, S_uM = 450, S_over_Km = 9   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_substrate_for_fraction_of_vmax', Km=2.0, f=0.25)
# -> S = 0.666667 mol/m^3, S_uM = 666.667, S_over_Km = 0.333333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Substrate for a fraction of Vmax: Km = 50 uM, f = 90%	SOLVED	S = 0.45 mol/m^3, S_uM = 450, S_over_Km = 9
Substrate for a fraction of Vmax: Km = 2000 uM, f = 25%	SOLVED	S = 0.666667 mol/m^3, S_uM = 666.667, S_over_Km = 0.333333
Substrate for a fraction of Vmax: f = 90%	NEEDS_INPUT	cmb_substrate_for_fraction_of_vmax needs Km (Michaelis constant). Example: solve('cmb_substrate_for_fraction_of_vmax', K
Substrate for a fraction of Vmax: Km = 50 uM, f = 90%, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Substrate for a fraction of Vmax: Km = 50 uM, f = 90%	SOLVED	S = 0.45 mol/m^3, S_uM = 450, S_over_Km = 9

Turnover number kcat and catalytic efficiency kcat/Km, `cmb_kcat_catalytic_efficiency`

The turnover number kcat = Vmax/[E]t, the catalytic efficiency kcat/Km (compare with the diffusion limit of about $1e8 - 1e9$ per M per s) and the time per turnover.

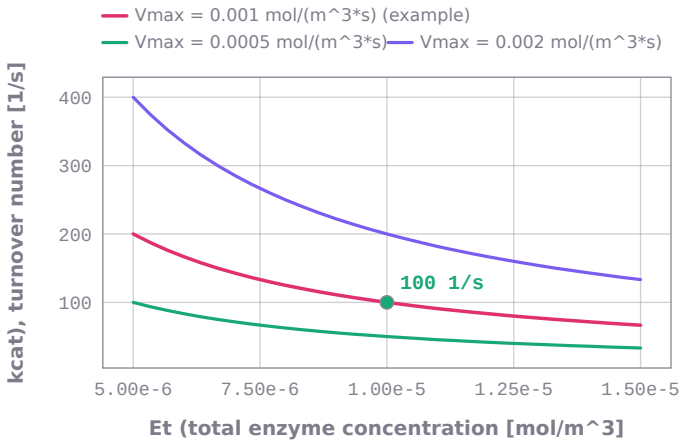
$$k_{cat} = \frac{V_{max}}{[E]_t}, \quad \eta = \frac{k_{cat}}{K_m}$$

input	meaning	condition
`Vmax`	maximum rate	
`Et`	total enzyme concentration	

input	meaning	condition
`Km`	Michaelis constant	

output	meaning
`kcat`	turnover number
`kcat_per_s`	kcat in $1/s$
`efficiency`	catalytic efficiency kcat/Km
`efficiency_per_M_s`	kcat/Km in 1/(M s)
`turnover_time`	time per catalytic event 1/kcat

- **Method:** kcat = Vmax / [E]t; efficiency = kcat / Km
- **Assumes:** Vmax, Et, $Km > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



kcat (turnover number) against Et (total enzyme concentration) from 5.0000e-06 mol/m^3 to 1.5000e-05 mol/m^3. One curve for each value of Vmax (maximum rate): 0.001 mol/(m^3*s), 0.0005 mol/(m^3*s), 0.002 mol/(m^3*s). Other inputs as in the worked example: Km = 0.025 mol/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, Et = 1.0000e-05 mol/m^3, kcat = 100 1/s.

```
cell_molecular_biology_ai999x.solve('cmb_kcat_catalytic_efficiency', Vmax=0.001, Et=1e-05, Km=0.025)
# -> kcat = 100 1/s, kcat_per_s = 100, efficiency = 4000 m^3/(mol*s), efficiency_per_M_s = 4e+06, turnover_time = 0.01
s   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_kcat_catalytic_efficiency', Vmax=5e-06, Et=2e-06, Km=1.5)
# -> kcat = 2.5 1/s, kcat_per_s = 2.5, efficiency = 1.66667 m^3/(mol*s), efficiency_per_M_s = 1666.67, turnover_time =
0.4 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
kcat and catalytic efficiency: Vmax = 60 uM/min, Et = 10 nM, Km = 25 uM	SOLVED	kcat = 100 1/s, kcat_per_s = 100, efficiency = 4000 m^3/(mol*s), efficiency_per_M_s = 4e+06, turnover_time = 0.01 s
kcat and catalytic efficiency: Vmax = 0.3 uM/min, Et = 2 nM, Km = 1500 uM	SOLVED	kcat = 2.5 1/s, kcat_per_s = 2.5, efficiency = 1.66667 m^3/(mol*s), efficiency_per_M_s = 1666.67, turnover_time = 0.4 s
kcat and catalytic efficiency: Et = 10 nM, Km = 25 uM	NEEDS_INPUT	cmb_kcat_catalytic_efficiency needs Vmax (maximum rate). Example: solve('cmb_kcat_catalytic_efficiency', Vmax='0.001 mol
kcat and catalytic efficiency: Vmax = 60 uM/min, Et = 10 nM, Km = 25 uM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Catalytic efficiency: Vmax = 60 uM/min, Et = 10 nM, Km = 25 uM	SOLVED	kcat = 100 1/s, kcat_per_s = 100, efficiency = 4000 m^3/(mol*s), efficiency_per_M_s = 4e+06, turnover_time = 0.01 s

Km and Vmax from a Lineweaver-Burk plot, `cmb_lineweaver_burk_fit`

Km and Vmax from initial-rate data by the double-reciprocal (Lineweaver-Burk) least-squares line 1/v = (Km/Vmax)(1/S) + 1/Vmax.

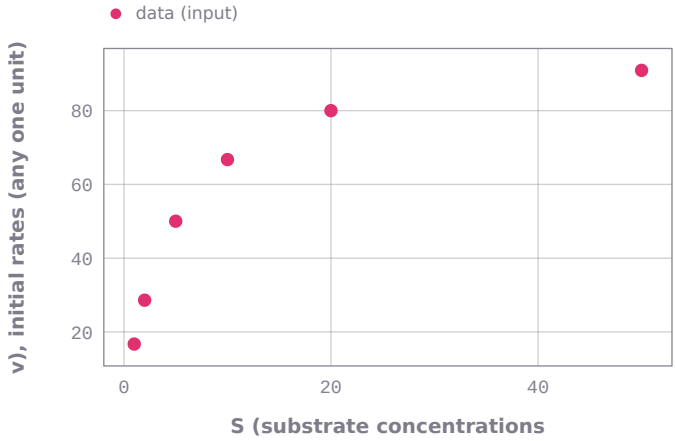
$$\frac{1}{v} = \frac{K_m}{V_{max}}\frac{1}{[S]} + \frac{1}{V_{max}}$$

input	meaning	condition
`S`	substrate concentrations (any one unit, e.g. uM)	
`v`	initial rates (any one unit)	

output	meaning
`Km`	Michaelis constant (unit of S)
`Vmax`	maximum rate (unit of v)
`slope`	slope Km/Vmax

output	meaning
`intercept`	intercept 1/Vmax
`r2`	r^2 of the double-reciprocal line

- **Method:** least squares $1/v = a + b/S$; Vmax = 1/a; $Km = b/a$
- **Assumes:** at least 3 points; S, $v > 0$; positive slope and intercept
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data v (initial rates (any one unit)) against S (substrate concentrations (any one unit, e.g. uM)) as given in the worked example (6 points). Also returned: Km = 4.98227, Vmax = 99.8888, slope = 0.0498781, intercept = 0.0100111, r2 = 1.

```
cell_molecular_biology_ai999x.solve('cmb_lineweaver_burk_fit', S=[1.0, 2.0, 5.0, 10.0, 20.0, 50.0], v=[16.7, 28.6,
50.0, 66.7, 80.0, 90.9])
# -> Km = 4.98227, Vmax = 99.8888, slope = 0.0498781, intercept = 0.0100111, r2 = 1   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_lineweaver_burk_fit', S=[0.5, 1.0, 2.0, 4.0], v=[2.1, 3.4, 5.1, 6.6])
# -> Km = 1.77206, Vmax = 9.52251, slope = 0.186092, intercept = 0.105014, r2 = 0.999775   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lineweaver-Burk fit: S = [1, 2, 5, 10, 20, 50], v = [16.7, 28.6, 50, 66.7, 80, 90.9]	SOLVED	Km = 4.98227, Vmax = 99.8888, slope = 0.0498781, intercept = 0.0100111, r2 = 1
Lineweaver-Burk fit: S = [0.5, 1, 2, 4], v = [2.1, 3.4, 5.1, 6.6]	SOLVED	Km = 1.77206, Vmax = 9.52251, slope = 0.186092, intercept = 0.105014, r2 = 0.999775
Lineweaver-Burk fit: v = [16.7, 28.6, 50, 66.7, 80, 90.9]	NEEDS_INPUT	cmb_lineweaver_burk_fit needs S (substrate concentrations (any one unit, e.g. uM)). Example: solve('cmb_lineweaver_burk_
Lineweaver-Burk fit: S = [1, 2, 5, 10, 20, 50], v = [16.7, 28.6, 50, 66.7, 80, 90.9], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Km and Vmax by nonlinear least squares, `cmb_michaelis_menten_nonlinear_fit`

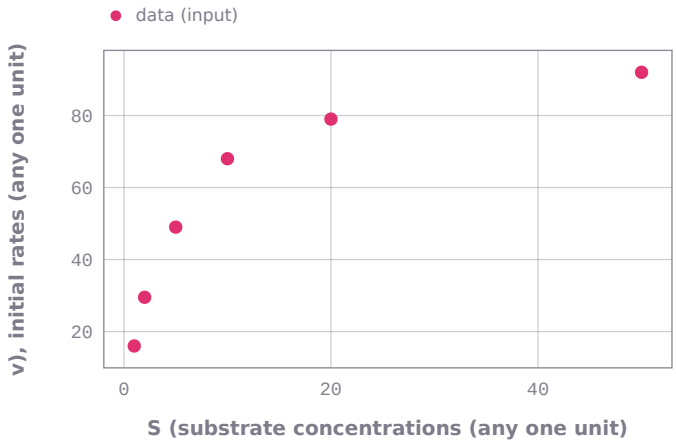
Km and Vmax fitted directly to initial-rate data by nonlinear least squares on v = Vmax S/(Km + S) (no linearizing transform), with the residual sum of squares and r^2.

$$\min_{V,K} \sum_i \left(v_i - \frac{V S_i}{K + S_i} \right)^2$$

input	meaning	condition
`S`	substrate concentrations (any one unit)	
`v`	initial rates (any one unit)	

output	meaning
`Km`	Michaelis constant (unit of S)
`Vmax`	maximum rate (unit of v)
`rss`	residual sum of squares
`r2`	coefficient of determination

- **Method:** Vmax profiled out (Vmax = sum v g / sum g^2, $g = S/(Km + S)$); the stationary point in Km found by bisection
- **Assumes:** at least 3 points; S, $v > 0$; an interior least-squares minimum
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data v (initial rates (any one unit)) against S (substrate concentrations (any one unit)) as given in the worked example (6 points). Also returned: Km = 5.07928, Vmax = 100.714, rss = 5.86753, r2 = 0.998642.

```
cell_molecular_biology_ai999x.solve('cmb_michaelis_menten_nonlinear_fit', S=[1.0, 2.0, 5.0, 10.0, 20.0, 50.0], v=[16.0, 29.5, 49.0, 68.0, 79.0, 92.0])
# -> Km = 5.07928, Vmax = 100.714, rss = 5.86753, r2 = 0.998642   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_michaelis_menten_nonlinear_fit', S=[0.5, 1.0, 2.0, 4.0, 8.0], v=[2.0, 3.5, 5.0, 6.7, 7.9])
# -> Km = 1.8574, Vmax = 9.75371, rss = 0.0172641, r2 = 0.999234   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nonlinear Michaelis-Menten fit: S = [1, 2, 5, 10, 20, 50], v = [16, 29.5, 49, 68, 79, 92]	SOLVED	Km = 5.07928, Vmax = 100.714, rss = 5.86753, r2 = 0.998642
Nonlinear Michaelis-Menten fit: S = [0.5, 1, 2, 4, 8], v = [2, 3.5, 5, 6.7, 7.9]	SOLVED	Km = 1.8574, Vmax = 9.75371, rss = 0.0172641, r2 = 0.999234
Nonlinear Michaelis-Menten fit: v = [16, 29.5, 49, 68, 79, 92]	NEEDS_INPUT	cmb_michaelis_menten_nonlinear_fit needs S (substrate concentrations (any one unit)). Example: solve('cmb_michaelis_ment
Nonlinear Michaelis-Menten fit: S = [1, 2, 5, 10, 20, 50], v = [16, 29.5, 49, 68, 79, 92], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Competitive inhibition rate, `cmb_competitive_inhibition`

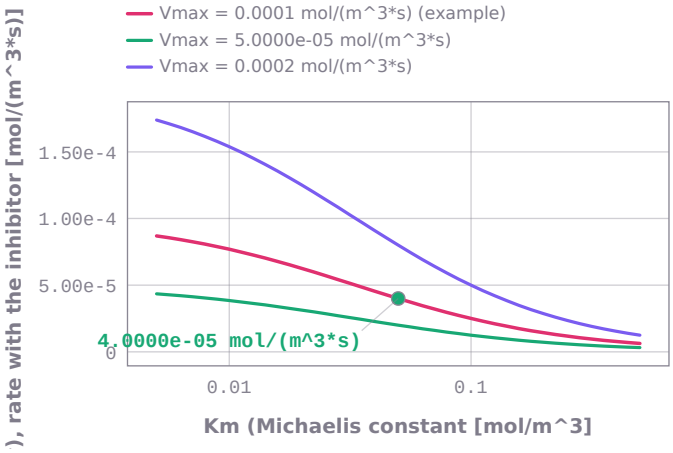
The rate of an enzyme with a competitive inhibitor (binding free enzyme only): Km rises by $(1 + I/Ki)$, Vmax unchanged.

$$v = \frac{V_{max}[S]}{K_m(1 + [I]/K_i) + [S]}$$

input	meaning	condition
`Vmax`	maximum rate without inhibitor	
`Km`	Michaelis constant	
`S`	substrate concentration	
`I`	inhibitor concentration	
`Ki`	inhibition (dissociation) constant	

output	meaning
`v`	rate with the inhibitor
`Km_app`	apparent Km
`Vmax_app`	apparent Vmax
`percent_inhibition`	per cent inhibition at this S

- **Method:** v = Vmax S / (Km (1 + I/Ki) + S)
- **Assumes:** Vmax, Km, $Ki > 0$; S, $I \geq 0$; rapid-equilibrium inhibitor binding
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v (rate with the inhibitor) against Km (Michaelis constant) from 0.005 mol/m^3 to 0.5 mol/m^3 on a logarithmic axis. One curve for each value of Vmax (maximum rate without inhibitor): 0.0001 mol/(m^3*s), 5.0000e-05 mol/(m^3*s), 0.0002 mol/(m^3*s). Other inputs as in the worked example: S = 0.1 mol/m^3, I = 0.02 mol/m^3, Ki = 0.01 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, Km = 0.05 mol/m^3, v = 4.0000e-05 mol/(m^3*s).

```
cell_molecular_biology_ai999x.solve('cmb_competitive_inhibition', Vmax=0.0001, Km=0.05, S=0.1, I=0.02, Ki=0.01)
# -> v = 4e-05 mol/(m^3*s), Km_app = 0.15 mol/m^3, Vmax_app = 0.0001 mol/(m^3*s), percent_inhibition = 40   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_competitive_inhibition', Vmax=2e-06, Km=1.0, S=0.5, I=0.5, Ki=0.25)
# -> v = 2.85714e-07 mol/(m^3*s), Km_app = 3 mol/m^3, Vmax_app = 2e-06 mol/(m^3*s), percent_inhibition = 57.1429
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Competitive inhibition rate: Vmax = 6 uM/min, Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM	SOLVED	v = 4e-05 mol/(m^3*s), Km_app = 0.15 mol/m^3, Vmax_app = 0.0001 mol/(m^3*s), percent_inhibition = 40
Competitive inhibition rate: Vmax = 0.12 uM/min, Km = 1000 uM, S = 500 uM, I = 500 uM, Ki = 250 uM	SOLVED	v = 2.85714e-07 mol/(m^3*s), Km_app = 3 mol/m^3, Vmax_app = 2e-06 mol/(m^3*s), percent_inhibition = 57.1429
Competitive inhibition rate: Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM	NEEDS_INPUT	cmb_competitive_inhibition needs Vmax (maximum rate without inhibitor). Example: solve('cmb_competitive_inhibition', Vma
Competitive inhibition rate: Vmax = 6 uM/min, Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Uncompetitive inhibition rate, `cmb_uncompetitive_inhibition`

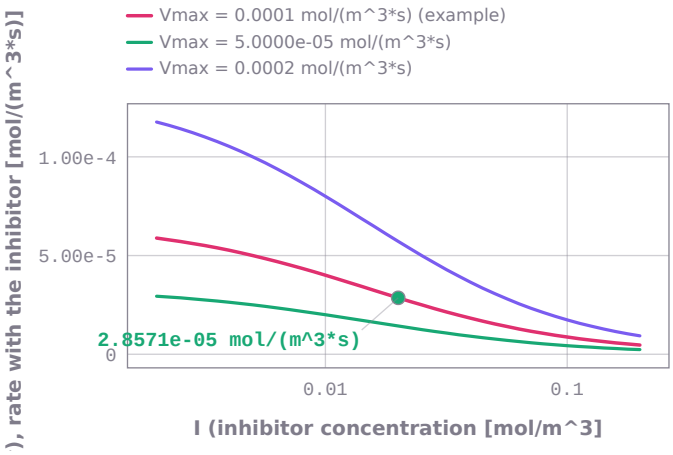
The rate of an enzyme with an uncompetitive inhibitor (binding the enzyme-substrate complex only): Km and Vmax both fall by $(1 + I/Ki)$.

$$v = \frac{V_{max}[S]}{K_m + [S](1 + [I]/K_i)}$$

input	meaning	condition
`Vmax`	maximum rate without inhibitor	
`Km`	Michaelis constant	
`S`	substrate concentration	
`I`	inhibitor concentration	
`Ki`	inhibition (dissociation) constant	

output	meaning
`v`	rate with the inhibitor
`Km_app`	apparent Km
`Vmax_app`	apparent Vmax
`percent_inhibition`	per cent inhibition at this S

- **Method:** v = Vmax S / (Km + S $(1 + I/Ki)$)
- **Assumes:** Vmax, Km, $Ki > 0$; S, $I \geq 0$; rapid-equilibrium inhibitor binding
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v (rate with the inhibitor) against I (inhibitor concentration) from 0.002 mol/m^3 to 0.2 mol/m^3 on a logarithmic axis. One curve for each value of Vmax (maximum rate without inhibitor): 0.0001 mol/(m^3*s), 5.0000e-05 mol/(m^3*s), 0.0002 mol/(m^3*s). Other inputs as in the worked example: Km = 0.05 mol/m^3, S = 0.1 mol/m^3, Ki = 0.01 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, I = 0.02 mol/m^3, v = 2.8571e-05 mol/(m^3*s).

```
cell_molecular_biology_ai999x.solve('cmb_uncompetitive_inhibition', Vmax=0.0001, Km=0.05, S=0.1, I=0.02, Ki=0.01)
# -> v = 2.85714e-05 mol/(m^3*s), Km_app = 0.0166667 mol/m^3, Vmax_app = 3.33333e-05 mol/(m^3*s), percent_inhibition =
57.1429   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_uncompetitive_inhibition', Vmax=2e-06, Km=1.0, S=0.5, I=0.5, Ki=0.25)
# -> v = 4e-07 mol/(m^3*s), Km_app = 0.333333 mol/m^3, Vmax_app = 6.66667e-07 mol/(m^3*s), percent_inhibition = 40
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Uncompetitive inhibition rate: Vmax = 6 uM/min, Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM	SOLVED	v = 2.85714e-05 mol/(m^3*s), Km_app = 0.0166667 mol/m^3, Vmax_app = 3.33333e-05 mol/(m^3*s), percent_inhibition = 57.1429
Uncompetitive inhibition rate: Vmax = 0.12 uM/min, Km = 1000 uM, S = 500 uM, I = 500 uM, Ki = 250 uM	SOLVED	v = 4e-07 mol/(m^3*s), Km_app = 0.333333 mol/m^3, Vmax_app = 6.66667e-07 mol/(m^3*s), percent_inhibition = 40
Uncompetitive inhibition rate: Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM	NEEDS_INPUT	cmb_uncompetitive_inhibition needs Vmax (maximum rate without inhibitor). Example: solve('cmb_uncompetitive_inhibition',
Uncompetitive inhibition rate: Vmax = 6 uM/min, Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Noncompetitive inhibition rate, `cmb_noncompetitive_inhibition`

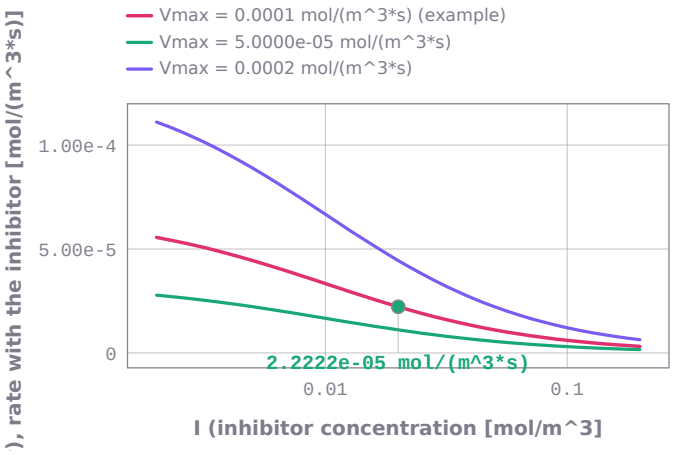
The rate of an enzyme with a pure noncompetitive inhibitor (equal affinity for E and ES): Vmax falls by $(1 + I/Ki)$, Km unchanged.

$$v = \frac{V_{max}[S]}{(K_m + [S])(1 + [I]/K_i)}$$

input	meaning	condition
`Vmax`	maximum rate without inhibitor	
`Km`	Michaelis constant	
`S`	substrate concentration	
`I`	inhibitor concentration	
`Ki`	inhibition (dissociation) constant	

output	meaning
`v`	rate with the inhibitor
`Km_app`	apparent Km
`Vmax_app`	apparent Vmax
`percent_inhibition`	per cent inhibition at this S

- **Method:** v = Vmax S / ((Km + S)$(1 + I/Ki)$)
- **Assumes:** Vmax, Km, $Ki > 0$; S, $I \geq 0$; rapid-equilibrium inhibitor binding
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v (rate with the inhibitor) against I (inhibitor concentration) from 0.002 mol/m^3 to 0.2 mol/m^3 on a logarithmic axis. One curve for each value of Vmax (maximum rate without inhibitor): 0.0001 mol/(m^3*s), 5.0000e-05 mol/(m^3*s), 0.0002 mol/(m^3*s). Other inputs as in the worked example: Km = 0.05 mol/m^3, S = 0.1 mol/m^3, Ki = 0.01 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, I = 0.02 mol/m^3, v = 2.2222e-05 mol/(m^3*s).

```
cell_molecular_biology_ai999x.solve('cmb_noncompetitive_inhibition', Vmax=0.0001, Km=0.05, S=0.1, I=0.02, Ki=0.01)
# -> v = 2.22222e-05 mol/(m^3*s), Km_app = 0.05 mol/m^3, Vmax_app = 3.33333e-05 mol/(m^3*s), percent_inhibition =
66.6667   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_noncompetitive_inhibition', Vmax=2e-06, Km=1.0, S=0.5, I=0.5, Ki=0.25)
# -> v = 2.22222e-07 mol/(m^3*s), Km_app = 1 mol/m^3, Vmax_app = 6.66667e-07 mol/(m^3*s), percent_inhibition = 66.6667
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Noncompetitive inhibition rate: Vmax = 6 uM/min, Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM	SOLVED	v = 2.22222e-05 mol/(m^3*s), Km_app = 0.05 mol/m^3, Vmax_app = 3.33333e-05 mol/(m^3*s), percent_inhibition = 66.6667
Noncompetitive inhibition rate: Vmax = 0.12 uM/min, Km = 1000 uM, S = 500 uM, I = 500 uM, Ki = 250 uM	SOLVED	v = 2.22222e-07 mol/(m^3*s), Km_app = 1 mol/m^3, Vmax_app = 6.66667e-07 mol/(m^3*s), percent_inhibition = 66.6667
Noncompetitive inhibition rate: Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM	NEEDS_INPUT	cmb_noncompetitive_inhibition needs Vmax (maximum rate without inhibitor). Example: solve('cmb_noncompetitive_inhibition
Noncompetitive inhibition rate: Vmax = 6 uM/min, Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Mixed inhibition rate, `cmb_mixed_inhibition`

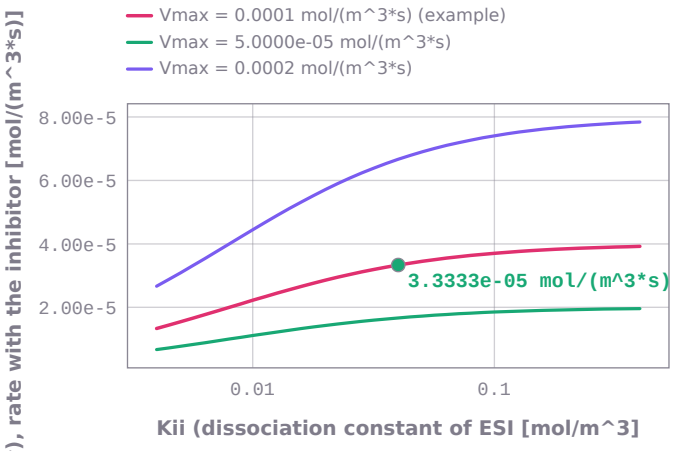
The rate, apparent Km and apparent Vmax of an enzyme with a mixed inhibitor that binds free enzyme (Ki) and the enzyme-substrate complex (Kii) with different affinities.

$$v = \frac{V_{max}[S]}{K_m\left(1+\frac{[I]}{K_i}\right)+[S]\left(1+\frac{[I]}{K_{ii}}\right)}$$

input	meaning	condition
`Vmax`	maximum rate without inhibitor	
`Km`	Michaelis constant	
`S`	substrate concentration	
`I`	inhibitor concentration	
`Ki`	dissociation constant of EI	
`Kii`	dissociation constant of ESI	

output	meaning
`v`	rate with the inhibitor
`Km_app`	apparent Km
`Vmax_app`	apparent Vmax
`alpha`	Kii / Ki

- **Method:** v = Vmax S / (Km $(1 + I/Ki) + S$ (1 + I/Kii)); Km_app = Km (1 + I/Ki)/(1 + I/Kii); Vmax_app = Vmax/(1 + I/Kii)
- **Assumes:** Vmax, Km, Ki, Kii > 0; S, $I \geq 0$; rapid equilibrium
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v (rate with the inhibitor) against Kii (dissociation constant of ESI) from 0.004 mol/m^3 to 0.4 mol/m^3 on a logarithmic axis. One curve for each value of Vmax (maximum rate without inhibitor): 0.0001 mol/(m^3*s), 5.0000e-05 mol/(m^3*s), 0.0002 mol/(m^3*s). Other inputs as in the worked example: Km = 0.05 mol/m^3, S = 0.1 mol/m^3, I = 0.02 mol/m^3, Ki = 0.01 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, Kii = 0.04 mol/m^3, v = 3.3333e-05 mol/(m^3*s).

```
cell_molecular_biology_ai999x.solve('cmb_mixed_inhibition', Vmax=0.0001, Km=0.05, S=0.1, I=0.02, Ki=0.01, Kii=0.04)
# -> v = 3.33333e-05 mol/(m^3*s), Km_app = 0.1 mol/m^3, Vmax_app = 6.66667e-05 mol/(m^3*s), alpha = 4   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_mixed_inhibition', Vmax=2e-06, Km=1.0, S=0.5, I=0.5, Ki=1.0, Kii=0.2)
# -> v = 3.07692e-07 mol/(m^3*s), Km_app = 0.428571 mol/m^3, Vmax_app = 5.71429e-07 mol/(m^3*s), alpha = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mixed inhibition rate: Vmax = 6 uM/min, Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM, Kii = 40 uM	SOLVED	v = 3.33333e-05 mol/(m^3*s), Km_app = 0.1 mol/m^3, Vmax_app = 6.66667e-05 mol/(m^3*s), alpha = 4
Mixed inhibition rate: Vmax = 0.12 uM/min, Km = 1000 uM, S = 500 uM, I = 500 uM, Ki = 1000 uM, Kii = 200 uM	SOLVED	v = 3.07692e-07 mol/(m^3*s), Km_app = 0.428571 mol/m^3, Vmax_app = 5.71429e-07 mol/(m^3*s), alpha = 0.2
Mixed inhibition rate: Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM, Kii = 40 uM	NEEDS_INPUT	cmb_mixed_inhibition needs Vmax (maximum rate without inhibitor). Example: solve('cmb_mixed_inhibition', Vmax='0.0001 mo
Mixed inhibition rate: Vmax = 6 uM/min, Km = 50 uM, S = 100 uM, I = 20 uM, Ki = 10 uM, Kii = 40 uM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Inhibition constant from IC50 (Cheng-Prusoff), `cmb_ic50_to_ki`

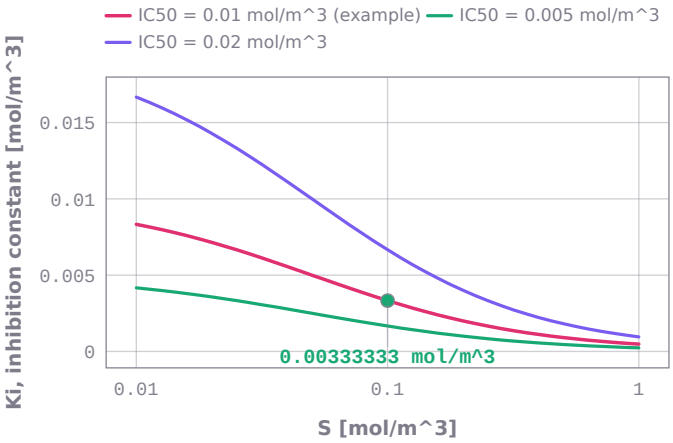
The inhibition constant Ki from a measured IC50 at substrate concentration S for a competitive, uncompetitive or noncompetitive inhibitor (Cheng-Prusoff relations).

$$K_i = \frac{IC_{50}}{1 + [S]/K_m} \ \text{(competitive)}$$

input	meaning	condition
`mode`	mechanism of inhibition	
`IC50`	inhibitor concentration giving 50 % inhibition	
`S`	substrate concentration of the assay	
`Km`	Michaelis constant	

output	meaning
`Ki`	inhibition constant
`Ki_uM`	Ki in uM

- **Method:** competitive $Ki = IC50/(1 + S/Km)$; uncompetitive $Ki = IC50/(1 + Km/S)$; noncompetitive $Ki = IC50$
- **Assumes:** IC50, S, $Km > 0$; inhibitor not depleted (I >> E)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ki (inhibition constant) against S (substrate concentration of the assay) from 0.01 mol/m^3 to 1 mol/m^3 on a logarithmic axis. One curve for each value of IC50 (inhibitor concentration giving 50 % inhibition): 0.01 mol/m^3, 0.005 mol/m^3, 0.02 mol/m^3. Other inputs as in the worked example: mode = competitive, Km = 0.05 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, S = 0.1 mol/m^3, Ki = 0.00333333 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_ic50_to_ki', mode='competitive', IC50=0.01, S=0.1, Km=0.05)
# -> Ki = 0.00333333 mol/m^3, Ki_uM = 3.33333   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_ic50_to_ki', mode='uncompetitive', IC50=0.5, S=1.0, Km=3.0)
# -> Ki = 0.125 mol/m^3, Ki_uM = 125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ki from IC50: mode = competitive, IC50 = 10 uM, S = 100 uM, Km = 50 uM	SOLVED	Ki = 0.00333333 mol/m^3, Ki_uM = 3.33333
Ki from IC50: mode = uncompetitive, IC50 = 500 uM, S = 1000 uM, Km = 3000 uM	SOLVED	Ki = 0.125 mol/m^3, Ki_uM = 125
Ki from IC50: IC50 = 10 uM, S = 100 uM, Km = 50 uM	NEEDS_INPUT	cmb_ic50_to_ki needs mode (mechanism of inhibition). Example: solve('cmb_ic50_to_ki', mode='competitive', IC50='0.01 mol
Ki from IC50: mode = competitive, IC50 = 10 uM, S = 100 uM, Km = 50 uM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Ki from IC50: mode = Competitive, IC50 = 10 uM, S = 0.1 mM, Km = 50 uM	SOLVED	Ki = 0.00333333 mol/m^3, Ki_uM = 3.33333

Fractional saturation by the Hill equation, `cmb_hill_equation`

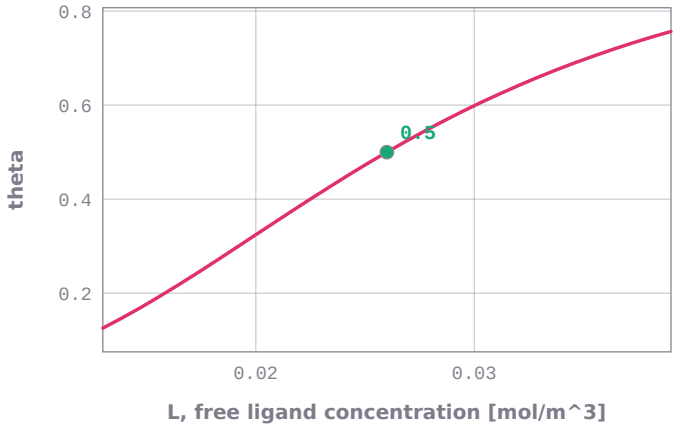
The fractional saturation (or response) of a cooperative binding protein at ligand concentration L, with half-saturation K and Hill coefficient n.

$$\theta = \frac{[L]^n}{K_{0.5}^n + [L]^n}$$

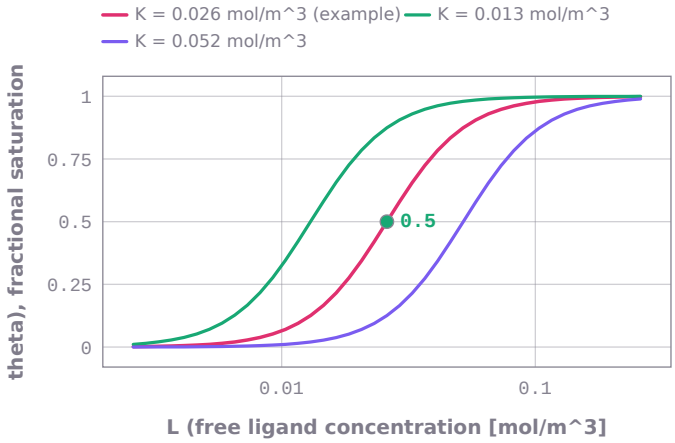
input	meaning	condition
`L`	free ligand concentration	
`K`	ligand concentration for half saturation (K0.5)	
`n`	Hill coefficient	

output	meaning
`theta`	fractional saturation
`theta_percent`	saturation in per cent

- **Method:** $\theta = L^n/(K^n + L^n)$
- **Assumes:** L, K, $n > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



theta (fractional saturation) against L (free ligand concentration), with K = 0.026 mol/m^3, n = 2.8. 31 points computed by cell-molecular-biology-ai999x; marked: L = 0.026, theta = 0.5.



theta (fractional saturation) against L (free ligand concentration) from 0.0026 mol/m^3 to 0.26 mol/m^3 on a logarithmic axis. One curve for each value of K (ligand concentration for half saturation (K0.5)): 0.026 mol/m^3, 0.013 mol/m^3, 0.052 mol/m^3. Other inputs as in the worked example: n = 2.8. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, L = 0.026 mol/m^3, theta = 0.5.

```
cell_molecular_biology_ai999x.solve('cmb_hill_equation', L=0.026, K=0.026, n=2.8)
# -> theta = 0.5, theta_percent = 50   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_hill_equation', L=0.01, K=0.02, n=2.0)
# -> theta = 0.2, theta_percent = 20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hill equation: L = 26 uM, K = 26 uM, n = 2.8	SOLVED	theta = 0.5, theta_percent = 50
Hill equation: L = 10 uM, K = 20 uM, n = 2	SOLVED	theta = 0.2, theta_percent = 20
Hill equation: K = 26 uM, n = 2.8	NEEDS_INPUT	cmb_hill_equation needs L (free ligand concentration). Example: solve('cmb_hill_equation', L='0.026 mol/m^3', K='0.026 m
Hill equation: L = 26 uM, K = 26 uM, n = 2.8, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Ligand concentration for a given saturation (Hill), `cmb_hill_ligand_for_fraction`

The ligand concentration that gives a fractional saturation theta for a cooperative protein with half-saturation K and Hill coefficient n.

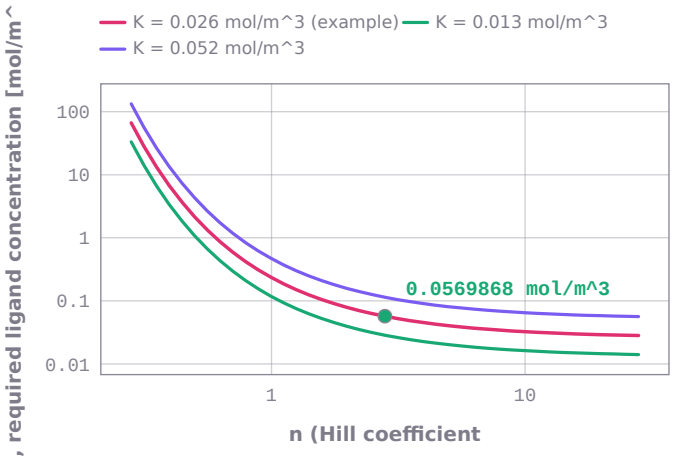
$$[L] = K_{0.5}\left(\frac{\theta}{1-\theta}\right)^{1/n}$$

input	meaning	condition
`K`	half-saturation concentration K0.5	
`n`	Hill coefficient	
`theta`	fractional saturation (0 to 1)	

output	meaning
`L`	required ligand concentration

output	meaning
`L_uM`	L in uM

- **Method:** $L = K$ (theta/(1 - theta))^$(1/n)$
- **Assumes:** K, $n > 0$; $0 < \theta < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



L (required ligand concentration) against n (Hill coefficient) from 0.28 to 28 on a logarithmic axis, logarithmic output axis. One curve for each value of K (half-saturation concentration K0.5): 0.026 mol/m^3, 0.013 mol/m^3, 0.052 mol/m^3. Other inputs as in the worked example: theta = 0.9. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, n = 2.8, L = 0.0569868 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_hill_ligand_for_fraction', K=0.026, n=2.8, theta=0.9)
# -> L = 0.0569868 mol/m^3, L_uM = 56.9868   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_hill_ligand_for_fraction', K=0.005, n=1.5, theta=0.2)
# -> L = 0.00198425 mol/m^3, L_uM = 1.98425   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hill ligand concentration for a saturation: K = 26 uM, n = 2.8, theta = 90%	SOLVED	L = 0.0569868 mol/m^3, L_uM = 56.9868
Hill ligand concentration for a saturation: K = 5 uM, n = 1.5, theta = 20%	SOLVED	L = 0.00198425 mol/m^3, L_uM = 1.98425
Hill ligand concentration for a saturation: n = 2.8, theta = 90%	NEEDS_INPUT	cmb_hill_ligand_for_fraction needs K (half-saturation concentration K0.5). Example: solve('cmb_hill_ligand_for_fraction'
Hill ligand concentration for a saturation: K = 26 uM, n = 2.8, theta = 90%, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Ligand for a fractional saturation: K = 26 uM, n = 2.8, theta = 90%	SOLVED	L = 0.0569868 mol/m^3, L_uM = 56.9868

Hill coefficient from EC10 and EC90, `cmb_hill_coefficient_from_ec`

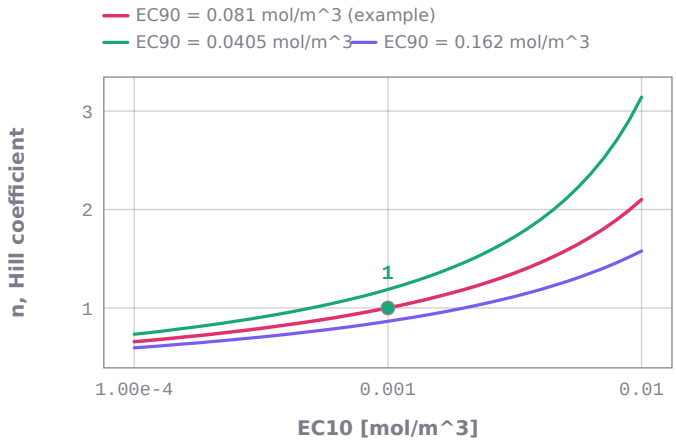
The Hill coefficient $n = ln\,81/ln$(EC90/EC10) and the EC50 (their geometric mean) of a Hill dose-response curve (81-fold range for n = 1).

$$n_H = \frac{\ln 81}{\ln(EC_{90}/EC_{10})}$$

input	meaning	condition
`EC10`	concentration giving 10 % of the maximum	
`EC90`	concentration giving 90 % of the maximum	

output	meaning
`n`	Hill coefficient
`EC50`	half-maximal concentration
`ratio`	$EC90/EC10$

- **Method:** $n = ln\,81/ln$(EC90/EC10); $EC50 = sqrt$(EC10 EC90)
- **Assumes:** $0 < EC10 < EC90$; a Hill curve
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n (Hill coefficient) against EC10 (concentration giving 10 % of the maximum) from 0.0001 mol/m^3 to 0.01 mol/m^3 on a logarithmic axis. One curve for each value of EC90 (concentration giving 90 % of the maximum): 0.081 mol/m^3, 0.0405 mol/m^3, 0.162 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, EC10 = 0.001 mol/m^3, n = 1.

```
cell_molecular_biology_ai999x.solve('cmb_hill_coefficient_from_ec', EC10=0.001, EC90=0.081)
# -> n = 1, EC50 = 0.009 mol/m^3, ratio = 81   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_hill_coefficient_from_ec', EC10=0.01, EC90=0.09)
# -> n = 2, EC50 = 0.03 mol/m^3, ratio = 9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hill coefficient from EC10 and EC90: EC10 = 1 uM, EC90 = 81 uM	SOLVED	n = 1, EC50 = 0.009 mol/m^3, ratio = 81
Hill coefficient from EC10 and EC90: EC10 = 10 uM, EC90 = 90 uM	SOLVED	n = 2, EC50 = 0.03 mol/m^3, ratio = 9
Hill coefficient from EC10 and EC90: EC90 = 81 uM	NEEDS_INPUT	cmb_hill_coefficient_from_ec needs EC10 (concentration giving 10 % of the maximum). Example: solve('cmb_hill_coefficient
Hill coefficient from EC10 and EC90: EC10 = 1 uM, EC90 = 81 uM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Fraction of receptor bound at a free ligand concentration, `cmb_fraction_bound`

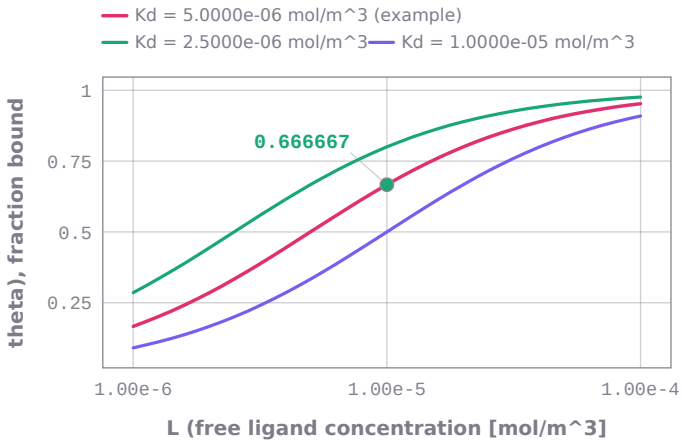
The fractional occupancy L/(Kd + L) of a single-site receptor at free ligand concentration L, and the bound concentration for a receptor total Rt (ligand not depleted).

$$\theta = \frac{[L]}{K_d + [L]}$$

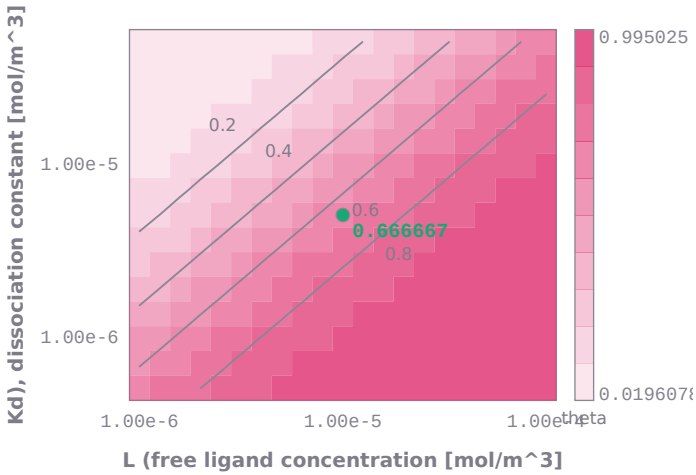
input	meaning	condition
`L`	free ligand concentration	
`Kd`	dissociation constant	
`Rt`	total receptor concentration (optional)	

output	meaning
`theta`	fraction bound
`theta_percent`	per cent bound
`bound`	bound complex concentration (when Rt is given)

- **Method:** $\theta = L/(Kd + L)$
- **Assumes:** L, $Kd > 0$; single independent site; ligand in excess (free = total)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



theta (fraction bound) against L (free ligand concentration) from 1.0000e-06 mol/m^3 to 0.0001 mol/m^3 on a logarithmic axis. One curve for each value of Kd (dissociation constant): 5.0000e-06 mol/m^3, 2.5000e-06 mol/m^3, 1.0000e-05 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, L = 1.0000e-05 mol/m^3, theta = 0.666667.



theta (fraction bound) over L (free ligand concentration) from 1.0000e-06 mol/m^3 to 0.0001 mol/m^3 and Kd (dissociation constant) from 5.0000e-07 mol/m^3 to 5.0000e-05 mol/m^3 (logarithmic axes): each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 0.0196078 to dark 0.995025); iso-lines at theta = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, L = 1.0000e-05 mol/m^3, Kd = 5.0000e-06 mol/m^3, theta = 0.666667.

```
cell_molecular_biology_ai999x.solve('cmb_fraction_bound', L=1e-05, Kd=5e-06)
# -> theta = 0.666667, theta_percent = 66.6667   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_fraction_bound', L=0.0002, Kd=0.001, Rt=5e-05)
# -> theta = 0.166667, theta_percent = 16.6667, bound = 8.33333e-06 mol/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fraction bound: L = 10 nM, Kd = 5 nM	SOLVED	theta = 0.666667, theta_percent = 66.6667
Fraction bound: L = 200 nM, Kd = 1000 nM, Rt = 50 nM	SOLVED	theta = 0.166667, theta_percent = 16.6667, bound = 8.33333e-06 mol/m^3
Fraction bound: Kd = 5 nM	NEEDS_INPUT	cmb_fraction_bound needs L (free ligand concentration). Example: solve('cmb_fraction_bound', L='1e-05 mol/m^3', Kd='5e-0
Fraction bound: L = 10 nM, Kd = 5 nM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Complex concentration with ligand depletion (quadratic binding), `cmb_ligand_depletion_binding`

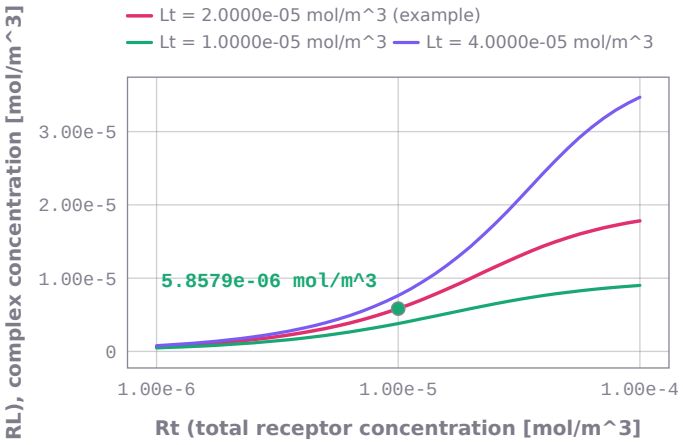
The concentration of receptor-ligand complex when neither component is in excess (exact quadratic solution), with the fractions bound and the free ligand.

$$[RL] = \frac{(R_t + L_t + K_d) - \sqrt{(R_t + L_t + K_d)^2 - 4R_tL_t}}{2}$$

input	meaning	condition
`Rt`	total receptor concentration	
`Lt`	total ligand concentration	
`Kd`	dissociation constant	

output	meaning
`RL`	complex concentration
`fraction_receptor_bound`	RL / Rt
`free_ligand`	free ligand Lt - RL
`fraction_ligand_bound`	RL / Lt

- **Method:** RL = [(Rt + Lt + Kd) - sqrt($(Rt + Lt + Kd)^2 - 4$ Rt Lt)] / 2
- **Assumes:** Rt, Lt, $Kd > 0$; single site, 1:1 binding at equilibrium
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



RL (complex concentration) against Rt (total receptor concentration) from 1.0000e-06 mol/m^3 to 0.0001 mol/m^3 on a logarithmic axis. One curve for each value of Lt (total ligand concentration): 2.0000e-05 mol/m^3, 1.0000e-05 mol/m^3, 4.0000e-05 mol/m^3. Other inputs as in the worked example: Kd = 9.9999999999999e-06 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, Rt = 1.0000e-05 mol/m^3, RL = 5.8579e-06 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_ligand_depletion_binding', Rt=1e-05, Lt=2e-05, Kd=1e-05)
# -> RL = 5.85786e-06 mol/m^3, fraction_receptor_bound = 0.585786, free_ligand = 1.41421e-05 mol/m^3,
fraction_ligand_bound = 0.292893   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_ligand_depletion_binding', Rt=0.0001, Lt=5e-05, Kd=2e-06)
# -> RL = 4.81432e-05 mol/m^3, fraction_receptor_bound = 0.481432, free_ligand = 1.85678e-06 mol/m^3,
fraction_ligand_bound = 0.962864   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Quadratic binding with ligand depletion: Rt = 10 nM, Lt = 20 nM, Kd = 10 nM	SOLVED	RL = 5.85786e-06 mol/m^3, fraction_receptor_bound = 0.585786, free_ligand = 1.41421e-05 mol/m^3, fraction_ligand_bound = 0.292893
Quadratic binding with ligand depletion: Rt = 100 nM, Lt = 50 nM, Kd = 2 nM	SOLVED	RL = 4.81432e-05 mol/m^3, fraction_receptor_bound = 0.481432, free_ligand = 1.85678e-06 mol/m^3, fraction_ligand_bound = 0.962864
Quadratic binding with ligand depletion: Lt = 20 nM, Kd = 10 nM	NEEDS_INPUT	cmb_ligand_depletion_binding needs Rt (total receptor concentration). Example: solve('cmb_ligand_depletion_binding', Rt=
Quadratic binding with ligand depletion: Rt = 10 nM, Lt = 20 nM, Kd = 10 nM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Dissociation constant from association and dissociation rates, `cmb_kd_from_rates`

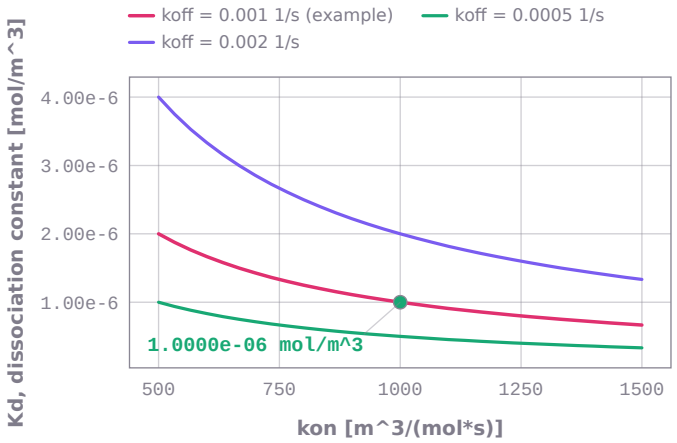
The equilibrium dissociation constant Kd = koff/kon, the association constant, the complex half-life and the residence time.

$$K_d = \frac{k_{off}}{k_{on}}, \quad t_{1/2} = \frac{\ln 2}{k_{off}}$$

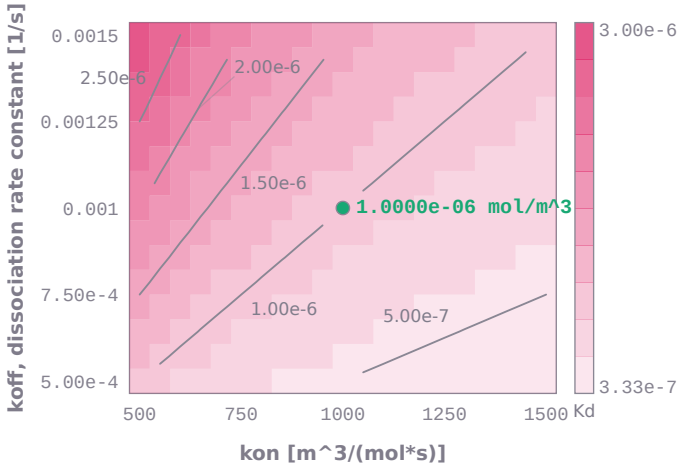
input	meaning	condition
`kon`	association rate constant	
`koff`	dissociation rate constant	

output	meaning
`Kd`	dissociation constant
`Kd_nM`	Kd in nM
`Ka_per_M`	association constant in $1/M$
`half_life`	complex half-life ln2/koff
`residence_time`	mean residence time 1/koff

- **Method:** Kd = koff/kon; $t_1/2 = ln$ 2/koff; tau = 1/koff
- **Assumes:** kon, koff > 0; simple 1:1 binding
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Kd (dissociation constant) against kon (association rate constant) from 500 m^3/(mol*s) to 1500 m^3/(mol*s). One curve for each value of koff (dissociation rate constant): 0.001 1/s, 0.0005 1/s, 0.002 1/s. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, kon = 1000 m^3/(mol*s), Kd = 1.0000e-06 mol/m^3.



Kd (dissociation constant) over kon (association rate constant) from 500 m^3/(mol*s) to 1500 m^3/(mol*s) and koff (dissociation rate constant) from 0.0005 1/s to 0.0015 1/s: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 3.3333e-07 mol/m^3 to dark 3.0000e-06 mol/m^3); iso-lines at Kd = 5.00e-7, 1.00e-6, 1.50e-6, 2.00e-6, 2.50e-6 mol/m^3. Marked: the worked example, kon = 1000 m^3/(mol*s), koff = 0.001 1/s, Kd = 1.0000e-06 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_kd_from_rates', kon=1000.0, koff=0.001)
# -> Kd = 1e-06 mol/m^3, Kd_nM = 1, Ka_per_M = 1e+09, half_life = 693.147 s, residence_time = 1000 s   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_kd_from_rates', kon=500.0, koff=0.2)
# -> Kd = 0.0004 mol/m^3, Kd_nM = 400, Ka_per_M = 2.5e+06, half_life = 3.46574 s, residence_time = 5 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kd from rate constants: kon = 1000000 1/(M*s), koff = 0.001 1/s	SOLVED	Kd = 1e-06 mol/m^3, Kd_nM = 1, Ka_per_M = 1e+09, half_life = 693.147 s, residence_time = 1000 s
Kd from rate constants: kon = 500000 1/(M*s), koff = 0.2 1/s	SOLVED	Kd = 0.0004 mol/m^3, Kd_nM = 400, Ka_per_M = 2.5e+06, half_life = 3.46574 s, residence_time = 5 s
Kd from rate constants: koff = 0.001 1/s	NEEDS_INPUT	cmb_kd_from_rates needs kon (association rate constant). Example: solve('cmb_kd_from_rates', kon='1000 m^3/(mol*s)', kof
Kd from rate constants: kon = 1000000 1/(M*s), koff = 0.001 1/s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Kd from rate constants: kon = 1e6 1/(M*s), koff = 0.001 1/s	SOLVED	Kd = 1e-06 mol/m^3, Kd_nM = 1, Ka_per_M = 1e+09, half_life = 693.147 s, residence_time = 1000 s

Kd and Bmax from a Scatchard plot, `cmb_scatchard_fit`

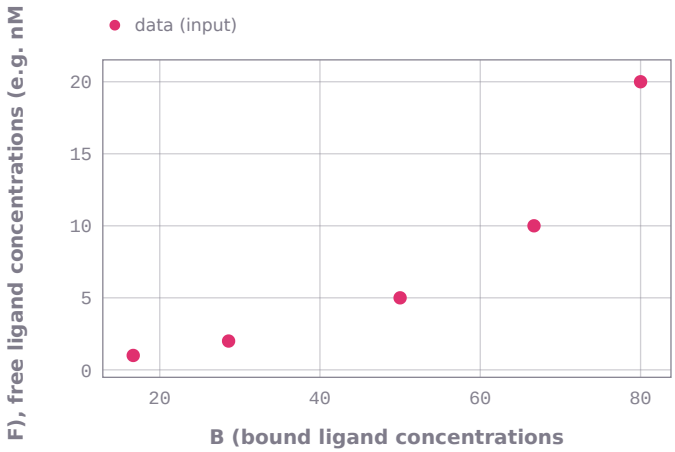
The dissociation constant and the total binding sites from saturation-binding data by the Scatchard least-squares line B/F = Bmax/Kd - B/Kd.

$$\frac{B}{F} = \frac{B_{max}}{K_d} - \frac{B}{K_d}$$

input	meaning	condition
`B`	bound ligand concentrations (any one unit, e.g. nM or fmol/mg)	
`F`	free ligand concentrations (e.g. nM)	

output	meaning
`Kd`	dissociation constant (unit of F)
`Bmax`	total binding sites (unit of B)
`slope`	slope -1/Kd
`intercept`	intercept Bmax/Kd
`r2`	r^2 of the Scatchard line

- **Method:** least squares $B/F = a + m$ B; $Kd = -1/m$; Bmax = -a/m
- **Assumes:** at least 3 points; B, $F > 0$; negative slope (one class of sites)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data F (free ligand concentrations (e.g. nM)) against B (bound ligand concentrations (any one unit, e.g. nM or fmol/mg)) as given in the worked example (5 points). Also returned: Kd = 4.98649, Bmax = 99.9304, slope = -0.200542, intercept = 20.0402, r2 = 0.999997.

```
cell_molecular_biology_ai999x.solve('cmb_scatchard_fit', B=[16.7, 28.6, 50.0, 66.7, 80.0], F=[1.0, 2.0, 5.0, 10.0, 20.0])
# -> Kd = 4.98649, Bmax = 99.9304, slope = -0.200542, intercept = 20.0402, r2 = 0.999997   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_scatchard_fit', B=[9.0, 16.5, 28.0, 41.0], F=[0.5, 1.0, 2.0, 4.0])
# -> Kd = 4.14005, Bmax = 84.432, slope = -0.241543, intercept = 20.394, r2 = 0.992706   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Scatchard fit: B = [16.7, 28.6, 50, 66.7, 80], F = [1, 2, 5, 10, 20]	SOLVED	Kd = 4.98649, Bmax = 99.9304, slope = -0.200542, intercept = 20.0402, r2 = 0.999997
Scatchard fit: B = [9, 16.5, 28, 41], F = [0.5, 1, 2, 4]	SOLVED	Kd = 4.14005, Bmax = 84.432, slope = -0.241543, intercept = 20.394, r2 = 0.992706
Scatchard fit: F = [1, 2, 5, 10, 20]	NEEDS_INPUT	cmb_scatchard_fit needs B (bound ligand concentrations (any one unit, e.g. nM or fmol/mg)). Example: solve('cmb_scatchar
Scatchard fit: B = [16.7, 28.6, 50, 66.7, 80], F = [1, 2, 5, 10, 20], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Standard binding free energy from Kd, `cmb_binding_free_energy`

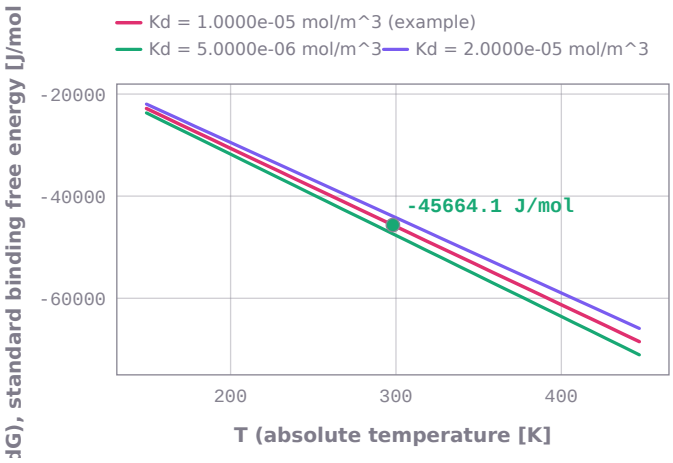
The standard free energy of binding $dG = RT$ ln($Kd/1$ M) (negative for favourable binding) at temperature T.

$$\Delta G^\circ = RT \ln \frac{K_d}{1\ \mathrm{M}}$$

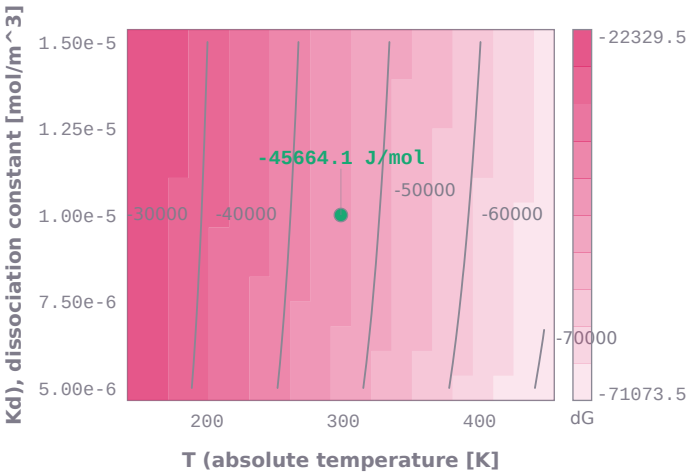
input	meaning	condition
`Kd`	dissociation constant	
`T`	absolute temperature (degC is converted)	

output	meaning
`dG`	standard binding free energy
`dG_kJ_mol`	dG in kJ/mol
`dG_kcal_mol`	dG in kcal/mol

- **Method:** $dG = R$ T ln($Kd/1$ M) = -R T ln Ka
- **Assumes:** Kd, $T > 0$; standard state 1 M
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dG (standard binding free energy) against T (absolute temperature (degC is converted)) from 149.075 K to 447.225 K. One curve for each value of Kd (dissociation constant): 1.0000e-05 mol/m^3, 5.0000e-06 mol/m^3, 2.0000e-05 mol/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, T = 298.15 K, dG = -45664.1 J/mol.



dG (standard binding free energy) over T (absolute temperature (degC is converted)) from 149.075 K to 447.225 K and Kd (dissociation constant) from 5.0000e-06 mol/m^3 to 1.5000e-05 mol/m^3: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light -71073.5 J/mol to dark -22329.5 J/mol); iso-lines at dG = -70000, -60000, -50000, -40000, -30000 J/mol. Marked: the worked example, T = 298.15 K, Kd = 1.0000e-05 mol/m^3, dG = -45664.1 J/mol.

```
cell_molecular_biology_ai999x.solve('cmb_binding_free_energy', Kd=1e-05, T=298.15)
# -> dG = -45664.1 J/mol, dG_kJ_mol = -45.6641, dG_kcal_mol = -10.914   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_binding_free_energy', Kd=0.0025, T=310.15)
# -> dG = -33263.6 J/mol, dG_kJ_mol = -33.2636, dG_kcal_mol = -7.95019   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Binding free energy: Kd = 10 nM, T = 298.15 K	SOLVED	dG = -45664.1 J/mol, dG_kJ_mol = -45.6641, dG_kcal_mol = -10.914
Binding free energy: Kd = 2500 nM, T = 310.15 K	SOLVED	dG = -33263.6 J/mol, dG_kJ_mol = -33.2636, dG_kcal_mol = -7.95019
Binding free energy: T = 298.15 K	NEEDS_INPUT	cmb_binding_free_energy needs Kd (dissociation constant). Example: solve('cmb_binding_free_energy', Kd='1e-05 mol/m^3',
Binding free energy: Kd = 10 nM, T = 298.15 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Binding free energy: Kd = 10 nM, T = 25 degC	SOLVED	dG = -45664.1 J/mol, dG_kJ_mol = -45.6641, dG_kcal_mol = -10.914

Receptor occupancy with a competing ligand, `cmb_competitive_occupancy`

The fractions of a receptor occupied by a ligand and by a competitor binding the same site, the free fraction and the apparent Kd of the ligand in the competitor's presence (no depletion).

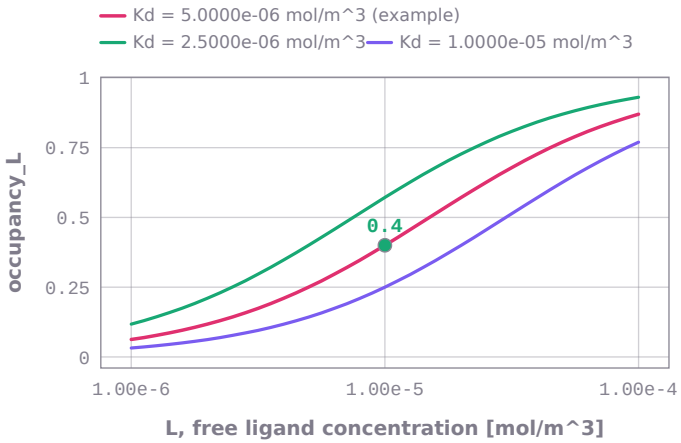
$$\theta_L = \frac{[L]/K_d}{1 + [L]/K_d + [I]/K_i}$$

input	meaning	condition
L	free ligand concentration	
Kd	ligand dissociation constant	

input	meaning	condition
`I`	free competitor concentration	
`Ki`	competitor dissociation constant	

output	meaning
`occupancy_L`	fraction occupied by the ligand
`occupancy_I`	fraction occupied by the competitor
`free_fraction`	fraction of free receptor
`Kd_app`	apparent Kd of the ligand Kd $(1 + I/Ki)$

- **Method:** occupancy_L = (L/Kd) / (1 + L/Kd + I/Ki)
- **Assumes:** Kd, $Ki > 0$; L, $I \geq 0$; one site, mutually exclusive binding, no depletion
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



occupancy_L (fraction occupied by the ligand) against L (free ligand concentration) from 1.0000e-06 mol/m^3 to 0.0001 mol/m^3 on a logarithmic axis. One curve for each value of Kd (ligand dissociation constant): 5.0000e-06 mol/m^3, 2.5000e-06 mol/m^3, 1.0000e-05 mol/m^3. Other inputs as in the worked example: I = 1.9999999999999998e-05 mol/m^3, Ki = 9.999999999999999e-06 mol/m^3. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, L = 1.0000e-05 mol/m^3, occupancy_L = 0.4.

```
cell_molecular_biology_ai999x.solve('cmb_competitive_occupancy', L=1e-05, Kd=5e-06, I=2e-05, Ki=1e-05)
# -> occupancy_L = 0.4, occupancy_I = 0.4, free_fraction = 0.2, Kd_app = 1.5e-05 mol/m^3   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_competitive_occupancy', L=0.0001, Kd=0.0001, I=0.001, Ki=5e-05)
# -> occupancy_L = 0.0454545, occupancy_I = 0.909091, free_fraction = 0.0454545, Kd_app = 0.0021 mol/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Competitive receptor occupancy: L = 10 nM, Kd = 5 nM, I = 20 nM, Ki = 10 nM	SOLVED	occupancy_L = 0.4, occupancy_I = 0.4, free_fraction = 0.2, Kd_app = 1.5e-05 mol/m^3
Competitive receptor occupancy: L = 100 nM, Kd = 100 nM, I = 1000 nM, Ki = 50 nM	SOLVED	occupancy_L = 0.0454545, occupancy_I = 0.909091, free_fraction = 0.0454545, Kd_app = 0.0021 mol/m^3
Competitive receptor occupancy: Kd = 5 nM, I = 20 nM, Ki = 10 nM	NEEDS_INPUT	cmb_competitive_occupancy needs L (free ligand concentration). Example: solve('cmb_competitive_occupancy', L='1e-05 mol/
Competitive receptor occupancy: L = 10 nM, Kd = 5 nM, I = 20 nM, Ki = 10 nM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Fractional saturation in the MWC (concerted) model, `cmb_mwc_saturation`

The fractional saturation and the fraction of molecules in the R state of an allosteric protein with n sites in the Monod-Wyman-Changeux model (L0 = [T]/[R] without ligand, c = KR/KT).

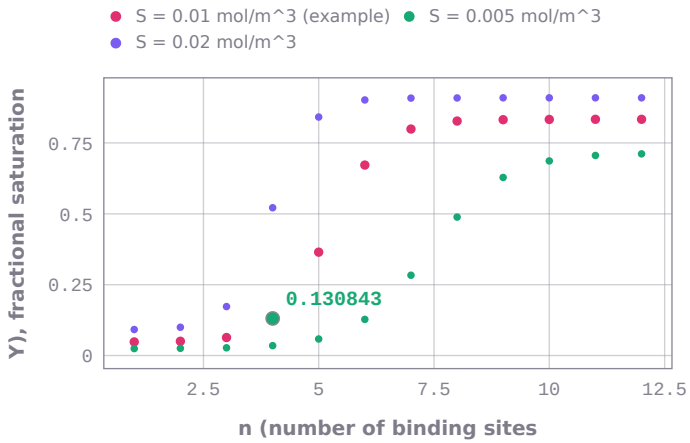
$$Y = \frac{\alpha(1+\alpha)^{n-1} + L_0 c\alpha(1+c\alpha)^{n-1}}{(1+\alpha)^n + L_0(1+c\alpha)^n}$$

input	meaning	condition
`S`	ligand concentration	
`KR`	dissociation constant of a site in the R state	
`c`	ratio KR/KT $(0 < c < 1)$	
`L0`	allosteric constant [T0]/[R0]	
`n`	number of binding sites	

output	meaning
`Y`	fractional saturation

output	meaning
`R_fraction`	fraction of molecules in the R state

- **Method:** $\alpha = S/KR$; Y = [alpha $(1+\alpha)^{n-1} + L_0$ c alpha (1 + c alpha)^(n-1)] / [$(1+\alpha)^n + L_0$ (1 + c alpha)^n]
- **Assumes:** S, KR, c, $L_0 > 0$; $1 \le n \le 12$; concerted R/T transition
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Y (fractional saturation) against n (number of binding sites) from 1 to 12. One curve for each value of S (ligand concentration): 0.01 mol/m^3, 0.005 mol/m^3, 0.02 mol/m^3. Other inputs as in the worked example: KR = 0.002 mol/m^3, c = 0.01, L0 = 9000.0. 12 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, n = 4, Y = 0.130843.

```
cell_molecular_biology_ai999x.solve('cmb_mwc_saturation', S=0.01, KR=0.002, c=0.01, L0=9000.0, n=4)
# -> Y = 0.130843, R_fraction = 0.105921   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_mwc_saturation', S=0.5, KR=0.1, c=0.1, L0=100.0, n=2)
# -> Y = 0.402299, R_fraction = 0.137931   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MWC fractional saturation: S = 10 uM, KR = 2 uM, c = 0.01, L0 = 9000, n = 4	SOLVED	Y = 0.130843, R_fraction = 0.105921
MWC fractional saturation: S = 500 uM, KR = 100 uM, c = 0.1, L0 = 100, n = 2	SOLVED	Y = 0.402299, R_fraction = 0.137931
MWC fractional saturation: KR = 2 uM, c = 0.01, L0 = 9000, n = 4	NEEDS_INPUT	cmb_mwc_saturation needs S (ligand concentration). Example: solve('cmb_mwc_saturation', S='0.01 mol/m^3', KR='0.002 mol/
MWC fractional saturation: S = 10 uM, KR = 2 uM, c = 0.01, L0 = 9000, n = 4, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Rate at another temperature from Q10, `cmb_q10_temperature`

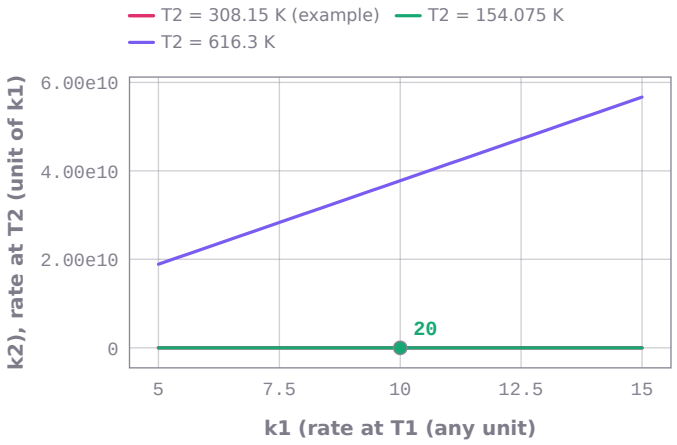
A biological rate at temperature T2 from its value at T1 and the temperature coefficient Q10, and the equivalent Arrhenius activation energy over that interval.

$$k_2 = k_1 Q_{10}^{(T_2 - T_1)/10}, \quad E_a = \frac{R \ln(k_2/k_1)}{1/T_1 - 1/T_2}$$

input	meaning	condition
`k1`	rate at T1 (any unit)	
`T1`	first temperature (degC is converted)	
`T2`	second temperature (degC is converted)	
`Q10`	temperature coefficient Q10	

output	meaning
`k2`	rate at T2 (unit of k1)
`ratio`	k_2/k_1
`Ea`	equivalent activation energy
`Ea_kJ_mol`	activation energy in kJ/mol

- **Method:** $k_2 = k_1$ Q10^((T2 - T1)/10); $Ea = R$ ln(k2/k1) / (1/T1 - 1/T2)
- **Assumes:** k1, $Q10 > 0$; $T_1 \neq T_2$, both > 0 K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



k2 (rate at T2 (unit of k1)) against k1 (rate at T1 (any unit)) from 5 to 15. One curve for each value of T2 (second temperature (degC is converted)): 308.15 K, 154.075 K, 616.3 K. Other inputs as in the worked example: T1 = 298.15 K, Q10 = 2.0. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, k1 = 10, k2 = 20.

```
cell_molecular_biology_ai999x.solve('cmb_q10_temperature', k1=10.0, T1=298.15, T2=308.15, Q10=2.0)
# -> k2 = 20, ratio = 2, Ea = 52948.9 J/mol, Ea_kJ_mol = 52.9489   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_q10_temperature', k1=4.5, T1=310.15, T2=293.15, Q10=2.5)
# -> k2 = 0.947795, ratio = 0.210621, Ea = 69267.4 J/mol, Ea_kJ_mol = 69.2674   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Q10 temperature coefficient: k1 = 10, T1 = 298.15 K, T2 = 308.15 K, Q10 = 2	SOLVED	k2 = 20, ratio = 2, Ea = 52948.9 J/mol, Ea_kJ_mol = 52.9489
Q10 temperature coefficient: k1 = 4.5, T1 = 310.15 K, T2 = 293.15 K, Q10 = 2.5	SOLVED	k2 = 0.947795, ratio = 0.210621, Ea = 69267.4 J/mol, Ea_kJ_mol = 69.2674
Q10 temperature coefficient: T1 = 298.15 K, T2 = 308.15 K, Q10 = 2	NEEDS_INPUT	cmb_q10_temperature needs k1 (rate at T1 (any unit)). Example: solve('cmb_q10_temperature', k1=10, T1='298.15 K', T2='30
Q10 temperature coefficient: k1 = 10, T1 = 298.15 K, T2 = 308.15 K, Q10 = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Rate from Q10: k1 = 10, T1 = 25 degC, T2 = 35 degC, Q10 = 2	SOLVED	k2 = 20, ratio = 2, Ea = 52948.9 J/mol, Ea_kJ_mol = 52.9489

Enzyme activity and specific activity from a spectrophotometric assay, `cmb_enzyme_assay_activity`

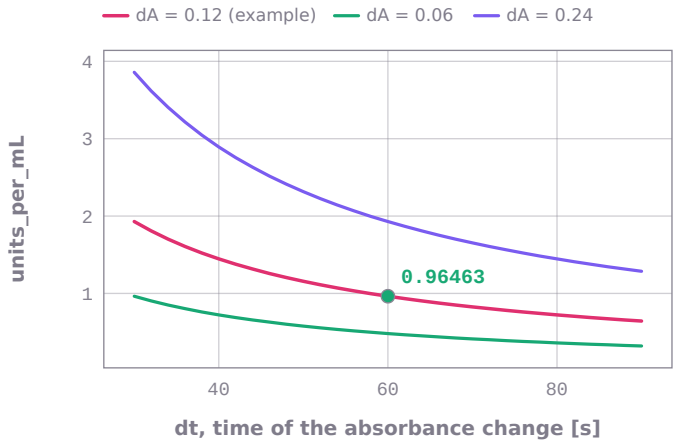
The enzyme activity (units = umol/min) in a cuvette from the absorbance change over a time (Beer-Lambert), the activity per mL of the enzyme sample and, with its protein concentration, the specific activity (U/mg).

$$\mathrm{U/mL} = \frac{(\Delta A/\Delta t)\, V_{assay}}{\varepsilon \ell V_{enz}}$$

input	meaning	condition
`dA`	absorbance change over dt	
`dt`	time of the absorbance change	
`eps`	molar absorptivity of the measured product or substrate (NADH 6220 L/(mol cm))	
`l`	path length	
`V_assay`	total assay volume	
`V_enzyme`	volume of enzyme sample added	
`protein`	protein concentration of the enzyme sample (optional)	

output	meaning
`rate`	rate of concentration change in the cuvette
`units`	activity in the cuvette in umol/min (U)
`units_per_mL`	activity of the enzyme sample in U/mL
`specific_activity`	specific activity in U/mg (when protein is given)

- **Method:** rate = |dA|/dt / (eps l); U = rate x V_assay (umol/min); U/mL = U / V_enzyme; $U/mg = (U/mL)/(mg/mL)$
- **Assumes:** $dA \neq 0$; dt, eps, l, volumes > 0; V_enzyme <= V_assay; linear initial rate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



units_per_mL (activity of the enzyme sample in U/mL) against dt (time of the absorbance change) from 30 s to 90 s. One curve for each value of dA (absorbance change over dt): 0.12, 0.06, 0.24. Other inputs as in the worked example: eps = 622.0 m^2/mol, l = 0.01 m, V_assay = 1e-06 m^3, V_enzyme = 2e-08 m^3, protein = 0.5 kg/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, dt = 60 s, units_per_mL = 0.96463.

```
cell_molecular_biology_ai999x.solve('cmb_enzyme_assay_activity', dA=0.12, dt=60.0, eps=622.0, l=0.01, V_assay=1e-06,
V_enzyme=2e-08, protein=0.5)
# -> rate = 0.000321543 mol/(m^3*s), units = 0.0192926, units_per_mL = 0.96463, specific_activity = 1.92926   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_enzyme_assay_activity', dA=-0.3, dt=120.0, eps=622.0, l=0.01, V_assay=3e-06,
V_enzyme=5e-08)
# -> rate = 0.000401929 mol/(m^3*s), units = 0.0723473, units_per_mL = 1.44695   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Enzyme assay activity: dA = 0.12, dt = 1 min, eps = 6220 L/(mol*cm), l = 1 cm, V_assay = 1 mL, V_enzyme = 20 uL, protein = 0.5 mg/mL	SOLVED	rate = 0.000321543 mol/(m^3*s), units = 0.0192926, units_per_mL = 0.96463, specific_activity = 1.92926
Enzyme assay activity: dA = -0.3, dt = 2 min, eps = 6220 L/(mol*cm), l = 1 cm, V_assay = 3 mL, V_enzyme = 50 uL	SOLVED	rate = 0.000401929 mol/(m^3*s), units = 0.0723473, units_per_mL = 1.44695
Enzyme assay activity: dt = 1 min, eps = 6220 L/(mol*cm), l = 1 cm, V_assay = 1 mL, V_enzyme = 20 uL, protein = 0.5 mg/mL	NEEDS_INPUT	cmb_enzyme_assay_activity needs dA (absorbance change over dt). Example: solve('cmb_enzyme_assay_activity', dA=0.12, dt=
Enzyme assay activity: dA = 0.12, dt = 1 min, eps = 6220 L/(mol*cm), l = 1 cm, V_assay = 1 mL, V_enzyme = 20 uL, protein = 0.5 mg/mL, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Enzyme assay activity: dA = 0.12, dt = 60 s, eps = 6220 L/(mol*cm), l = 1 cm, V_assay = 1 mL, V_enzyme = 20 uL, protein = 0.5 mg/mL	SOLVED	rate = 0.000321543 mol/(m^3*s), units = 0.0192926, units_per_mL = 0.96463, specific_activity = 1.92926

Buffer pH by the Henderson-Hasselbalch equation, `cmb_henderson_hasselbalch`

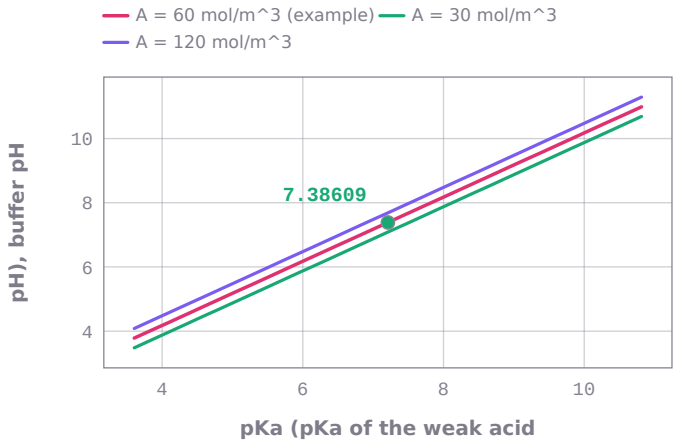
The pH of a buffer from the pKa and the concentrations of the conjugate base and the weak acid.

$$\mathrm{pH} = \mathrm{p}K_a + \log_{10}\frac{[\mathrm{A}^-]}{[\mathrm{HA}]}$$

input	meaning	condition
`pKa`	pKa of the weak acid	
`A`	conjugate base concentration	
`HA`	weak acid concentration	

output	meaning
`pH`	buffer pH
`ratio`	[A-]/[HA]
`fraction_base`	fraction in the base form

- **Method:** pH = pKa + log10([A-]/[HA])
- **Assumes:** A, $HA > 0$; buffer concentrations well above [H+] and [OH-]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



pH (buffer pH) against pKa (pKa of the weak acid) from 3.605 to 10.815. One curve for each value of A (conjugate base concentration): 60 mol/m^3, 30 mol/m^3, 120 mol/m^3. Other inputs as in the worked example: HA = 40.0 mol/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, pKa = 7.21, pH = 7.38609.

```
cell_molecular_biology_ai999x.solve('cmb_henderson_hasselbalch', pKa=7.21, A=60.0, HA=40.0)
# -> pH = 7.38609, ratio = 1.5, fraction_base = 0.6   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_henderson_hasselbalch', pKa=4.76, A=10.0, HA=100.0)
# -> pH = 3.76, ratio = 0.1, fraction_base = 0.0909091   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Henderson-Hasselbalch: pKa = 7.21, A = 60 mM, HA = 40 mM	SOLVED	pH = 7.38609, ratio = 1.5, fraction_base = 0.6
Henderson-Hasselbalch: pKa = 4.76, A = 10 mM, HA = 100 mM	SOLVED	pH = 3.76, ratio = 0.1, fraction_base = 0.0909091
Henderson-Hasselbalch: A = 60 mM, HA = 40 mM	NEEDS_INPUT	cmb_henderson_hasselbalch needs pKa (pKa of the weak acid). Example: solve('cmb_henderson_hasselbalch', pKa=7.21, A='60
Henderson-Hasselbalch: pKa = 7.21, A = 60 mM, HA = 40 mM, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Buffer pH: pKa = 7.21, A = 0.06 M, HA = 40 mM	SOLVED	pH = 7.38609, ratio = 1.5, fraction_base = 0.6

Recipe for a buffer of given pH and concentration, `cmb_buffer_preparation`

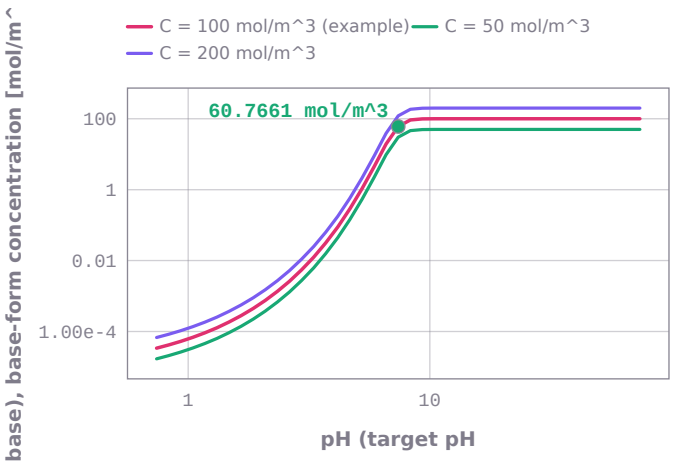
The concentrations, amounts and (with molar masses) masses of the weak acid and its conjugate base needed for a volume V of buffer at total concentration C and a target pH.

$$\frac{[\mathrm{A^-}]}{[\mathrm{HA}]} = 10^{\mathrm{pH}-\mathrm{p}K_a}$$

input	meaning	condition
`C`	total buffer concentration	
`V`	volume of buffer	
`pH`	target pH	
`pKa`	pKa of the buffer acid	
`MW_acid`	molar mass of the acid form (optional)	
`MW_base`	molar mass of the base form (optional)	

output	meaning
`C_base`	base-form concentration
`C_acid`	acid-form concentration
`n_base`	amount of base form
`n_acid`	amount of acid form
`ratio`	[A-]/[HA]
`m_acid_g`	grams of the acid form (when MW_acid is given)
`m_base_g`	grams of the base form (when MW_base is given)

- **Method:** r = 10^(pH - pKa); C_base = C $r/(1+r)$; C_acid = C - C_base; $n = C$ V; $m = n$ M
- **Assumes:** C, $V > 0$; Henderson-Hasselbalch valid (pH within about 1 of pKa)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C_base (base-form concentration) against pH (target pH) from 0.74 to 74 on a logarithmic axis, logarithmic output axis. One curve for each value of C (total buffer concentration): 100 mol/m^3, 50 mol/m^3, 200 mol/m^3. Other inputs as in the worked example: V = 0.001 m^3, pKa = 7.21, MW_acid = 0.13799 kg/mol, MW_base = 0.14196 kg/mol. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, pH = 7.4, C_base = 60.7661 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_buffer_preparation', C=100.0, V=0.001, pH=7.4, pKa=7.21, MW_acid=0.13799,
MW_base=0.14196)
# -> C_base = 60.7661 mol/m^3, C_acid = 39.2339 mol/m^3, n_base = 0.0607661 mol, n_acid = 0.0392339 mol, ratio =
1.54882, m_acid_g = 5.41388, m_base_g = 8.62636   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_buffer_preparation', C=50.0, V=0.0005, pH=4.5, pKa=4.76)
# -> C_base = 17.7324 mol/m^3, C_acid = 32.2676 mol/m^3, n_base = 0.00886619 mol, n_acid = 0.0161338 mol, ratio =
0.549541   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Buffer preparation: C = 100 mM, V = 1000 mL, pH = 7.4, pKa = 7.21, MW_acid = 137.99 g/mol, MW_base = 141.96 g/mol	SOLVED	C_base = 60.7661 mol/m^3, C_acid = 39.2339 mol/m^3, n_base = 0.0607661 mol, n_acid = 0.0392339 mol, ratio = 1.54882, m_acid_g = 5.41388, m_base_g = 8.62636
Buffer preparation: C = 50 mM, V = 500 mL, pH = 4.5, pKa = 4.76	SOLVED	C_base = 17.7324 mol/m^3, C_acid = 32.2676 mol/m^3, n_base = 0.00886619 mol, n_acid = 0.0161338 mol, ratio = 0.549541
Buffer preparation: V = 1000 mL, pH = 7.4, pKa = 7.21, MW_acid = 137.99 g/mol, MW_base = 141.96 g/mol	NEEDS_INPUT	cmb_buffer_preparation needs C (total buffer concentration). Example: solve('cmb_buffer_preparation', C='100 mol/m^3', V
Buffer preparation: C = 100 mM, V = 1000 mL, pH = 7.4, pKa = 7.21, MW_acid = 137.99 g/mol, MW_base = 141.96 g/mol, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Buffer recipe: C = 0.1 M, V = 1 L, pH = 7.4, pKa = 7.21, MW_acid = 137.99 g/mol, MW_base = 141.96 g/mol	SOLVED	C_base = 60.7661 mol/m^3, C_acid = 39.2339 mol/m^3, n_base = 0.0607661 mol, n_acid = 0.0392339 mol, ratio = 1.54882, m_acid_g = 5.41388, m_base_g = 8.62636

Buffer capacity of a weak-acid buffer, `cmb_buffer_capacity`

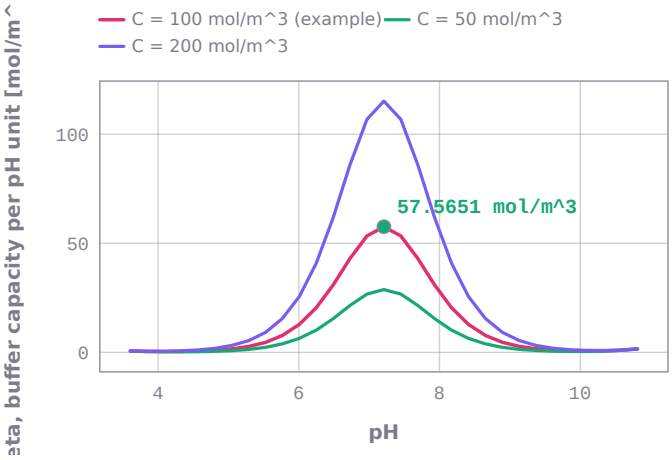
The buffer capacity beta = dCb/dpH (strong base per unit pH change) of a monoprotic buffer of total concentration C at a pH, including water, and the buffer's own contribution (25 degC, Kw = 1e-14).

$$\beta = 2.303\left([\mathrm{H^+}] + \frac{K_w}{[\mathrm{H^+}]} + \frac{C K_a [\mathrm{H^+}]}{(K_a + [\mathrm{H^+}])^2}\right)$$

input	meaning	condition
`C`	total buffer concentration	
`pKa`	pKa of the buffer acid	
`pH`	pH	

output	meaning
`beta`	buffer capacity per pH unit
`beta_mM`	buffer capacity in mM per pH unit
`beta_buffer_mM`	contribution of the buffer alone in mM per pH unit

- **Method:** $\beta = 2.303$ ([H+] + Kw/[H+] + C Ka [H+] / (Ka + [H+])^2)
- **Assumes:** $C > 0$; monoprotic acid; $Kw = 1e-14$ (25 degC), ideal solution
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



beta (buffer capacity per pH unit) against pH (pH) from 3.605 to 10.815. One curve for each value of C (total buffer concentration): 100 mol/m^3, 50 mol/m^3, 200 mol/m^3. Other inputs as in the worked example: pKa = 7.21. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, pH = 7.21, beta = 57.5651 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_buffer_capacity', C=100.0, pKa=7.21, pH=7.21)
# -> beta = 57.5651 mol/m^3, beta_mM = 57.5651, beta_buffer_mM = 57.5646   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_buffer_capacity', C=20.0, pKa=4.76, pH=5.5)
# -> beta = 6.00565 mol/m^3, beta_mM = 6.00565, beta_buffer_mM = 5.99836   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Buffer capacity: C = 100 mM, pKa = 7.21, pH = 7.21	SOLVED	beta = 57.5651 mol/m^3, beta_mM = 57.5651, beta_buffer_mM = 57.5646
Buffer capacity: C = 20 mM, pKa = 4.76, pH = 5.5	SOLVED	beta = 6.00565 mol/m^3, beta_mM = 6.00565, beta_buffer_mM = 5.99836
Buffer capacity: pKa = 7.21, pH = 7.21	NEEDS_INPUT	cmb_buffer_capacity needs C (total buffer concentration). Example: solve('cmb_buffer_capacity', C='100 mol/m^3', pKa=7.2
Buffer capacity: C = 100 mM, pKa = 7.21, pH = 7.21, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Buffer pH after adding strong acid or base, `cmb_buffer_after_addition`

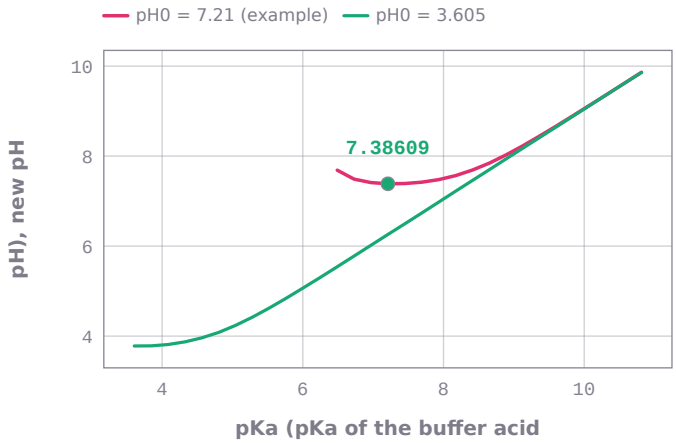
The pH of a buffer (total concentration C, volume V, initial pH0) after adding an amount of strong acid or strong base, by Henderson-Hasselbalch on the new amounts.

$$\mathrm{pH} = \mathrm{p}K_a + \log_{10}\frac{n_{A^-} \pm n}{n_{HA} \mp n}$$

input	meaning	condition
`C`	total buffer concentration	
`V`	buffer volume	
`pH0`	initial pH	
`pKa`	pKa of the buffer acid	
`n_add`	amount of strong acid or base added	
`added`	strong acid or strong base	

output	meaning
`pH`	new pH
`delta_pH`	change in pH
`n_base`	base form after the addition
`n_acid`	acid form after the addition

- **Method:** split C V by the initial ratio 10^(pH0 - pKa); base converts HA to A- (acid the reverse); pH = pKa + log10(n_A/n_HA)
- **Assumes:** C, V, n_add > 0; the addition smaller than the form it consumes; Henderson-Hasselbalch valid
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



pH (new pH) against pKa (pKa of the buffer acid) from 3.605 to 10.815. One curve for each value of pH0 (initial pH): 7.21, 3.605. Other inputs as in the worked example: C = 100.0 mol/m^3, V = 9.999999999999999e-05 m^3, n_add = 0.001 mol, added = base. 19 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, pKa = 7.21, pH = 7.38609.

```
cell_molecular_biology_ai999x.solve('cmb_buffer_after_addition', C=100.0, V=0.0001, pH0=7.21, pKa=7.21, n_add=0.001,
added='base')
# -> pH = 7.38609, delta_pH = 0.176091, n_base = 0.006 mol, n_acid = 0.004 mol   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_buffer_after_addition', C=50.0, V=0.0005, pH0=4.76, pKa=4.76, n_add=0.002,
added='acid')
# -> pH = 4.61982, delta_pH = -0.140179, n_base = 0.0105 mol, n_acid = 0.0145 mol   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Buffer pH after adding acid or base: C = 100 mM, V = 100 mL, pH0 = 7.21, pKa = 7.21, n_add = 1 mmol, added = base	SOLVED	pH = 7.38609, delta_pH = 0.176091, n_base = 0.006 mol, n_acid = 0.004 mol
Buffer pH after adding acid or base: C = 50 mM, V = 500 mL, pH0 = 4.76, pKa = 4.76, n_add = 2 mmol, added = acid	SOLVED	pH = 4.61982, delta_pH = -0.140179, n_base = 0.0105 mol, n_acid = 0.0145 mol
Buffer pH after adding acid or base: V = 100 mL, pH0 = 7.21, pKa = 7.21, n_add = 1 mmol, added = base	NEEDS_INPUT	cmb_buffer_after_addition needs C (total buffer concentration). Example: solve('cmb_buffer_after_addition', C='100 mol/m
Buffer pH after adding acid or base: C = 100 mM, V = 100 mL, pH0 = 7.21, pKa = 7.21, n_add = 1 mmol, added = base, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Buffer after addition: C = 100 mM, V = 100 mL, pH0 = 7.21, pKa = 7.21, n_add = 1 mmol, added = strong base	INVALID_INPUT	added: one of acid, base; got 'strong base'.
Buffer after addition: C = 100 mM, V = 100 mL, pH0 = 7.21, pKa = 7.21, n_add = 1 mmol, added = Base	SOLVED	pH = 7.38609, delta_pH = 0.176091, n_base = 0.006 mol, n_acid = 0.004 mol

Dilution of a stock solution (C1 V1 = C2 V2), `cmb_c1v1_dilution`

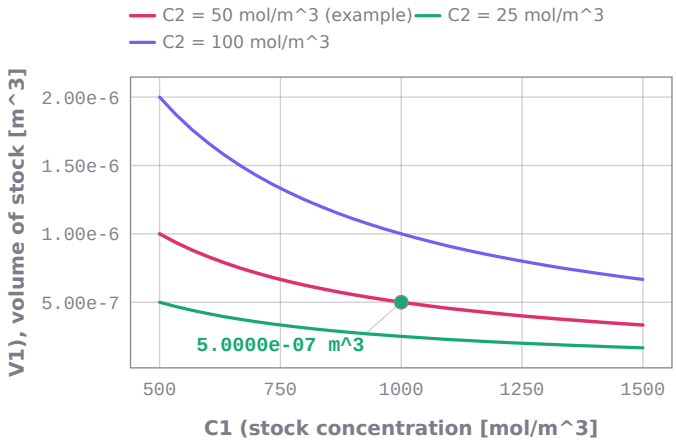
The volume of stock and of diluent needed to make a volume V2 at concentration C2 from a stock at C1.

$$C_1 V_1 = C_2 V_2$$

input	meaning	condition
`C1`	stock concentration	
`C2`	final concentration	
`V2`	final volume	

output	meaning
`V1`	volume of stock
`V1_uL`	volume of stock in uL
`V_diluent`	volume of diluent
`dilution_factor`	dilution factor C_1/C_2

- **Method:** $V_1 = C_2\,V_2/C_1$; diluent = V2 - V1
- **Assumes:** C1, C2, $V_2 > 0$; $C_2 \le C_1$; volumes additive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V1 (volume of stock) against C1 (stock concentration) from 500 mol/m^3 to 1500 mol/m^3. One curve for each value of C2 (final concentration): 50 mol/m^3, 25 mol/m^3, 100 mol/m^3. Other inputs as in the worked example: V2 = 9.999999999999999e-06 m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, C1 = 1000 mol/m^3, V1 = 5.0000e-07 m^3.

```
cell_molecular_biology_ai999x.solve('cmb_c1v1_dilution', C1=1000.0, C2=50.0, V2=1e-05)
# -> V1 = 5e-07 m^3, V1_uL = 500, V_diluent = 9.5e-06 m^3, dilution_factor = 20   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_c1v1_dilution', C1=5000.0, C2=150.0, V2=0.0005)
# -> V1 = 1.5e-05 m^3, V1_uL = 15000, V_diluent = 0.000485 m^3, dilution_factor = 33.3333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
C1V1 dilution: C1 = 1000 mM, C2 = 50 mM, V2 = 10 mL	SOLVED	V1 = 5e-07 m^3, V1_uL = 500, V_diluent = 9.5e-06 m^3, dilution_factor = 20
C1V1 dilution: C1 = 5000 mM, C2 = 150 mM, V2 = 500 mL	SOLVED	V1 = 1.5e-05 m^3, V1_uL = 15000, V_diluent = 0.000485 m^3, dilution_factor = 33.3333
C1V1 dilution: C2 = 50 mM, V2 = 10 mL	NEEDS_INPUT	cmb_c1v1_dilution needs C1 (stock concentration). Example: solve('cmb_c1v1_dilution', C1='1000 mol/m^3', C2='50 mol/m^3'
C1V1 dilution: C1 = 1000 mM, C2 = 50 mM, V2 = 10 mL, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
C1V1 dilution: C1 = 1 M, C2 = 50 mM, V2 = 10 mL	SOLVED	V1 = 5e-07 m^3, V1_uL = 500, V_diluent = 9.5e-06 m^3, dilution_factor = 20

Serial dilution series, `cmb_serial_dilution`

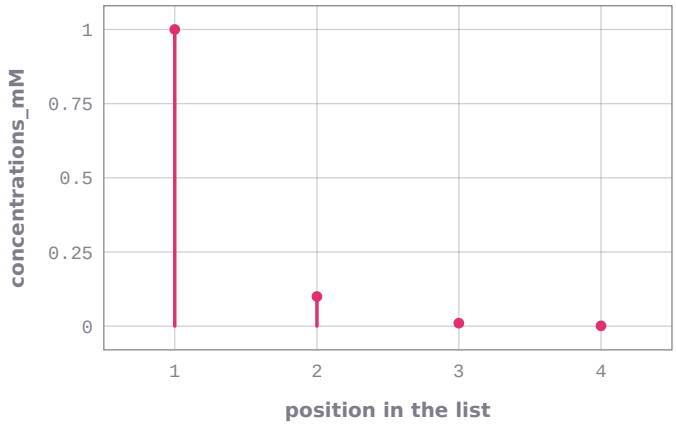
The concentrations after each step of a serial dilution with a constant factor, the final concentration and total dilution, and (with the tube volume) the transfer and diluent volumes per step.

$$C_k = \frac{C_0}{f^k}$$

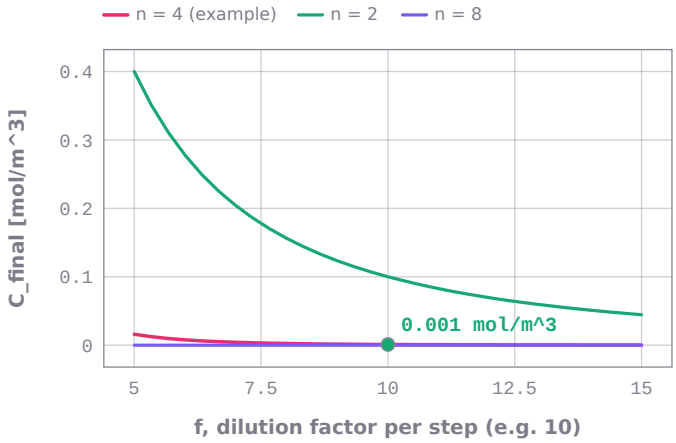
input	meaning	condition
`C0`	starting concentration	
`f`	dilution factor per step (e.g. 10)	
`n`	number of dilution steps	
`V`	final volume in each tube (optional)	

output	meaning
`C_final`	concentration after n steps
`total_dilution`	total dilution factor f^n
`concentrations_mM`	concentration after each step in mM
`transfer`	volume carried to the next tube (when V is given)
`diluent`	diluent per tube (when V is given)

- **Method:** $C_k = C_0 / f^k$; transfer = V/f, diluent = V - V/f
- **Assumes:** $C_0 > 0$; $f > 1$; $1 \le n \le 30$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



concentration after each step in mM (concentrations_mM) returned by cell-molecular-biology-ai999x for the worked example, one value for each position in the list (4 values, first 1, last 0.001, largest 1). Inputs: C0 = 10.0 mol/m^3, f = 10.0, n = 4.



C_final (concentration after n steps) against f (dilution factor per step (e.g. 10)) from 5 to 15. One curve for each value of n (number of dilution steps): 4, 2, 8. Other inputs as in the worked example: C0 = 10.0 mol/m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, f = 10, C_final = 0.001 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_serial_dilution', C0=10.0, f=10.0, n=4)
# -> C_final = 0.001 mol/m^3, total_dilution = 10000, concentrations_mM = [1.0, 0.1, 0.01, 0.001]   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_serial_dilution', C0=1.0, f=2.0, n=6, V=1e-07)
# -> C_final = 0.015625 mol/m^3, total_dilution = 64, concentrations_mM = [0.5, 0.25, 0.125, 0.0625, 0.03125,
0.015625], transfer = 5e-08 m^3, diluent = 5e-08 m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Serial dilution: C0 = 10 mM, f = 10, n = 4 steps	SOLVED	C_final = 0.001 mol/m^3, total_dilution = 10000, concentrations_mM = [1.0, 0.1, 0.01, 0.001]
Serial dilution: C0 = 1 mM, f = 2, n = 6 steps, V = 100 uL	SOLVED	C_final = 0.015625 mol/m^3, total_dilution = 64, concentrations_mM = [0.5, 0.25, 0.125, 0.0625, 0.03125, 0.015625], transfer = 5e-08 m^3, diluent = 5e-08 m^3
Serial dilution: f = 10, n = 4 steps	NEEDS_INPUT	cmb_serial_dilution needs C0 (starting concentration). Example: solve('cmb_serial_dilution', C0='10 mol/m^3', f=10, n=4)
Serial dilution: C0 = 10 mM, f = 10, n = 4 steps, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Serial dilution: C0 = 10 mM, f = 10, n = 4 steps	SOLVED	C_final = 0.001 mol/m^3, total_dilution = 10000, concentrations_mM = [1.0, 0.1, 0.01, 0.001]

Mass of solute for a solution of given molarity, `cmb_mass_for_molarity`

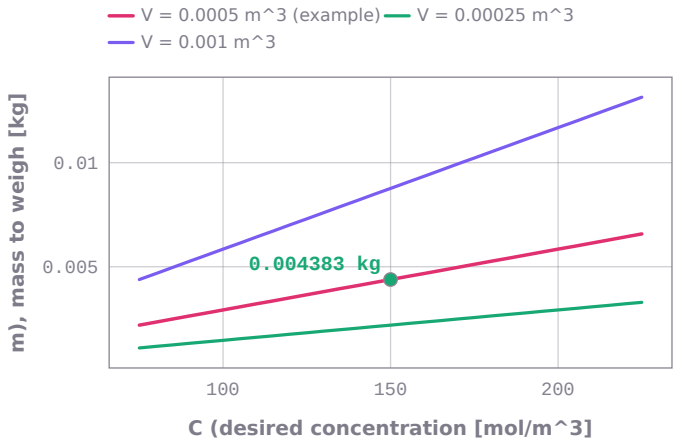
The mass of a compound to weigh out for a volume V of solution at molar concentration C.

$$m = C\,V\,M$$

input	meaning	condition
`C`	desired concentration	
`V`	volume of solution	
`MW`	molar mass of the compound	

output	meaning
`m`	mass to weigh
`m_g`	mass in grams
`m_mg`	mass in milligrams
`n`	amount of substance

- **Method:** $m = C$ V MW
- **Assumes:** C, V, $MW > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



m (mass to weigh) against C (desired concentration) from 75 mol/m^3 to 225 mol/m^3. One curve for each value of V (volume of solution): 0.0005 m^3, 0.00025 m^3, 0.001 m^3. Other inputs as in the worked example: MW = 0.05844 kg/mol. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, C = 150 mol/m^3, m = 0.004383 kg.

```
cell_molecular_biology_ai999x.solve('cmb_mass_for_molarity', C=150.0, V=0.0005, MW=0.05844)
# -> m = 0.004383 kg, m_g = 4.383, m_mg = 4383, n = 0.075 mol   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_mass_for_molarity', C=0.5, V=0.0001, MW=66.43)
# -> m = 0.0033215 kg, m_g = 3.3215, m_mg = 3321.5, n = 5e-05 mol   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mass for a molar solution: C = 150 mM, V = 500 mL, MW = 58.44 g/mol	SOLVED	m = 0.004383 kg, m_g = 4.383, m_mg = 4383, n = 0.075 mol
Mass for a molar solution: C = 0.5 mM, V = 100 mL, MW = 66430 g/mol	SOLVED	m = 0.0033215 kg, m_g = 3.3215, m_mg = 3321.5, n = 5e-05 mol
Mass for a molar solution: V = 500 mL, MW = 58.44 g/mol	NEEDS_INPUT	cmb_mass_for_molarity needs C (desired concentration). Example: solve('cmb_mass_for_molarity', C='150 mol/m^3', V='0.000
Mass for a molar solution: C = 150 mM, V = 500 mL, MW = 58.44 g/mol, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Mass for molarity: C = 0.15 M, V = 500 mL, MW = 58.44 g/mol	SOLVED	m = 0.004383 kg, m_g = 4.383, m_mg = 4383, n = 0.075 mol

Molar concentration from mass and volume, `cmb_molarity_from_mass`

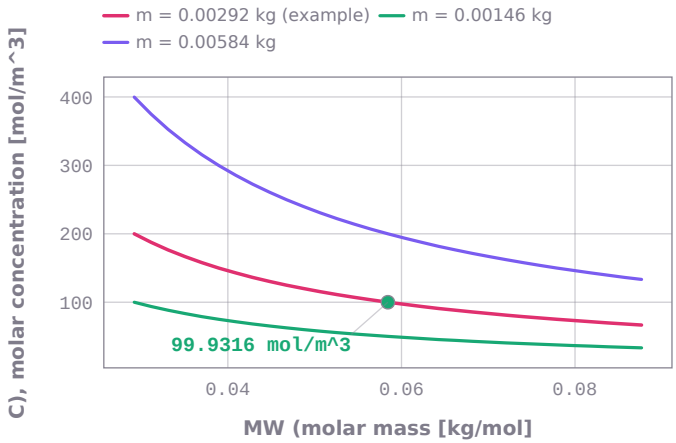
The molar concentration of a solution made by dissolving a mass of compound to a final volume V.

$$C = \frac{m}{MV}$$

input	meaning	condition
`m`	mass of solute	
`MW`	molar mass	
`V`	final volume	

output	meaning
`C`	molar concentration
`C_mM`	concentration in mM
`C_M`	concentration in M
`n`	amount of substance

- **Method:** C = m / (MW V)
- **Assumes:** m, MW, $V > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (molar concentration) against MW (molar mass) from 0.02922 kg/mol to 0.08766 kg/mol. One curve for each value of m (mass of solute): 0.00292 kg, 0.00146 kg, 0.00584 kg. Other inputs as in the worked example: V = 0.0005 m^3. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, MW = 0.05844 kg/mol, C = 99.9316 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_molarity_from_mass', m=0.00292, MW=0.05844, V=0.0005)
# -> C = 99.9316 mol/m^3, C_mM = 99.9316, C_M = 0.0999316, n = 0.0499658 mol   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_molarity_from_mass', m=1e-05, MW=66.43, V=1e-06)
# -> C = 0.150534 mol/m^3, C_mM = 0.150534, C_M = 0.000150534, n = 1.50534e-07 mol   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Molarity from mass: m = 2920 mg, MW = 58.44 g/mol, V = 500 mL	SOLVED	C = 99.9316 mol/m^3, C_mM = 99.9316, C_M = 0.0999316, n = 0.0499658 mol
Molarity from mass: m = 10 mg, MW = 66430 g/mol, V = 1 mL	SOLVED	C = 0.150534 mol/m^3, C_mM = 0.150534, C_M = 0.000150534, n = 1.50534e-07 mol
Molarity from mass: MW = 58.44 g/mol, V = 500 mL	NEEDS_INPUT	cmb_molarity_from_mass needs m (mass of solute). Example: solve('cmb_molarity_from_mass', m='0.00292 kg', MW='0.05844 kg
Molarity from mass: m = 2920 mg, MW = 58.44 g/mol, V = 500 mL, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Molarity from mass: m = 2.92 g, MW = 58.44 g/mol, V = 500 mL	SOLVED	C = 99.9316 mol/m^3, C_mM = 99.9316, C_M = 0.0999316, n = 0.0499658 mol

Molarity of a per cent (w/v) solution, `cmb_percent_wv_to_molarity`

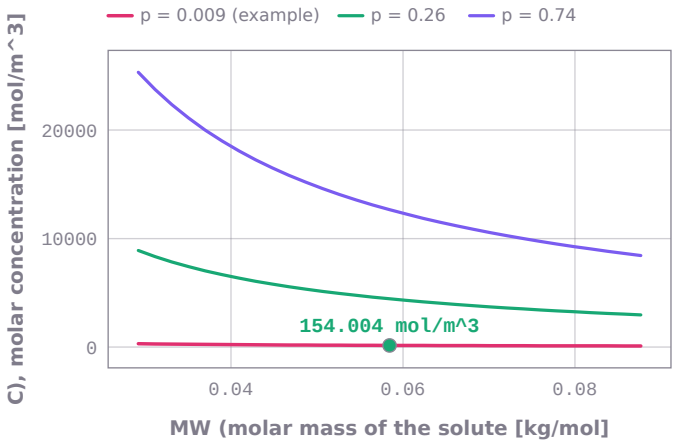
The molar concentration and grams per litre of a solution given in per cent weight per volume (g per 100 mL), e.g. 0.9 % NaCl.

$$C = \frac{10 \times (\%\, w/v)}{M}$$

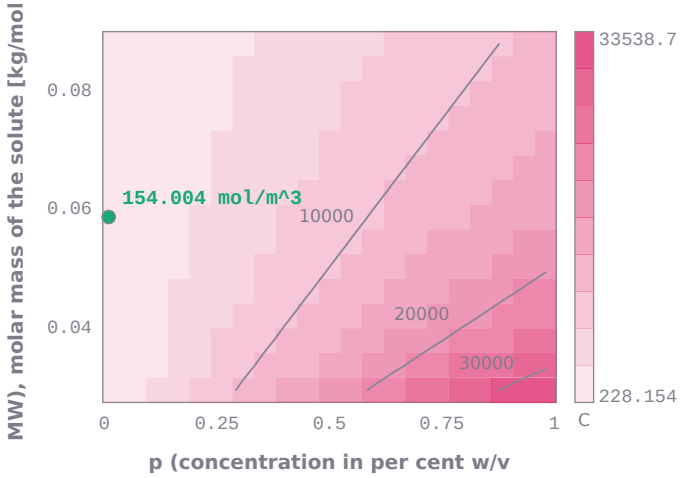
input	meaning	condition
`p`	concentration in per cent w/v (written with %, e.g. 0.9%)	
`MW`	molar mass of the solute	

output	meaning
`C`	molar concentration
`C_mM`	concentration in mM
`C_M`	concentration in M
`mass_concentration`	mass concentration
`g_per_L`	grams per litre

- **Method:** $g/L = 10$ x (% w/v); $C = (g/L)/MW$
- **Assumes:** $0 < p < 100$ %; $MW > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (molar concentration) against MW (molar mass of the solute) from 0.02922 kg/mol to 0.08766 kg/mol. One curve for each value of p (concentration in per cent w/v (written with %, e.g. 0.9%)): 0.009, 0.26, 0.74. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, MW = 0.05844 kg/mol, C = 154.004 mol/m^3.



C (molar concentration) over p (concentration in per cent w/v (written with %, e.g. 0.9%)) from 0.02 to 0.98 and MW (molar mass of the solute) from 0.02922 kg/mol to 0.08766 kg/mol: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 228.154 mol/m^3 to dark 33538.7 mol/m^3); iso-lines at C = 10000, 20000, 30000 mol/m^3. Marked: the worked example, p = 0.009, MW = 0.05844 kg/mol, C = 154.004 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_percent_wv_to_molarity', p=0.009, MW=0.05844)
# -> C = 154.004 mol/m^3, C_mM = 154.004, C_M = 0.154004, mass_concentration = 9 kg/m^3, g_per_L = 9   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_percent_wv_to_molarity', p=0.05, MW=0.18016)
# -> C = 277.531 mol/m^3, C_mM = 277.531, C_M = 0.277531, mass_concentration = 50 kg/m^3, g_per_L = 50   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Per cent w/v to molarity: p = 0.9%, MW = 58.44 g/mol	SOLVED	C = 154.004 mol/m^3, C_mM = 154.004, C_M = 0.154004, mass_concentration = 9 kg/m^3, g_per_L = 9
Per cent w/v to molarity: p = 5%, MW = 180.16 g/mol	SOLVED	C = 277.531 mol/m^3, C_mM = 277.531, C_M = 0.277531, mass_concentration = 50 kg/m^3, g_per_L = 50
Per cent w/v to molarity: MW = 58.44 g/mol	NEEDS_INPUT	cmb_percent_wv_to_molarity needs p (concentration in per cent w/v (written with %, e.g. 0.9%)). Example: solve('cmb_perc
Per cent w/v to molarity: p = 0.9%, MW = 58.44 g/mol, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Percent w/v to molarity: p = 0.9%, MW = 58.44 g/mol	SOLVED	C = 154.004 mol/m^3, C_mM = 154.004, C_M = 0.154004, mass_concentration = 9 kg/m^3, g_per_L = 9
Percent w/v to molarity: p = 0.9, MW = 58.44 g/mol	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Ionic strength of a solution, `cmb_ionic_strength`

The ionic strength $I = (1/2)$ sum c z^2 of a solution from the concentration and charge of each ion.

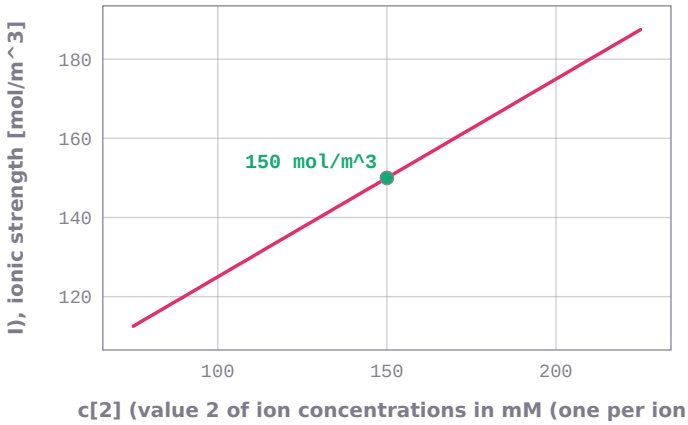
$$I = \frac{1}{2} \sum_i c_i z_i^2$$

input	meaning	condition
c	ion concentrations in mM (one per ion)	

input	meaning	condition
`z`	charge of each ion (e.g. 1, -1, 2)	

output	meaning
`I`	ionic strength
`I_mM`	ionic strength in mM
`I_M`	ionic strength in M

- **Method:** $I = 1/2$ sum c_i z_i^2
- **Assumes:** one charge per concentration; charges non-zero whole numbers
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



I (ionic strength) as value 2 of c (ion concentrations in mM (one per ion)) moves from 75 to 225, the other values as given ([150.0, 150.0]); 41 runs of cell-molecular-biology-ai999x. Other inputs: z = [1.0, -1.0]. Marked: the worked example, c[2] = 150, I = 150 mol/m^3.

```
cell_molecular_biology_ai999x.solve('cmb_ionic_strength', c=[150.0, 150.0], z=[1.0, -1.0])
# -> I = 150 mol/m^3, I_mM = 150, I_M = 0.15   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_ionic_strength', c=[10.0, 20.0, 5.0], z=[2.0, -1.0, 1.0])
# -> I = 32.5 mol/m^3, I_mM = 32.5, I_M = 0.0325   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ionic strength: c = [150, 150], z = [1, -1]	SOLVED	I = 150 mol/m^3, I_mM = 150, I_M = 0.15
Ionic strength: c = [10, 20, 5], z = [2, -1, 1]	SOLVED	I = 32.5 mol/m^3, I_mM = 32.5, I_M = 0.0325
Ionic strength: z = [1, -1]	NEEDS_INPUT	cmb_ionic_strength needs c (ion concentrations in mM (one per ion)). Example: solve('cmb_ionic_strength', c=[150.0, 150.
Ionic strength: c = [150, 150], z = [1, -1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Relative centrifugal force from rotor speed, `cmb_centrifuge_rcf`

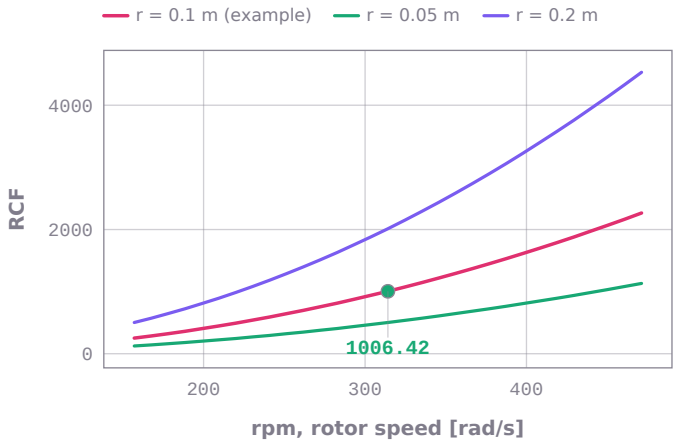
The relative centrifugal force (x g) at radius r for a rotor speed in rpm.

$$\mathrm{RCF} = \frac{\omega^2 r}{g} = 1.118 \times 10^{-5}\, r_{cm}\, N_{rpm}^2$$

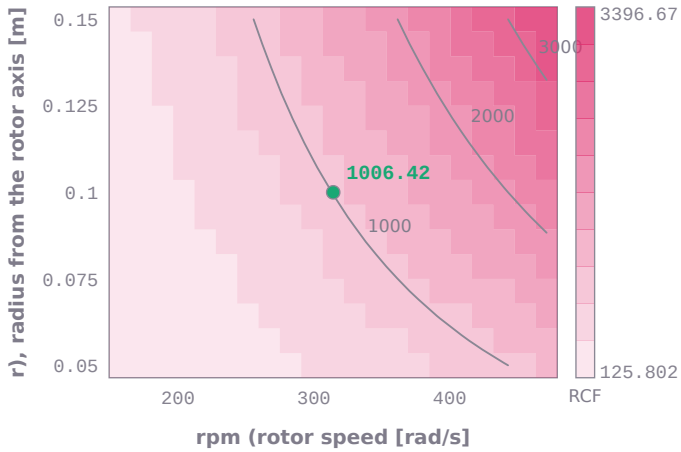
input	meaning	condition
`rpm`	rotor speed	
`r`	radius from the rotor axis	

output	meaning
`RCF`	relative centrifugal force in multiples of g
`rpm_value`	rotor speed in rpm

- **Method:** RCF = omega^2 $r/g = 1.118e-5$ r(cm) rpm^2
- **Assumes:** rpm, $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



RCF (relative centrifugal force in multiples of g) against rpm (rotor speed) from 157.08 rad/s to 471.239 rad/s. One curve for each value of r (radius from the rotor axis): 0.1 m, 0.05 m, 0.2 m. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, rpm = 314.159 rad/s, RCF = 1006.42.



RCF (relative centrifugal force in multiples of g) over rpm (rotor speed) from 157.08 rad/s to 471.239 rad/s and r (radius from the rotor axis) from 0.05 m to 0.15 m: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 125.802 to dark 3396.67); iso-lines at RCF = 1000, 2000, 3000. Marked: the worked example, rpm = 314.159 rad/s, r = 0.1 m, RCF = 1006.42.

```
cell_molecular_biology_ai999x.solve('cmb_centrifuge_rcf', rpm=314.1592653589793, r=0.1)
# -> RCF = 1006.42, rpm_value = 3000   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_centrifuge_rcf', rpm=1466.0765716752367, r=0.084)
# -> RCF = 18410.8, rpm_value = 14000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Relative centrifugal force: rpm = 3000 rpm, r = 10 cm	SOLVED	RCF = 1006.42, rpm_value = 3000
Relative centrifugal force: rpm = 14000 rpm, r = 8.4 cm	SOLVED	RCF = 18410.8, rpm_value = 14000
Relative centrifugal force: r = 10 cm	NEEDS_INPUT	cmb_centrifuge_rcf needs rpm (rotor speed). Example: solve('cmb_centrifuge_rcf', rpm='314.159 rad/s', r='0.1 m').
Relative centrifugal force: rpm = 3000 rpm, r = 10 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem

Rotor speed for a required centrifugal force, `cmb_centrifuge_rpm`

The rotor speed (rpm) that gives a required relative centrifugal force (x g) at radius r.

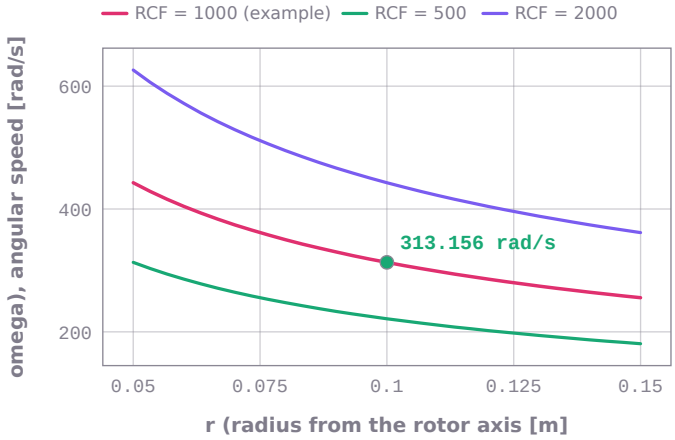
$$N_{rpm} = \frac{60}{2\pi}\sqrt{\frac{\mathrm{RCF}\,g}{r}}$$

input	meaning	condition
`RCF`	required relative centrifugal force in multiples of g	
`r`	radius from the rotor axis	

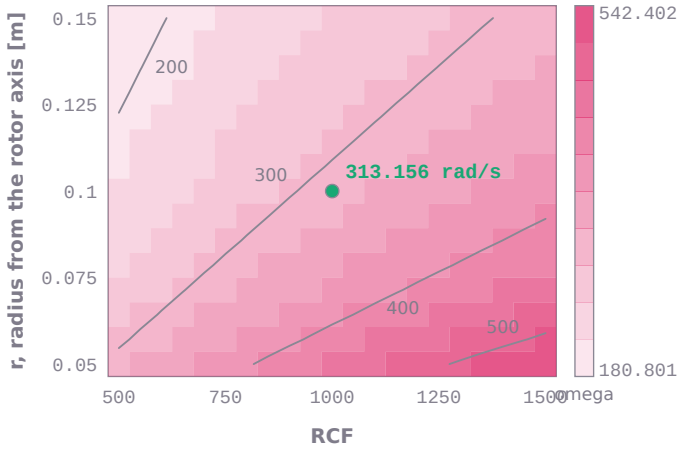
output	meaning
`omega`	angular speed

output	meaning
`rpm_value`	rotor speed in rpm

- **Method:** rpm = sqrt(RCF g / r) x 60 / (2 pi)
- **Assumes:** RCF, $r > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



omega (angular speed) against r (radius from the rotor axis) from 0.05 m to 0.15 m. One curve for each value of RCF (required relative centrifugal force in multiples of g): 1000, 500, 2000. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, r = 0.1 m, omega = 313.156 rad/s.



omega (angular speed) over RCF (required relative centrifugal force in multiples of g) from 500 to 1500 and r (radius from the rotor axis) from 0.05 m to 0.15 m: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 180.801 rad/s to dark 542.402 rad/s); iso-lines at omega = 200, 300, 400, 500 rad/s. Marked: the worked example, RCF = 1000, r = 0.1 m, omega = 313.156 rad/s.

```
cell_molecular_biology_ai999x.solve('cmb_centrifuge_rpm', RCF=1000.0, r=0.1)
# -> omega = 313.156 rad/s, rpm_value = 2990.42   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_centrifuge_rpm', RCF=16000.0, r=0.084)
# -> omega = 1366.72 rad/s, rpm_value = 13051.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Centrifuge speed for an RCF: RCF = 1000 g, r = 10 cm	SOLVED	omega = 313.156 rad/s, rpm_value = 2990.42
Centrifuge speed for an RCF: RCF = 16000 g, r = 8.4 cm	SOLVED	omega = 1366.72 rad/s, rpm_value = 13051.2
Centrifuge speed for an RCF: r = 10 cm	NEEDS_INPUT	cmb_centrifuge_rpm needs RCF (required relative centrifugal force in multiples of g). Example: solve('cmb_centrifuge_rpm
Centrifuge speed for an RCF: RCF = 1000 g, r = 10 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Rotor speed for an RCF: RCF = 1000 g, r = 10 cm	SOLVED	omega = 313.156 rad/s, rpm_value = 2990.42

Time to pellet a particle in a centrifuge (k-factor), `cmb_sedimentation_time`

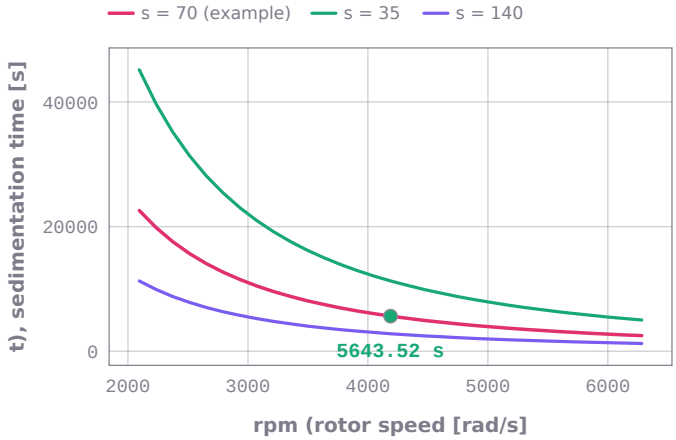
The time to sediment a particle of sedimentation coefficient s from r_min to r_max at a rotor speed, and the rotor's k-factor (t in hours = k / s).

$$t = \frac{\ln(r_{max}/r_{min})}{\omega^2 s}, \quad k = \frac{\ln(r_{max}/r_{min})}{\omega^2} \cdot \frac{10^{13}}{3600}$$

input	meaning	condition
`s`	sedimentation coefficient in svedbergs	
`rpm`	rotor speed	
`r_min`	starting (minimum) radius	
`r_max`	radius of the pellet (maximum radius)	

output	meaning
`t`	sedimentation time
`t_h`	time in hours
`k_factor`	k-factor $(t_h = k/s)$

- **Method:** $dr/dt = s\,\omega^2$ r: $t = ln$(r_max/r_min) / (omega^2 s); $k = ln$(r_max/r_min) / omega^2 x $1e13/3600$
- **Assumes:** s, rpm > 0; 0 < r_min < r_max; constant s (no diffusion)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (sedimentation time) against rpm (rotor speed) from 2094.4 rad/s to 6283.19 rad/s. One curve for each value of s (sedimentation coefficient in svedbergs): 70, 35, 140. Other inputs as in the worked example: r_min = 0.05 m, r_max = 0.1 m. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, rpm = 4188.79 rad/s, t = 5643.52 s.

```
cell_molecular_biology_ai999x.solve('cmb_sedimentation_time', s=70.0, rpm=4188.790204786391, r_min=0.05, r_max=0.1)
# -> t = 5643.52 s, t_h = 1.56764, k_factor = 109.735   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_sedimentation_time', s=4.4, rpm=6283.185307179586, r_min=0.06, r_max=0.085)
# -> t = 20051.6 s, t_h = 5.56989, k_factor = 24.5075   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sedimentation time: s = 70 S, rpm = 40000 rpm, r_min = 5 cm, r_max = 10 cm	SOLVED	t = 5643.52 s, t_h = 1.56764, k_factor = 109.735
Sedimentation time: s = 4.4 S, rpm = 60000 rpm, r_min = 6 cm, r_max = 8.5 cm	SOLVED	t = 20051.6 s, t_h = 5.56989, k_factor = 24.5075
Sedimentation time: rpm = 40000 rpm, r_min = 5 cm, r_max = 10 cm	NEEDS_INPUT	cmb_sedimentation_time needs s (sedimentation coefficient in svedbergs). Example: solve('cmb_sedimentation_time', s=70,
Sedimentation time: s = 70 S, rpm = 40000 rpm, r_min = 5 cm, r_max = 10 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Sedimentation time: s = 70 S, rpm = 40000 rpm, r_min = 5 cm, r_max = 10 cm	SOLVED	t = 5643.52 s, t_h = 1.56764, k_factor = 109.735

Light microscope: total magnification and resolution limit, `cmb_microscope_resolution`

The total magnification of a compound light microscope (objective x eyepiece) and its resolution limit d = lambda / (2 NA) (Abbe).

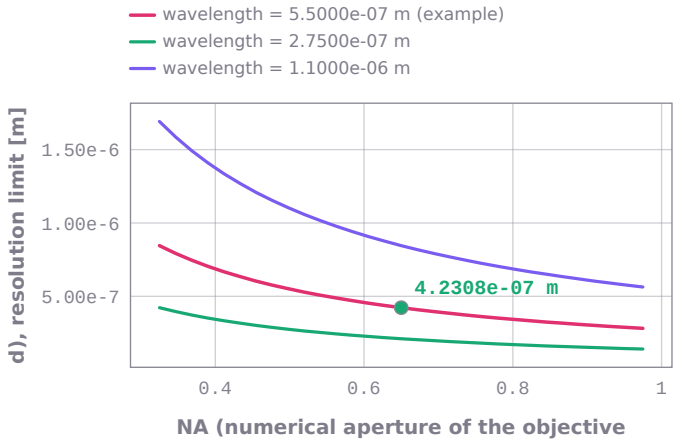
$$M = M_{obj}\,M_{oc}, \quad d = \frac{\lambda}{2\,\mathrm{NA}}$$

input	meaning	condition
`M_obj`	magnification of the objective lens (e.g. 40 for 40x)	
`M_oc`	magnification of the eyepiece (ocular) lens (e.g. 10 for 10x)	
`wavelength`	wavelength of the light	
`NA`	numerical aperture of the objective	

output	meaning
`M`	total magnification

output	meaning
`d`	resolution limit (smallest resolvable separation)
`d_nm`	resolution limit in nanometres
`d_um`	resolution limit in micrometres

- **Method:** M = M_objective x M_eyepiece; d = lambda / (2 NA) (Abbe)
- **Assumes:** magnifications > 0; wavelength > 0; $\mathbf{0 < NA \leq 1.6}$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



d (resolution limit (smallest resolvable separation)) against NA (numerical aperture of the objective) from 0.325 to 0.975. One curve for each value of wavelength (wavelength of the light): 5.5000e-07 m, 2.7500e-07 m, 1.1000e-06 m. Other inputs as in the worked example: M_obj = 40.0, M_oc = 10.0. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, NA = 0.65, d = 4.2308e-07 m.

```
cell_molecular_biology_ai999x.solve('cmb_microscope_resolution', M_obj=40.0, M_oc=10.0, wavelength=5.5e-07, NA=0.65)
# -> M = 400, d = 4.23077e-07 m, d_nm = 423.077, d_um = 0.423077   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_microscope_resolution', M_obj=100.0, M_oc=10.0, wavelength=4.5e-07, NA=1.25)
# -> M = 1000, d = 1.8e-07 m, d_nm = 180, d_um = 0.18   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Microscope resolution: M_obj = 40 x, M_oc = 10 x, wavelength = 550 nm, NA = 0.65	SOLVED	M = 400, d = 4.23077e-07 m, d_nm = 423.077, d_um = 0.423077
Microscope resolution: M_obj = 100 x, M_oc = 10 x, wavelength = 450 nm, NA = 1.25	SOLVED	M = 1000, d = 1.8e-07 m, d_nm = 180, d_um = 0.18
Microscope resolution: M_oc = 10 x, wavelength = 550 nm, NA = 0.65	NEEDS_INPUT	cmb_microscope_resolution needs M_obj (magnification of the objective lens (e.g. 40 for 40x)). Example: solve('cmb_micro
Microscope resolution: M_obj = 40 x, M_oc = 10 x, wavelength = 550 nm, NA = 0.65, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Microscope resolution: M_obj = 40 x, M_oc = 10 x, wavelength = 0.55 um, NA = 0.65	SOLVED	M = 400, d = 4.23077e-07 m, d_nm = 423.077, d_um = 0.423077
Microscope resolution: M_obj = 40, M_oc = 10, wavelength = 550 nm, NA = 2	INVALID_INPUT	Light microscope: total magnification and resolution limit: the numerical aperture of a light microscope cannot exceed a

Actual size of a specimen from a micrograph, `cmb_specimen_size`

The actual size of an object from its size measured on an image (drawing or micrograph), by the magnification or by the image's scale bar; also the magnification of the image.

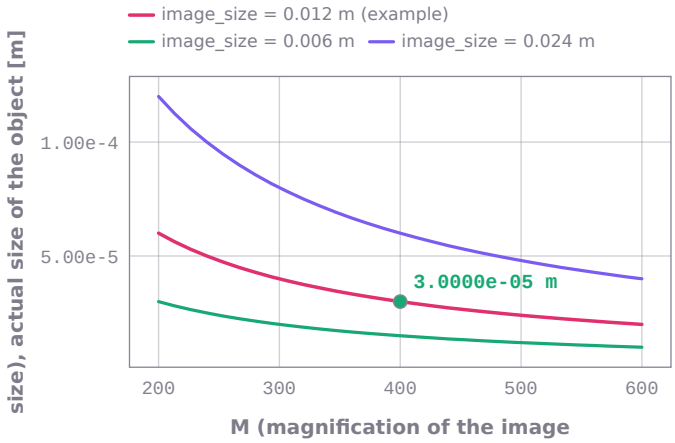
$$\text{actual size} = \frac{\text{image size}}{M}, \quad M = \frac{\text{bar on image}}{\text{bar label}}$$

input	meaning	condition
`image_size`	size of the object measured on the image	
`M`	magnification of the image (give this or the scale bar)	
`bar_image`	length of the scale bar measured on the image (optional, with bar_label)	
`bar_label`	length the scale bar stands for (optional, with bar_image)	

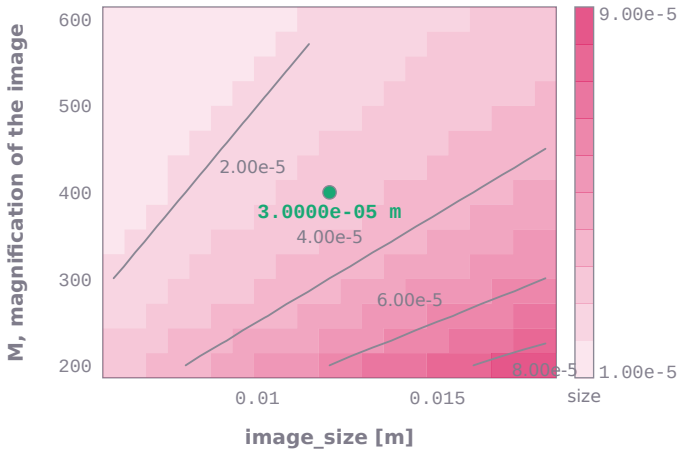
output	meaning
`size`	actual size of the object
`size_um`	actual size in micrometres
`M_image`	magnification of the image

- **Method:** actual size = image size / M; with a scale bar M = bar length on the image / length it represents

- **Assumes:** all lengths > 0; either the magnification or both scale-bar lengths, not both
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



size (actual size of the object) against M (magnification of the image (give this or the scale bar)) from 200 to 600. One curve for each value of image_size (size of the object measured on the image): 0.012 m, 0.006 m, 0.024 m. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, M = 400, size = 3.0000e-05 m.



size (actual size of the object) over image_size (size of the object measured on the image) from 0.006 m to 0.018 m and M (magnification of the image (give this or the scale bar)) from 200 to 600: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light 1.0000e-05 m to dark 9.0000e-05 m); iso-lines at size = 2.00e-5, 4.00e-5, 6.00e-5, 8.00e-5 m. Marked: the worked example, image_size = 0.012 m, M = 400, size = 3.0000e-05 m.

```
cell_molecular_biology_ai999x.solve('cmb_specimen_size', image_size=0.012, M=400.0)
# -> size = 3e-05 m, size_um = 30, M_image = 400   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_specimen_size', image_size=0.018, bar_image=0.025, bar_label=1e-05)
# -> size = 7.2e-06 m, size_um = 7.2, M_image = 2500   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Specimen size from a micrograph: image_size = 12 mm, M = 400 x	SOLVED	size = 3e-05 m, size_um = 30, M_image = 400
Specimen size from a micrograph: image_size = 18 mm, bar_image = 25 mm, bar_label = 10 um	SOLVED	size = 7.2e-06 m, size_um = 7.2, M_image = 2500
Specimen size from a micrograph: M = 400 x	NEEDS_INPUT	cmb_specimen_size needs image_size (size of the object measured on the image). Example: solve('cmb_specimen_size', image
Specimen size from a micrograph: image_size = 12 mm, M = 400 x, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Actual size from magnification: image_size = 1.2 cm, M = 400 x	SOLVED	size = 3e-05 m, size_um = 30, M_image = 400
Specimen size from a micrograph: image_size = 18 mm, M = 400, bar_image = 25 mm, bar_label = 10 um	INVALID_INPUT	Actual size of a specimen from a micrograph: give either the magnification M or the scale bar (bar_image and bar_label),

Water potential of a plant cell or solution (solute plus pressure potential), `cmb_water_potential`

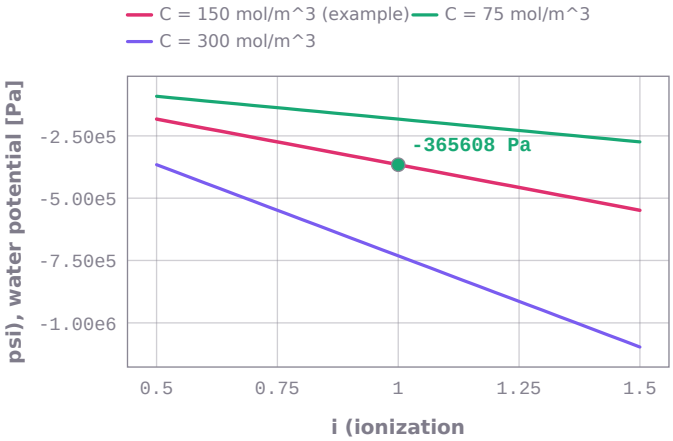
The solute potential $psi_s = -i$ C R T and the water potential $\psi = psi_s + psi_p$ of a solution or cell (pure water at atmospheric pressure = 0).

$$\Psi_s = -iCRT, \quad \Psi = \Psi_s + \Psi_p$$

input	meaning	condition
`i`	ionization (van 't Hoff) constant: 1 for sucrose, about 2 for NaCl	
`C`	molar concentration of the solute	
`T`	absolute temperature (degC is converted)	
`psi_p`	pressure potential (turgor; 0 for an open solution)	

output	meaning
`psi_s`	solute (osmotic) potential
`psi_s_MPa`	solute potential in MPa
`psi`	water potential
`psi_MPa`	water potential in MPa

- **Method:** $psi_s = -i$ C R T; $\psi = psi_s + psi_p$
- **Assumes:** $i > 0$, $T > 0$; $C \geq 0$; ideal dilute solution
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



psi (water potential) against i (ionization (van 't Hoff) constant: 1 for sucrose, about 2 for NaCl) from 0.5 to 1.5. One curve for each value of C (molar concentration of the solute): 150 mol/m^3, 75 mol/m^3, 300 mol/m^3. Other inputs as in the worked example: T = 293.15 K, psi_p = 0.0 Pa. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, i = 1, psi = -365608 Pa.

```
cell_molecular_biology_ai999x.solve('cmb_water_potential', i=1.0, C=150.0, T=293.15, psi_p=0.0)
# -> psi_s = -365608 Pa, psi_s_MPa = -0.365608, psi = -365608 Pa, psi_MPa = -0.365608   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_water_potential', i=2.0, C=100.0, T=298.15, psi_p=300000.0)
# -> psi_s = -495791 Pa, psi_s_MPa = -0.495791, psi = -195791 Pa, psi_MPa = -0.195791   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Water potential: i = 1, C = 150 mM, T = 293.15 K, psi_p = 0 MPa	SOLVED	psi_s = -365608 Pa, psi_s_MPa = -0.365608, psi = -365608 Pa, psi_MPa = -0.365608
Water potential: i = 2, C = 100 mM, T = 298.15 K, psi_p = 0.3 MPa	SOLVED	psi_s = -495791 Pa, psi_s_MPa = -0.495791, psi = -195791 Pa, psi_MPa = -0.195791
Water potential: C = 150 mM, T = 293.15 K, psi_p = 0 MPa	NEEDS_INPUT	cmb_water_potential needs i (ionization (van 't Hoff) constant: 1 for sucrose, about 2 for NaCl). Example: solve('cmb_wa
Water potential: i = 1, C = 150 mM, T = 293.15 K, psi_p = 0 MPa, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Water potential: i = 1, C = 0.15 M, T = 20 degC, psi_p = 0 MPa	SOLVED	psi_s = -365608 Pa, psi_s_MPa = -0.365608, psi = -365608 Pa, psi_MPa = -0.365608
Water potential: i = 0, C = 0.15 M, T = 293.15 K, psi_p = 0 MPa	INVALID_INPUT	Water potential of a plant cell or solution (solute plus pressure potential): the van 't Hoff factor i and T must be a p

Direction of water movement between a cell and its surroundings, `cmb_water_movement`

The water potentials of a cell and of the surrounding solution and the direction of net water movement (from higher to lower water potential).

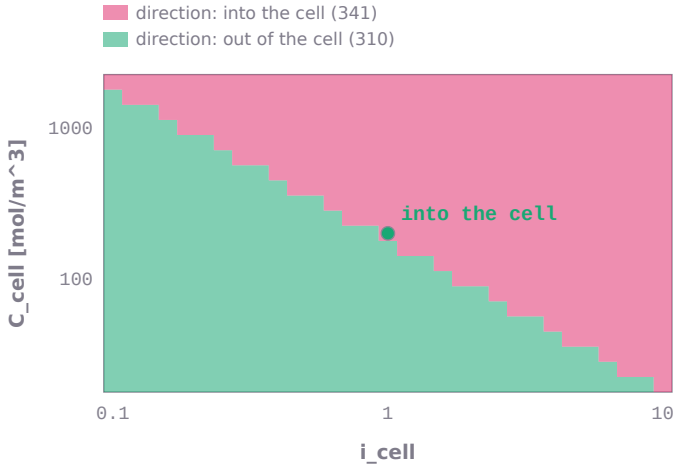
$$\Psi = -iCRT + \Psi_p, \quad \text{water moves toward lower } \Psi$$

input	meaning	condition
`i_cell`	ionization constant of the cell's solute	
`C_cell`	solute concentration in the cell	
`P_cell`	pressure potential of the cell (turgor)	
`i_out`	ionization constant of the outside solute	
`C_out`	solute concentration outside	

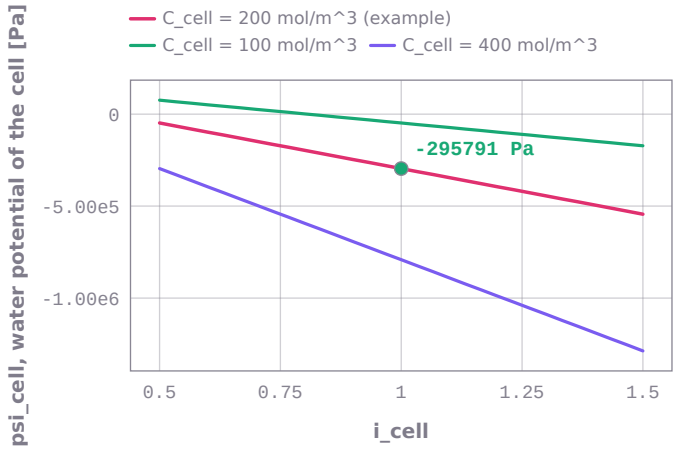
input	meaning	condition
`P_out`	pressure potential outside (0 for an open solution)	
`T`	absolute temperature (degC is converted)	

output	meaning
`psi_cell`	water potential of the cell
`psi_out`	water potential outside
`dpsi`	outside minus cell water potential
`dpsi_MPa`	outside minus cell water potential in MPa
`direction`	net water movement: into the cell, out of the cell or no net movement

- **Method:** $\psi = -i$ C R T + psi_p for each side; water moves from the higher to the lower water potential
- **Assumes:** $i > 0$, $T > 0$; concentrations >= 0; equal within $1e-9 = no$ net movement
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



net water movement: into the cell, out of the cell or no net movement (direction) over i_cell (ionization constant of the cell's solute) from 0.1 to 10 and C_cell (solute concentration in the cell) from 20 mol/m^3 to 2000 mol/m^3 (logarithmic axes): each of the 31 × 21 cells is one run of cell-molecular-biology-ai999x, coloured by the verdict it returned, into the cell: 341 cells, out of the cell: 310 cells. Other inputs as in the worked example: P_cell = 200000.0 Pa, i_out = 1.0, C_out = 100.0 mol/m^3, P_out = 0.0 Pa, T = 298.15 K. Marked: the worked example, i_cell = 1, C_cell = 200 mol/m^3 → into the cell.



psi_cell (water potential of the cell) against i_cell (ionization constant of the cell's solute) from 0.5 to 1.5. One curve for each value of C_cell (solute concentration in the cell): 200 mol/m^3, 100 mol/m^3, 400 mol/m^3. Other inputs as in the worked example: P_cell = 200000.0 Pa, i_out = 1.0, C_out = 100.0 mol/m^3, P_out = 0.0 Pa, T = 298.15 K. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, i_cell = 1, psi_cell = -295791 Pa.

```
cell_molecular_biology_ai999x.solve('cmb_water_movement', i_cell=1.0, C_cell=200.0, P_cell=200000.0, i_out=1.0,
C_out=100.0, P_out=0.0, T=298.15)
# -> psi_cell = -295791 Pa, psi_out = -247896 Pa, dpsi = 47895.7 Pa, dpsi_MPa = 0.0478957, direction = into the cell
(SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_water_movement', i_cell=1.0, C_cell=300.0, P_cell=0.0, i_out=2.0, C_out=200.0,
P_out=0.0, T=310.15)
# -> psi_cell = -773619 Pa, psi_out = -1.03149e+06 Pa, dpsi = -257873 Pa, dpsi_MPa = -0.257873, direction = out of the
cell   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Water movement between a cell and its surroundings: i_cell = 1, C_cell = 200 mM, P_cell = 0.2 MPa, i_out = 1, C_out = 100 mM, P_out = 0 MPa, T = 298.15 K	SOLVED	psi_cell = -295791 Pa, psi_out = -247896 Pa, dpsi = 47895.7 Pa, dpsi_MPa = 0.0478957, direction = into the cell

question	outcome	result
Water movement between a cell and its surroundings: i_cell = 1, C_cell = 300 mM, P_cell = 0 MPa, i_out = 2, C_out = 200 mM, P_out = 0 MPa, T = 310.15 K	SOLVED	psi_cell = -773619 Pa, psi_out = -1.03149e+06 Pa, dpsi = -257873 Pa, dpsi_MPa = -0.257873, direction = out of the cell
Water movement between a cell and its surroundings: C_cell = 200 mM, P_cell = 0.2 MPa, i_out = 1, C_out = 100 mM, P_out = 0 MPa, T = 298.15 K	NEEDS_INPUT	cmb_water_movement needs i_cell (ionization constant of the cell's solute). Example: solve('cmb_water_movement', i_cell=
Water movement between a cell and its surroundings: i_cell = 1, C_cell = 200 mM, P_cell = 0.2 MPa, i_out = 1, C_out = 100 mM, P_out = 0 MPa, T = 298.15 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Direction of water movement: i_cell = 1, C_cell = 0.2 M, P_cell = 0.2 MPa, i_out = 1, C_out = 0.1 M, P_out = 0 MPa, T = 25 degC	SOLVED	psi_cell = -295791 Pa, psi_out = -247896 Pa, dpsi = 47895.7 Pa, dpsi_MPa = 0.0478957, direction = into the cell
Direction of water movement: i_cell = 1, C_cell = -200 mM, P_cell = 0.2 MPa, i_out = 1, C_out = 100 mM, P_out = 0 MPa, T = 298.15 K	INVALID_INPUT	Direction of water movement between a cell and its surroundings: the concentrations must be zero or positive.

Free-energy change of a reaction at cellular concentrations, `cmb_reaction_free_energy`

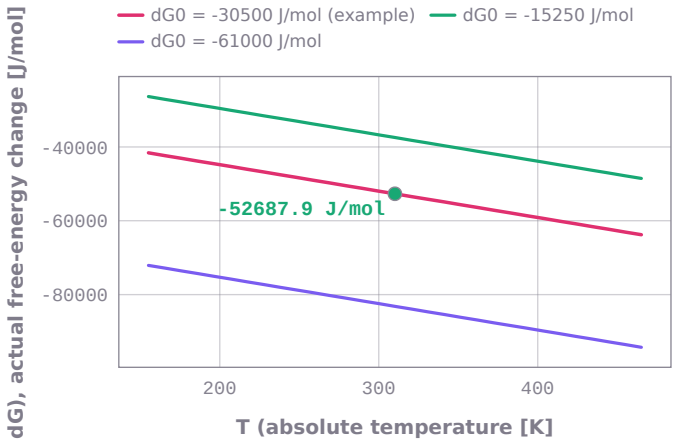
The actual free-energy change $dG = dG0'$ + RT ln Q of a biochemical reaction at the concentrations in the cell (standard state 1 M), its equilibrium constant and the direction it proceeds.

$$\Delta G = \Delta G^{\circ\prime} + RT \ln Q, \quad Q = \frac{\prod [P]}{\prod [R]}$$

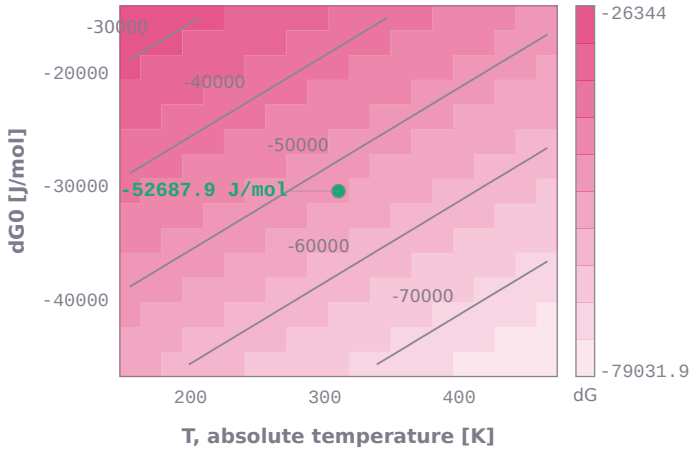
input	meaning	condition
`dG0`	standard free-energy change dG0' (biochemical standard state, pH 7)	
`T`	absolute temperature (degC is converted)	
`products`	product concentrations in mM (list a species twice for a coefficient of 2; omit water and H+)	
`reactants`	reactant concentrations in mM (list a species twice for a coefficient of 2; omit water and H+)	

output	meaning
`Q`	reaction quotient (concentrations relative to 1 M)
`dG`	actual free-energy change
`dG_kJ_mol`	actual free-energy change in kJ/mol
`K_eq`	equilibrium constant exp(-dG0'/RT)
`direction`	forward $(dG < 0)$, reverse $(dG > 0)$ or at equilibrium

- **Method:** Q = prod(products / 1 M) / prod(reactants / 1 M); $dG = dG0'$ + R T ln Q; $K = exp$(-dG0'/RT)
- **Assumes:** $T > 0$; |dG0'| <= 500 kJ/mol; 0 < each concentration <= 10 M; ideal (activities = concentrations / 1 M)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dG (actual free-energy change) against T (absolute temperature (degC is converted)) from 155.075 K to 465.225 K. One curve for each value of dG0 (standard free-energy change dG0' (biochemical standard state, pH 7)): -30500 J/mol, -15250 J/mol, -61000 J/mol. Other inputs as in the worked example: products = [0.25, 1.65], reactants = [2.25]. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, T = 310.15 K, dG = -52687.9 J/mol.



dG (actual free-energy change) over T (absolute temperature (degC is converted)) from 155.075 K to 465.225 K and dG0 (standard free-energy change dG0' (biochemical standard state, pH 7)) from -45750 J/mol to -15250 J/mol: each of the 21 × 15 cells is one run of cell-molecular-biology-ai999x, shaded by its value (light -79031.9 J/mol to dark -26344 J/mol); iso-lines at dG = -70000, -60000, -50000, -40000, -30000 J/mol. Other inputs as in the worked example: products = [0.25, 1.65], reactants = [2.25]. Marked: the worked example, T = 310.15 K, dG0 = -30500 J/mol, dG = -52687.9 J/mol.

```
cell_molecular_biology_ai999x.solve('cmb_reaction_free_energy', dG0=-30500.0, T=310.15, products=[0.25, 1.65],
reactants=[2.25])
# -> Q = 0.000183333, dG = -52687.9 J/mol, dG_kJ_mol = -52.6879, K_eq = 136971, direction = forward   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_reaction_free_energy', dG0=5000.0, T=298.15, products=[1.0], reactants=
[1000.0])
# -> Q = 0.001, dG = -12124 J/mol, dG_kJ_mol = -12.124, K_eq = 0.133057, direction = forward   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Free energy at cellular concentrations: dG0 = -30.5 kJ/mol, T = 310.15 K, products = [0.25, 1.65], reactants = [2.25]	SOLVED	Q = 0.000183333, dG = -52687.9 J/mol, dG_kJ_mol = -52.6879, K_eq = 136971, direction = forward
Free energy at cellular concentrations: dG0 = 5 kJ/mol, T = 298.15 K, products = [1], reactants = [1000]	SOLVED	Q = 0.001, dG = -12124 J/mol, dG_kJ_mol = -12.124, K_eq = 0.133057, direction = forward
Free energy at cellular concentrations: T = 310.15 K, products = [0.25, 1.65], reactants = [2.25]	NEEDS_INPUT	cmb_reaction_free_energy needs dG0 (standard free-energy change dG0' (biochemical standard state, pH 7)). Example: solve
Free energy at cellular concentrations: dG0 = -30.5 kJ/mol, T = 310.15 K, products = [0.25, 1.65], reactants = [2.25], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Reaction free energy: dG0 = -30.5 kJ/mol, T = 37 degC, products = [0.25, 1.65], reactants = [2.25]	SOLVED	Q = 0.000183333, dG = -52687.9 J/mol, dG_kJ_mol = -52.6879, K_eq = 136971, direction = forward
Reaction free energy: dG0 = -30.5 kJ/mol, T = 310.15 K, products = [0, 1.65], reactants = [2.25]	INVALID_INPUT	Free-energy change of a reaction at cellular concentrations: concentrations must be above 0 and at most 10 M (10000 mM).

Free-energy change of a redox reaction from reduction potentials, `cmb_redox_free_energy`

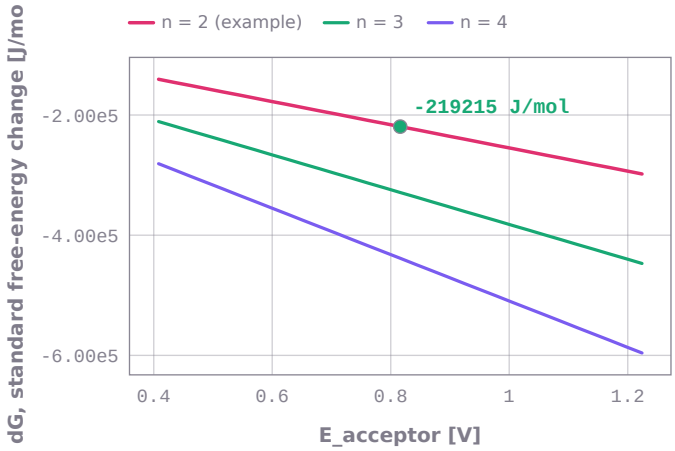
The standard free-energy change dG0' = -n F dE0' of electron transfer from a donor couple to an acceptor couple, from their standard reduction potentials.

$$\Delta G^{o\prime} = -nF\,\Delta E^{o\prime}, \quad \Delta E^{o\prime} = E^{o\prime}_{acceptor} - E^{o\prime}_{donor}$$

input	meaning	condition
`E_acceptor`	standard reduction potential of the electron acceptor couple (E0')	
`E_donor`	standard reduction potential of the electron donor couple (E0')	
`n`	number of electrons transferred	

output	meaning
`dE`	potential difference E0'(acceptor) - E0'(donor)
`dE_mV`	potential difference in mV
`dG`	standard free-energy change
`dG_kJ_mol`	standard free-energy change in kJ/mol
`direction`	favourable as written $(dG < 0)$, unfavourable or at equilibrium

- **Method:** dE0' = E0'(acceptor) - E0'(donor); dG0' = -n F dE0'
- **Assumes:** n a whole number >= 1; |E0'| <= 3 V; standard (pH 7) conditions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dG (standard free-energy change) against E_acceptor (standard reduction potential of the electron acceptor couple (E0')) from 0.408 V to 1.224 V. One curve for each value of n (number of electrons transferred): 2, 3, 4. Other inputs as in the worked example: E_donor = -0.32 V. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, E_acceptor = 0.816 V, dG = -219215 J/mol.

```
cell_molecular_biology_ai999x.solve('cmb_redox_free_energy', E_acceptor=0.816, E_donor=-0.32, n=2)
# -> dE = 1.136 V, dE_mV = 1136, dG = -219215 J/mol, dG_kJ_mol = -219.215, direction = favourable as written   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_redox_free_energy', E_acceptor=0.045, E_donor=-0.219, n=2)
# -> dE = 0.264 V, dE_mV = 264, dG = -50944.3 J/mol, dG_kJ_mol = -50.9443, direction = favourable as written   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Redox free energy: E_acceptor = 0.816 V, E_donor = -0.32 V, n = 2	SOLVED	dE = 1.136 V, dE_mV = 1136, dG = -219215 J/mol, dG_kJ_mol = -219.215, direction = favourable as written
Redox free energy: E_acceptor = 0.045 V, E_donor = -0.219 V, n = 2	SOLVED	dE = 0.264 V, dE_mV = 264, dG = -50944.3 J/mol, dG_kJ_mol = -50.9443, direction = favourable as written
Redox free energy: E_donor = -0.32 V, n = 2	NEEDS_INPUT	cmb_redox_free_energy needs E_acceptor (standard reduction potential of the electron acceptor couple (E0')). Example: so
Redox free energy: E_acceptor = 0.816 V, E_donor = -0.32 V, n = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Redox free energy: E_acceptor = 816 mV, E_donor = -320 mV, n = 2	SOLVED	dE = 1.136 V, dE_mV = 1136, dG = -219215 J/mol, dG_kJ_mol = -219.215, direction = favourable as written
Redox free energy: E_acceptor = 0.816 V, E_donor = -0.32 V, n = 0	INVALID_INPUT	n: must be at least 1, got 0.

ATP yield per glucose: aerobic respiration versus fermentation, `cmb_atp_yield`

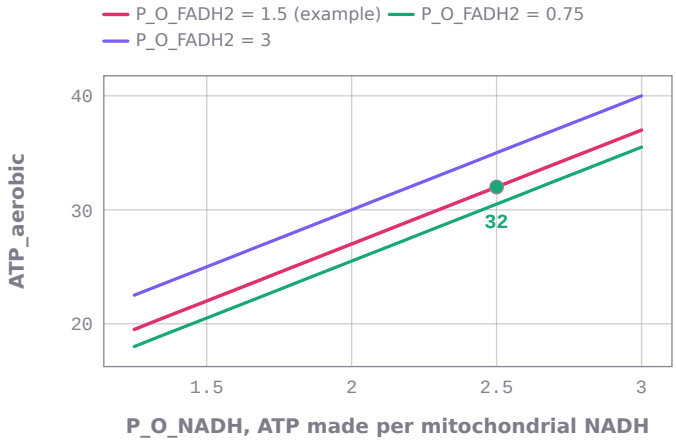
The ATP made per glucose by aerobic respiration (substrate-level plus oxidative phosphorylation from NADH and FADH2 at stated P/O ratios and the shuttle for cytosolic NADH) and by fermentation, and optionally the energy efficiency of each.

$$N_{ATP} = 4 + 8\,(P/O)_{NADH} + 2\,(P/O)_{FADH_2} + 2\,(P/O)_{shuttle}, \quad \eta = \frac{N_{ATP}\,\Delta G_{ATP}}{\Delta G_{glucose}}$$

input	meaning	condition
`P_O_NADH`	ATP made per mitochondrial NADH (P/O ratio, e.g. 2.5 or 3)	
`P_O_FADH2`	ATP made per FADH2 (P/O ratio, e.g. 1.5 or 2)	
`shuttle`	shuttle that carries the 2 cytosolic NADH into the mitochondrion	
`dG_ATP`	free energy stored per mol of ATP, a positive magnitude (optional, e.g. 30.5 kJ/mol)	
`dG_glucose`	free energy released by complete oxidation of glucose, a positive magnitude (optional, e.g. 2870 kJ/mol)	

output	meaning
`ATP_aerobic`	ATP per glucose by aerobic respiration
`ATP_fermentation`	ATP per glucose by fermentation
`ratio`	aerobic yield / fermentation yield
`ATP_oxidative`	ATP from oxidative phosphorylation
`ATP_substrate_level`	ATP (and GTP) from substrate-level phosphorylation, net
`NADH_total`	NADH made per glucose
`FADH2_total`	FADH2 made per glucose
`efficiency_aerobic`	fraction of glucose's free energy stored in ATP, aerobic (when energies are given)
`efficiency_fermentation`	fraction of glucose's complete-oxidation free energy stored in ATP, fermentation (when energies are given)

- **Method:** ATP = 4 (substrate level, net) + 8 P/O(NADH) + 2 P/O(FADH2) + 2 P/O(shuttle); fermentation = 2; efficiency = ATP dG_ATP / dG_glucose
- **Assumes:** $0 < P/O \leq 3$; both energies or neither; efficiency <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ATP_aerobic (ATP per glucose by aerobic respiration) against P_O_NADH (ATP made per mitochondrial NADH (P/O ratio, e.g. 2.5 or 3)) from 1.25 to 3.75. One curve for each value of P_O_FADH2 (ATP made per FADH2 (P/O ratio, e.g. 1.5 or 2)): 1.5, 0.75, 3. Other inputs as in the worked example: shuttle = malate-aspartate, dG_ATP = 30500.0 J/mol, dG_glucose = 2870000.0 J/mol. 22 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, P_O_NADH = 2.5, ATP_aerobic = 32.

```
cell_molecular_biology_ai999x.solve('cmb_atp_yield', P_O_NADH=2.5, P_O_FADH2=1.5, shuttle='malate-aspartate',
dG_ATP=30500.0, dG_glucose=2870000.0)
# -> ATP_aerobic = 32, ATP_fermentation = 2, ratio = 16, ATP_oxidative = 28, ATP_substrate_level = 4, NADH_total = 10,
FADH2_total = 2, efficiency_aerobic = 0.34007, efficiency_fermentation = 0.0212544   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_atp_yield', P_O_NADH=2.5, P_O_FADH2=1.5, shuttle='glycerol-phosphate')
# -> ATP_aerobic = 30, ATP_fermentation = 2, ratio = 15, ATP_oxidative = 26, ATP_substrate_level = 4, NADH_total = 10,
FADH2_total = 2   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_atp_yield', P_O_NADH=3.0, P_O_FADH2=2.0, shuttle='malate-aspartate')
# -> ATP_aerobic = 38, ATP_fermentation = 2, ratio = 19, ATP_oxidative = 34, ATP_substrate_level = 4, NADH_total = 10,
FADH2_total = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ATP yield per glucose: P_O_NADH = 2.5, P_O_FADH2 = 1.5, shuttle = malate-aspartate, dG_ATP = 30.5 kJ/mol, dG_glucose = 2870 kJ/mol	SOLVED	ATP_aerobic = 32, ATP_fermentation = 2, ratio = 16, ATP_oxidative = 28, ATP_substrate_level = 4, NADH_total = 10, FADH2_total = 2, efficiency_aerobic = 0.34007, efficiency_fermentation = 0.0212544
ATP yield per glucose: P_O_NADH = 2.5, P_O_FADH2 = 1.5, shuttle = glycerol-phosphate	SOLVED	ATP_aerobic = 30, ATP_fermentation = 2, ratio = 15, ATP_oxidative = 26, ATP_substrate_level = 4, NADH_total = 10, FADH2_total = 2
ATP yield per glucose: P_O_NADH = 3, P_O_FADH2 = 2, shuttle = malate-aspartate	SOLVED	ATP_aerobic = 38, ATP_fermentation = 2, ratio = 19, ATP_oxidative = 34, ATP_substrate_level = 4, NADH_total = 10, FADH2_total = 2
ATP yield per glucose: P_O_FADH2 = 1.5, shuttle = malate-aspartate, dG_ATP = 30.5 kJ/mol, dG_glucose = 2870 kJ/mol	NEEDS_INPUT	cmb_atp_yield needs P_O_NADH (ATP made per mitochondrial NADH (P/O ratio, e.g. 2.5 or 3)). Example: solve('cmb_atp_yield
ATP yield per glucose: P_O_NADH = 2.5, P_O_FADH2 = 1.5, shuttle = malate-aspartate, dG_ATP = 30.5 kJ/mol, dG_glucose = 2870 kJ/mol, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
ATP per glucose: P_O_NADH = 2.5, P_O_FADH2 = 1.5, shuttle = malate aspartate, dG_ATP = 30.5 kJ/mol, dG_glucose = 2870 kJ/mol	SOLVED	ATP_aerobic = 32, ATP_fermentation = 2, ratio = 16, ATP_oxidative = 28, ATP_substrate_level = 4, NADH_total = 10, FADH2_total = 2, efficiency_aerobic = 0.34007, efficiency_fermentation = 0.0212544
ATP yield per glucose: P_O_NADH = 5, P_O_FADH2 = 1.5, shuttle = malate-aspartate	INVALID_INPUT	ATP yield per glucose: aerobic respiration versus fermentation: P/O ratios must be at most 3.

D-value (decimal reduction time) from survivor counts, `cmb_decimal_reduction_time`

The number of log10 reductions and the D-value (time for a 90 % kill) of a treatment from the viable counts before and after a stated time, assuming first-order (log-linear) death.

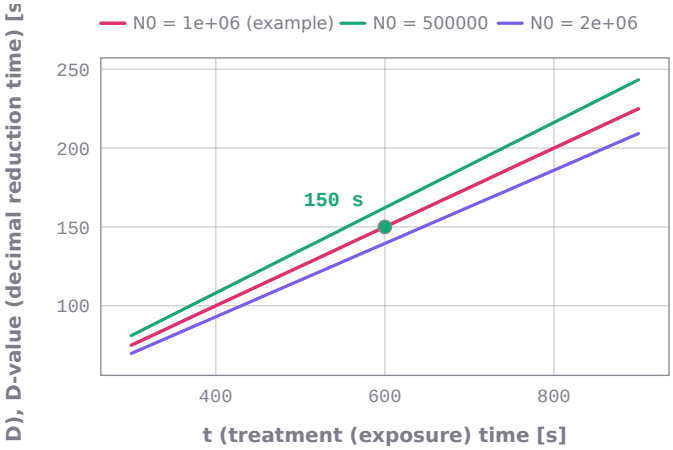
$$D = \frac{t}{\log_{10}(N_0/N)}, \quad k = \frac{\ln 10}{D}$$

input	meaning	condition
`N0`	initial viable count (CFU/mL or CFU)	

input	meaning	condition
`N`	surviving viable count after the treatment (same units as N0)	
`t`	treatment (exposure) time	

output	meaning
`log_reduction`	number of log10 reductions log10(N0/N)
`D`	D-value (decimal reduction time)
`D_min`	D-value in minutes
`k_per_min`	first-order death rate constant in 1/min
`survival_fraction`	surviving fraction N/N_0

- **Method:** log reductions = log10(N0/N); $D = t/log10$(N0/N); $k = ln\ 10/D$
- **Assumes:** $0 < N < N_0$; $t > 0$; first-order (log-linear) killing
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



D (D-value (decimal reduction time)) against t (treatment (exposure) time) from 300 s to 900 s. One curve for each value of N0 (initial viable count (CFU/mL or CFU)): 1e+06, 500000, 2e+06. Other inputs as in the worked example: N = 100.0. 31 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, t = 600 s, D = 150 s.

```
cell_molecular_biology_ai999x.solve('cmb_decimal_reduction_time', N0=1000000.0, N=100.0, t=600.0)
# -> log_reduction = 4, D = 150 s, D_min = 2.5, k_per_min = 0.921034, survival_fraction = 0.0001   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_decimal_reduction_time', N0=50000000.0, N=50000.0, t=270.0)
# -> log_reduction = 3, D = 90 s, D_min = 1.5, k_per_min = 1.53506, survival_fraction = 0.001   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
D-value from survivor counts: N0 = 1000000, N = 100, t = 10 min	SOLVED	log_reduction = 4, D = 150 s, D_min = 2.5, k_per_min = 0.921034, survival_fraction = 0.0001
D-value from survivor counts: N0 = 50000000, N = 50000, t = 4.5 min	SOLVED	log_reduction = 3, D = 90 s, D_min = 1.5, k_per_min = 1.53506, survival_fraction = 0.001
D-value from survivor counts: N = 100, t = 10 min	NEEDS_INPUT	cmb_decimal_reduction_time needs N0 (initial viable count (CFU/mL or CFU)). Example: solve('cmb_decimal_reduction_time',
D-value from survivor counts: N0 = 1000000, N = 100, t = 10 min, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Decimal reduction time: N0 = 1e6, N = 100, t = 10 min	SOLVED	log_reduction = 4, D = 150 s, D_min = 2.5, k_per_min = 0.921034, survival_fraction = 0.0001
D-value: N0 = 1000, N = 5000, t = 10 min	INVALID_INPUT	D-value (decimal reduction time) from survivor counts: the surviving count N must be below the initial count N0.

z-value from D-values at two temperatures, `cmb_z_value`

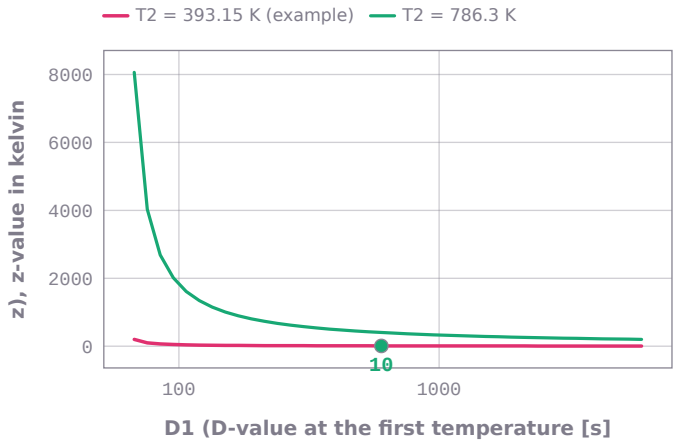
The z-value (temperature rise that reduces the D-value ten-fold) from the D-values measured at two temperatures, and the fold change of D per 10 degrees.

$$z = \frac{T_2 - T_1}{\log_{10}(D_1/D_2)}$$

input	meaning	condition
`D1`	D-value at the first temperature	
`T1`	first temperature (degC is converted)	
`D2`	D-value at the second temperature	
`T2`	second temperature (degC is converted)	

output	meaning
`z`	z-value in kelvin (= degrees Celsius of difference)
`fold_per_10_degrees`	factor by which D falls for a 10-degree rise

- **Method:** $z = (T_2 - T_1)/log10$(D1 / D2)
- **Assumes:** D-values and temperatures > 0; $T_1 \neq T_2$; D falls as T rises $(z > 0)$; log-linear thermal death curve
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



z (z-value in kelvin (= degrees Celsius of difference)) against D1 (D-value at the first temperature) from 60 s to 6000 s on a logarithmic axis. One curve for each value of T2 (second temperature (degC is converted)): 393.15 K, 786.3 K. Other inputs as in the worked example: T1 = 383.15 K, D2 = 60.0 s. 40 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, D1 = 600 s, z = 10.

```
cell_molecular_biology_ai999x.solve('cmb_z_value', D1=600.0, T1=383.15, D2=60.0, T2=393.15)
# -> z = 10, fold_per_10_degrees = 10   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_z_value', D1=240.0, T1=385.15, D2=30.0, T2=394.15)
# -> z = 9.96578, fold_per_10_degrees = 10.0794   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
z-value: D1 = 10 min, T1 = 383.15 K, D2 = 1 min, T2 = 393.15 K	SOLVED	z = 10, fold_per_10_degrees = 10
z-value: D1 = 4 min, T1 = 385.15 K, D2 = 0.5 min, T2 = 394.15 K	SOLVED	z = 9.96578, fold_per_10_degrees = 10.0794
z-value: T1 = 383.15 K, D2 = 1 min, T2 = 393.15 K	NEEDS_INPUT	cmb_z_value needs D1 (D-value at the first temperature). Example: solve('cmb_z_value', D1='600 s', T1='383.15 K', D2='60
z-value: D1 = 10 min, T1 = 383.15 K, D2 = 1 min, T2 = 393.15 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
z-value: D1 = 10 min, T1 = 110 degC, D2 = 1 min, T2 = 120 degC	SOLVED	z = 10, fold_per_10_degrees = 10
z-value: D1 = 1 min, T1 = 110 degC, D2 = 10 min, T2 = 120 degC	INVALID_INPUT	z-value from D-values at two temperatures: the D-value must fall as the temperature rises (z > 0).

D-value at a new temperature and time to a target kill (z-value), `cmb_thermal_death_time`

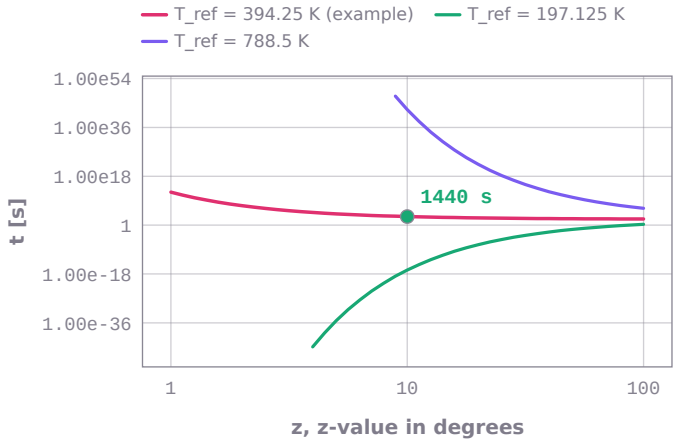
The D-value at temperature T from a reference D-value and the z-value, and the holding time needed for a stated number of log10 reductions at T.

$$D_T = D_{ref}\, 10^{(T_{ref}-T)/z}, \quad t = n\, D_T$$

input	meaning	condition
`D_ref`	D-value at the reference temperature	
`T_ref`	reference temperature (degC is converted)	
`z`	z-value in degrees (a temperature difference: degC = K)	
`T`	process temperature (degC is converted)	
`log_reductions`	number of log10 reductions required (e.g. 12)	

output	meaning
`D_T`	D-value at T
`D_T_min`	D-value at T in minutes
`t`	holding time for the required log reductions at T
`t_min`	holding time in minutes
`lethal_rate`	lethal rate 10^((T - T_ref)/z) relative to the reference temperature

- **Method:** D_T = D_ref x 10^((T_ref - T)/z); $t = n$ D_T
- **Assumes:** D_ref, z, log reductions > 0; |T - T_ref| / $z \leq 50$; log-linear thermal death
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (holding time for the required log reductions at T) against z (z-value in degrees (a temperature difference: degC = K)) from 1 to 100 on a logarithmic axis, logarithmic output axis. One curve for each value of T_ref (reference temperature (degC is converted)): 394.25 K, 197.125 K, 788.5 K. Other inputs as in the worked example: D_ref = 12.0 s, T = 384.25 K, log_reductions = 12.0. 41 points per curve computed by cell-molecular-biology-ai999x; marked: the worked example, z = 10, t = 1440 s.

```
cell_molecular_biology_ai999x.solve('cmb_thermal_death_time', D_ref=12.0, T_ref=394.25, z=10.0, T=384.25,
log_reductions=12.0)
# -> D_T = 120 s, D_T_min = 2, t = 1440 s, t_min = 24, lethal_rate = 0.1   (SOLVED)
```

```
cell_molecular_biology_ai999x.solve('cmb_thermal_death_time', D_ref=15.0, T_ref=394.25, z=10.0, T=399.25,
log_reductions=5.0)
# -> D_T = 4.74342 s, D_T_min = 0.0790569, t = 23.7171 s, t_min = 0.395285, lethal_rate = 3.16228   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thermal death time: D_ref = 0.2 min, T_ref = 394.25 K, z = 10 degC, T = 384.25 K, log_reductions = 12	SOLVED	D_T = 120 s, D_T_min = 2, t = 1440 s, t_min = 24, lethal_rate = 0.1
Thermal death time: D_ref = 0.25 min, T_ref = 394.25 K, z = 10 degC, T = 399.25 K, log_reductions = 5	SOLVED	D_T = 4.74342 s, D_T_min = 0.0790569, t = 23.7171 s, t_min = 0.395285, lethal_rate = 3.16228
Thermal death time: T_ref = 394.25 K, z = 10 degC, T = 384.25 K, log_reductions = 12	NEEDS_INPUT	cmb_thermal_death_time needs D_ref (D-value at the reference temperature). Example: solve('cmb_thermal_death_time', D_re
Thermal death time: D_ref = 0.2 min, T_ref = 394.25 K, z = 10 degC, T = 384.25 K, log_reductions = 12, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: cell_molecular_biology_ai999x.tem
Thermal death time: D_ref = 0.2 min, T_ref = 394.25 K, z = 10 degC, T = 384.25 K, log_reductions = 12	SOLVED	D_T = 120 s, D_T_min = 2, t = 1440 s, t_min = 24, lethal_rate = 0.1
Thermal death time: D_ref = 0.2 min, T_ref = 121.1 degC, z = 0 degC, T = 111.1 degC, log_reductions = 12	INVALID_INPUT	D-value at a new temperature and time to a target kill (z-value): D_ref, T_ref, z, T and the number of log reductions mu